CAN Tool

Handbuch / Manual / Manuel / Manual / 用户手册 / คู่มือ

Ausgabedatum: 2026-10-29

Enthaltene Sprachen

Deutsch: Handbuch – Ausgabedatum: 2026-10-29

English: Manual – Issue date: 2026-10-29

Français: Manuel – Date d'édition : 2026-10-29

Español: Manual – Fecha de emisión: 2026-10-29

中文(简体): 用户手册 – 发布日期:2026-10-29

ไทย: คู่มือ – วันที่เผยแพร่: 2026-10-29

Deutsch – Handbuch

CAN Tool · Ausgabedatum: 2026-10-29

1. Allgemeines und Projekterzeugung

2. Bedatung vor dem Start

3.1 Rubrik Projekt

3.2 Rubrik Daten

3.3 Rubrik Trace

3.4 Rubrik Testsequenz

3.5 Rubrik Graph

3.6 Rubrik Logging

3.7 Rubrik Diagnose

3.8 Rubrikzeile: Dark Mode und Sprache

3.9 Rubrik Hilfe

3.10 Rubrik XCP

3.11 Rubrik GUI Builder

Anhang A: Befehlsreferenz Testsequenz

Anhang B: Hinweise und Grenzen

Anhang C: vFlash-Pack und Manifest

1. Allgemeine Beschreibung und Projekterzeugung

1.1 Wofuer das Tool gedacht ist

Das CAN Tool ist ein Python-Werkzeug zum Testen und Analysieren von CAN- und CAN-FD-Bussen. Es arbeitet wahlweise mit dem **Vector VN1630A / VN7572** oder dem **PEAK-System PCAN-USB Pro FD** (beide ueber python-can) oder - ohne Hardware - mit einer eingebauten **Simulation**.

Kernfunktionen:

- Verwaltung mehrerer **Projekte** (jedes mit eigenen DBC-, CDD-, Sequenz- und Logdateien).
- **DBC/CDD-Konvertierung** nach JSON oder YAML vor der eigentlichen Nutzung (schnelles Laden, einfache Pruefung).
- **Zyklisches Senden** und Empfangen von Botschaften, Signalzugriff in physikalischen Werten.
- **Trace** aller gesendeten und empfangenen Frames, optional mit Signaldekodierung.
- **Testsequenzen** (JSON): Signale setzen, auf Reaktionen warten, pruefen, Diagnose ausfuehren.
- **Liniengraph** ausgewaehlter Signale.
- Umschaltbares Erscheinungsbild (**Dark Mode**) und **sechs Sprachen** (Deutsch, Englisch, Franzoesisch, Spanisch, Chinesisch/Mandarin, Thai).
- **Parquet-Logging** mit Filter.
- **Diagnose** ueber ISO-TP (ISO 15765-2) mit UDS-Services aus der CDD.

1.2 Datenfluss im Ueberblick

```
dbc/*.dbc  --(Konvertierung)-->  cache/*.dbc.json|yaml  ---+
cdd/*.cdd  --(Konvertierung)-->  cache/*.cdd.json|yaml  ---+--> Rubrik "Daten": Auswahl TX / RX
                                                           |        |
                                                           |        +--> Trace   (alle Frames)
                                                           |        +--> Graph   (gewaehlte Signale)
                                                           |        +--> Logging (gewaehlte Signale, Filter)
sequences/*.json  ------------------------------------------+--> Testsequenz (setzt/prueft Signale, Diagnose)
```

Zur Laufzeit wird immer nur der **Cache** (JSON/YAML) gelesen. Aendert sich eine DBC/CDD-Datei, wird sie beim naechsten Oeffnen des Projekts automatisch neu konvertiert (Pruefsumme der Quelldatei).

1.3 Installation und Start

```
pip install -r requirements.txt
python main.py                       # GUI
python main.py --project demo        # GUI mit einem bestimmten Projekt
python main.py --list                # Projekte auflisten
python main.py --convert demo        # nur DBC/CDD konvertieren (ohne GUI)
python main.py --convert demo --format yaml --force
```

Treiber: Vector benoetigt die *Vector XL Driver Library* (Teil der Vector-Treiber), PEAK den *PCAN-Basic*-Treiber. Ohne Hardware genuegt die Simulation.

1.4 Ordnerstruktur

```
can_tool/
  main.py  gui.py  can_manager.py  dbc_manager.py  cdd_manager.py
  iso_tp_manager.py  sequence_runner.py  parquet_logger.py  project_manager.py
  help_content.py  i18n.py
  lang/               Sprachdateien (de.yaml, en.yaml, fr.yaml, es.yaml, zh.yaml, th.yaml)
  tools/check_lang.py Pruefung der Sprachdateien
  projects/
    <Projektname>/
      project.json      Einstellungen des Projekts (wird vom Tool geschrieben)
      dbc/              Ihre DBC-Dateien (*.dbc)
      cdd/              Ihre Diagnosebeschreibungen (*.cdd)
      sequences/        Testsequenzen (*.json)
      cache/            automatisch erzeugt: konvertierte DBC/CDD (JSON oder YAML)
      logs/             Parquet-Logdateien
```

1.5 Projekt erzeugen

In der GUI

1. Rubrik **Projekt** oeffnen und **Neu ...** klicken.
2. Projektnamen eingeben. Erlaubt sind Buchstaben, Ziffern, Unterstrich, Bindestrich und Leerzeichen (max. 64 Zeichen, erstes Zeichen Buchstabe oder Ziffer). Existiert der Name schon, wird nichts ueberschrieben.
3. Das Tool legt `projects/<name>/` mit den Unterordnern `dbc, cdd, sequences, cache, logs` und einer `project.json` mit Standardwerten (Simulation, 500 kbit/s, JSON-Cache) an und wechselt in das neue Projekt.
4. **DBC/CDD importieren ...**: Dateien werden in `dbc/` bzw. `cdd/` kopiert und sofort konvertiert. Alternativ die Dateien von Hand in die Ordner legen und **Neu konvertieren** klicken (oder das Projekt neu oeffnen).
5. Hardware einstellen (siehe Kapitel 2.3) und **Verbinden**.

Per Datei

Einen Projektordner mit der obigen Struktur anlegen und eine `project.json` mit mindestens `{}` hineinlegen - fehlende Einstellungen werden mit Standardwerten ergaenzt. Das Demo-Projekt `projects/demo` dient als Vorlage.

Projekte wechseln

Auswahl in der Projekt-Liste. Beim Wechsel (und beim Beenden) wird die Verbindung getrennt, ein laufendes Logging gestoppt und das bisherige Projekt gespeichert. Gespeichert werden: Hardware, Sendeliste inkl. Signalwerten und Zyklen, Empfangsauswahl, Logging-Filter, Graph-Auswahl, Diagnose-Einstellungen und die zuletzt geladene Sequenz. Das zyklische Senden ist nach dem Oeffnen eines Projekts immer **ausgeschaltet** (Sicherheit am echten Bus).

1.6 Typischer Arbeitsablauf

1. Projekt anlegen oder waehlen, DBC (und CDD) importieren.
2. Hardware einstellen, **Verbinden**.
3. Rubrik **Daten**: Sende- und Empfangsbotschaften aus der DBC waehlen, Signalwerte und Zyklen setzen.
4. Zyklisches Senden einschalten; in **Trace** und **Graph** das Ergebnis beobachten.
5. Bei Bedarf **Logging** starten und/oder eine **Testsequenz** erstellen und ausfuehren.
6. **Diagnose**: Services senden, Antworten pruefen.

2. Bedatung von Dateien vor dem Start

Bevor Sie verbinden, muessen die Projektdateien vorbereitet sein. Dieses Kapitel beschreibt, was in welcher Datei stehen muss und welche Angaben das Tool auswertet.

2.1 DBC-Dateien (`dbc/*.dbc`)

Standard-DBC-Dateien aus CANdb++, CANdb Editor, SavvyCAN u. a. Mehrere DBC-Dateien pro Projekt sind moeglich (deren Summe bildet die Datenbasis). Zeichensatz UTF-8 oder Windows-1252.

Ausgewertet werden:

DBC-Element	Bedeutung im Tool
`BO_`	Botschaft: ID (Bit 31 gesetzt = Extended-ID), Name, Laenge (DLC > 8 = CAN FD), Sender.
`SG_`	Signal: Startbit, Laenge, Byte-Reihenfolge (Intel/Motorola), Vorzeichen, Faktor, Offset, Min/Max, Einheit, Empfaenger.
`SG_ ... M / m0, m1`	Einfaches Multiplexing (ein Multiplexor, mehrere Gruppen). Extended Multiplexing wird **nicht** unterstuetzt (Warnung).
`VAL_`	Wertetabelle: Text je Rohwert. Im Tool als Dropdown beim Senden und als Text beim Empfang.
`SIG_VALTYPE_`	Gleitkomma-Signale (32/64 Bit).
`CM_`	Kommentare zu Botschaften/Signalen.
`BA_ "GenMsgCycleTime"`	Standard-Zykluszeit (ms) einer Botschaft; ohne Angabe 100 ms.
`BA_ "VFrameFormat"`	Frame-Format; Werte mit *FD* kennzeichnen CAN-FD-Botschaften.

Beispiel:

```
VERSION "1.0"
NS_ :
BS_:
BU_: Tester ECU

BO_ 512 EngineData: 8 ECU
 SG_ RPM : 0|16@1+ (0.25,0) [0|8000] "rpm" Tester
 SG_ CoolantTemp : 16|8@1+ (1,-40) [-40|130] "degC" Tester

VAL_ 257 StartButton 0 "Aus" 1 "Ein" ;
BA_DEF_ BO_ "GenMsgCycleTime" INT 0 65535;
BA_ "GenMsgCycleTime" BO_ 512 10;
```

Hinweise zur Bedatung

- Botschafts- und Signalnamen werden in Testsequenzen verwendet - sprechende, eindeutige Namen waehlen.
- Doppelte Botschaftsnamen ueber mehrere DBC-Dateien: die erste Definition bleibt, die zweite wird mit Warnung ignoriert.
- Gleiche ID mit unterschiedlichen Namen: Beim Empfang wird dem ersten Namen zugeordnet (Warnung).
- Die Sender-Knoten (`BU_`) steuern in der Simulation, welche Botschaften das virtuelle Steuergeraet zyklisch sendet (Standard: Sender `ECU`, einstellbar in `sim.traffic_senders`). Sinnvolle Min/Max-Werte in der DBC geben realistische Simulationswerte.
- Warnungen der Konvertierung stehen in der Rubrik **Projekt** im Feld *Konvertierte Dateien / Warnungen*.

Der Cache (`cache/*.dbc.json` oder `.yaml`)

Beim Oeffnen des Projekts wird jede DBC automatisch in den Cache geschrieben (Format in der Rubrik **Projekt** waehlbar). Nur wenn sich die Quelldatei geaendert hat, wird neu konvertiert; **Neu konvertieren** erzwingt es. Eigene Aenderungen im Cache werden bei einer neuen Konvertierung ueberschrieben - Korrekturen gehoeren in die DBC.

```
{
  "format_version": 1,
  "nodes": ["Tester", "ECU"],
  "messages": {
    "EngineData": {
      "id": 512, "extended": false, "dlc": 8, "fd": false, "sender": "ECU",
      "cycle_time_ms": 10, "comment": "", "multiplexor": null,
      "signals": {
        "RPM": {"start": 0, "length": 16, "byte_order": "little", "signed": false,
                "factor": 0.25, "offset": 0, "min": 0, "max": 8000, "unit": "rpm",
                "receivers": ["Tester"], "choices": {}, "comment": ""}
      }
    }
  },
  "warnings": [], "source": "engine.dbc", "source_sha1": "..."
}
```

2.2 Diagnosebeschreibung (`cdd/*.cdd`)

Die CDD beschreibt Diagnose-Services. Sie wird wie die DBC nach `cache/*.cdd.json|yaml` konvertiert.

Wichtige Einschraenkung: Vollstaendig unterstuetzt wird das einfache XML-Format `<DIAGDATA>` (siehe Beispiel und `projects/demo/cdd/diag.cdd`). Originale CANdelaStudio-CDD-Dateien werden nur **heuristisch** gelesen (Servicenamen und, falls auffindbar, Request-Bytes). Pruefen Sie in dem Fall die erzeugte Cache-Datei (Rubrik **Projekt**, Warnungen) und ergaenzen Sie Adressen, Requests und Antwortparameter dort oder im einfachen Format.

```
<?xml version="1.0" encoding="UTF-8"?>
<DIAGDATA>
  <ECU name="Engine_ECU" request_id="0x7E0" response_id="0x7E8" extended="false" fd="false">
    <SERVICE name="ExtendedSession" request="10 {session}" description="Select diagnostic session">
      <PARAM name="session" bytes="1" default="03"/>
    </SERVICE>
    <SERVICE name="ReadVIN" request="22 F1 90" description="Read VIN">
```

```
      <RESPONSE prefix_len="3">
        <PARAM name="VIN" offset="0" length="17" type="ascii"/>
      </RESPONSE>
    </SERVICE>
    <SERVICE name="ReadSpeed" request="22 F4 0D" description="Read speed">
      <RESPONSE prefix_len="3">
        <PARAM name="Speed" offset="0" length="1" type="uint" factor="1" unit="km/h"/>
      </RESPONSE>
    </SERVICE>
  </ECU>
</DIAGDATA>
```

Element / Attribut	Bedeutung
`ECU name, request_id, response_id`	Name des Steuergeraets und CAN-IDs (Tester -> ECU, ECU -> Tester), z. B. `0x7E0`/`0x7E8`.
`ECU extended, fd`	`true`: 29-Bit-IDs bzw. CAN FD (ISO-TP mit bis zu 64 Byte je Frame).
`SERVICE name`	Name des Services (in Rubrik Diagnose und in Sequenzschritt `diag` verwendet).
`SERVICE request`	Request-Bytes in Hex. Platzhalter in geschweiften Klammern, z. B. `10 {session}`.
`SERVICE description`	Freitext, wird in der Serviceliste angezeigt.
`PARAM`	Direkt im SERVICE: Eingabeparameter fuer einen Platzhalter: `name`, `bytes` (Breite), `default` (Hex).
`RESPONSE prefix_len`	Anzahl Bytes am Anfang der Antwort, die uebersprungen werden (Service-ID + DID, z. B. 3 bei `62 F1 90`).
`RESPONSE/PARAM`	Antwortparameter: `name`, `offset` und `length` in Bytes (ab Ende des Praefix), `type` = `ascii | uint | int | hex`, optional `factor`, `offset_val`, `unit`.

Der entsprechende Cache-Eintrag (zum manuellen Nachbessern):

```
{
  "format_version": 1,
  "ecus": {
    "Engine_ECU": {
      "request_id": "0x7E0", "response_id": "0x7E8", "extended": false, "fd": false,
      "services": {
        "ReadVIN": {
          "request": "22 F1 90", "description": "Read VIN", "params": [],
          "response": {"prefix_len": 3,
                       "params": [{"name": "VIN", "offset": 0, "length": 17, "type": "ascii"}]}
        }
      }
    }
  }
}
```

2.3 Hardware und Verbindung

Einstellungen in der Rubrik **Projekt** (werden in `project.json` unter `hardware` gespeichert):

Einstellung	Vector VN1630A / VN7572	PEAK PCAN-USB Pro FD	Simulation
Schnittstelle	`vector` (XL-Treiber)	`pcan` (PCAN-Basic)	`simulation`
Kanal	Kanalindex, 0-basiert (z. B. `0`). Zuordnung ueber die *Vector Hardware Config*; bei gesetztem App-Namen gilt die dort zugeordnete Kanalnummer.	`PCAN_USBBUS1`, `PCAN_USBBUS2` ...	nicht benoetigt
Bitrate	Nominal-Bitrate (kbit/s)	Nominal-Bitrate (kbit/s); bei FD fest 500k/2M (80-MHz-Takt)	ohne Bedeutung
CAN FD / Daten-Bitrate	Haken setzen und Daten-Bitrate eintragen	Haken setzen; abweichendes Timing in `hardware.extra`	Haken fuer FD-Frames
App-Name	Name aus der Vector Hardware Config (leer = Standard von python-can)	-	-

Vector VN7572 (PCIe-Karte): Auch die Vector-Schnittstellenkarte VN7572 wird als Typ `vector` ueber den Vector-XL-Treiber angesprochen (nur Windows), genau wie die VN1630A. Die Karte muss mit installiertem *Vector Driver Setup* in der *Vector Hardware Config* sichtbar sein. Die Kanalnummer ist 0-basiert (0 = CH1, 1 = CH2 ...); mit **Hardware erkennen** werden die gefundenen Kanaele samt Seriennummer aufgelistet. Ein anderes Programm (CANalyzer, CANoe, vFlash) darf den Kanal nicht belegen.

- Alle Teilnehmer am Bus muessen dieselbe Bitrate nutzen; bei CAN FD zusaetzlich dieselbe Daten-Bitrate.
- Auf einem Tisch-Aufbau mit zwei Geraeten muessen die Busenden mit 120 Ohm terminiert sein.
- `hardware.extra` (nur in `project.json`) reicht beliebige Parameter an python-can durch, z. B. `{"nom_brp": 5}` fuer PEAK.
- **Simulation**: *Spiegeln* liefert die eigenen Sendebotschaften als Empfang zurueck; *virtuelles Steuergeraet* beantwortet UDS-Requests (u. a. 10, 11, 14, 19, 22, 27, 3E) auf den Adressen der Diagnose-Einstellung und sendet die Botschaften der ECU-Sender zyklisch.

2.4 Sequenzdateien (`sequences/*.json`)

Eine Sequenz besteht aus Name, Beschreibung und einer Liste von Schritten. Sie koennen sie in der Rubrik **Testsequenz** erstellen oder die Datei direkt bearbeiten. Befehlsreferenz: Anhang A.

```
{
  "name": "startup",
  "description": "Simulate engine start and check run-up",
  "steps": [
    {"type": "enable_tx", "message": "DriverCmd", "enabled": true},
    {"type": "set_signal", "message": "DriverCmd", "signal": "StartButton", "value": 1},
    {"type": "wait_for", "direction": "RX", "message": "EngineData", "signal": "RPM",
     "op": ">", "value": 100, "timeout_ms": 3000},
    {"type": "diag", "service": "ReadVIN", "expect": "62 F1 90"}
```

```
  ]
}
```

2.5 Die Projektdatei (`project.json`)

Wird vom Tool geschrieben; bei Bedarf manuell editierbar (Tool dabei beenden). Fehlende Schluessel werden mit Standardwerten ergaenzt.

```
{
  "hardware": {"type": "vector", "channel": "0", "bitrate": 500000, "fd": true,
               "data_bitrate": 2000000, "app_name": "", "extra": {}},
  "cache_format": "json",
  "tx": [{"message": "DriverCmd", "cycle_ms": 50, "values": {"ThrottleCmd": 0}, "signals": null}],
  "rx": {"EngineData": null, "VehicleSpeed": ["Speed"]},
  "logging": {"selection": {"TX": {}, "RX": {"EngineData": null}},
              "on_change_only": false, "min_interval_ms": 0, "log_raw": false},
  "graph": ["RX:EngineData.RPM"],
  "diag": {"cdd": "diag.cdd.json", "ecu": "Engine_ECU", "request_id": "", "response_id": "",
           "extended": false, "fd": false, "padding": 170},
  "sequence": "startup.json",
  "sim": {"ecu": true, "traffic_senders": ["ECU"]}
}
```

Schluessel	Bedeutung
`hardware`	Verbindungsparameter (Kapitel 2.3); Bitraten in bit/s.
`cache_format`	`json` oder `yaml` fuer die konvertierten Dateien.
`tx`	Sendeliste: Botschaft, Zyklus (ms), Signalwerte, `signals` = Auswahl fuer Graph/Logging (`null` = alle).
`rx`	Empfangsauswahl: Botschaft -> Liste der gewaehlten Signale oder `null` (alle).
`logging`	Logging-Filter und -Optionen (Rubrik Logging).
`graph`	Schluessel der im Graph angehakten Signale, z. B. `RX:Msg.Signal`.
`diag`	CDD-Datei, ECU, optionale Adress-Ueberschreibung (`request_id`, `response_id`), `extended`, `fd`, `padding`.
`sequence`	Zuletzt geladene Sequenz (Dateiname in `sequences/`).
`sim`	Simulation: `ecu` (virtuelles Steuergeraet), `traffic_senders` (Sender-Knoten, die zyklisch simuliert werden).

2.6 Checkliste vor dem ersten Verbinden

1. DBC-Datei(en) im Ordner `dbc/`; in der Rubrik Projekt erscheinen die erwarteten Botschaften (Anzahl) und keine unerwarteten Warnungen.
2. Optional CDD im Ordner `cdd/`; in der Rubrik Diagnose erscheinen ECU und Services.
3. Hardware-Treiber installiert, Geraet angeschlossen, Kanal und Bitraten passend zum Bus.
4. Bei CAN FD: Haken *CAN FD* und Daten-Bitrate; Gegenstellen ebenfalls FD-faehig.
5. Sequenzen unter `sequences/` vorhanden bzw. in der Rubrik Testsequenz erstellt.

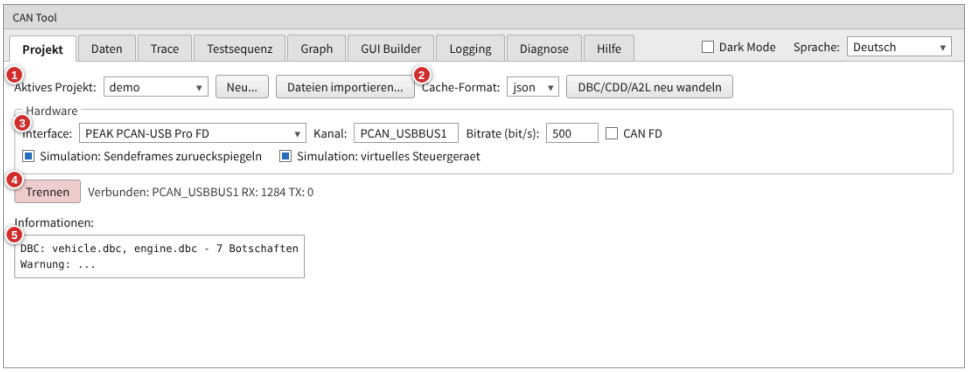
Haeufige Probleme

Symptom	Moegliche Ursache / Abhilfe
Keine Botschaften in der Daten-Rubrik	DBC nicht im Ordner `dbc/` oder Dateiendung nicht `.dbc`; Warnungen in der Rubrik Projekt lesen.
Verbinden schlaegt fehl	python-can/Treiber fehlt, falscher Kanal, Geraet von anderer Anwendung belegt (z. B. CANalyzer/PCAN-View).
Frames sichtbar, aber nicht dekodiert	ID/Extended-Kennzeichnung der DBC stimmt nicht mit dem Bus ueberein; Botschaft fehlt in der DBC.
Falsche Signalwerte	Byte-Reihenfolge (Intel/Motorola), Startbit, Faktor/Offset in der DBC pruefen.
Diagnose: Timeout	Request-/Response-ID, Extended/FD und Padding in der Rubrik Diagnose pruefen; Steuergeraet in passender Session?
Logging nicht verfuegbar	Python-Paket `pyarrow` fehlt (`pip install pyarrow`).

3.1 Rubrik "Projekt"

Betriebssystem: Das Tool erkennt beim Start selbststaendig, ob es unter **Windows** oder **Linux** laeuft (Anzeige in der Titelzeile). Die Hardware-Auswahl passt sich an: *Vector* (XL-Treiber) nur unter Windows, *SocketCAN* nur unter Linux, *PEAK* und *Simulation* auf beiden. Nicht nutzbare Eintraege sind markiert und gesperrt. Die Schaltflaeche **Hardware suchen** listet angeschlossene Schnittstellen auf. Alle uebrigen Funktionen (Trace, Sequenzen, Logging, Diagnose, vFlash, Hilfe/PDF) sind auf beiden Systemen identisch; Projektordner sind zwischen Windows und Linux austauschbar.

Bildschirmbild mit Erklaerungen



Schematische Darstellung der Oberflaeche; die Nummern verweisen auf die Erklaerungen darunter.

1. Projektauswahl, neues Projekt anlegen und DBC/CDD importieren.
2. Cache-Format (json/yaml) und erneute Umwandlung von DBC/CDD.
3. Hardware: Schnittstelle, Kanal (z. B. PCAN_USBBUS1), Bitrate und CAN FD.
4. Verbinden bzw. Trennen mit Statusanzeige (Empfangs- und Sendezaehler).
5. Informationsfeld: geladene DBC-Dateien, Botschaftsanzahl und Warnungen.

Re-Init HW: Neben *Verbinden* schliesst dieser Button die Schnittstelle und oeffnet sie mit denselben Einstellungen neu. Er hilft, wenn die angeschlossene Hardware haengt. Die Sendeliste bleibt erhalten (zyklisch aktive Botschaften senden danach weiter); ein laufendes Logging wird beendet.

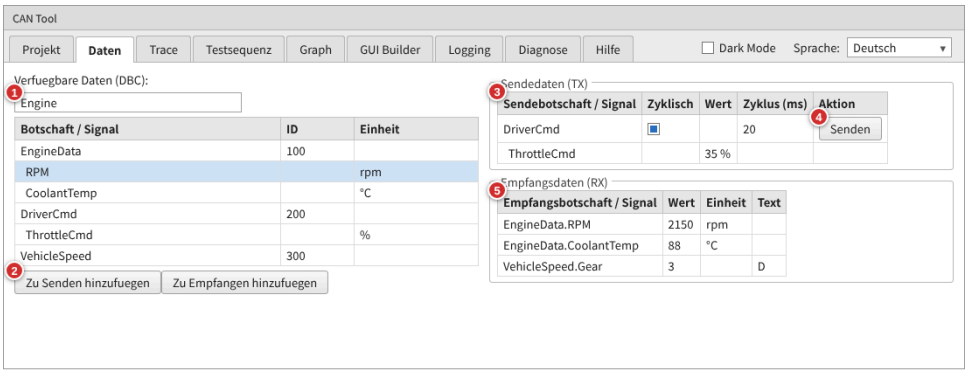
Projekte verwalten, DBC/CDD einbinden, Hardware einstellen und die Verbindung herstellen.

Bedienelement	Funktion
Aktives Projekt	Auswahlliste aller Projekte (`projects/*/project.json`). Ein Wechsel trennt die Verbindung, stoppt das Logging, speichert das alte und laedt das neue Projekt.
Neu ...	Legt ein Projekt mit Ordnerstruktur und Standardeinstellungen an (Kapitel 1.5).
DBC/CDD importieren ...	Kopiert gewaehlte `.dbc`/`.cdd`-Dateien in das Projekt und konvertiert sie.
Cache-Format	`json` oder `yaml`; bei Aenderung wird neu konvertiert.
Neu konvertieren	Erzwingt die Konvertierung aller DBC/CDD und laedt das Projekt neu.
Schnittstelle / Kanal / Bitrate / CAN FD / Daten-Bitrate / App-Name	Hardware-Parameter (Kapitel 2.3). Waehrend der Verbindung gesperrt.
Simulation: Spiegeln	Eigene Sendebotschaften erscheinen zusaetzlich als Empfang.
Simulation: virtuelles Steuergeraet	Startet beim Verbinden ein UDS-faehiges Software-Steuergeraet mit Testverkehr.
Verbinden / Trennen	Oeffnet bzw. schliesst die Schnittstelle. Beim Trennen stoppt ein laufendes Logging.
Statuszeile	Verbindungszustand, Anzahl empfangener (RX) und gesendeter (TX) Frames, letzter Fehler.
Konvertierte Dateien / Warnungen	Liste der Cache-Dateien, Anzahl der Botschaften, Warnungen aus DBC/CDD.

3.2 Rubrik "Daten"

Hier wird festgelegt, **welche** Botschaften gesendet und empfangen werden und **welche Signale** fuer Graph, Logging und Anzeige gelten - jeweils aus der Summe aller DBC-Daten des Projekts.

Bildschirmbild mit Erklaerungen



Schematische Darstellung der Oberflaeche; die Nummern verweisen auf die Erklaerungen darunter.

1. Filter und Liste aller Botschaften und Signale aus der DBC.
2. Auswahl zu den Sendedaten (TX) oder Empfangsdaten (RX) hinzufuegen.
3. Sendedaten: zyklisch ein/aus, Signalwerte und Zykluszeit einstellen.
4. Einzelsendung: sendet die Botschaft sofort einmal.
5. Empfangsdaten mit den aktuellen Livewerten der Signale.

Linke Seite: Verfuegbare Daten

- Baum aller Botschaften mit ihren Signalen (ID, Sender, Einheit). Das **Filterfeld** sucht in Botschaftsname, ID und Signalname.
- **-> Senden (TX)** uebernimmt die markierten Botschaften in die Sendeliste (Doppelklick auf eine Botschaft macht dasselbe). Mehrfachauswahl ist moeglich; markiert man ein Signal, wird dessen Botschaft uebernommen.
- **-> Empfang (RX)** uebernimmt die markierten Botschaften in die Empfangsliste.

Rechts oben: Senden

Spalte	Bedeutung
Sendebotschaft / Signal	Botschaft (mit ID) und darunter alle ihre Signale.
Zyklisch	Haken an der Botschaft schaltet das zyklische Senden ein/aus (nur bei bestehender Verbindung wird tatsaechlich gesendet).
Wert	Physikalischer Signalwert (Eingabefeld bzw. Auswahlliste bei Signalen mit Wertetabelle). Aenderungen wirken ab dem naechsten Frame.
Zyklus [ms]	Zykluszeit der Botschaft; Vorbelegung aus der DBC (`GenMsgCycleTime`), sonst 100 ms. 0 = nicht zyklisch.
Auswahl	Haken am Signal: Signal gehoert zur Auswahl fuer Graph und Logging.
Senden (Schaltflaeche)	Sendet die Botschaft genau einmal mit den aktuellen Werten.

Entfernen loescht die Botschaft der markierten Zeile aus der Sendeliste.

Rechts unten: Empfangen

Spalte	Bedeutung
Empfangsbotschaft / Signal	Botschaft (mit ID) und ihre Signale.
Wert / Einheit / Text	Zuletzt empfangener Wert (Aktualisierung alle 200 ms), Einheit und - falls vorhanden - der Text der Wertetabelle.
Auswahl	Haken am Signal: Signal gehoert zur Auswahl fuer Graph und Logging. Nicht angehakte Signale werden nicht gespeichert.

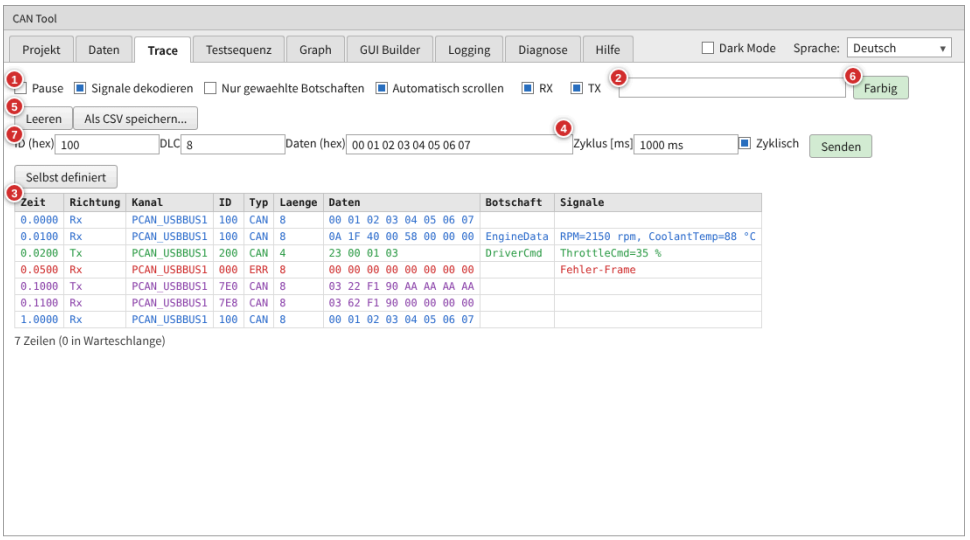
Hinweise

- Die Auswahl (Haken) bestimmt, was **Graph** und **Logging** sehen. Der **Trace** zeigt dagegen alle Frames (optional nur die gewaehlten Botschaften).
- Die Auswahl und alle Sendewerte werden mit dem Projekt gespeichert; das zyklische Senden ist nach dem Laden aus.
- Eine Botschaft kann gleichzeitig in der TX- und RX-Liste stehen (z. B. mit Spiegelung in der Simulation).

3.3 Rubrik "Trace"

Zeitlicher Ablauf aller gesendeten (Tx, farbig hervorgehoben) und empfangenen (Rx) Frames.

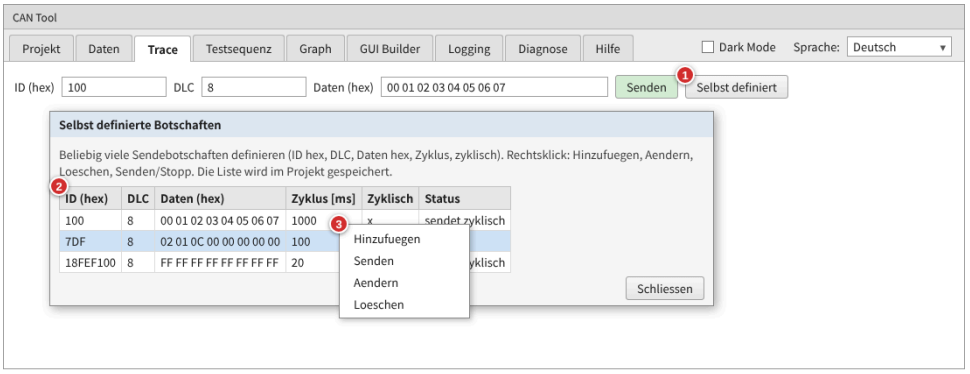
Bildschirmbild mit Erklaerungen



Schematische Darstellung der Oberflaeche; die Nummern verweisen auf die Erklaerungen darunter.

1. Optionen: Pause, Signale dekodieren, nur gewaehlte Botschaften, Autoscroll.
2. Filter nach ID oder Botschaftsname.
3. Trace-Tabelle: Zeit, Richtung (Rx/Tx), Kanal, ID, Typ, Laenge und Rohdaten. Auch ohne DBC/CDD.
4. Dekodierte Botschaftsnamen und Signalwerte (wenn eine DBC geladen ist).
5. Trace leeren oder als CSV-Datei speichern.
6. Farbmodus: Rx blau, Tx gruen, Fehler rot, ISO-TP violett.
7. Sende-Zeile: ID (hex), DLC, Daten, Zykluszeit, Haken fuer zyklisches Senden und Senden-Button.

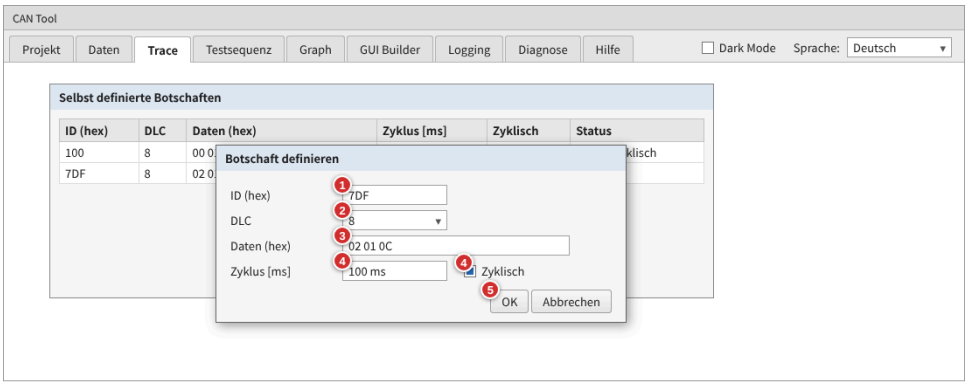
Bildschirmbild mit Erklaerungen



Schematische Darstellung der Oberflaeche; die Nummern verweisen auf die Erklaerungen darunter.

1. Der Button Selbst definiert oeffnet das Fenster fuer eigene Sendebotschaften.
2. Tabelle der Sendebotschaften: ID (hex), DLC, Daten (hex), Zyklus [ms], zyklisch und Status.
3. Rechtsklick-Menue: Hinzufuegen, Senden/Stopp, Aendern, Loeschen.

Bildschirmbild mit Erklaerungen



Schematische Darstellung der Oberflaeche; die Nummern verweisen auf die Erklaerungen darunter.

1. ID (hex): CAN-ID, auch Extended (> 7FF).
2. DLC: Datenlaenge, auch CAN-FD-Werte.
3. Daten (hex): Bytes mit oder ohne Leerzeichen; fehlende Bytes werden mit 00 aufgefuellt.
4. Zyklus [ms] und Haken Zyklisch: einmalig oder zyklisch senden.
5. OK uebernimmt die Botschaft in die Liste, Abbrechen verwirft die Eingabe.

Spalte	Inhalt
Zeit [s]	Zeit relativ zum ersten angezeigten Frame (seit Verbinden bzw. *Leeren*).
Rx/Tx, Kanal	Richtung und Schnittstelle.
ID, Typ, Len	CAN-ID (hex, 8-stellig bei Extended), Typ (CAN, FD, FD/BRS) und Datenlaenge.
Daten	Nutzdaten in Hex.
Botschaft, Signale	Name aus der DBC und dekodierte Signale mit Einheit (wenn *Dekodieren* aktiv).
Farbmodus	Schaltflaeche (umschaltbar). Aktiv: Rx-Botschaften mittelblau, Tx-Botschaften mittelgruen, fehlerhafte Botschaften (Bus-Fehlerframes, Typ ERR) rot und ISO-TP-Botschaften (Diagnose-Request-/Response-ID, auch 0x7DF) mittelviolett. Inaktiv: Darstellung wie bisher.

Bedienelement	Funktion
Pause	Haelt die Anzeige an. Frames werden weiter gepuffert (bis 50 000) und nach dem Aufheben nachgeliefert.
Dekodieren	Schaltet die Signaldekodierung ein/aus (aus = schneller bei hoher Buslast).
Nur gewaehlte Botschaften	Zeigt nur Botschaften, die in der Rubrik Daten in der TX- bzw. RX-Liste stehen.
Autoscroll	Folgt automatisch dem neuesten Eintrag.
Rx / Tx	Richtungen ein-/ausblenden.
Filterfeld	Kommagetrennte IDs in Hex (z. B. `100, 200`) und/oder ein Teil eines Botschaftsnamens (z. B. `engine`).
Leeren	Loescht Anzeige und Puffer.
Als CSV speichern ...	Exportiert die aktuell sichtbaren Zeilen (Trennzeichen `;`).

Aktualisieren: Mit dem Haken *Aktualisieren* steht jede Botschaft (Richtung, Kanal, ID) nur einmal in der Tabelle, in der Reihenfolge des ersten Auftretens. Bei erneutem Auftreten werden Zeitstempel, Daten und dekodierte Signale dieser Zeile aktualisiert und die Spalte *Anzahl* erhoeht. Ohne Haken werden alle Frames scrollend hintereinander geschrieben. Beim Umschalten wird die Tabelle geleert; Autoscroll wirkt nur im scrollenden Modus.

Diffzeit: Zusaetzlich zu *Aktualisieren* zeigt der Haken *Diffzeit* in der Zeitspalte statt der absoluten Zeit den Abstand zum vorigen Auftreten derselben Botschaft (Richtung, Kanal, ID), also die Zykluszeit; beim ersten Auftreten steht 0. Die Spaltenueberschrift wechselt zu *Diffzeit [s]*. Beim Umschalten wird die Tabelle geleert.

Sende-Zeile: Unter der Optionsleiste sendet eine Zeile Rohframes (Standard: ID 100 hex, DLC 8, Daten 00 01 02 03 04 05 06 07, 1000 ms). **Senden** schickt den Frame einmal; mit Haken **Zyklisch** startet es das zyklische Senden, ein weiterer Klick (**Stopp**) beendet es. IDs ueber 7FF werden als Extended-Frames gesendet, DLC groesser 8 als CAN FD (FD muss in den Verbindungseinstellungen aktiv sein). Der Frame erscheint im Trace als Tx.

Selbst definierte Botschaften (Multi-Senden)

Die Sende-Zeile sendet genau eine Botschaft. Fuer **mehrere Botschaften** dient der Button **Selbst definiert**: Er oeffnet ein eigenes Fenster, das neben dem Trace geoeffnet bleibt. Dort werden beliebig viele Sendebotschaften definiert und einzeln gesendet. Die Liste wird im Projekt gespeichert (`custom_tx`) und steht beim naechsten Oeffnen des Projekts wieder zur Verfuegung.

Botschaft definieren und senden

1. Button **Selbst definiert** in der Sende-Zeile klicken: das Fenster "Selbst definierte Botschaften" oeffnet sich.
2. Mit **Rechtsklick** in die Tabelle den Eintrag **Hinzufuegen** waehlen.
3. Im Dialog "Botschaft definieren" ID (hex), DLC, Daten (hex), Zyklus [ms] eingeben und ggf. **Zyklisch** anhaken; mit **OK** uebernehmen. Ungueltige Eingaben (z. B. unzulaessiger DLC oder zu viele Datenbytes) werden mit einer Meldung abgelehnt, nichts wird gespeichert.
4. Rechtsklick auf die Zeile und **Senden** waehlen: ohne Haken Zyklisch wird die Botschaft einmal gesendet, mit Haken startet das zyklische Senden (Status: "sendet zyklisch"). Derselbe Menuepunkt heisst dann **Stopp**.

Feld	Bedeutung
ID (hex)	CAN-ID in Hex, ohne Praefix (z. B. `7DF`). IDs groesser als 7FF werden automatisch als Extended-ID (29 Bit) gesendet.
DLC	Laenge der Daten: 0 bis 8 sowie 12, 16, 20, 24, 32, 48, 64. Werte ueber 8 werden als CAN FD gesendet (Hardware und Projekt muessen CAN FD unterstuetzen).
Daten (hex)	Hex-Bytes mit oder ohne Leerzeichen. Zu kurze Eingaben werden mit 00 auf den DLC aufgefuellt, zu lange sind ein Fehler.
Zyklus [ms]	Sendeabstand von 1 bis 3 600 000 ms; wirkt nur bei Zyklisch.
Zyklisch	Ohne Haken wird einmal gesendet, mit Haken zyklisch bis zum Stopp.

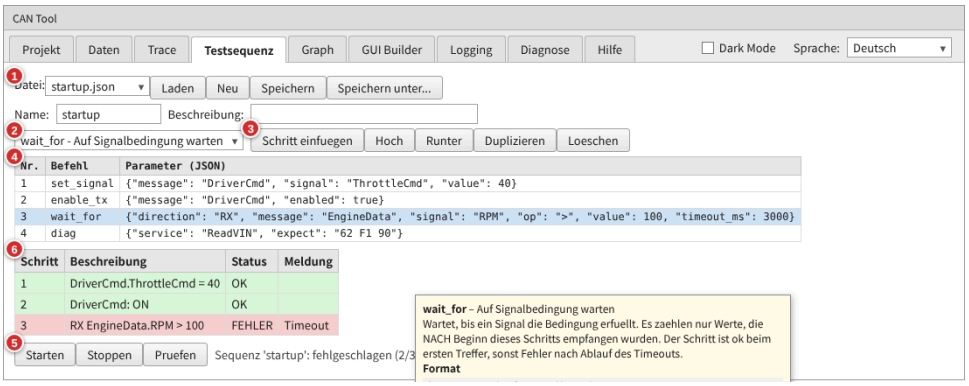
Rechtsklick-Menue: Hinzufuegen (auch auf freier Flaeche), Senden bzw. Stopp, Aendern (auch per Doppelklick) und Loeschen. Wird ein laufender Eintrag geaendert oder geloescht, stoppt er zuvor. Zum Senden muss die Verbindung hergestellt sein; beim Trennen stoppt das zyklische Senden aller Eintraege. Mehrere Eintraege mit derselben ID teilen sich einen Sendeauftrag. Gesendete Botschaften erscheinen im Trace als Tx.

Hinweise: Es werden maximal 5 000 Zeilen angezeigt (aeltere rollen heraus). Die Anzeige wird nur aktualisiert, solange die Rubrik sichtbar ist; im Hintergrund puffert das Tool bis zu 50 000 Frames. Fuer lueckenlose Aufzeichnung dient das **Logging**.

3.4 Rubrik "Testsequenz"

Erstellen, Speichern, Laden und Ausfuehren von Testsequenzen. Es ist immer **genau eine Sequenz** geladen/aktiv; mehrere Sequenzen liegen als Dateien im Ordner `sequences/` des Projekts.

Bildschirmbild mit Erklaerungen



Schematische Darstellung der Oberflaeche; die Nummern verweisen auf die Erklaerungen darunter.

1. Testsequenz-Datei waehlen, laden, neu anlegen, speichern (jeweils eine Sequenz).
2. Befehls-Dropdown: beim Ueberfahren eines Eintrags erscheint der Tooltip mit Format, Elementen und Beispiel.
3. Schritt einfuegen, verschieben, duplizieren oder loeschen.
4. Schrittliste: Befehl und Parameter als JSON, direkt in der Tabelle editierbar.
5. Starten, Stoppen und statische Pruefung der Sequenz; darunter das Gesamtergebnis.
6. Ergebnistabelle je Schritt: gruen = bestanden, rot = fehlgeschlagen, mit Meldung.

Datei

Bedienelement	Funktion
Datei-Liste + Laden	Waehlt eine Sequenz aus `sequences/` und laedt sie in den Editor.
Neu	Leere Sequenz (nicht gespeichert, bis *Speichern*).
Speichern / Speichern unter ...	Schreibt die Sequenz als JSON. Der zuletzt gespeicherte/geladene Dateiname wird im Projekt vermerkt und beim naechsten Oeffnen wieder geladen.
Name / Beschreibung	Metadaten der Sequenz.

Schritte bearbeiten

Die Schritte stehen in einer Tabelle (Nr., Typ, Parameter als JSON). Der **Parameter-Text ist direkt editierbar** (Doppelklick). Ungueltiges JSON wird abgelehnt und der alte Wert wiederhergestellt; der Typ eines Schritts ist fest.

Bedienelement	Funktion
Befehls-Dropdown	Waehlt den Schritttyp. **Beim Ueberfahren eines Eintrags mit der Maus erscheint ein Tooltip** mit Format, Beschreibung aller Elemente und einem Beispiel; darunter steht zusaetzlich die Kurzbeschreibung des markierten Befehls.
Schritt hinzufuegen	Fuegt den Befehl mit einer Vorlage hinter der markierten Zeile (bzw. am Ende) ein.
Hoch / Runter / Duplizieren / Loeschen	Markierten Schritt verschieben, kopieren oder entfernen.
Signal-Auswahl + Signal in Schritt uebernehmen	Listet alle Signale der DBC als `Message.Signal` (mit Tipp-Vervollstaendigung) und traegt Botschaft (und bei `set_signal`, `wait_for`, `check` auch Signal) in den markierten Schritt ein.
Tooltip in der Tabelle	Auch ueber der Typ-Spalte eines Schritts erscheint der Befehlstooltip.

Pruefen und Ausfuehren

Bedienelement	Funktion
Pruefen	Statische Pruefung: unbekannte Schritttypen, Operatoren, Botschaften und Signale (gegen die DBC).
Start	Fuehrt die Sequenz aus (nur bei bestehender Verbindung). Vor dem Start wird automatisch geprueft. Waehrend des Laufs ist der Editor gesperrt.
Stopp	Bricht die Sequenz ab (auch ein laufendes `wait`); Status *aborted*.
Ergebnistabelle	Je Schritt: Nummer (Schleifen: z. B. `3.2.1`), Beschreibung, Status (gruen = ok, rot = fail/error) und Meldung (z. B. gemessener Wert, Antwort-Bytes).

Ablaufregeln: Die Schritte laufen nacheinander. Der **erste fehlgeschlagene Schritt beendet die Sequenz** (Status *failed*). Nach Ende zeigt die Statuszeile z. B. *startup: passed (10/10 Schritte ok)*. Zyklisch gestartete Botschaften laufen nach dem Ende weiter, sofern nicht per `enable_tx` gestoppt.

Befehle im Ueberblick

Befehl	Titel	Kurzbeschreibung

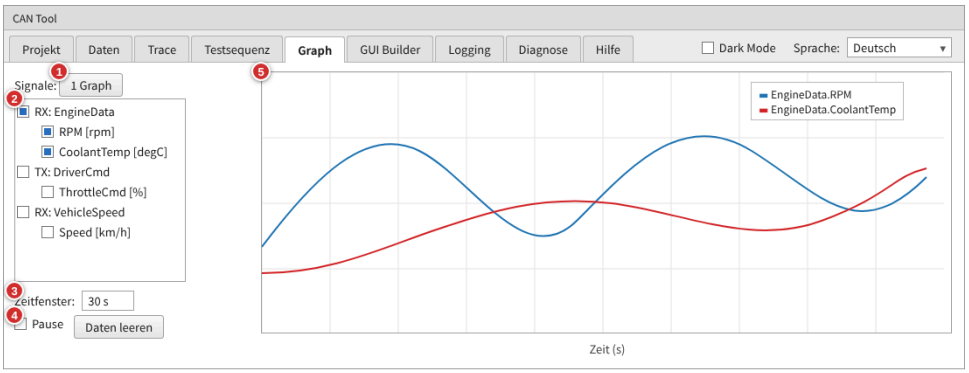
`set_signal`	Signalwert setzen	Setzt den physikalischen Wert eines Signals in einer Sendebotschaft. Der neue Wert geht mit dem naechsten (zyklischen oder einzelnen) Frame auf den Bus.
`enable_tx`	Zyklisches Senden ein-/ausschalten	Startet oder stoppt das zyklische Senden einer Botschaft mit ihrem eingestellten Zyklus.
`set_cycle`	Zykluszeit aendern	Aendert die Zykluszeit einer Sendebotschaft (wirkt sofort, auch waehrend sie zyklisch sendet).
`send_once`	Botschaft einmal senden	Sendet genau einen Frame mit den aktuellen Signalwerten, unabhaengig vom zyklischen Senden.
`send_raw`	Rohframe senden	Sendet einen Rohframe (ID + Daten) auf den CAN, einmalig oder zyklisch. Mit cyclic=true und enabled=false wird das zyklische Senden dieser ID beendet.
`wait`	Feste Zeit warten	Pausiert die Sequenz fuer eine feste Zeit. Zyklisches Senden und Empfang laufen weiter.
`wait_for`	Auf Signalbedingung warten	Wartet, bis ein Signal die Bedingung erfuellt. Es zaehlen nur Werte, die NACH Beginn dieses Schritts empfangen wurden. Der Schritt ist ok beim ersten Treffer, sonst Fehler nach Ablauf des Timeouts.
`check`	Signalwert pruefen	Prueft sofort den zuletzt empfangenen Wert eines Signals (kein Warten). Der Schritt schlaegt fehl, wenn die Bedingung nicht erfuellt ist oder seit Sequenzstart noch kein Wert empfangen wurde.
`diag`	Diagnose-Service aus der CDD	Sendet einen in der CDD definierten Diagnose-Service per ISO-TP und wertet die Antwort aus.
`diag_raw`	Diagnose-Request in Hex	Sendet einen beliebigen UDS-Request (Hex-Bytes) per ISO-TP und wertet die Antwort aus.
`loop`	Schleife	Fuehrt die enthaltenen Schritte mehrfach nacheinander aus.
`comment`	Kommentar	Tut nichts; dient nur der Dokumentation der Sequenz.

Die vollstaendige Beschreibung jedes Befehls mit Format, Elementen und Beispiel steht in Anhang A.

3.5 Rubrik "Graph"

Die Rubrik zeigt Signalverlaeufe live als Liniendiagramm (Zeitachse: Sekunden relativ zu jetzt, links = Vergangenheit, rechts = jetzt). Mit dem Schalter oberhalb der Auswahl wird zwischen **1 Graph** und **n Graphen** umgeschaltet.

Bildschirmbild mit Erklaerungen



Schematische Darstellung der Oberflaeche; die Nummern verweisen auf die Erklaerungen darunter.

1. Schalter 1 Graph / n Graphen (hier: 1 Graph).
2. Signalauswahl, nach Botschaft gruppiert; Haken an der Botschaft waehlt alle Signale, Einheit in eckigen Klammern.
3. Zeitfenster in Sekunden.
4. Pause friert die Anzeige ein, Daten leeren verwirft die Verlaeufe.
5. Gemeinsames Diagramm mit Legende; Zeitachse in Sekunden relativ zu jetzt.

Bildschirmbild mit Erklaerungen



Schematische Darstellung der Oberflaeche; die Nummern verweisen auf die Erklaerungen darunter.

1. Schalter steht auf n Graphen.
2. Die Auswahl bleibt nach Botschaft gruppiert; hier sind drei Signale gewaehlt.
3. Jedes Signal hat einen eigenen Graph mit eigener Wertachse, beschriftet mit Botschaft.Signal und Einheit.
4. Gemeinsame Zeitachse: Zoomen und Verschieben wirkt auf alle Graphen; Beschriftung nur unten.

Auswahl nach Botschaft gruppiert

Die Auswahl zeigt alle in der Rubrik Daten gewaehlten Signale, gruppiert nach Richtung und Botschaft (`RX` = empfangen, `TX` = gesendet). Unter jeder Botschaft stehen ihre Signale mit Einheit in eckigen Klammern. Der Haken an der Botschaft kennt drei Zustaende: alle Signale gewaehlt, teilweise gewaehlt (Teilzustand) oder keines gewaehlt. Ein Klick darauf waehlt alle Signale der Botschaft oder entfernt sie wieder.

Darstellung "1 Graph"

Alle gewaehlten Signale werden in einem gemeinsamen Diagramm mit Legende (Farbzuordnung) gezeichnet. Vorteil: direkter Vergleich der Kurven. Bei sehr unterschiedlichen Wertebereichen werden kleine Signale flach - dann ist "n Graphen" besser geeignet.

Darstellung "n Graphen"

Jedes gewaehlte Signal erhaelt ein eigenes Diagramm mit eigener Wertachse, beschriftet mit Botschaft.Signal und Einheit; die Diagramme stehen untereinander. Alle Diagramme teilen sich dieselbe **Zeitbasis**: Zoomen und Verschieben der Zeitachse in einem Diagramm wirkt auf alle, die Zeitbeschriftung erscheint nur am untersten Diagramm.

Bedienelemente

Bedienelement	Funktion
Schalter 1 Graph / n Graphen	Wechselt die Darstellungsart; die Anzeige wird sofort neu aufgebaut. Die Einstellung wird im Projekt gespeichert (`graph_multi`).

Signalauswahl	Haken setzen = Kurve anzeigen. Der Haken an der Botschaft waehlt bzw. entfernt alle ihre Signale. Die Auswahl wird im Projekt gespeichert (`graph`).
Zeitfenster	Sichtbarer Zeitraum in Sekunden (1 bis 3600).
Pause	Friert die Anzeige ein; Daten werden im Hintergrund weiter gesammelt.
Daten leeren	Verwirft alle gesammelten Signalverlaeufe.
Diagramm	Maus: Zoomen mit Mausrad, Verschieben mit gedrueckter Taste. In "n Graphen" gilt dies gemeinsam fuer alle Diagramme.

Hinweise: Je Signal werden bis zu 50 000 Punkte vorgehalten. Nur Signale, die in der Rubrik Daten angehakt sind, werden aufgezeichnet. Signale erscheinen erst, wenn Daten empfangen/gesendet wurden. Die Anzeige aktualisiert sich nur bei sichtbarer Rubrik.

3.6 Rubrik "Logging"

Aufgeloeste Signale: Ist eine DBC, A2L oder CDD eingebunden, enthaelt die Parquet-Datei nicht nur die Botschaft als Hex, sondern die Signale mit **Rohwert, Roh-Hex, physikalischem Wert, Text (Wertetabelle) und Einheit**. Spalten der Signaldatei: `timestamp, direction, message, signal, value, source, can_id, raw, raw_hex, text, unit, message_hex`. `source` ist `CAN` (DBC-Botschaften), `XCP` (gelesene A2L-Werte der Rubrik XCP) oder `UDS` (dekodierte Antwortparameter der Rubrik Diagnose aus der CDD), solange das Logging laeuft. Die Rohdatei (`_raw.parquet`) enthaelt zusaetzlich die Spalte `signals` mit allen Signalen des Frames als Text, z. B. `RPM=0x1F40/2000 rpm`.

Bildschirmbild mit Erklaerungen



Schematische Darstellung der Oberflaeche; die Nummern verweisen auf die Erklaerungen darunter.

1. Auswahl der zu loggenden Botschaften und Signale (Filter aus der Summe der gewaehlten Daten).
2. Alle oder keine Signale markieren.
3. Name der Parquet-Datei (liegt im Ordner logs/ des Projekts).
4. Optionen: nur bei Wertaenderung, Mindestabstand je Signal, Rohdaten zusaetzlich.
5. Logging starten bzw. stoppen; die Statuszeile zeigt Zeilenzahl und Dateigroesse.

Zeichnet ausgewaehlte Signale im **Parquet**-Format auf (benoetigt `pyarrow`). Dateien liegen im Ordner `logs/` des Projekts.

Bedienelement	Funktion
Datenbaum	Zeigt die in der Rubrik Daten gewaehlten Botschaften/Signale. **Filter:** nur angehakte Signale werden aufgezeichnet (Haken an der Botschaft setzt/entfernt alle ihre Signale). Die Auswahl wird im Projekt gespeichert.
Alle / Keine	Alle Signale an- bzw. abwaehlen.
Dateiname	Vorbelegt mit *Projekt_Datum_Uhrzeit.parquet*; Endung wird bei Bedarf ergaenzt.
Mindestabstand je Signal	Pro Signal hoechstens ein Eintrag pro Zeitraum (ms) - reduziert die Datenmenge bei schnellen Botschaften. 0 = jeder Frame.
Nur bei Wertaenderung	Ein Signal wird nur geschrieben, wenn sich sein Wert gegenueber dem letzten geschriebenen aendert.
Rohframes	Schreibt zusaetzlich <*Name*>*_raw.parquet* mit **allen** Frames des Busses (unabhaengig von DBC und Signalauswahl, inkl. Fehlerframes). Spalten: Zeit, Richtung, Kanal, ID, Extended, FD, BRS, Fehler, DLC, Daten (binaer), Daten (Hex-Text), Botschaftsname. Logging ist damit auch ohne Signalauswahl moeglich. **Rohdaten-IDs** begrenzen die Frames, z. B. `100, 7E0-7EF` (hex, leer = alle).
Logging starten / stoppen	Startet bzw. beendet die Aufzeichnung (nur bei Verbindung). Beim Trennen oder Projektwechsel wird automatisch gestoppt und die Datei sauber abgeschlossen.
Statuszeile	Dateiname, geschriebene Zeilen, Dateigroesse.

Dateiformat

Spalte	Typ	Inhalt
timestamp	float64	Zeitstempel (Epoch-Sekunden)
direction	string	`RX` oder `TX`
message	string	Botschaftsname
signal	string	Signalname
value	float64	Physikalischer Wert

Das Langformat (eine Zeile je Signalwert) vermeidet Luecken bei unterschiedlichen Zykluszeiten. Daten werden gepuffert und alle 5 Sekunden bzw. ab 20 000 Zeilen auf die Platte geschrieben (Kompression zstd).

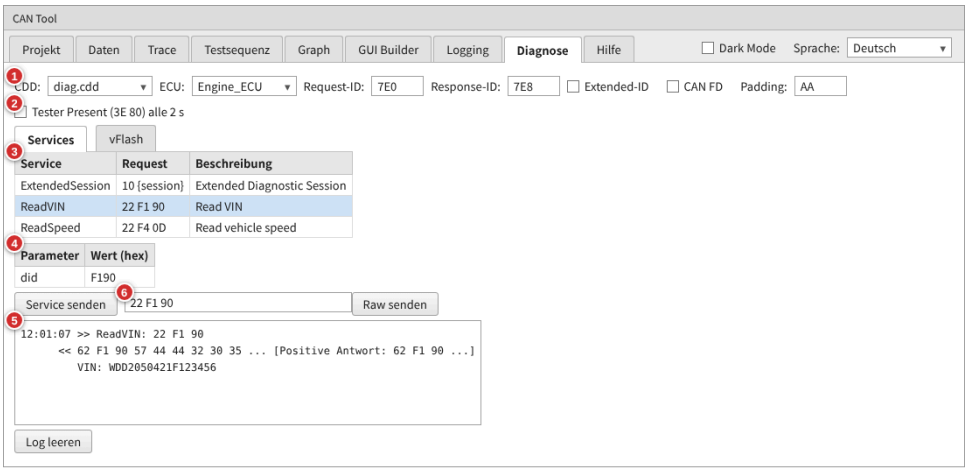
Auswertung in Python (siehe auch Anhang B):

```
import pandas as pd
df = pd.read_parquet("projects/demo/logs/demo_20261020_071500.parquet")
wide = df.pivot_table(index="timestamp", columns=["direction", "message", "signal"], values="value")
```

3.7 Rubrik "Diagnose"

Diagnose-Services (UDS) ueber **ISO-TP** (ISO 15765-2) auf CAN bzw. CAN FD. Die Services stammen aus der konvertierten CDD; zusaetzlich koennen beliebige Requests in Hex gesendet werden.

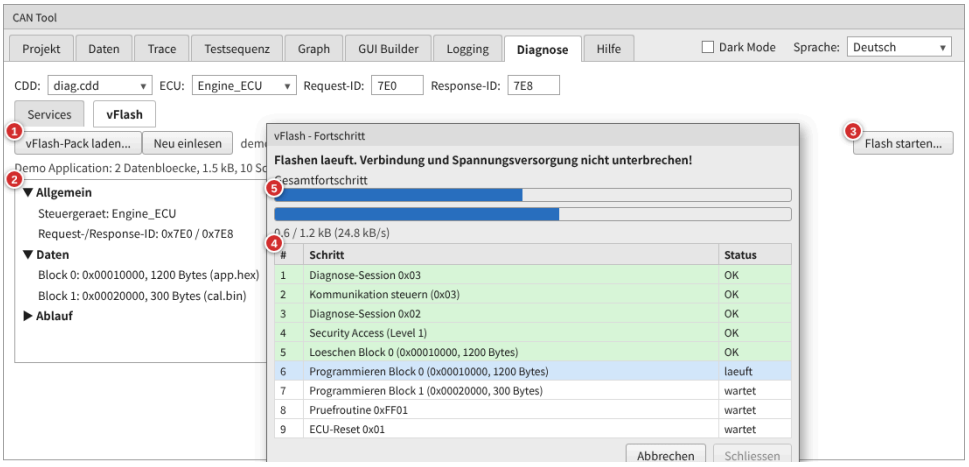
Bildschirmbild mit Erklaerungen



Schematische Darstellung der Oberflaeche; die Nummern verweisen auf die Erklaerungen darunter.

1. Diagnose-Konfiguration: CDD, ECU, Request-/Response-ID, Extended-ID, CAN FD, Padding.
2. Tester Present (3E 80) wird alle 2 s automatisch gesendet.
3. Services aus der CDD mit Request-Vorlage und Beschreibung.
4. Parameter des gewaehlten Services (Hex-Werte editierbar), danach „Service senden“.
5. Antwort-Log: gesendete Requests, Antworten (positiv/negativ) und dekodierte Werte.
6. Roh-Request in Hex senden (z. B. 22 F1 90) ohne CDD-Definition.

Bildschirmbild mit Erklaerungen



Schematische Darstellung der Oberflaeche; die Nummern verweisen auf die Erklaerungen darunter.

1. vFlash-Pack laden (.vflashpack/.zip, Manifest, HEX/S19/BIN); "Neu einlesen" liest die Datei erneut.
2. Interpretation des Packs: Allgemein (ECU, IDs, Seed&Key), Daten (Bloecke mit Adresse und Laenge), Ablauf (Schritte) und Hinweise.
3. Flash starten: nach einer Sicherheitsabfrage laeuft der Ablauf ueber die gewaehlte Hardware.
4. Fortschrittsfenster: je Schritt Status (wartet, laeuft, OK, FEHLER, uebersprungen) mit Details.
5. Gesamtfortschritt, Fortschritt des laufenden Datenblocks mit Datenrate und Protokoll; "Abbrechen" stoppt zwischen zwei Bloecken.

Konfiguration (obere Zeile)

Feld	Bedeutung
CDD	Konvertierte Diagnosebeschreibung aus dem Projekt-Cache.
ECU	Steuergeraet aus der CDD; ein Wechsel uebernimmt dessen Adressen.
Request-ID / Response-ID	CAN-IDs in Hex (Tester -> ECU / ECU -> Tester). Vorbelegt aus der CDD, hier ueberschreibbar.
Extended-ID	29-Bit-Adressierung.
CAN FD	ISO-TP mit FD-Frames (bis 64 Byte je Frame, mit Bitraten-Umschaltung). Muss zur Gegenstelle passen.
Padding	Fuellbyte in Hex (Standard AA); leer = ohne Auffuellen bei Classic-CAN.
Tester Present	Sendet alle 2 s `3E 80` (ohne Antwort), um eine Diagnosesession offen zu halten.
Reiter vFlash	Laedt ein vFlash-Pack (oder Manifest/HEX/S19/BIN), zeigt die Interpretation und fuehrt den Flashablauf mit der gewaehlten Hardware aus, mit Fortschrittsfenster (Anhang C).

Services senden

1. Service in der Liste markieren (Name, Request, Beschreibung).
2. Falls der Service Parameter hat, in der Tabelle *Parameter / Wert (hex)* den Wert eintragen (Platzhalter im Request).
3. **Service senden**. Der Versand laeuft im Hintergrund; die Oberflaeche bleibt bedienbar.
4. Das **Log** zeigt Request (>>), Antwort (<<) mit Auswertung (positiv/negativ, bei negativen Antworten NRC mit Klartext) sowie die dekodierten Antwortparameter der CDD (z. B. *VIN: WDD...*).

Raw senden: Request als Hex-Bytes eintragen (z. B. `22 F1 86`) und Enter druecken oder die Schaltflaeche nutzen.

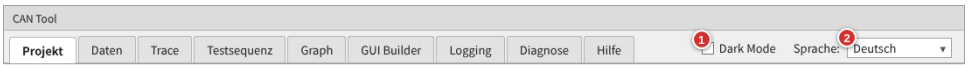
Technische Details

- Segmentierung: Single Frame, First Frame, Consecutive Frames und Flow Control, bis 4095 Byte (laengere ueber den erweiterten First Frame).
- Beim Empfang laengerer Antworten sendet das Tool Flow Control mit Blocksize 0 und STmin 0.
- Antwort-Timeout 2 s; bei *ResponsePending* (NRC 0x78) wird bis zu 10 s weiter gewartet.
- Requests ohne Antwort (Sub-Function mit Bit 7, z. B. `3E 80`) werden als gesendet protokolliert.
- In Testsequenzen verwenden `diag` und `diag_raw` dieselbe Adress-Konfiguration.

3.8 Rubrikzeile: Dark Mode und Sprache

Am rechten Rand der Rubrikzeile (neben den Reitern) befinden sich zwei Bedienelemente, die fuer alle Rubriken gelten.

Bildschirmbild mit Erklaerungen



Schematische Darstellung der Oberflaeche; die Nummern verweisen auf die Erklaerungen darunter.

1. Dark Mode: schaltet zwischen hellem und dunklem Erscheinungsbild um (wird gespeichert).
2. Sprache: Dropdown mit allen vorhandenen Sprachdateien; die Umschaltung gilt sofort, auch fuer die Hilfe.

Bedienelement	Funktion
Dark Mode	Haken schaltet zwischen hellem und dunklem Erscheinungsbild um. Betroffen sind Oberflaeche, Trace, Graph, Ergebnistabelle der Sequenz und diese Hilfe. Die Einstellung wird gespeichert und beim naechsten Start verwendet.
Sprache	Dropdown-Menue mit Deutsch (Primaersprache), English, Francais, Espanol, Chinesisch (Mandarin, vereinfacht) und Thai. Die Umschaltung wirkt sofort ohne Neustart. Die Auswahl wird gespeichert.

Was wird uebersetzt?

- Alle Beschriftungen, Schaltflaechen, Tabellenkoepfe, Tooltips, Dialoge und Meldungen der Oberflaeche.
- Die Befehlsreferenz der Testsequenz (Tooltips im Dropdown, Anhang A) und alle Kapitel dieser Hilfe.
- Fehler- und Warnmeldungen der Programmteile (z. B. Konvertierung, Verbindung, Sequenzlauf).
- **Nicht** uebersetzt werden Daten aus Ihren Dateien: Botschafts- und Signalnamen, Einheiten und Wertetabellen aus DBC/CDD, Dateinamen, Projektnamen und der Inhalt von Sequenzdateien.
- Bereits geschriebene Protokollzeilen (Diagnose-Log, Ergebnistabelle einer Sequenz) behalten die Sprache, in der sie entstanden sind.

Fuer Chinesisch und Thai muss auf dem Rechner eine Schriftart mit den entsprechenden Zeichen vorhanden sein (bei gaengigen Windows-, macOS- und Linux-Installationen der Fall).

Sprachdateien

Jede Sprache liegt in einer eigenen Datei im Ordner `lang/` (YAML; JSON ist ebenfalls moeglich):

Datei	Sprache
`lang/de.yaml`	Deutsch - Primaersprache und Rueckfallebene: fehlt in einer Sprache ein Text, wird der deutsche verwendet.
`lang/en.yaml`	English
`lang/fr.yaml`	Francais
`lang/es.yaml`	Espanol
`lang/zh.yaml`	Chinesisch (Mandarin, vereinfachte Schriftzeichen)
`lang/th.yaml`	Thai

Aufbau jeder Datei: Abschnitte `meta` (Anzeigename, Reihenfolge im Dropdown), `ui` (Oberflaeche), `core` (Meldungen), `steps` (Befehlsreferenz) und `help` (Kapitel dieser Hilfe). Platzhalter in geschweiften Klammern (z. B. `{name}`) und Platzhalter in doppelten eckigen Klammern in den Hilfetexten duerfen nicht veraendert werden.

Eine Sprache hinzufuegen oder aendern

1. `lang/de.yaml` kopieren und als `lang/<code>.yaml` speichern (z. B. `it.yaml`).
2. Im Abschnitt `meta` den Anzeigenamen (`name`) und die Position (`order`) setzen; alle Texte uebersetzen.
3. Mit `python tools/check_lang.py it` pruefen: Das Werkzeug meldet fehlende Schluessel, abweichende Platzhalter und beschaedigte HTML-Struktur.
4. Programm neu starten - die Sprache erscheint automatisch im Dropdown.

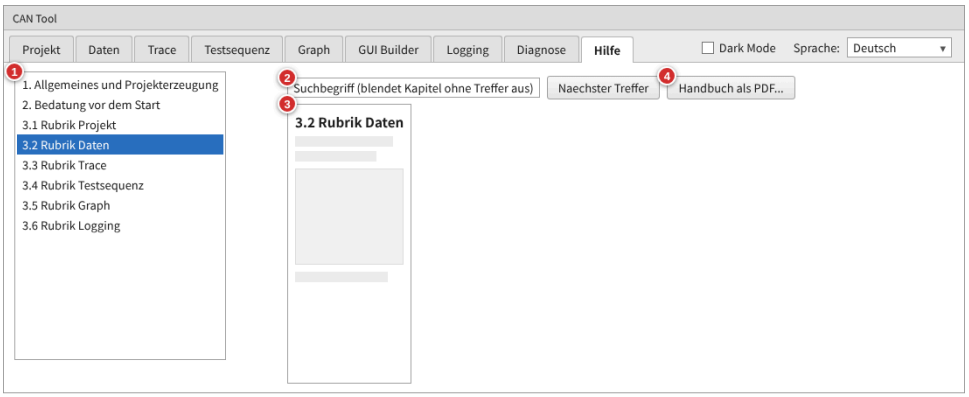
Kommandozeile

Auch die Ausgaben von `main.py` sind uebersetzt: `python main.py --lang en --list`. Ohne `--lang` wird die zuletzt in der GUI gewaehlte Sprache verwendet.

3.9 Rubrik "Hilfe"

Diese Rubrik. Links stehen die Kapitel, rechts der Text.

Bildschirmbild mit Erklaerungen



Schematische Darstellung der Oberflaeche; die Nummern verweisen auf die Erklaerungen darunter.

1. Kapitelliste der Hilfe.
2. Suchfeld: blendet Kapitel ohne Treffer aus; „Naechster Treffer“ springt weiter.
3. Kapiteltext mit kommentierten Abbildungen.
4. Handbuch als PDF: gibt die Hilfe in allen verfuegbaren Sprachen aus (mit Deckblatt und Ausgabedatum) und bietet sie zum Speichern an.

- **Suchfeld**: blendet Kapitel aus, in denen der Begriff nicht vorkommt, und markiert die erste Fundstelle im gewaehlten Kapitel. **Naechster Treffer** springt zur naechsten Fundstelle. Leeres Feld zeigt wieder alle Kapitel.
- Verweise im Text (z. B. auf den Anhang) lassen sich anklicken.
- Die Befehlsbeschreibungen entsprechen den Tooltips im Befehls-Dropdown der Rubrik Testsequenz.
- Die Hilfe erscheint in der gewaehlten Sprache (Kapitel 3.8) und wird beim Umschalten der Sprache sofort ausgetauscht; Hell/Dunkel-Umschaltung gilt auch hier.

3.10 Rubrik "XCP"

Messen und Verstellen von Steuergeraete-Variablen per **XCP on CAN** auf Basis einer **A2L-Datei** (ASAP2). Die A2L wird wie DBC und CDD konvertiert: Quelldateien liegen im Projektordner `a2l/`, zur Laufzeit wird nur der Cache `cache/<name>.a2l.json|yaml` gelesen.

Einrichtung

1. A2L ueber **Importieren** (Rubrik Projekt) in das Projekt kopieren; sie wird automatisch konvertiert (`python main.py --convert demo` geht auch ohne GUI).
2. CAN verbinden (Rubrik Projekt). XCP nutzt dieselbe Schnittstelle.
3. Rubrik XCP: A2L waehlen, **Master-ID** (Kommandos an das Steuergeraet) und **Slave-ID** (Antworten) pruefen. Sie werden aus dem XCP-Teil der A2L (`IF_DATA XCP / XCP_ON_CAN`) vorbelegt und koennen geaendert werden.
4. **XCP verbinden**. Im Log stehen alle Kommandos (->) und Antworten (<-) als Hex.

Bedienelement	Funktion
Liste links	Alle Messgroessen (MEASUREMENT) und Parameter (CHARACTERISTIC vom Typ VALUE) der A2L; Filter nach Namen; Doppelklick oder *Zur Beobachtung* uebernimmt sie.
Beobachtung	Zeigt je Eintrag Rohwert und physikalischen Wert mit Einheit. Die Auswahl wird im Projekt gespeichert.
Zyklisch lesen	Liest alle beobachteten Eintraege im eingestellten Intervall (ms). Es wird per SHORT_UPLOAD gepollt; DAQ-Listen werden nicht verwendet.
Einmal lesen	Liest die Beobachtung einmalig.
Wert schreiben	Schreibt den eingegebenen Wert (physikalisch, bei Textumrechnung auch der Text, z. B. *RUN*) in die gewaehlte Zeile (SET_MTA + DOWNLOAD) und liest sie danach zurueck.

Interpretation der A2L

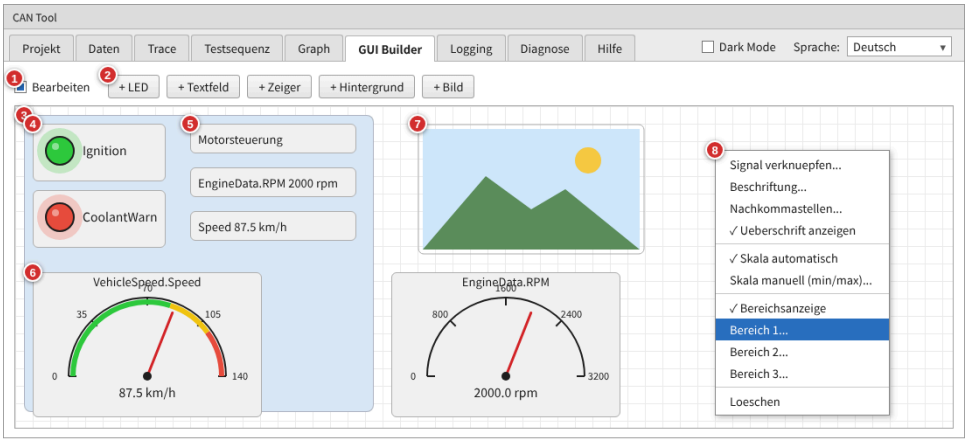
- Datentypen UBYTE, SBYTE, UWORD, SWORD, ULONG, SLONG, A_UINT64, A_INT64, FLOAT32_IEEE, FLOAT64_IEEE; Byte-Reihenfolge laut CONNECT-Antwort des Steuergeraets.
- Umrechnung IDENTICAL, LINEAR, RAT_FUNC (ohne quadratische Terme) und TAB_VERB (Textwerte); BIT_MASK wird beim Lesen ausmaskiert.
- Nicht unterstuetzte Objekte (Kurven, Kennfelder, Felder, Messgroessen ohne Adresse) erscheinen ausgegraut.

Mit Hardware *Simulation* startet das Tool einen simulierten XCP-Slave, der die A2L des Projekts abbildet (Demo: `projects/demo/a2l/demo.a2l`). Nicht enthalten: DAQ/STIM, Seed&Key, Kalibrierseiten und Flash-Programmierung. Ob ein echtes Steuergeraet alle Zugriffe erlaubt, haengt von dessen Schutzstufen ab.

3.11 Rubrik "GUI Builder"

Hier entsteht eine eigene Anzeige aus einfachen Elementen fuer vorhandene Signale. Die Elemente dienen nur der **Ansicht**; es gibt keine Werteeingabe. Verknuepfen lassen sich die Signale, die in der Rubrik Daten ausgewaehlt sind (empfangen oder gesendet). Aufbau und Verknuepfungen werden im Projekt gespeichert (`builder`).

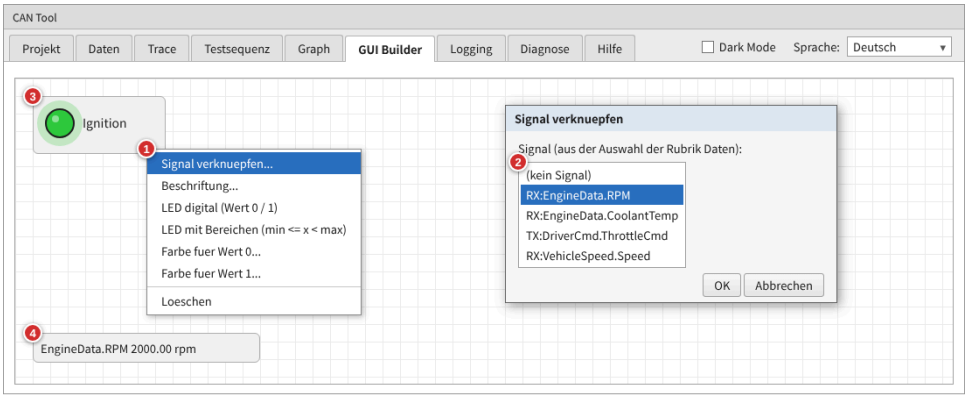
Bildschirmbild mit Erklaerungen



Schematische Darstellung der Oberflaeche; die Nummern verweisen auf die Erklaerungen darunter.

1. Haken Bearbeiten: Raster, Hinzufuegen, Verschieben und Groessenaenderung moeglich.
2. Buttons zum Hinzufuegen von LED, Textfeld, Zeiger, Hintergrund und Bild.
3. Hintergrundelement mit waehlbarer Farbe: fasst Elemente optisch zusammen und liegt hinter ihnen.
4. LEDs: digital (Wert 0 / 1) und mit Bereichen (Farbe je nach Bereich min <= x < max).
5. Textfelder: links ohne Signal (reiner Text), rechts mit Signal (Wert und Einheit live).
6. Zeiger mit 180-Grad-Skala, automatisch oder manuell skaliert, optional mit Bereichsanzeige.
7. Bilderrahmen: das gewaehlte Bild wird automatisch auf den Rahmen skaliert.
8. Rechtsklick-Menue des Zeigers mit Signal, Ueberschrift, Skala, Bereichsanzeige und Loeschen.

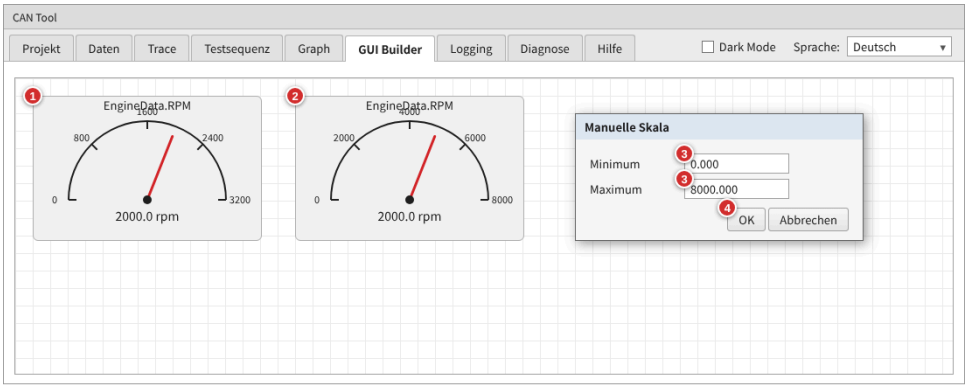
Bildschirmbild mit Erklaerungen



Schematische Darstellung der Oberflaeche; die Nummern verweisen auf die Erklaerungen darunter.

1. Rechtsklick auf die LED: Signal verknuepfen, Beschriftung, Modus digital/Bereiche, Farben, Loeschen.
2. Auswahl: Signale aus der Rubrik Daten als RX:/TX:Botschaft.Signal, oder (kein Signal).
3. LED: Farbe je nach Signalwert (digital 0 / 1 oder nach Bereich 1 bis 3).
4. Textfeld mit Signal: Text, Wert und Einheit; Nachkommastellen einstellbar.

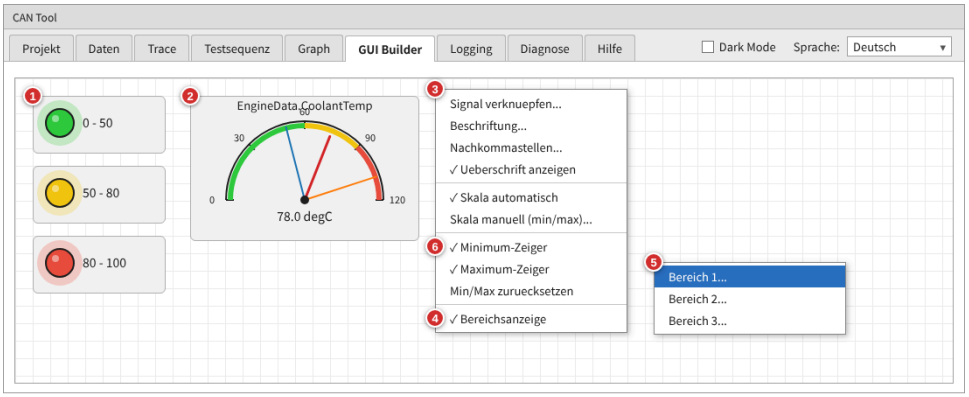
Bildschirmbild mit Erklaerungen



Schematische Darstellung der Oberflaeche; die Nummern verweisen auf die Erklaerungen darunter.

1. Zeiger mit automatischer Skala: Grenzen auf runde Zahlen, hier 0 bis 3200.
2. Zeiger mit manueller Skala, hier 0 bis 8000.
3. Dialog Manuelle Skala: Minimum und Maximum eingeben.
4. OK uebernimmt die Skala (nur wenn Minimum kleiner als Maximum), Abbrechen verwirft.

Bildschirmbild mit Erklaerungen



Schematische Darstellung der Oberflaeche; die Nummern verweisen auf die Erklaerungen darunter.

1. Drei LEDs mit Bereichen: je ein Wertebereich (min <= x < max) bestimmt die Farbe.
2. Zeiger mit Bereichsanzeige: drei farbige Bereiche auf der Skala (Bereich 1 bis 3).
3. Menue: Ueberschrift anzeigen schaltet die Beschriftung ein oder aus.
4. Menue: Bereichsanzeige ein- oder ausschalten.
5. Bereich 1 bis 3: je Minimum, Maximum (Regel Minimum <= x < Maximum), Farbe und Ein/Aus.
6. Menue: optionaler zweiter und dritter Zeiger fuer Minimum (blau) und Maximum (orange) sowie Zuruecksetzen.

Elemente

Element	Funktion
LED	Zeigt den Zustand eines Signals als Farbe. **Digital**: Wert 0 und Wert 1 haben je eine waehlbare Farbe. **Bereiche**: wie bei der Zeiger-Skala drei Bereiche (min <= x < max) mit je einer Farbe; liegt der Wert in keinem Bereich oder fehlt er, ist die LED grau.
Textfeld ohne Signal	Reines Textfeld mit frei waehlbarem Text, z. B. als Ueberschrift oder Beschriftung.
Textfeld mit Signal	Zeigt Text und aktuellen Wert des Signals mit Einheit; wird laufend aktualisiert. Die Nachkommastellen sind einstellbar.
Zeiger	Zeigerinstrument mit Skala ueber 180 Grad (Halbkreis): links Minimum, rechts Maximum; darunter der Zahlenwert mit Einheit.
Hintergrund	Einfaches farbiges Rechteck (Farbe waehlbar) ohne weitere Funktion. Es fasst andere Elemente optisch zusammen und liegt immer hinter den Funktionselementen, aber ueber dem Standardhintergrund.
Bild	Bilderrahmen mit frei waehlbarem Bild (PNG, JPG, BMP, GIF, SVG). Das Bild wird automatisch an den Rahmen angepasst (Seitenverhaeltnis bleibt erhalten) und folgt jeder Groessenaenderung.

Bedienung

Bedienelement	Funktion
Bearbeiten	Haken gesetzt: Raster sichtbar, Elemente koennen hinzugefuegt, verschoben, in der Groesse geaendert und geloescht werden. Ohne Haken ist die Anzeige gesperrt (reine Ansicht).
+ LED / + Textfeld / + Zeiger	Fuegt ein neues Element hinzu (auch per Rechtsklick auf die freie Flaeche an der Mausposition).
Ziehen	Verschiebt ein Element; die Ecke unten rechts aendert die Groesse.
Rechtsklick auf ein Element	Menue: Signal verknuepfen (oder "kein Signal"), Beschriftung bzw. Text, LED-Schwelle und -Farbe, Nachkommastellen, Skala (Zeiger), Loeschen.

Zeigerelement und Skala

Die Skala hat zwei Betriebsarten. **Automatisch** (Standard): Der Bereich waechst mit den beobachteten Werten und wird auf runde Zahlen gesetzt (z. B. 0 bis 100); Skalenstriche und Zahlen passen sich an. **Manuell**: Per **Rechtsklick** auf den Zeiger und **Skala manuell (min/max)** werden Minimum und Maximum vorgegeben (das Minimum muss kleiner als das Maximum sein). Mit dem Haken **Skala automatisch** im selben Menue wird wieder auf Automatik zurueckgeschaltet. Werte ausserhalb der Skala stellen den Zeiger am Anschlag dar.

Schnellstart

1. Haken **Bearbeiten** gesetzt lassen (Raster sichtbar).
2. **+ LED**, **+ Textfeld** oder **+ Zeiger** klicken; das Element erscheint auf der Zeichenflaeche (auch per Rechtsklick auf die freie Flaeche an der Mausposition).
3. Mit **Rechtsklick** auf das Element **Signal verknuepfen** waehlen und ein Signal aus der Liste auswaehlen.
4. Beschriftung, LED-Schwelle, Nachkommastellen oder Skala einstellen und das Element per Ziehen positionieren; die Ecke unten rechts aendert die Groesse.
5. Haken **Bearbeiten** entfernen: die Anzeige ist gesperrt und dient nur noch der Ansicht.

Signal verknuepfen

Die Liste zeigt die in der Rubrik Daten ausgewaehlten Signale als `RX:Msg.Signal` bzw. `TX:Msg.Signal`, dazu den Eintrag "(kein Signal)", der die Verknuepfung loest. Fehlt ein Signal, wird es zuerst in der Rubrik Daten ausgewaehlt. Die Werte erscheinen, sobald Daten empfangen bzw. gesendet werden; die Einheit stammt aus der DBC. Beim Wechsel des Signals wird die automatische Skala zurueckgesetzt.

LED und Textfeld einstellen

Einstellung	Wirkung
Beschriftung / Text	LED: Beschriftung neben der LED (ohne Eintrag der Signalname). Textfeld: freier Text; mit Signal steht er vor dem Wert.
LED-Modus	Digital (Wert 0 / Wert 1) oder Bereiche (Bereich 1 bis 3). Digital: Wert 0 ergibt Farbe 0, jeder andere Wert Farbe 1.
Farben / Bereiche	Digital: Farbe fuer Wert 0 und Wert 1. Bereiche: je Bereich Ein/Aus, Minimum, Maximum und Farbe nach der Regel **Minimum <= x < Maximum**; der erste passende Bereich bestimmt die Farbe. Dieselbe Einstellung wie bei der Bereichsanzeige des Zeigers.
Nachkommastellen	Anzahl der Dezimalstellen von 0 bis 9 fuer Textfeld und Zeiger.
Ueberschrift (Zeiger)	Der Haken **Ueberschrift anzeigen** blendet die Beschriftung ueber dem Zeiger ein oder aus; der Platz wird dann fuer die Skala genutzt.
Bereichsanzeige (Zeiger)	Der Haken **Bereichsanzeige** faerbt die Skala in bis zu drei Bereichen (Bereich 1 bis 3), je mit Minimum, Maximum, Farbe und Ein/Aus nach der Regel **Minimum <= x < Maximum**; die drei Bereiche stehen dann im Menue.
Min-/Max-Zeiger (Zeiger)	Mit den Haken **Minimum-Zeiger** und **Maximum-Zeiger** zeigt die Skala zusaetzlich einen duennen zweiten (blau) und dritten Zeiger (orange) fuer den kleinsten bzw. groessten Wert seit Beginn der Beobachtung. **Min/Max zuruecksetzen** beginnt neu; ein Signalwechsel oder das Umschalten der Haken setzt ebenfalls zurueck. Die Werte werden nicht im Projekt gespeichert, nur die Haken.

Skala des Zeigers im Detail

Automatisch: Bis der erste Wert vorliegt gilt 0 bis 100. Danach merkt sich der Zeiger kleinsten und groessten beobachteten Wert und rundet die Grenzen auf runde Zahlen (1, 2, 2,5, 5 mal Zehnerpotenz); liegen alle Werte nahe bei 0 und sind nicht negativ, beginnt die Skala bei 0. Die Skala waechst mit, schrumpft aber nicht. Mit dem Haken **Skala automatisch** im Rechtsklick-Menue (aus- und wieder einschalten) beginnt die Beobachtung neu. Die Skala zeigt fuenf Teilstriche mit Zahlen.

Manuell: Rechtsklick auf den Zeiger, **Skala manuell (min/max)**: im Dialog Minimum und Maximum eingeben (Dezimalstellen erlaubt). Ist das Minimum nicht kleiner als das Maximum, erscheint eine Meldung und nichts wird uebernommen. Die manuelle Skala gilt sofort, bleibt im Projekt gespeichert und schaltet die Automatik ab. Werte ausserhalb stellen den Zeiger am Anschlag dar.

Hintergrund und Bild

Die Elemente liegen in Ebenen: Standardhintergrund (Zeichenflaeche), darueber die Hintergrundelemente, darueber die Funktionselemente (LED, Textfeld, Zeiger, Bild). Hintergrundelemente werden beim Hinzufuegen, Verschieben und Laden automatisch hinter die uebrigen Elemente gelegt, so kann z. B. ein farbiger Rahmen mehrere Anzeigen zusammenfassen. Das gewaehlte Bild wird als Kopie im Projektordner (`images`) abgelegt, damit das Projekt austauschbar bleibt; ohne Bild erscheint "Kein Bild".

LED-Farben und Bereichsanzeige

Beispiel LED digital: Wert 0 = rot, Wert 1 = gruen. Beispiel LED mit Bereichen (Temperatur): Bereich 1: 0 bis 80 gruen, Bereich 2: 80 bis 100 gelb, Bereich 3: 100 bis 120 rot; ausserhalb ist die LED grau.

Bereichsanzeige des Zeigers: Es gibt drei Bereiche, jeder mit Minimum, Maximum und Farbe. Ein Wert x gehoert zu einem Bereich, wenn **Minimum <= x < Maximum** gilt (das Minimum gehoert dazu, das Maximum nicht). Beispiel: Bereich 1: 0 bis 80 gruen, Bereich 2: 80 bis 100 gelb, Bereich 3: 100 bis 120 rot. Jeder Bereich laesst sich einzeln ein- oder ausschalten; Teile ausserhalb der Skala werden abgeschnitten, das Minimum muss kleiner als das Maximum sein. Bei automatischer Skala passen sich die Baender der aktuellen Skala an.

Speicherung und Grenzen

Elemente, Positionen, Groessen, Verknuepfungen und Einstellungen werden im Projekt gespeichert (`builder`). Die Zeichenflaeche ist scrollbar. Der GUI Builder zeigt nur Werte an; es gibt keine Eingabe- oder Schaltelemente. Angezeigt werden nur Signale aus der Auswahl der Rubrik Daten.

Hinweis: Signale erscheinen erst, wenn Daten empfangen bzw. gesendet wurden; bis dahin zeigen Textfeld und Zeiger "--" bzw. Nullstellung, die LED ist aus.

Anhang A: Befehlsreferenz Testsequenz

Jeder Schritt ist ein JSON-Objekt mit dem Feld `type` und den unten genannten Elementen. Physikalische Werte sind Werte nach Faktor und Offset der DBC.

set_signal - Signalwert setzen

Setzt den physikalischen Wert eines Signals in einer Sendebotschaft. Der neue Wert geht mit dem naechsten (zyklischen oder einzelnen) Frame auf den Bus.

Format

```
{"type": "set_signal", "message": "<Botschaft>", "signal": "<Signal>", "value": <Zahl oder Text>}
```

Elemente

Element	Typ	Beschreibung
`message`	Text, Pflicht	Name der DBC-Botschaft. Fehlt sie in der Sendeliste, wird sie ergaenzt (zunaechst nicht zyklisch aktiv).
`signal`	Text, Pflicht	Name des Signals innerhalb der Botschaft.
`value`	Zahl oder Text, Pflicht	Physikalischer Wert (nach Faktor/Offset, z. B. 40 fuer 40 %). Bei Signalen mit Wertetabelle auch deren Text, z. B. "R". Werte ausserhalb des Signalbereichs werden begrenzt.

Beispiel

```
{
  "type": "set_signal",
  "message": "DriverCmd",
  "signal": "ThrottleCmd",
  "value": 40
}
```

Hinweis: Nicht angegebene Signale der Botschaft bleiben unveraendert (Startwert 0).

enable_tx - Zyklisches Senden ein-/ausschalten

Startet oder stoppt das zyklische Senden einer Botschaft mit ihrem eingestellten Zyklus.

Format

```
{"type": "enable_tx", "message": "<Botschaft>", "enabled": <true|false>}
```

Elemente

Element	Typ	Beschreibung
`message`	Text, Pflicht	Name der DBC-Botschaft (wird bei Bedarf zur Sendeliste ergaenzt; Zyklus aus der DBC, sonst 100 ms).
`enabled`	true/false, Pflicht	true = zyklisch senden, false = Senden stoppen.

Beispiel

```
{
  "type": "enable_tx",
  "message": "DriverCmd",
  "enabled": true
}
```

Hinweis: Das erste Telegramm geht sofort raus, danach im eingestellten Zyklus.

set_cycle - Zykluszeit aendern

Aendert die Zykluszeit einer Sendebotschaft (wirkt sofort, auch waehrend sie zyklisch sendet).

Format

```
{"type": "set_cycle", "message": "<Botschaft>", "cycle_ms": <Zahl>}
```

Elemente

Element	Typ	Beschreibung
`message`	Text, Pflicht	Name der DBC-Botschaft.
`cycle_ms`	Zahl, Pflicht	Zykluszeit in Millisekunden. 0 pausiert das zyklische Senden.

Beispiel

```
{
  "type": "set_cycle",
  "message": "DriverCmd",
  "cycle_ms": 20
}
```

send_once - Botschaft einmal senden

Sendet genau einen Frame mit den aktuellen Signalwerten, unabhaengig vom zyklischen Senden.

Format

```
{"type": "send_once", "message": "<Botschaft>"}
```

Elemente

Element	Typ	Beschreibung
`message`	Text, Pflicht	Name der DBC-Botschaft.

Beispiel

```
{
  "type": "send_once",
  "message": "DriverCmd"
}
```

send_raw - Rohframe senden

Sendet einen Rohframe (ID + Daten) auf den CAN, einmalig oder zyklisch. Mit cyclic=true und enabled=false wird das zyklische Senden dieser ID beendet.

Format

```
{"type": "send_raw", "id": "<hex>", "data": "<hex>", "dlc": <n>, "cyclic": <true|false>, "cycle_ms": <n>}
```

Elemente

Element	Typ	Beschreibung
`id`	Zahl oder Text, Pflicht	CAN-ID, hexadezimal (z. B. "0x100" oder "100").
`data`	Hex-Text, Pflicht	Nutzdaten als Hex-Bytes, z. B. "00 01 02 03 04 05 06 07".
`dlc`	Zahl, optional	Datenlaenge (0-8, 12, 16, 20, 24, 32, 48, 64). Standard: Anzahl der Datenbytes.
`cyclic`	true/false, optional	true = zyklisch senden, false = einmal senden (Standard).
`cycle_ms`	Zahl, optional	Zykluszeit in ms bei cyclic=true (Standard 100).
`enabled`	true/false, optional	false beendet das zyklische Senden dieser ID (nur mit cyclic=true, Standard true).
`extended`	true/false, optional	true erzwingt Extended-ID (Standard: automatisch ab 0x800).
`fd`	true/false, optional	true erzwingt CAN FD (Standard: automatisch bei DLC ueber 8).

Beispiel

```
{
  "type": "send_raw",
  "id": "0x100",
  "data": "00 01 02 03 04 05 06 07",
  "dlc": 8,
  "cyclic": true,
  "cycle_ms": 1000
}
```

Hinweis: Fehlende Datenbytes werden mit 00 aufgefuellt. ID ueber 0x7FF = Extended, DLC ueber 8 = CAN FD. Zyklisch gesendete Frames laufen nach dem Schritt weiter.

wait - Feste Zeit warten

Pausiert die Sequenz fuer eine feste Zeit. Zyklisches Senden und Empfang laufen weiter.

Format

```
{"type": "wait", "ms": <Zahl>}
```

Elemente

Element	Typ	Beschreibung
`ms`	Zahl, Pflicht	Wartezeit in Millisekunden.

Beispiel

```
{
  "type": "wait",
  "ms": 500
}
```

Hinweis: Ein Klick auf Stopp beendet auch ein laufendes Warten sofort.

wait_for - Auf Signalbedingung warten

Wartet, bis ein Signal die Bedingung erfuellt. Es zaehlen nur Werte, die NACH Beginn dieses Schritts empfangen wurden. Der Schritt ist ok beim ersten Treffer, sonst Fehler nach Ablauf des Timeouts.

Format

```
{"type": "wait_for", "direction": "RX", "message": "<Botschaft>", "signal": "<Signal>", "op": "<Operator>", "value": <Zahl>, "timeout_ms": <
```

Elemente

Element	Typ	Beschreibung
`direction`	"RX" oder "TX", Pflicht	Empfangene oder gesendete Botschaft auswerten.
`message`	Text, Pflicht	Name der DBC-Botschaft.
`signal`	Text, Pflicht	Name des Signals.
`op`	Text, Pflicht	Vergleich: == != < <= > >=
`value`	Zahl, Pflicht	Vergleichswert (physikalisch).
`timeout_ms`	Zahl, optional	Maximale Wartezeit in ms (Standard 3000).

Beispiel

```
{
  "type": "wait_for",
  "direction": "RX",
  "message": "EngineData",
  "signal": "RPM",
  "op": ">",
  "value": 100,
  "timeout_ms": 3000
}
```

Hinweis: Die Auswertung ist unabhaengig von der Signalauswahl in der Rubrik 'Daten'.

check - Signalwert pruefen

Prueft sofort den zuletzt empfangenen Wert eines Signals (kein Warten). Der Schritt schlaegt fehl, wenn die Bedingung nicht erfuellt ist oder seit Sequenzstart noch kein Wert empfangen wurde.

Format

```
{"type": "check", "direction": "RX", "message": "<Botschaft>", "signal": "<Signal>", "op": "<Operator>", "value": <Zahl>}
```

Elemente

Element	Typ	Beschreibung
`direction`	"RX" oder "TX", Pflicht	Empfangene oder gesendete Botschaft auswerten.
`message`	Text, Pflicht	Name der DBC-Botschaft.
`signal`	Text, Pflicht	Name des Signals.
`op`	Text, Pflicht	Vergleich: == != < <= > >=
`value`	Zahl, Pflicht	Vergleichswert (physikalisch).

Beispiel

```
{
  "type": "check",
  "direction": "RX",
  "message": "EngineData",
  "signal": "CoolantTemp",
  "op": "<",
  "value": 150
}
```

Hinweis: Direkt nach dem Start ist oft noch kein Wert da - vorher mit wait_for synchronisieren.

diag - Diagnose-Service aus der CDD

Sendet einen in der CDD definierten Diagnose-Service per ISO-TP und wertet die Antwort aus.

Format

```
{"type": "diag", "service": "<Servicename>", "params": {"<Parameter>": <Wert>}, "expect": "<Hex-Praefix>", "timeout_ms": <Zahl>}
```

Elemente

Element	Typ	Beschreibung
`service`	Text, Pflicht	Name des Services in der CDD (Rubrik Diagnose), z. B. ReadVIN.
`params`	Objekt, optional	Werte fuer {Platzhalter} im Request. Zahl = dezimal, Text = hexadezimal ("F186").
`expect`	Hex-Text, optional	Erwarteter Anfang der Antwort, z. B. "62 F1 90". Leer = jede positive Antwort genuegt.
`timeout_ms`	Zahl, optional	Antwort-Timeout in ms (Standard 1000). Bei 'ResponsePending' (NRC 0x78) wird laenger gewartet.

Beispiel

```
{
  "type": "diag",
  "service": "ExtendedSession",
  "params": {
    "session": 3
  },
  "expect": "50 03"
}
```

Hinweis: Adressierung (Request-/Response-ID, FD) kommt aus der Rubrik 'Diagnose'.

diag_raw - Diagnose-Request in Hex

Sendet einen beliebigen UDS-Request (Hex-Bytes) per ISO-TP und wertet die Antwort aus.

Format

```
{"type": "diag_raw", "request": "<Hex-Bytes>", "expect": "<Hex-Praefix>", "timeout_ms": <Zahl>}
```

Elemente

Element	Typ	Beschreibung
`request`	Hex-Text, Pflicht	Request-Bytes, z. B. "22 F1 86".
`expect`	Hex-Text, optional	Erwarteter Anfang der Antwort, z. B. "62 F1 86". Leer = jede positive Antwort genuegt.
`timeout_ms`	Zahl, optional	Antwort-Timeout in ms (Standard 1000).

Beispiel

```
{
  "type": "diag_raw",
  "request": "3E 00",
  "expect": "7E 00"
}
```

Hinweis: Requests ohne Antwort (Sub-Function mit gesetztem Bit 7, z. B. "3E 80") gelten als ok.

loop - Schleife

Fuehrt die enthaltenen Schritte mehrfach nacheinander aus.

Format

```
{"type": "loop", "count": <Zahl>, "steps": [ <Schritt>, <Schritt>, ... ]}
```

Elemente

Element	Typ	Beschreibung
`count`	Zahl, Pflicht	Anzahl der Durchlaeufe.
`steps`	Liste, Pflicht	Verschachtelte Schritte (beliebige Typen, auch weitere Schleifen).

Beispiel

```
{
  "type": "loop",
  "count": 3,
  "steps": [
    {
      "type": "set_signal",
      "message": "DriverCmd",
      "signal": "ThrottleCmd",
      "value": 20
    },
    {
      "type": "wait",
      "ms": 200
    }
  ]
}
```

Hinweis: Im Ergebnis erscheinen innere Schritte als z. B. 3.2.1 (Schritt 3, Durchlauf 2, Schritt 1).

comment - Kommentar

Tut nichts; dient nur der Dokumentation der Sequenz.

Format

```
{"type": "comment", "text": "<Text>"}
```

Elemente

Element	Typ	Beschreibung
`text`	Text, optional	Beliebiger Kommentar.

Beispiel

```
{
  "type": "comment",
  "text": "Hier beginnt der Hochlauftest"
}
```

Anhang B: Hinweise und Grenzen

- **Zeitgenauigkeit**: Zyklisches Senden erfolgt per Software-Timer (Genauigkeit im Bereich weniger Millisekunden), nicht hardware-getriggert.
- **DBC**: Extended Multiplexing und Umgebungsvariablen werden nicht unterstuetzt.
- **CDD**: Originale CANdelaStudio-Dateien nur heuristisch (Kapitel 2.2).
- **PEAK + CAN FD**: Bit-Timing fest vorbelegt (80 MHz, 500k/2M); Abweichungen ueber `hardware.extra`.
- **Sicherheit**: Zyklisches Senden startet nie automatisch beim Laden eines Projekts. Auf Fahrzeug- oder Pruefstandsbussen vor dem Verbinden Sendeliste und Werte pruefen.
- **Tastatur**: In Tabellen mit Doppelklick bearbeiten; im Raw-Diagnosefeld sendet Enter den Request.

Testen ohne Hardware

Das Demo-Projekt `demo` nutzt die Simulation mit virtuellem Steuergeraet. Damit lassen sich alle Rubriken (Senden/Empfangen, Trace, Graph, Logging, Sequenzen *startup* und *driving*, Diagnose) ohne Hardware ausprobieren. Die Kerntests laufen mit `python -m unittest discover -s tests -v`.

Anhang C: vFlash-Pack und Manifest

Der Reiter vFlash in der Rubrik Diagnose laedt ein Flash-Paket, interpretiert es und fuehrt es mit dem eigenen UDS-Flashablauf ueber die gewaehlte Hardware aus (Vector, PEAK, SocketCAN, Simulation). Die Vector-Software vFlash wird dafuer nicht benoetigt.

Unterstuetzte Quellen

- `*.vflashpack` oder `*.zip` mit `flashpack.json` (oder `flashpack.yaml`) und den Datendateien: vollstaendig unterstuetzt.
- Echtes vFlash-Pack ohne Manifest: Das interne Format ist nicht oeffentlich dokumentiert. HEX-, S-Record- und BIN-Dateien werden gefunden und das Projekt wird heuristisch gelesen. Ablauf, Adressen und Seed&Key muessen meist ueber ein Manifest ergaenzt werden.
- Einzelne Manifest-Datei oder Abbild (`.hex`, `.s19`, `.srec`, `.mot`, `.bin`) mit generischem Standardablauf.

Manifest flashpack.json

```
{
  "format": "canTool-flashpack-1",
  "name": "Demo Application",
  "ecu": "Engine_ECU",
  "request_id": "0x7E0",
  "response_id": "0x7E8",
  "extended": false,
  "fd": false,
  "security": {
    "type": "xor",
    "level": 1,
    "value": "0xFFFFFFFF"
  },
  "address_format": "0x44",
  "data": [
    {
      "file": "app.hex",
      "name": "Application"
    },
    {
      "file": "cal.bin",
      "address": "0x00020000",
      "name": "Calibration"
    }
  ],
  "steps": [
    {
      "type": "session",
      "session": 3
    },
    {
      "type": "comm_control",
      "control": 3,
      "comm_type": 1
    },
    {
      "type": "session",
      "session": 2
    },
    {
      "type": "security",
      "level": 1
    },
    {
      "type": "erase",
      "block": "all"
    },
    {
      "type": "download",
      "block": "all"
    },
    {
      "type": "check",
      "routine": "FF01"
    },
    {
      "type": "reset",
      "reset": 1
    }
  ]
}
```

Element	Bedeutung
`name`, `ecu`	Anzeigename und Steuergeraet.
`request_id`, `response_id`, `extended`, `fd`	ISO-TP-Adressierung; fehlt sie, gilt die Diagnose-Konfiguration der Rubrik Diagnose.
`security`	Seed&Key: `type` = `none`, `xor`, `add` oder `constant`, dazu `level` (Seed-Level, Key = Level + 1) und `value`.
`address_format`	addressAndLengthFormatIdentifier, z. B. `0x44` = 4 Byte Laenge und 4 Byte Adresse.
`max_chunk`	Obergrenze der Nutzdaten je Transfer-Request (sonst aus der Antwort auf 0x34).
`data`	Liste der Datendateien (`file`, `name`); fuer BIN-Dateien ist `address` Pflicht.
`steps`	Eigener Ablauf; fehlt er, gilt der Standardablauf.

Ablaufschritte (steps)

Schritt	Parameter und Wirkung

`session`	`session`: Diagnose-Session (0x10).
`comm_control`	`control`, `comm_type`: Kommunikation steuern (0x28).
`security`	`level`: Seed anfordern, Key berechnen und senden (0x27).
`erase`	`block` (`all` oder Nummer), `routine` (Standard FF00): Speicher loeschen (0x31).
`download`	`block`: Request Download, Transfer Data, Transfer Exit (0x34, 0x36, 0x37).
`check`, `routine`	`routine`, `data` (Hex), `control`, `timeout_ms`: Routine starten (0x31).
`request`	`data` (Hex), `expect` (erwartete Antwort-Anfangsbytes).
`reset`	`reset`: ECU-Reset (0x11).
`wait`, `comment`	`ms` Wartezeit bzw. `text` als Kommentar.

Standardablauf

Ohne `steps` werden nacheinander ausgefuehrt: Session 0x03, Kommunikation abschalten (0x28 03 01), Session 0x02, Security Access (falls `security` gesetzt), Loeschen und Programmieren aller Bloecke, ECU-Reset 0x11 01. Dieser Ablauf ist generisch und nicht herstellerspezifisch.

Sicherheitshinweise

Flashen loescht und ueberschreibt Speicher des Steuergeraets. Spannungsversorgung und Busverbindung duerfen waehrenddessen nicht unterbrochen werden. Aus Paketen wird nie Code ausgefuehrt; herstellerspezifische Seed&Key-DLLs werden nicht unterstuetzt. Bei einem Fehler stoppt der Ablauf, die restlichen Schritte werden uebersprungen.

English – Manual

CAN Tool · Issue date: 2026-10-29

1. General information and creating a project

2. Preparing the files before startup

3.1 Project tab

3.2 Data tab

3.3 Trace tab

3.4 Test sequence tab

3.5 Graph tab

3.6 Logging tab

3.7 Diagnostics tab

3.8 Tab bar: dark mode and language

3.9 Help tab

3.10 XCP tab

3.11 GUI Builder tab

Appendix A: Test sequence command reference

Appendix B: Notes and limits

Appendix C: vFlash pack and manifest

1. General description and creating a project

1.1 What the tool is for

The CAN Tool is a Python tool for testing and analyzing CAN and CAN FD buses. It works with either the **Vector VN1630A / VN7572** or the **PEAK-System PCAN-USB Pro FD** (both via python-can) or - without hardware - with a built-in **simulation**.

Core functions:

- Management of multiple **projects** (each with its own DBC, CDD, sequence and log files).
- **DBC/CDD conversion** to JSON or YAML before actual use (fast loading, easy inspection).
- **Cyclic transmission** and reception of messages, signal access in physical values.
- **Trace** of all transmitted and received frames, optionally with signal decoding.
- **Test sequences** (JSON): set signals, wait for reactions, check, run diagnostics.
- **Line graph** of selected signals.
- Switchable appearance (**dark mode**) and **six languages** (German, English, French, Spanish, Chinese/Mandarin, Thai).
- **Parquet logging** with filter.
- **Diagnostics** via ISO-TP (ISO 15765-2) with UDS services from the CDD.

1.2 Data flow overview

```
dbc/*.dbc  --(conversion)-->  cache/*.dbc.json|yaml  ---+
cdd/*.cdd  --(conversion)-->  cache/*.cdd.json|yaml  ---+--> "Data" tab: TX / RX selection
                                                        |        |
                                                        |        +--> Trace   (all frames)
                                                        |        +--> Graph   (selected signals)
                                                        |        +--> Logging (selected signals, filter)
sequences/*.json  ------------------------------------------+--> Test sequence (sets/checks signals, diagnostics)
```

At runtime only the **cache** (JSON/YAML) is ever read. If a DBC/CDD file changes, it is converted again automatically the next time the project is opened (checksum of the source file).

1.3 Installation and startup

```
pip install -r requirements.txt
python main.py                       # GUI
python main.py --project demo        # GUI with a specific project
python main.py --list                # list projects
python main.py --convert demo        # only convert DBC/CDD (without GUI)
python main.py --convert demo --format yaml --force
```

Drivers: Vector requires the *Vector XL Driver Library* (part of the Vector drivers), PEAK the *PCAN-Basic* driver. Without hardware the simulation is sufficient.

1.4 Folder structure

```
can_tool/
  main.py  gui.py  can_manager.py  dbc_manager.py  cdd_manager.py
  iso_tp_manager.py  sequence_runner.py  parquet_logger.py  project_manager.py
  help_content.py  i18n.py
  lang/               Language files (de.yaml, en.yaml, fr.yaml, es.yaml, zh.yaml, th.yaml)
  tools/check_lang.py Check of the language files
  projects/
    <project name>/
      project.json      Project settings (written by the tool)
      dbc/              Your DBC files (*.dbc)
      cdd/              Your diagnostic descriptions (*.cdd)
      sequences/        Test sequences (*.json)
      cache/            automatically generated: converted DBC/CDD (JSON or YAML)
      logs/             Parquet log files
```

1.5 Creating a project

In the GUI

1. Open the **Project** tab and click **New ...**.
2. Enter a project name. Letters, digits, underscore, hyphen and spaces are allowed (max. 64 characters, first character a letter or digit). If the name already exists, nothing is overwritten.
3. The tool creates `projects/<name>/` with the subfolders `dbc, cdd, sequences, cache, logs` and a `project.json` with default values (simulation, 500 kbit/s, JSON cache) and switches to the new project.
4. **Import DBC/CDD ...**: files are copied to `dbc/` or `cdd/` and converted immediately. Alternatively, place the files in the folders manually and click **Reconvert** (or reopen the project).
5. Configure the hardware (see chapter 2.3) and click **Connect**.

By file

Create a project folder with the structure above and put a `project.json` containing at least `{}` in it - missing settings are filled in with default values. The demo project `projects/demo` serves as a template.

Switching projects

Select from the project list. When switching (and on exit) the connection is closed, a running logging session is stopped and the previous project is saved. Saved are: hardware, transmit list incl. signal values and cycles, receive selection, logging filter, graph selection, diagnostic settings and the most recently loaded sequence. Cyclic transmission is always **switched off** after a project is opened (safety on a real bus).

1.6 Typical workflow

1. Create or select a project, import DBC (and CDD).
2. Configure the hardware, click **Connect**.
3. **Data** tab: select transmit and receive messages from the DBC, set signal values and cycles.
4. Switch on cyclic transmission; observe the result in **Trace** and **Graph**.
5. If needed, start **Logging** and/or create and run a **test sequence**.
6. **Diagnostics**: send services, check responses.

2. Preparing the files before startup

Before you connect, the project files must be prepared. This chapter describes what has to be contained in which file and which information the tool evaluates.

2.1 DBC files (`dbc/*.dbc`)

Standard DBC files from CANdb++, CANdb Editor, SavvyCAN and others. Multiple DBC files per project are possible (their sum forms the database). Character set UTF-8 or Windows-1252.

The following are evaluated:

DBC element	Meaning in the tool
`BO_`	Message: ID (bit 31 set = extended ID), name, length (DLC > 8 = CAN FD), sender.
`SG_`	Signal: start bit, length, byte order (Intel/Motorola), sign, factor, offset, min/max, unit, receivers.
`SG_ ... M / m0, m1`	Simple multiplexing (one multiplexor, several groups). Extended multiplexing is **not** supported (warning).
`VAL_`	Value table: text per raw value. In the tool shown as a dropdown when transmitting and as text when receiving.
`SIG_VALTYPE_`	Floating-point signals (32/64 bit).
`CM_`	Comments on messages/signals.
`BA_ "GenMsgCycleTime"`	Default cycle time (ms) of a message; 100 ms if not specified.
`BA_ "VFrameFormat"`	Frame format; values containing *FD* identify CAN FD messages.

Example:

```
VERSION "1.0"
NS_ :
BS_:
BU_: Tester ECU

BO_ 512 EngineData: 8 ECU
 SG_ RPM : 0|16@1+ (0.25,0) [0|8000] "rpm" Tester
 SG_ CoolantTemp : 16|8@1+ (1,-40) [-40|130] "degC" Tester

VAL_ 257 StartButton 0 "Aus" 1 "Ein" ;
BA_DEF_ BO_ "GenMsgCycleTime" INT 0 65535;
BA_ "GenMsgCycleTime" BO_ 512 10;
```

Notes on data preparation

- Message and signal names are used in test sequences - choose descriptive, unique names.
- Duplicate message names across several DBC files: the first definition is kept, the second is ignored with a warning.
- Same ID with different names: on reception it is assigned to the first name (warning).
- In the simulation, the sender nodes (`BU_`) control which messages the virtual ECU transmits cyclically (default: sender `ECU`, configurable in `sim.traffic_senders`). Sensible min/max values in the DBC give realistic simulation values.
- Conversion warnings are shown in the **Project** tab in the field *Converted files / warnings*.

The cache (`cache/*.dbc.json` or `.yaml`)

When the project is opened, every DBC is written to the cache automatically (format selectable in the **Project** tab). It is only converted again if the source file has changed; **Reconvert** forces it. Your own changes in the cache are overwritten by a new conversion - corrections belong in the DBC.

```
{
  "format_version": 1,
  "nodes": ["Tester", "ECU"],
  "messages": {
    "EngineData": {
      "id": 512, "extended": false, "dlc": 8, "fd": false, "sender": "ECU",
      "cycle_time_ms": 10, "comment": "", "multiplexor": null,
      "signals": {
        "RPM": {"start": 0, "length": 16, "byte_order": "little", "signed": false,
                "factor": 0.25, "offset": 0, "min": 0, "max": 8000, "unit": "rpm",
                "receivers": ["Tester"], "choices": {}, "comment": ""}
      }
    }
  },
  "warnings": [], "source": "engine.dbc", "source_sha1": "..."
}
```

2.2 Diagnostic description (`cdd/*.cdd`)

The CDD describes diagnostic services. Like the DBC, it is converted to `cache/*.cdd.json|yaml`.

Important limitation: The simple XML format `<DIAGDATA>` is fully supported (see the example and `projects/demo/cdd/diag.cdd`). Original CANdelaStudio CDD files are only read **heuristically** (service names and, if found, request bytes). In that case, check the generated cache file (**Project** tab, warnings) and add addresses, requests and response parameters there or in the simple format.

```
<?xml version="1.0" encoding="UTF-8"?>
<DIAGDATA>
  <ECU name="Engine_ECU" request_id="0x7E0" response_id="0x7E8" extended="false" fd="false">
    <SERVICE name="ExtendedSession" request="10 {session}" description="Select diagnostic session">
      <PARAM name="session" bytes="1" default="03"/>
    </SERVICE>
    <SERVICE name="ReadVIN" request="22 F1 90" description="Read VIN">
      <RESPONSE prefix_len="3">
        <PARAM name="VIN" offset="0" length="17" type="ascii"/>
```

```
      </RESPONSE>
    </SERVICE>
    <SERVICE name="ReadSpeed" request="22 F4 0D" description="Read speed">
      <RESPONSE prefix_len="3">
        <PARAM name="Speed" offset="0" length="1" type="uint" factor="1" unit="km/h"/>
      </RESPONSE>
    </SERVICE>
  </ECU>
</DIAGDATA>
```

Element / attribute	Meaning
`ECU name, request_id, response_id`	Name of the ECU and CAN IDs (tester -> ECU, ECU -> tester), e.g. `0x7E0`/`0x7E8`.
`ECU extended, fd`	`true`: 29-bit IDs or CAN FD (ISO-TP with up to 64 bytes per frame).
`SERVICE name`	Name of the service (used in the Diagnostics tab and in the sequence step `diag`).
`SERVICE request`	Request bytes in hex. Placeholders in curly braces, e.g. `10 {session}`.
`SERVICE description`	Free text, shown in the service list.
`PARAM`	Directly in SERVICE: input parameter for a placeholder: `name`, `bytes` (width), `default` (hex).
`RESPONSE prefix_len`	Number of bytes at the start of the response that are skipped (service ID + DID, e.g. 3 for `62 F1 90`).
`RESPONSE/PARAM`	Response parameter: `name`, `offset` and `length` in bytes (from the end of the prefix), `type` = `ascii | uint | int | hex`, optionally `factor`, `offset_val`, `unit`.

The corresponding cache entry (for manual refinement):

```
{
  "format_version": 1,
  "ecus": {
    "Engine_ECU": {
      "request_id": "0x7E0", "response_id": "0x7E8", "extended": false, "fd": false,
      "services": {
        "ReadVIN": {
          "request": "22 F1 90", "description": "Read VIN", "params": [],
          "response": {"prefix_len": 3,
                       "params": [{"name": "VIN", "offset": 0, "length": 17, "type": "ascii"}]}
        }
      }
    }
  }
}
```

2.3 Hardware and connection

Settings in the **Project** tab (stored in `project.json` under `hardware`):

Setting	Vector VN1630A / VN7572	PEAK PCAN-USB Pro FD	Simulation
Interface	`vector` (XL driver)	`pcan` (PCAN-Basic)	`simulation`
Channel	Channel index, 0-based (e.g. `0`). Assignment via the *Vector Hardware Config*; if an app name is set, the channel number assigned there applies.	`PCAN_USBBUS1`, `PCAN_USBBUS2` ...	not required
Bitrate	Nominal bitrate (kbit/s)	Nominal bitrate (kbit/s); with FD fixed 500k/2M (80 MHz clock)	irrelevant
CAN FD / data bitrate	Tick the checkbox and enter the data bitrate	Tick the checkbox; different timing in `hardware.extra`	Checkbox for FD frames
App name	Name from the Vector Hardware Config (empty = python-can default)	-	-

Vector VN7572 (PCIe card): The Vector interface card VN7572 is also used as type `vector` through the Vector XL driver (Windows only), exactly like the VN1630A. With *Vector Driver Setup* installed, the card must be visible in the *Vector Hardware Config*. The channel number is 0-based (0 = CH1, 1 = CH2 ...); **Detect hardware** lists the channels found together with their serial numbers. No other program (CANalyzer, CANoe, vFlash) may occupy the channel.

- All nodes on the bus must use the same bitrate; with CAN FD also the same data bitrate.
- On a bench setup with two devices, the bus ends must be terminated with 120 ohms.
- `hardware.extra` (only in `project.json`) passes arbitrary parameters through to python-can, e.g. `{"nom_brp": 5}` for PEAK.
- **Simulation**: *Echo* returns your own transmit messages as received; the *virtual ECU* answers UDS requests (among others 10, 11, 14, 19, 22, 27, 3E) on the addresses of the diagnostic settings and transmits the messages of the ECU senders cyclically.

2.4 Sequence files (`sequences/*.json`)

A sequence consists of a name, a description and a list of steps. You can create it in the **Test sequence** tab or edit the file directly. Command reference: Appendix A.

```
{
  "name": "startup",
  "description": "Simulate engine start and check run-up",
  "steps": [
    {"type": "enable_tx", "message": "DriverCmd", "enabled": true},
    {"type": "set_signal", "message": "DriverCmd", "signal": "StartButton", "value": 1},
    {"type": "wait_for", "direction": "RX", "message": "EngineData", "signal": "RPM",
     "op": ">", "value": 100, "timeout_ms": 3000},
    {"type": "diag", "service": "ReadVIN", "expect": "62 F1 90"}
  ]
}
```

2.5 The project file (`project.json`)

Written by the tool; can be edited manually if needed (close the tool first). Missing keys are filled in with default values.

```
{
  "hardware": {"type": "vector", "channel": "0", "bitrate": 500000, "fd": true,
               "data_bitrate": 2000000, "app_name": "", "extra": {}},
  "cache_format": "json",
  "tx": [{"message": "DriverCmd", "cycle_ms": 50, "values": {"ThrottleCmd": 0}, "signals": null}],
  "rx": {"EngineData": null, "VehicleSpeed": ["Speed"]},
  "logging": {"selection": {"TX": {}, "RX": {"EngineData": null}},
              "on_change_only": false, "min_interval_ms": 0, "log_raw": false},
  "graph": ["RX:EngineData.RPM"],
  "diag": {"cdd": "diag.cdd.json", "ecu": "Engine_ECU", "request_id": "", "response_id": "",
           "extended": false, "fd": false, "padding": 170},
  "sequence": "startup.json",
  "sim": {"ecu": true, "traffic_senders": ["ECU"]}
}
```

Key	Meaning
`hardware`	Connection parameters (chapter 2.3); bitrates in bit/s.
`cache_format`	`json` or `yaml` for the converted files.
`tx`	Transmit list: message, cycle (ms), signal values, `signals` = selection for graph/logging (`null` = all).
`rx`	Receive selection: message -> list of selected signals or `null` (all).
`logging`	Logging filter and options (Logging tab).
`graph`	Keys of the signals ticked in the graph, e.g. `RX:Msg.Signal`.
`diag`	CDD file, ECU, optional address override (`request_id`, `response_id`), `extended`, `fd`, `padding`.
`sequence`	Most recently loaded sequence (file name in `sequences/`).
`sim`	Simulation: `ecu` (virtual ECU), `traffic_senders` (sender nodes that are simulated cyclically).

2.6 Checklist before the first connection

1. DBC file(s) in the `dbc/` folder; the Project tab shows the expected messages (count) and no unexpected warnings.
2. Optionally a CDD in the `cdd/` folder; the Diagnostics tab shows the ECU and services.
3. Hardware driver installed, device connected, channel and bitrates matching the bus.
4. With CAN FD: tick *CAN FD* and set the data bitrate; the other nodes must also be FD-capable.
5. Sequences present under `sequences/` or created in the Test sequence tab.

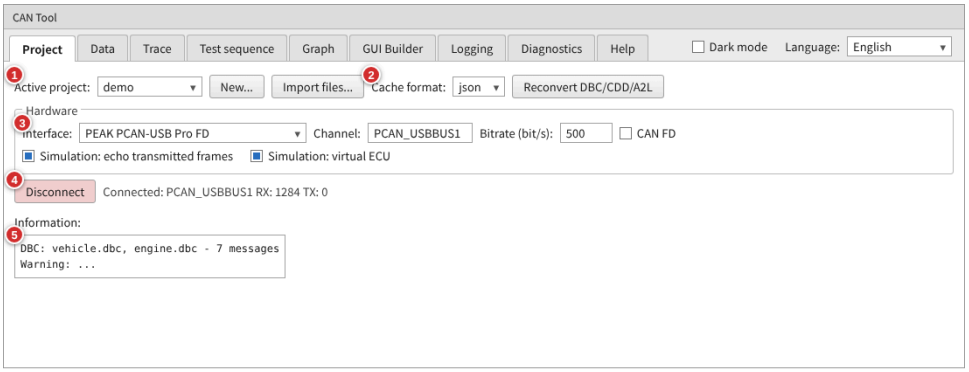
Common problems

Symptom	Possible cause / remedy
No messages in the Data tab	DBC not in the `dbc/` folder or file extension not `.dbc`; read the warnings in the Project tab.
Connect fails	python-can/driver missing, wrong channel, device in use by another application (e.g. CANalyzer/PCAN-View).
Frames visible but not decoded	ID/extended flag in the DBC does not match the bus; message missing in the DBC.
Wrong signal values	Check byte order (Intel/Motorola), start bit, factor/offset in the DBC.
Diagnostics: timeout	Check request/response ID, extended/FD and padding in the Diagnostics tab; is the ECU in the appropriate session?
Logging not available	Python package `pyarrow` is missing (`pip install pyarrow`).

3.1 "Project" tab

Operating system: At startup the tool detects on its own whether it runs on **Windows** or **Linux** (shown in the title bar). The hardware selection adapts: *Vector* (XL driver) on Windows only, *SocketCAN* on Linux only, *PEAK* and *Simulation* on both. Unusable entries are marked and disabled. The **Detect hardware** button lists connected interfaces. All other functions (trace, sequences, logging, diagnostics, vFlash, help/PDF) are identical on both systems; project folders can be exchanged between Windows and Linux.

Annotated screenshot



Schematic view of the user interface; the numbers refer to the explanations below.

1. Project selection, creating a new project and importing DBC/CDD files.
2. Cache format (json/yaml) and re-conversion of DBC/CDD.
3. Hardware: interface, channel (e.g. PCAN_USBBUS1), bit rate and CAN FD.
4. Connect or disconnect, with status display (receive and transmit counters).
5. Information field: loaded DBC files, number of messages and warnings.

Re-init HW: Next to *Connect*, this button closes the interface and reopens it with the same settings. It helps when the connected hardware hangs. The transmit list is kept (cyclically active messages continue afterwards); running logging is stopped.

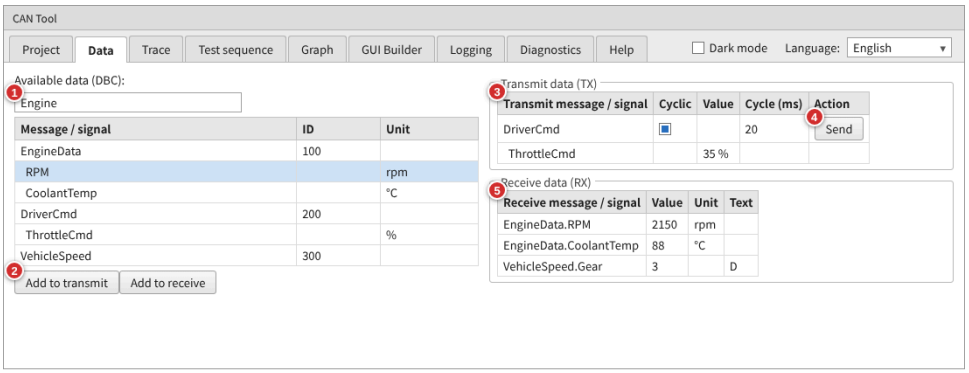
Manage projects, include DBC/CDD, configure the hardware and establish the connection.

Control	Function
Active project	Selection list of all projects (`projects/*/project.json`). Switching closes the connection, stops logging, saves the old project and loads the new one.
New ...	Creates a project with folder structure and default settings (chapter 1.5).
Import DBC/CDD ...	Copies selected `.dbc`/`.cdd` files into the project and converts them.
Cache format	`json` or `yaml`; changing it triggers a new conversion.
Reconvert	Forces conversion of all DBC/CDD and reloads the project.
Interface / channel / bitrate / CAN FD / data bitrate / app name	Hardware parameters (chapter 2.3). Locked while connected.
Simulation: echo	Your own transmit messages additionally appear as received.
Simulation: virtual ECU	On connect, starts a UDS-capable software ECU with test traffic.
Connect / disconnect	Opens or closes the interface. Disconnecting stops a running logging session.
Status line	Connection state, number of received (RX) and transmitted (TX) frames, last error.
Converted files / warnings	List of cache files, number of messages, warnings from DBC/CDD.

3.2 "Data" tab

This is where you define **which** messages are transmitted and received and **which signals** apply to the graph, logging and display - in each case from the sum of all DBC data of the project.

Annotated screenshot



Schematic view of the user interface; the numbers refer to the explanations below.

1. Filter and list of all messages and signals from the DBC.
2. Add the selection to the transmit data (TX) or receive data (RX).
3. Transmit data: cyclic on/off, set signal values and cycle time.
4. Single transmission: sends the message once, immediately.
5. Receive data with the current live values of the signals.

Left side: available data

- Tree of all messages with their signals (ID, sender, unit). The **filter field** searches in message name, ID and signal name.
- **-> Transmit (TX)** adds the selected messages to the transmit list (double-clicking a message does the same). Multiple selection is possible; if a signal is selected, its message is added.
- **-> Receive (RX)** adds the selected messages to the receive list.

Top right: transmit

Column	Meaning
Transmit message / signal	Message (with ID) and below it all its signals.
Cyclic	Checkbox on the message switches cyclic transmission on/off (frames are only actually sent while connected).
Value	Physical signal value (input field or selection list for signals with a value table). Changes take effect from the next frame.
Cycle [ms]	Cycle time of the message; preset from the DBC (`GenMsgCycleTime`), otherwise 100 ms. 0 = not cyclic.
Selection	Checkbox on the signal: the signal belongs to the selection for graph and logging.
Send (button)	Sends the message exactly once with the current values.

Remove deletes the message of the selected row from the transmit list.

Bottom right: receive

Column	Meaning
Receive message / signal	Message (with ID) and its signals.
Value / unit / text	Most recently received value (updated every 200 ms), unit and - if present - the value table text.
Selection	Checkbox on the signal: the signal belongs to the selection for graph and logging. Signals that are not ticked are not stored.

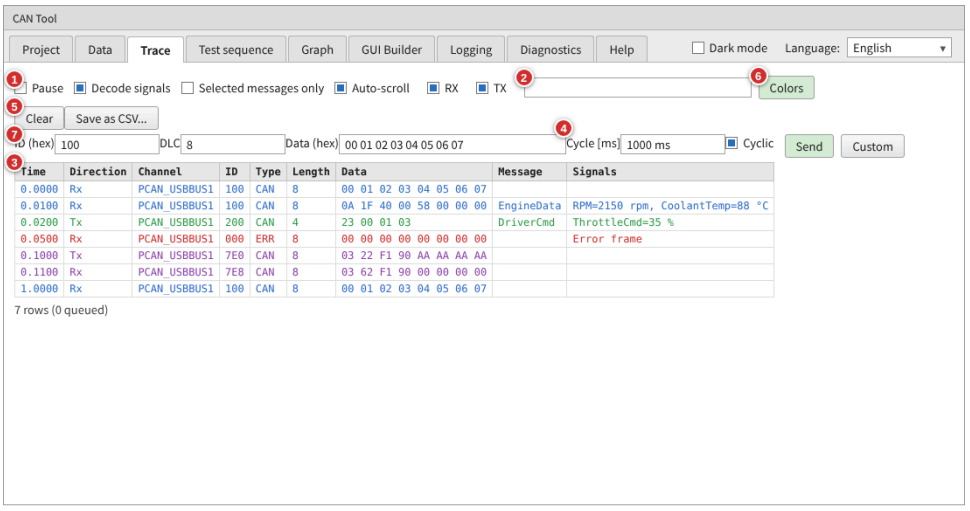
Notes

- The selection (checkboxes) determines what **Graph** and **Logging** see. The **Trace**, by contrast, shows all frames (optionally only the selected messages).
- The selection and all transmit values are saved with the project; cyclic transmission is off after loading.
- A message can be in the TX and RX lists at the same time (e.g. with echo in the simulation).

3.3 "Trace" tab

Chronological sequence of all transmitted (Tx, highlighted in color) and received (Rx) frames.

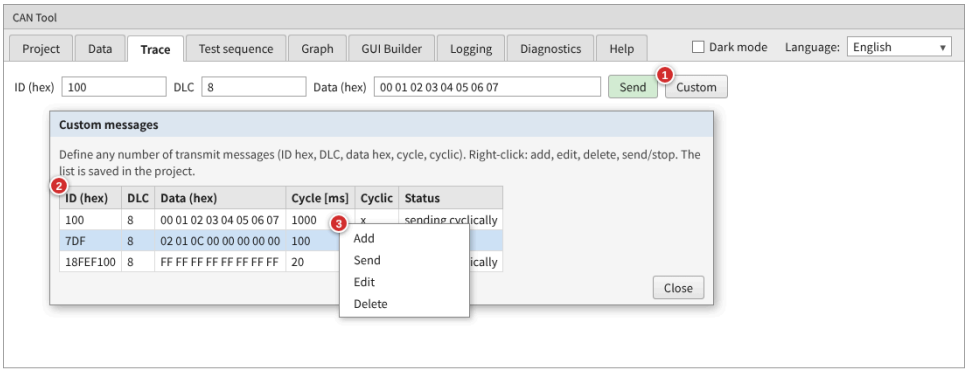
Annotated screenshot



Schematic view of the user interface; the numbers refer to the explanations below.

1. Options: pause, decode signals, selected messages only, autoscroll.
2. Filter by ID or message name.
3. Trace table: time, direction (Rx/Tx), channel, ID, type, length and raw data. Works without DBC/CDD, too.
4. Decoded message names and signal values (if a DBC is loaded).
5. Clear the trace or save it as a CSV file.
6. Color mode: Rx blue, Tx green, errors red, ISO-TP violet.
7. Send row: ID (hex), DLC, data, cycle time, checkbox for cyclic sending and Send button.

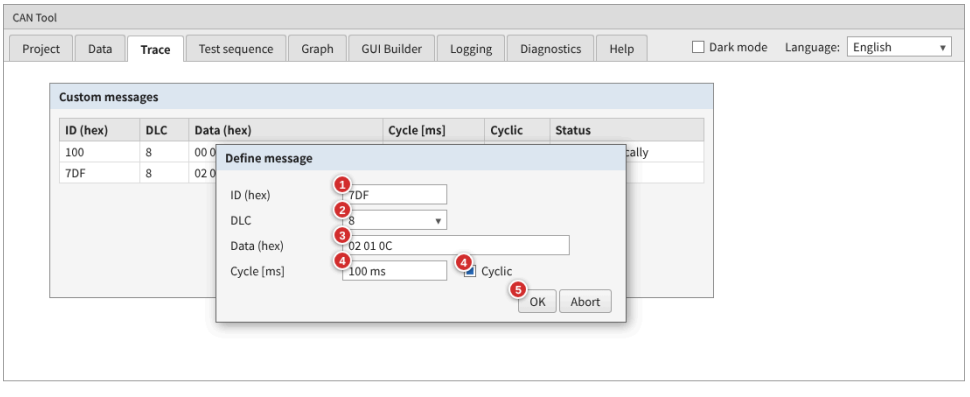
Annotated screenshot



Schematic view of the user interface; the numbers refer to the explanations below.

1. The Custom button opens the window for your own transmit messages.
2. Table of transmit messages: ID (hex), DLC, data (hex), cycle [ms], cyclic and status.
3. Right-click menu: add, send/stop, edit, delete.

Annotated screenshot



Schematic view of the user interface; the numbers refer to the explanations below.

1. ID (hex): CAN ID, also extended (> 7FF).

2. DLC: data length, also CAN FD values.
3. Data (hex): bytes with or without spaces; missing bytes are padded with 00.
4. Cycle [ms] and Cyclic check: send once or cyclically.
5. OK adds the message to the list, Cancel discards the input.

Column	Content
Time [s]	Time relative to the first displayed frame (since connecting or *Clear*).
Rx/Tx, channel	Direction and interface.
ID, type, len	CAN ID (hex, 8 digits for extended), type (CAN, FD, FD/BRS) and data length.
Data	Payload in hex.
Message, signals	Name from the DBC and decoded signals with unit (when *Decode* is active).
Color mode	Toggle button. On: Rx messages medium blue, Tx messages medium green, faulty messages (bus error frames, type ERR) red and ISO-TP messages (diagnostic request/response ID, also 0x7DF) medium violet. Off: display as before.

Control	Function
Pause	Freezes the display. Frames continue to be buffered (up to 50 000) and are delivered after resuming.
Decode	Switches signal decoding on/off (off = faster under high bus load).
Selected messages only	Shows only messages that are in the TX or RX list in the Data tab.
Auto-scroll	Automatically follows the newest entry.
Rx / Tx	Show/hide directions.
Filter field	Comma-separated IDs in hex (e.g. `100, 200`) and/or part of a message name (e.g. `engine`).
Clear	Deletes the display and the buffer.
Save as CSV ...	Exports the currently visible rows (separator `;`).

Update in place: With the *Update in place* box checked, each message (direction, channel, ID) appears only once in the table, in order of first occurrence. When it occurs again, the timestamp, data and decoded signals of that row are updated and the *Count* column is increased. Unchecked, all frames are written one after another, scrolling. Switching clears the table; autoscroll only applies in scrolling mode.

Diff time: In addition to *Update in place*, the *Diff time* box makes the time column show the interval since the previous occurrence of the same message (direction, channel, ID), i.e. the cycle time, instead of the absolute time; 0 on first occurrence. The column header changes to *Diff time [s]*. Switching clears the table.

Send row: Below the option bar, a row sends raw frames (default: ID 100 hex, DLC 8, data 00 01 02 03 04 05 06 07, 1000 ms). **Send** transmits the frame once; with **Cyclic** checked it starts cyclic transmission and another click (**Stop**) ends it. IDs above 7FF are sent as extended frames, DLC above 8 as CAN FD (FD must be enabled in the connection settings). The frame appears in the trace as Tx.

Custom messages (multi send)

The send row sends exactly one message. For **several messages** use the **Custom** button: it opens a separate window that stays open next to the trace. Any number of transmit messages can be defined there and sent individually. The list is saved in the project (`custom_tx`) and is available again the next time the project is opened.

Define and send a message

1. Click the **Custom** button in the send row: the window "Custom messages" opens.
2. **Right-click** in the table and choose **Add**.
3. In the dialog "Define message" enter ID (hex), DLC, data (hex), cycle [ms] and tick **Cyclic** if needed; confirm with **OK**. Invalid input (e.g. an illegal DLC or too many data bytes) is rejected with a message and nothing is saved.
4. Right-click the row and choose **Send**: without the Cyclic check the message is sent once, with it cyclic sending starts (status: "sending cyclically"). The same menu item then reads **Stop**.

Field	Meaning
ID (hex)	CAN ID in hex without prefix (e.g. `7DF`). IDs above 7FF are sent automatically as extended IDs (29 bit).
DLC	Data length: 0 to 8 and 12, 16, 20, 24, 32, 48, 64. Values above 8 are sent as CAN FD (hardware and project must support CAN FD).
Data (hex)	Hex bytes with or without spaces. Too short input is padded with 00 up to the DLC, too long input is an error.
Cycle [ms]	Send interval from 1 to 3 600 000 ms; only effective with Cyclic.
Cyclic	Unchecked: sent once; checked: sent cyclically until stopped.

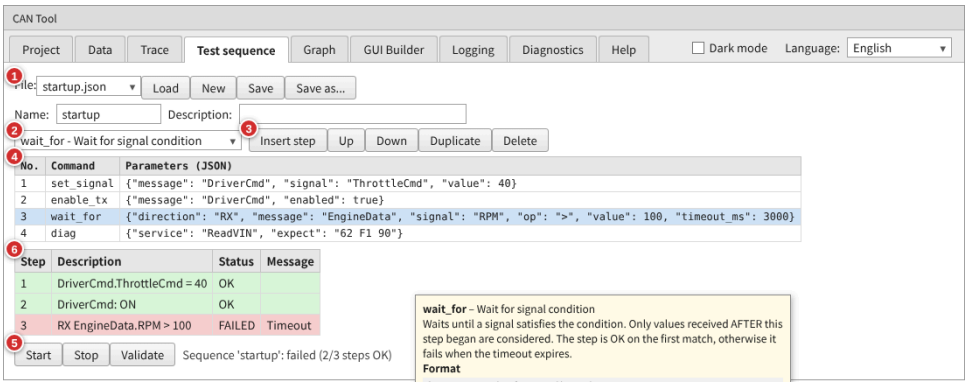
Right-click menu: Add (also on empty space), Send or Stop, Edit (also by double-click) and Delete. A running entry is stopped before it is edited or deleted. The connection must be established for sending; when it is closed, cyclic sending of all entries stops. Several entries with the same ID share one transmit job. Sent messages appear in the trace as Tx.

Notes: At most 5 000 rows are displayed (older ones scroll out). The display is only updated while the tab is visible; in the background the tool buffers up to 50 000 frames. For gap-free recording use **Logging**.

3.4 "Test sequence" tab

Create, save, load and run test sequences. There is always **exactly one sequence** loaded/active; multiple sequences exist as files in the project's `sequences/` folder.

Annotated screenshot



Schematic view of the user interface; the numbers refer to the explanations below.

1. Choose, load, create and save a test sequence file (one sequence at a time).
2. Command dropdown: hovering over an entry shows the tooltip with format, elements and example.
3. Insert, move, duplicate or delete a step.
4. Step list: command and parameters as JSON, editable directly in the table.
5. Start, stop and statically check the sequence; the overall result is shown next to them.
6. Result table per step: green = passed, red = failed, with message.

File

Control	Function
File list + Load	Selects a sequence from `sequences/` and loads it into the editor.
New	Empty sequence (not saved until *Save*).
Save / Save as ...	Writes the sequence as JSON. The most recently saved/loaded file name is recorded in the project and loaded again the next time it is opened.
Name / description	Metadata of the sequence.

Editing steps

The steps are in a table (No., type, parameters as JSON). The **parameter text is directly editable** (double-click). Invalid JSON is rejected and the old value is restored; the type of a step is fixed.

Control	Function
Command dropdown	Selects the step type. **Hovering over an entry with the mouse shows a tooltip** with the format, a description of all elements and an example; below it the summary of the selected command is also shown.
Insert step	Inserts the command with a template after the selected row (or at the end).
Up / Down / Duplicate / Delete	Move, copy or remove the selected step.
Signal selection + Apply to selected step	Lists all signals of the DBC as `Message.Signal` (with type-ahead completion) and enters the message (and, for `set_signal`, `wait_for`, `check`, also the signal) into the selected step.
Tooltip in the table	The command tooltip also appears over the type column of a step.

Validating and running

Control	Function
Validate	Static check: unknown step types, operators, messages and signals (against the DBC).
Start	Runs the sequence (only while connected). It is validated automatically before starting. The editor is locked during the run.
Stop	Aborts the sequence (also a running `wait`); status *aborted*.
Result table	Per step: number (loops: e.g. `3.2.1`), description, status (green = OK, red = fail/error) and message (e.g. measured value, response bytes).

Execution rules: The steps run one after another. The **first failed step ends the sequence** (status *failed*). Afterwards the status line shows e.g. *startup: passed (10/10 steps OK)*. Messages started cyclically keep running after the end, unless stopped via `enable_tx`.

Commands at a glance

Command	Title	Summary

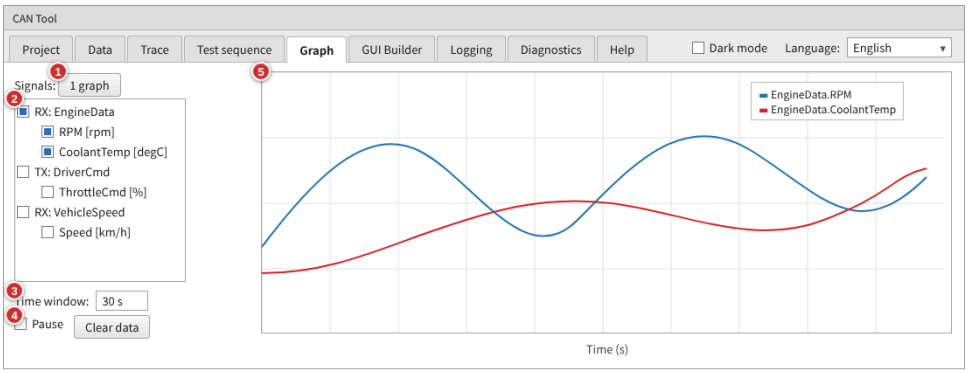
`set_signal`	Set signal value	Sets the physical value of a signal in a transmit message. The new value goes onto the bus with the next (cyclic or single) frame.
`enable_tx`	Switch cyclic transmit on/off	Starts or stops the cyclic transmission of a message at its configured cycle time.
`set_cycle`	Change cycle time	Changes the cycle time of a transmit message (takes effect immediately, even while it is transmitting cyclically).
`send_once`	Send message once	Sends exactly one frame with the current signal values, independent of cyclic transmission.
`send_raw`	Send raw frame	Sends a raw frame (ID + data) on the CAN bus, once or cyclically. With cyclic=true and enabled=false the cyclic transmission of this ID is stopped.
`wait`	Wait for a fixed time	Pauses the sequence for a fixed time. Cyclic transmission and reception continue.
`wait_for`	Wait for signal condition	Waits until a signal satisfies the condition. Only values received AFTER this step began are considered. The step is OK on the first match, otherwise it fails when the timeout expires.
`check`	Check signal value	Immediately checks the most recently received value of a signal (no waiting). The step fails if the condition is not met or if no value has been received since the sequence started.
`diag`	Diagnostic service from the CDD	Sends a diagnostic service defined in the CDD via ISO-TP and evaluates the response.
`diag_raw`	Diagnostic request in hex	Sends any UDS request (hex bytes) via ISO-TP and evaluates the response.
`loop`	Loop	Executes the contained steps several times in succession.
`comment`	Comment	Does nothing; serves only to document the sequence.

The complete description of each command with format, elements and example is in Appendix A.

3.5 "Graph" tab

This tab shows signal curves live as a line chart (time axis: seconds relative to now, left = past, right = now). The switch above the selection toggles between **1 graph** and **n graphs**.

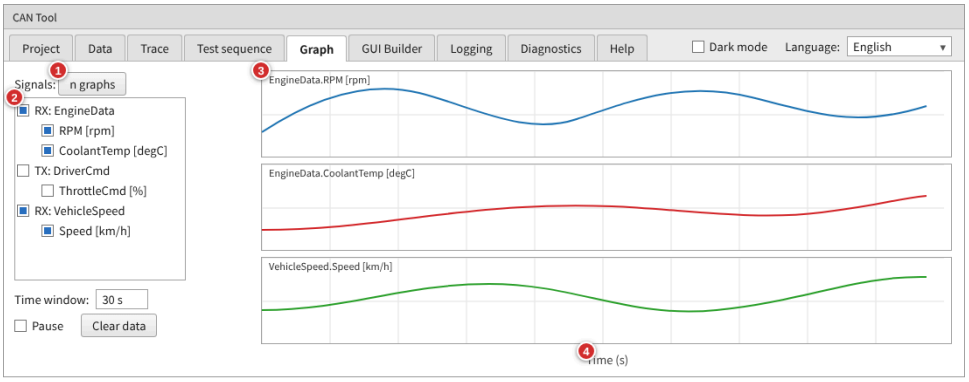
Annotated screenshot



Schematic view of the user interface; the numbers refer to the explanations below.

1. Switch 1 graph / n graphs (here: 1 graph).
2. Signal selection grouped by message; the check box on a message selects all its signals, unit in square brackets.
3. Time window in seconds.
4. Pause freezes the display, Clear data discards the curves.
5. Common chart with legend; time axis in seconds relative to now.

Annotated screenshot



Schematic view of the user interface; the numbers refer to the explanations below.

1. The switch is set to n graphs.
2. The selection stays grouped by message; here three signals are selected.
3. Each signal has its own graph with its own value axis, labelled Message.Signal and unit.
4. Shared time axis: zooming and panning affects all graphs; labels only at the bottom.

Selection grouped by message

The selection lists all signals chosen in the Data tab, grouped by direction and message (`RX` = received, `TX` = sent). Below each message its signals are shown with their unit in square brackets. The check box on the message has three states: all signals selected, partly selected (partial state) or none. Clicking it selects all signals of the message or removes them again.

Display "1 graph"

All selected signals are drawn in one common chart with a legend (colour assignment). Advantage: direct comparison of the curves. With very different value ranges small signals look flat - then "n graphs" is the better choice.

Display "n graphs"

Each selected signal gets its own chart with its own value axis, labelled Message.Signal with unit; the charts are stacked. All charts share the same **time base**: zooming and panning the time axis in one chart affects all of them, and the time labels appear only on the bottom chart.

Controls

Control	Function
Switch 1 graph / n graphs	Changes the display mode; the view is rebuilt immediately. The setting is saved in the project (`graph_multi`).
Signal selection	Tick = show curve. The check box on a message selects or removes all of its signals. The selection is saved in the project (`graph`).
Time window	Visible period in seconds (1 to 3600).

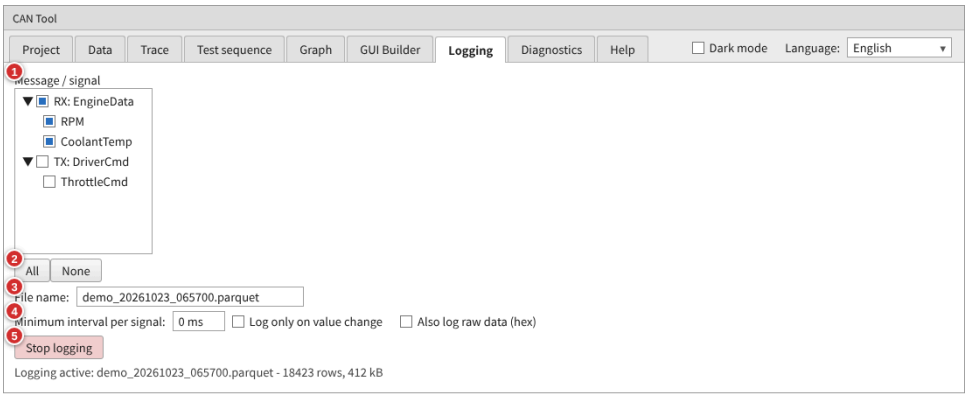
Pause	Freezes the display; data keeps being collected in the background.
Clear data	Discards all collected signal curves.
Chart	Mouse: zoom with the wheel, pan with a pressed button. In "n graphs" this applies to all charts together.

Notes: Up to 50 000 points are kept per signal. Only signals that are ticked in the Data tab are recorded. Signals only appear once data has been received/sent. The display only updates while the tab is visible.

3.6 "Logging" tab

Resolved signals: When a DBC, A2L or CDD is loaded, the Parquet file contains not only the message as hex but the signals with **raw value, raw hex, physical value, text (value table) and unit**. Columns of the signal file: `timestamp, direction, message, signal, value, source, can_id, raw, raw_hex, text, unit, message_hex`. `source` is `CAN` (DBC messages), `XCP` (A2L values read in the XCP tab) or `UDS` (decoded response parameters of the Diagnostics tab from the CDD) while logging is running. The raw file (`_raw.parquet`) additionally has the column `signals` with all signals of the frame as text, e.g. `RPM=0x1F40/2000 rpm`.

Annotated screenshot



Schematic view of the user interface; the numbers refer to the explanations below.

1. Selection of the messages and signals to log (filter from the set of chosen data).
2. Select all or no signals.
3. Name of the Parquet file (stored in the project's logs/ folder).
4. Options: only on change, minimum interval per signal, additionally raw data.
5. Start or stop logging; the status line shows row count and file size.

Records selected signals in **Parquet** format (requires `pyarrow`). Files are located in the project's `logs/` folder.

Control	Function
Data tree	Shows the messages/signals selected in the Data tab. **Filter:** only ticked signals are recorded (ticking the message sets/clears all its signals). The selection is saved in the project.
All / None	Select or deselect all signals.
File name	Preset with *Project_Date_Time.parquet*; the extension is added if needed.
Minimum interval per signal	At most one entry per signal per time period (ms) - reduces the data volume for fast messages. 0 = every frame.
Only on value change	A signal is only written if its value differs from the last written one.
Raw frames	Additionally writes *<name>_raw.parquet* with **all** frames on the bus (independent of DBC and signal selection, including error frames). Columns: time, direction, channel, ID, extended, FD, BRS, error, DLC, data (binary), data (hex text), message name. Logging is therefore possible without a signal selection. **Raw IDs** limit the frames, e.g. `100, 7E0-7EF` (hex, empty = all).
Start / stop logging	Starts or ends the recording (only while connected). On disconnect or project switch it is stopped automatically and the file is closed cleanly.
Status line	File name, rows written, file size.

File format

Column	Type	Content
timestamp	float64	Timestamp (epoch seconds)
direction	string	`RX` or `TX`
message	string	Message name
signal	string	Signal name
value	float64	Physical value

The long format (one row per signal value) avoids gaps with different cycle times. Data is buffered and written to disk every 5 seconds or at 20 000 rows (zstd compression).

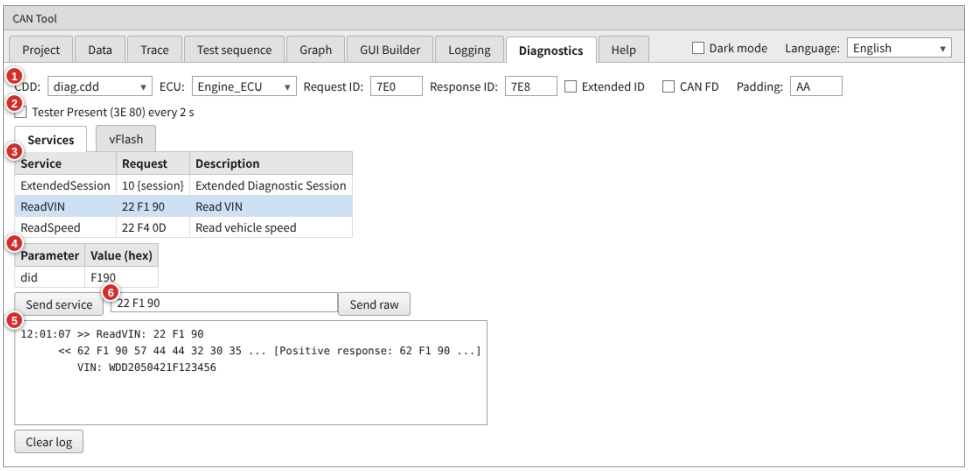
Evaluation in Python (see also Appendix B):

```
import pandas as pd
df = pd.read_parquet("projects/demo/logs/demo_20261020_071500.parquet")
wide = df.pivot_table(index="timestamp", columns=["direction", "message", "signal"], values="value")
```

3.7 "Diagnostics" tab

Diagnostic services (UDS) via **ISO-TP** (ISO 15765-2) on CAN or CAN FD. The services come from the converted CDD; in addition, any requests can be sent in hex.

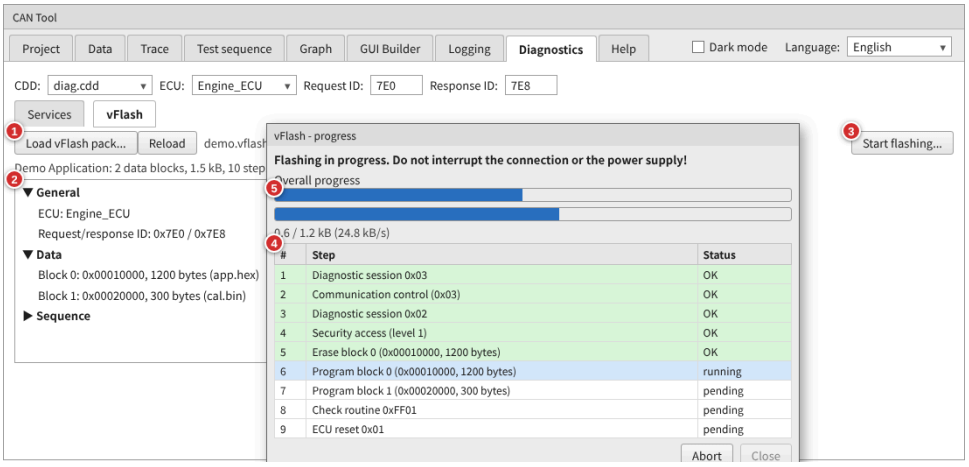
Annotated screenshot



Schematic view of the user interface; the numbers refer to the explanations below.

1. Diagnostic configuration: CDD, ECU, request/response ID, extended ID, CAN FD, padding.
2. Tester Present (3E 80) is sent automatically every 2 s.
3. Services from the CDD with request template and description.
4. Parameters of the selected service (hex values editable), then “Send service”.
5. Response log: sent requests, responses (positive/negative) and decoded values.
6. Send a raw request in hex (e.g. 22 F1 90), no CDD definition needed.

Annotated screenshot



Schematic view of the user interface; the numbers refer to the explanations below.

1. Load a vFlash pack (.vflashpack/.zip, manifest, HEX/S19/BIN); "Reload" reads the file again.
2. Interpretation of the pack: General (ECU, IDs, seed&key), Data (blocks with address and length), Sequence (steps) and Notes.
3. Start flashing: after a safety prompt the sequence runs on the selected hardware.
4. Progress window: status per step (pending, running, OK, FAILED, skipped) with details.
5. Overall progress, progress of the current data block with data rate, and log; "Abort" stops between two blocks.

Configuration (top row)

Field	Meaning
CDD	Converted diagnostic description from the project cache.
ECU	ECU from the CDD; changing it adopts its addresses.
Request ID / response ID	CAN IDs in hex (tester -> ECU / ECU -> tester). Preset from the CDD, can be overridden here.
Extended ID	29-bit addressing.
CAN FD	ISO-TP with FD frames (up to 64 bytes per frame, with bitrate switching). Must match the other node.
Padding	Fill byte in hex (default AA); empty = no padding with classic CAN.
Tester Present	Sends `3E 80` every 2 s (without response) to keep a diagnostic session open.
vFlash tab	Loads a vFlash pack (or manifest/HEX/S19/BIN), shows the interpretation and runs the flash sequence on the selected hardware, with a progress window (Appendix C).

Sending services

1. Select a service in the list (name, request, description).
2. If the service has parameters, enter the value in the *Parameter / Value (hex)* table (placeholders in the request).
3. **Send service**. Sending runs in the background; the interface remains usable.
4. The **log** shows the request (>>), the response (<<) with evaluation (positive/negative, for negative responses the NRC in plain text) and the decoded response parameters of the CDD (e.g. *VIN: WDD...*).

Send raw: enter the request as hex bytes (e.g. `22 F1 86`) and press Enter or use the button.

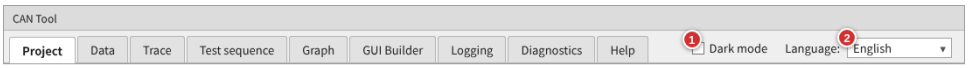
Technical details

- Segmentation: single frame, first frame, consecutive frames and flow control, up to 4095 bytes (longer via the extended first frame).
- When receiving longer responses, the tool sends flow control with block size 0 and STmin 0.
- Response timeout 2 s; with *ResponsePending* (NRC 0x78) it keeps waiting for up to 10 s.
- Requests without a response (sub-function with bit 7, e.g. `3E 80`) are logged as sent.
- In test sequences, `diag` and `diag_raw` use the same address configuration.

3.8 Tab bar: dark mode and language

At the right edge of the tab bar (next to the tabs) there are two controls that apply to all tabs.

Annotated screenshot



Schematic view of the user interface; the numbers refer to the explanations below.

1. Dark mode: switches between light and dark appearance (remembered).
2. Language: dropdown listing all available language files; switching takes effect immediately, including the help.

Control	Function
Dark mode	Checkbox switches between light and dark appearance. This affects the interface, trace, graph, the sequence result table and this help. The setting is saved and used at the next startup.
Language	Dropdown menu with Deutsch (primary language), English, Francais, Espanol, Chinese (Mandarin, simplified) and Thai. Switching takes effect immediately without a restart. The selection is saved.

What is translated?

- All labels, buttons, table headers, tooltips, dialogs and messages of the interface.
- The test sequence command reference (tooltips in the dropdown, Appendix A) and all chapters of this help.
- Error and warning messages of the program components (e.g. conversion, connection, sequence run).
- Data from your files is **not** translated: message and signal names, units and value tables from DBC/CDD, file names, project names and the content of sequence files.
- Log lines already written (diagnostic log, result table of a sequence) keep the language in which they were created.

For Chinese and Thai, a font containing the corresponding characters must be present on the computer (the case with common Windows, macOS and Linux installations).

Language files

Each language is in its own file in the `lang/` folder (YAML; JSON is also possible):

File	Language
`lang/de.yaml`	Deutsch - primary language and fallback: if a text is missing in a language, the German one is used.
`lang/en.yaml`	English
`lang/fr.yaml`	Francais
`lang/es.yaml`	Espanol
`lang/zh.yaml`	Chinese (Mandarin, simplified characters)
`lang/th.yaml`	Thai

Structure of each file: sections `meta` (display name, order in the dropdown), `ui` (interface), `core` (messages), `steps` (command reference) and `help` (chapters of this help). Placeholders in curly braces (e.g. `{name}`) and placeholders in double square brackets in the help texts must not be changed.

Adding or changing a language

1. Copy `lang/de.yaml` and save it as `lang/<code>.yaml` (e.g. `it.yaml`).
2. In the `meta` section set the display name (`name`) and the position (`order`); translate all texts.
3. Check with `python tools/check_lang.py it`: the tool reports missing keys, deviating placeholders and damaged HTML structure.
4. Restart the program - the language appears automatically in the dropdown.

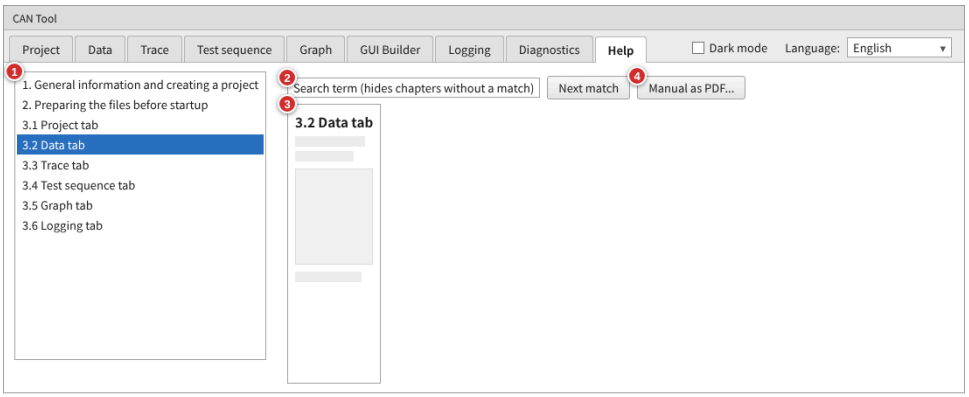
Command line

The output of `main.py` is translated too: `python main.py --lang en --list`. Without `--lang`, the language last selected in the GUI is used.

3.9 "Help" tab

This tab. The chapters are on the left, the text on the right.

Annotated screenshot



Schematic view of the user interface; the numbers refer to the explanations below.

1. List of help chapters.
2. Search box: hides chapters without a match; “Next match” jumps on.
3. Chapter text with annotated figures.
4. Manual as PDF: exports the help in all available languages (with cover page and issue date) and offers it for saving.

- **Search field**: hides chapters in which the term does not occur and highlights the first match in the selected chapter. **Next match** jumps to the next occurrence. An empty field shows all chapters again.
- References in the text (e.g. to the appendix) can be clicked.
- The command descriptions correspond to the tooltips in the command dropdown of the Test sequence tab.
- The help appears in the selected language (chapter 3.8) and is replaced immediately when the language is switched; the light/dark switch applies here too.

3.10 "XCP" tab

Measure and adjust ECU variables via **XCP on CAN** based on an **A2L file** (ASAP2). The A2L is converted like DBC and CDD: source files live in the project folder `a2l/`; at runtime only the cache `cache/<name>.a2l.json|yaml` is read.

Setup

1. Copy the A2L into the project with **Import** (Project tab); it is converted automatically (`python main.py --convert demo` also works without the GUI).
2. Connect CAN (Project tab). XCP uses the same interface.
3. XCP tab: choose the A2L and check the **master ID** (commands to the ECU) and **slave ID** (responses). They are pre-filled from the XCP part of the A2L (`IF_DATA XCP / XCP_ON_CAN`) and can be changed.
4. **Connect XCP**. The log shows all commands (->) and responses (<-) as hex.

Control	Function
List on the left	All measurements (MEASUREMENT) and parameters (CHARACTERISTIC of type VALUE) of the A2L; filter by name; double-click or *Add to watch* takes them over.
Watch	Shows raw value and physical value with unit for each entry. The selection is saved with the project.
Read cyclically	Reads all watched entries at the set interval (ms). Polling uses SHORT_UPLOAD; DAQ lists are not used.
Read once	Reads the watch list once.
Write value	Writes the entered value (physical; for text conversions also the text, e.g. *RUN*) to the selected row (SET_MTA + DOWNLOAD) and reads it back.

Interpretation of the A2L

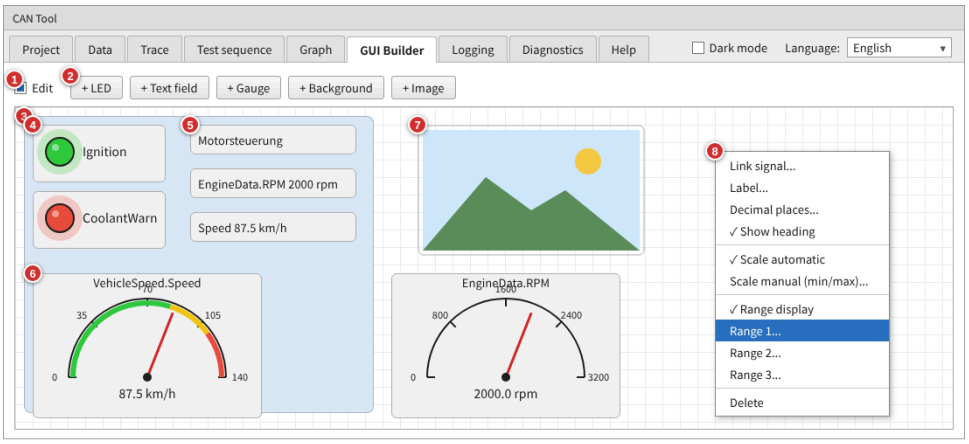
- Data types UBYTE, SBYTE, UWORD, SWORD, ULONG, SLONG, A_UINT64, A_INT64, FLOAT32_IEEE, FLOAT64_IEEE; byte order according to the ECU's CONNECT response.
- Conversion IDENTICAL, LINEAR, RAT_FUNC (without quadratic terms) and TAB_VERB (text values); BIT_MASK is masked out when reading.
- Unsupported objects (curves, maps, arrays, measurements without an address) appear greyed out.

With *Simulation* hardware the tool starts a simulated XCP slave that mirrors the project's A2L (demo: `projects/demo/a2l/demo.a2l`). Not included: DAQ/STIM, seed&key, calibration pages and flash programming. Whether a real ECU permits all accesses depends on its protection levels.

3.11 "GUI Builder" tab

Build your own display from simple elements for existing signals. The elements are **view only**; there is no value input. You can link the signals selected in the Data tab (received or sent). The layout and links are saved in the project (`builder`).

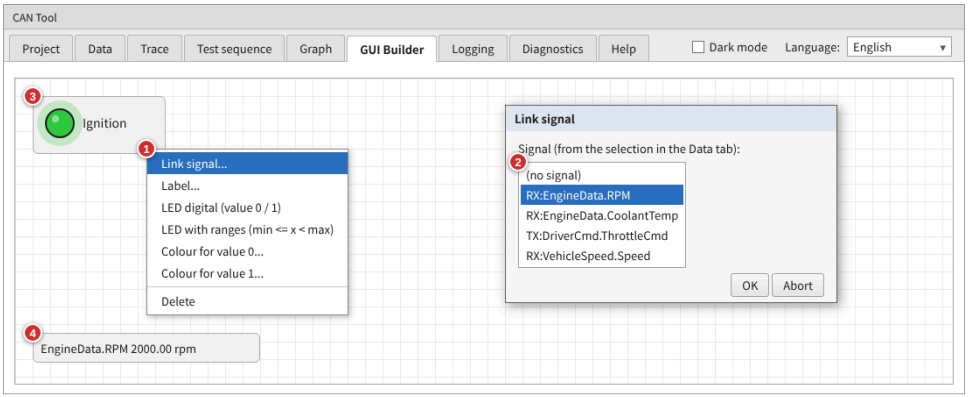
Annotated screenshot



Schematic view of the user interface; the numbers refer to the explanations below.

1. Edit check box: grid, adding, moving and resizing are possible.
2. Buttons to add LED, text field, gauge, background and image.
3. Background element with selectable colour: groups elements visually and lies behind them.
4. LEDs: digital (value 0 / 1) and with ranges (colour depends on range min <= x < max).
5. Text fields: left without signal (plain text), right with signal (value and unit live).
6. Gauge with 180 degree scale, automatic or manual scaling, optionally with range display.
7. Picture frame: the chosen image is scaled automatically to the frame.
8. Right-click menu of the gauge: signal, heading, scale, range display and delete.

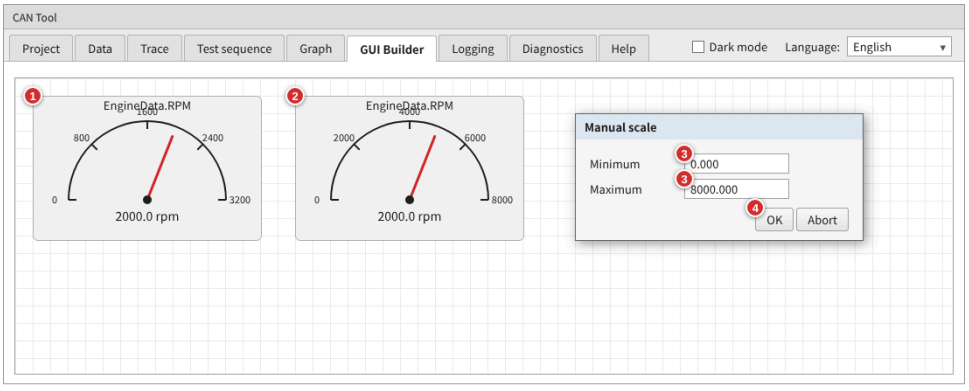
Annotated screenshot



Schematic view of the user interface; the numbers refer to the explanations below.

1. Right-click the LED: link signal, label, digital/ranges mode, colours, delete.
2. Selection: signals from the Data tab as RX:/TX:Message.Signal, or (no signal).
3. LED: colour depends on the signal value (digital 0 / 1 or by range 1 to 3).
4. Text field with signal: text, value and unit; decimal places adjustable.

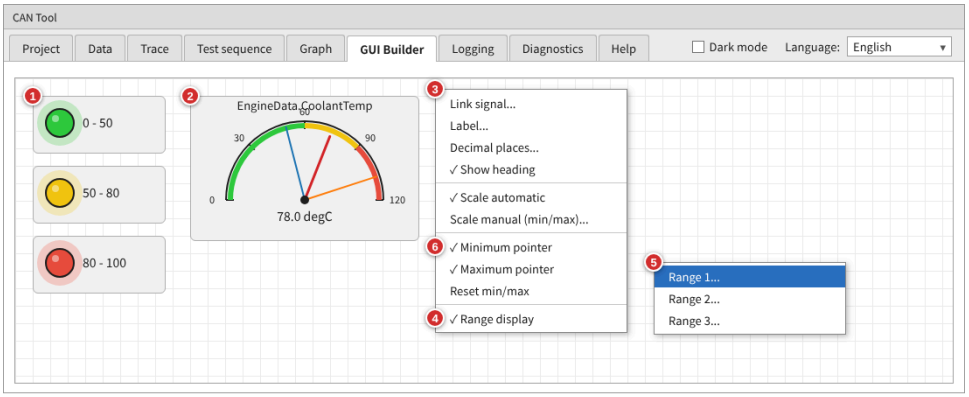
Annotated screenshot



Schematic view of the user interface; the numbers refer to the explanations below.

1. Gauge with automatic scale: limits on round numbers, here 0 to 3200.
2. Gauge with manual scale, here 0 to 8000.
3. Manual scale dialog: enter minimum and maximum.
4. OK applies the scale (only if minimum is smaller than maximum), Cancel discards.

Annotated screenshot



Schematic view of the user interface; the numbers refer to the explanations below.

1. Three LEDs with ranges: each value range (min <= x < max) decides the colour.
2. Gauge with range display: three coloured ranges on the scale (range 1 to 3).
3. Menu: Show heading switches the label on or off.
4. Menu: switch the range display on or off.
5. Range 1 to 3: each with minimum, maximum (rule minimum <= x < maximum), colour and on/off.
6. Menu: optional second and third pointer for minimum (blue) and maximum (orange), and reset.

Elements

Element	Function
LED	Shows the state of a signal as a colour. **Digital**: value 0 and value 1 each have a selectable colour. **Ranges**: as with the gauge scale, three ranges (min <= x < max) with one colour each; if the value is in no range or missing, the LED is grey.
Text field without signal	Plain text field with freely chosen text, e.g. as a heading or label.
Text field with signal	Shows text and the current signal value with unit; updated continuously. The decimal places are adjustable.
Gauge	Pointer instrument with a 180 degree scale (half circle): minimum on the left, maximum on the right; the numeric value with unit is shown below.
Background	Simple coloured rectangle (selectable colour) without any further function. It visually groups other elements and always lies behind the functional elements but above the default background.
Image	Picture frame with a freely selectable image (PNG, JPG, BMP, GIF, SVG). The image is scaled automatically to the frame (aspect ratio is kept) and follows every size change.

Operation

Control	Function
Edit	Checked: grid visible; elements can be added, moved, resized and deleted. Unchecked: the display is locked (view only).
+ LED / + Text field / + Gauge	Adds a new element (also by right-clicking the empty area, at the mouse position).
Drag	Moves an element; the bottom-right corner changes its size.
Right-click on an element	Menu: link signal (or "no signal"), label or text, LED threshold and colour, decimal places, scale (gauge), delete.

Gauge and scale

The scale has two modes. **Automatic** (default): the range grows with the observed values and is set to round numbers (e.g. 0 to 100); ticks and numbers adapt. **Manual**: **right-click** the gauge and choose **Scale manual (min/max)** to set minimum and maximum (the minimum must be smaller than the maximum). The **Scale automatic** check in the same menu switches back to automatic. Values outside the scale put the pointer at the end stop.

Quick start

1. Keep **Edit** checked (grid visible).
2. Click **+ LED**, **+ Text field** or **+ Gauge**; the element appears on the canvas (also by right-clicking the empty area, at the mouse position).
3. **Right-click** the element, choose **Link signal** and pick a signal from the list.
4. Set label, LED threshold, decimal places or scale and position the element by dragging; the bottom-right corner changes its size.
5. Uncheck **Edit**: the display is locked and used for viewing only.

Link a signal

The list shows the signals selected in the Data tab as `RX:Msg.Signal` or `TX:Msg.Signal`, plus the entry "(no signal)" which removes the link. If a signal is missing, select it in the Data tab first. Values appear as soon as data is received or sent; the unit comes from the DBC. When the signal is changed, the automatic scale is reset.

Configure LED and text field

Setting	Effect
Label / text	LED: label next to the LED (signal name if empty). Text field: free text; with a signal it precedes the value.
LED mode	Digital (value 0 / value 1) or ranges (range 1 to 3). Digital: value 0 gives colour 0, any other value colour 1.
Colours / ranges	Digital: colour for value 0 and value 1. Ranges: per range on/off, minimum, maximum and colour following the rule **minimum <= x < maximum**; the first matching range decides the colour. Same setting as the range display of the gauge.
Decimal places	Number of decimals from 0 to 9 for text field and gauge.
Heading (gauge)	The check **Show heading** shows or hides the label above the gauge; the space is then used for the scale.
Range display (gauge)	The **Range display** check colours the scale in up to three ranges (range 1 to 3), each with minimum, maximum, colour and on/off following the rule **minimum <= x < maximum**; the three ranges then appear in the menu.
Min/max pointer (gauge)	With the checks **Minimum pointer** and **Maximum pointer** the scale shows an additional thin second (blue) and third pointer (orange) for the smallest and largest value since the observation began. **Reset min/max** starts over; changing the signal or toggling the checks also resets. The values are not saved in the project, only the checks.

Gauge scale in detail

Automatic: Until the first value arrives, 0 to 100 applies. Afterwards the gauge remembers the smallest and largest observed value and rounds the limits to round numbers (1, 2, 2.5, 5 times a power of ten); if all values are non-negative and close to 0, the scale starts at 0. The scale grows but never shrinks. The **Scale automatic** check in the right-click menu (switch off and on again) restarts the observation. The scale shows five ticks with numbers.

Manual: right-click the gauge, **Scale manual (min/max)**: enter minimum and maximum in the dialog (decimals allowed). If the minimum is not smaller than the maximum, a message appears and nothing is applied. The manual scale takes effect immediately, stays saved in the project and switches the automatic mode off. Values outside put the pointer at the end stop.

Background and image

The elements lie in layers: default background (canvas), above it the background elements, above those the functional elements (LED, text field, gauge, image). Background elements are placed behind the other elements automatically when added, moved and loaded, so a coloured panel can group several displays. The chosen image is copied into the project folder (`images`) so the project stays portable; without an image "No image" is shown.

LED colours and range display

LED digital example: value 0 = red, value 1 = green. LED with ranges example (temperature): range 1: 0 to 80 green, range 2: 80 to 100 yellow, range 3: 100 to 120 red; outside the LED is grey.

Range display of the gauge: there are three ranges, each with minimum, maximum and colour. A value x belongs to a range if **minimum <= x < maximum** holds (the minimum is included, the maximum is not). Example: range 1: 0 to 80 green, range 2: 80 to 100 yellow, range 3: 100 to 120 red. Each range can be switched on or off individually; parts outside the scale are cut off, and the minimum must be smaller than the maximum. With automatic scaling the bands follow the current scale.

Saving and limits

Elements, positions, sizes, links and settings are saved in the project (`builder`). The canvas is scrollable. The GUI Builder only displays values; there are no input or switch elements. Only signals from the selection in the Data tab can be shown.

Note: signals only appear once data has been received or sent; until then the text field and gauge show "--" or zero position and the LED is off.

Appendix A: Test sequence command reference

Each step is a JSON object with the field `type` and the elements listed below. Physical values are values after factor and offset of the DBC.

set_signal - Set signal value

Sets the physical value of a signal in a transmit message. The new value goes onto the bus with the next (cyclic or single) frame.

Format

```
{"type": "set_signal", "message": "<message>", "signal": "<signal>", "value": <number or text>}
```

Elements

Element	Type	Description
`message`	text, required	Name of the DBC message. If it is missing from the transmit list, it is added (initially not cyclically active).
`signal`	text, required	Name of the signal within the message.
`value`	number or text, required	Physical value (after factor/offset, e.g. 40 for 40 %). For signals with a value table, its text may also be used, e.g. "R". Values outside the signal range are clamped.

Example

```
{
  "type": "set_signal",
  "message": "DriverCmd",
  "signal": "ThrottleCmd",
  "value": 40
}
```

Note: Signals of the message that are not specified remain unchanged (initial value 0).

enable_tx - Switch cyclic transmit on/off

Starts or stops the cyclic transmission of a message at its configured cycle time.

Format

```
{"type": "enable_tx", "message": "<message>", "enabled": <true|false>}
```

Elements

Element	Type	Description
`message`	text, required	Name of the DBC message (added to the transmit list if needed; cycle from the DBC, otherwise 100 ms).
`enabled`	true/false, required	true = transmit cyclically, false = stop transmitting.

Example

```
{
  "type": "enable_tx",
  "message": "DriverCmd",
  "enabled": true
}
```

Note: The first frame is sent immediately, then at the configured cycle time.

set_cycle - Change cycle time

Changes the cycle time of a transmit message (takes effect immediately, even while it is transmitting cyclically).

Format

```
{"type": "set_cycle", "message": "<message>", "cycle_ms": <number>}
```

Elements

Element	Type	Description
`message`	text, required	Name of the DBC message.
`cycle_ms`	number, required	Cycle time in milliseconds. 0 pauses cyclic transmission.

Example

```
{
  "type": "set_cycle",
  "message": "DriverCmd",
  "cycle_ms": 20
}
```

send_once - Send message once

Sends exactly one frame with the current signal values, independent of cyclic transmission.

Format

```
{"type": "send_once", "message": "<message>"}
```

Elements

Element	Type	Description
`message`	text, required	Name of the DBC message.

Example

```
{
  "type": "send_once",
  "message": "DriverCmd"
}
```

send_raw - Send raw frame

Sends a raw frame (ID + data) on the CAN bus, once or cyclically. With cyclic=true and enabled=false the cyclic transmission of this ID is stopped.

Format

```
{"type": "send_raw", "id": "<hex>", "data": "<hex>", "dlc": <n>, "cyclic": <true|false>, "cycle_ms": <n>}
```

Elements

Element	Type	Description
`id`	number or text, required	CAN ID, hexadecimal (e.g. "0x100" or "100").
`data`	hex text, required	Payload as hex bytes, e.g. "00 01 02 03 04 05 06 07".
`dlc`	number, optional	Data length (0-8, 12, 16, 20, 24, 32, 48, 64). Default: number of data bytes.
`cyclic`	true/false, optional	true = send cyclically, false = send once (default).
`cycle_ms`	number, optional	Cycle time in ms when cyclic=true (default 100).
`enabled`	true/false, optional	false stops the cyclic transmission of this ID (only with cyclic=true, default true).
`extended`	true/false, optional	true forces an extended ID (default: automatic above 0x7FF).
`fd`	true/false, optional	true forces CAN FD (default: automatic when DLC is above 8).

Example

```
{
  "type": "send_raw",
  "id": "0x100",
  "data": "00 01 02 03 04 05 06 07",
  "dlc": 8,
  "cyclic": true,
  "cycle_ms": 1000
}
```

Note: Missing data bytes are padded with 00. ID above 0x7FF = extended, DLC above 8 = CAN FD. Cyclically sent frames keep running after the step.

wait - Wait for a fixed time

Pauses the sequence for a fixed time. Cyclic transmission and reception continue.

Format

```
{"type": "wait", "ms": <number>}
```

Elements

Element	Type	Description
`ms`	number, required	Wait time in milliseconds.

Example

```
{
  "type": "wait",
  "ms": 500
}
```

Note: Clicking Stop also ends a running wait immediately.

wait_for - Wait for signal condition

Waits until a signal satisfies the condition. Only values received AFTER this step began are considered. The step is OK on the first match, otherwise it fails when the timeout expires.

Format

```
{"type": "wait_for", "direction": "RX", "message": "<message>", "signal": "<signal>", "op": "<operator>", "value": <number>, "timeout_ms": <
```

Elements

Element	Type	Description

`direction`	"RX" or "TX", required	Evaluate the received or the transmitted message.
`message`	text, required	Name of the DBC message.
`signal`	text, required	Name of the signal.
`op`	text, required	Comparison: == != < <= > >=
`value`	number, required	Comparison value (physical).
`timeout_ms`	number, optional	Maximum wait time in ms (default 3000).

Example

```
{
  "type": "wait_for",
  "direction": "RX",
  "message": "EngineData",
  "signal": "RPM",
  "op": ">",
  "value": 100,
  "timeout_ms": 3000
}
```

Note: The evaluation is independent of the signal selection in the 'Data' tab.

check - Check signal value

Immediately checks the most recently received value of a signal (no waiting). The step fails if the condition is not met or if no value has been received since the sequence started.

Format

```
{"type": "check", "direction": "RX", "message": "<message>", "signal": "<signal>", "op": "<operator>", "value": <number>}
```

Elements

Element	Type	Description
`direction`	"RX" or "TX", required	Evaluate the received or the transmitted message.
`message`	text, required	Name of the DBC message.
`signal`	text, required	Name of the signal.
`op`	text, required	Comparison: == != < <= > >=
`value`	number, required	Comparison value (physical).

Example

```
{
  "type": "check",
  "direction": "RX",
  "message": "EngineData",
  "signal": "CoolantTemp",
  "op": "<",
  "value": 150
}
```

Note: Right after the start there is often no value yet - synchronize first with wait_for.

diag - Diagnostic service from the CDD

Sends a diagnostic service defined in the CDD via ISO-TP and evaluates the response.

Format

```
{"type": "diag", "service": "<service name>", "params": {"<parameter>": <value>}, "expect": "<hex prefix>", "timeout_ms": <number>}
```

Elements

Element	Type	Description
`service`	text, required	Name of the service in the CDD (Diagnostics tab), e.g. ReadVIN.
`params`	object, optional	Values for {Platzhalter} in the request. Number = decimal, text = hexadecimal ("F186").
`expect`	hex text, optional	Expected beginning of the response, e.g. "62 F1 90". Empty = any positive response is accepted.
`timeout_ms`	number, optional	Response timeout in ms (default 1000). With 'ResponsePending' (NRC 0x78) it waits longer.

Example

```
{
  "type": "diag",
  "service": "ExtendedSession",
  "params": {
    "session": 3
  },
  "expect": "50 03"
}
```

Note: Addressing (request/response ID, FD) comes from the 'Diagnostics' tab.

diag_raw - Diagnostic request in hex

Sends any UDS request (hex bytes) via ISO-TP and evaluates the response.

Format

```
{"type": "diag_raw", "request": "<hex bytes>", "expect": "<hex prefix>", "timeout_ms": <number>}
```

Elements

Element	Type	Description
`request`	hex text, required	Request bytes, e.g. "22 F1 86".
`expect`	hex text, optional	Expected beginning of the response, e.g. "62 F1 86". Empty = any positive response is accepted.
`timeout_ms`	number, optional	Response timeout in ms (default 1000).

Example

```
{
  "type": "diag_raw",
  "request": "3E 00",
  "expect": "7E 00"
}
```

Note: Requests without a response (sub-function with bit 7 set, e.g. "3E 80") count as OK.

loop - Loop

Executes the contained steps several times in succession.

Format

```
{"type": "loop", "count": <number>, "steps": [ <step>, <step>, ... ]}
```

Elements

Element	Type	Description
`count`	number, required	Number of iterations.
`steps`	list, required	Nested steps (any types, including further loops).

Example

```
{
  "type": "loop",
  "count": 3,
  "steps": [
    {
      "type": "set_signal",
      "message": "DriverCmd",
      "signal": "ThrottleCmd",
      "value": 20
    },
    {
      "type": "wait",
      "ms": 200
    }
  ]
}
```

Note: In the result, inner steps appear as e.g. 3.2.1 (step 3, iteration 2, step 1).

comment - Comment

Does nothing; serves only to document the sequence.

Format

```
{"type": "comment", "text": "<text>"}
```

Elements

Element	Type	Description
`text`	text, optional	Any comment.

Example

```
{
  "type": "comment",
  "text": "Hier beginnt der Hochlauftest"
}
```

Appendix B: Notes and limits

- **Timing accuracy**: Cyclic transmission is driven by a software timer (accuracy in the range of a few milliseconds), not hardware-triggered.
- **DBC**: Extended multiplexing and environment variables are not supported.
- **CDD**: Original CANdelaStudio files only heuristically (chapter 2.2).
- **PEAK + CAN FD**: Bit timing is preset (80 MHz, 500k/2M); deviations via `hardware.extra`.
- **Safety**: Cyclic transmission never starts automatically when a project is loaded. On vehicle or test bench buses, check the transmit list and values before connecting.
- **Keyboard**: Edit in tables by double-clicking; in the raw diagnostic field, Enter sends the request.

Testing without hardware

The demo project `demo` uses the simulation with a virtual ECU. This lets you try all tabs (transmit/receive, trace, graph, logging, sequences *startup* and *driving*, diagnostics) without hardware. The core tests run with `python -m unittest discover -s tests -v`.

Appendix C: vFlash pack and manifest

The vFlash tab in the Diagnostics section loads a flash package, interprets it and runs it with the built-in UDS flash sequence on the selected hardware (Vector, PEAK, SocketCAN, simulation). The Vector software vFlash is not required.

Supported sources

- `*.vflashpack` or `*.zip` with `flashpack.json` (or `flashpack.yaml`) and the data files: fully supported.
- Genuine vFlash pack without a manifest: the internal format is not publicly documented. HEX, S-record and BIN files are found and the project is read heuristically. Sequence, addresses and seed&key usually have to be added through a manifest.
- Single manifest file or image (`.hex`, `.s19`, `.srec`, `.mot`, `.bin`) with the generic default sequence.

Manifest flashpack.json

```
{
  "format": "canTool-flashpack-1",
  "name": "Demo Application",
  "ecu": "Engine_ECU",
  "request_id": "0x7E0",
  "response_id": "0x7E8",
  "extended": false,
  "fd": false,
  "security": {
    "type": "xor",
    "level": 1,
    "value": "0xFFFFFFFF"
  },
  "address_format": "0x44",
  "data": [
    {
      "file": "app.hex",
      "name": "Application"
    },
    {
      "file": "cal.bin",
      "address": "0x00020000",
      "name": "Calibration"
    }
  ],
  "steps": [
    {
      "type": "session",
      "session": 3
    },
    {
      "type": "comm_control",
      "control": 3,
      "comm_type": 1
    },
    {
      "type": "session",
      "session": 2
    },
    {
      "type": "security",
      "level": 1
    },
    {
      "type": "erase",
      "block": "all"
    },
    {
      "type": "download",
      "block": "all"
    },
    {
      "type": "check",
      "routine": "FF01"
    },
    {
      "type": "reset",
      "reset": 1
    }
  ]
}
```

Element	Meaning
`name`, `ecu`	Display name and ECU.
`request_id`, `response_id`, `extended`, `fd`	ISO-TP addressing; if missing, the diagnostic configuration of the Diagnostics section applies.
`security`	Seed&key: `type` = `none`, `xor`, `add` or `constant`, plus `level` (seed level, key = level + 1) and `value`.
`address_format`	addressAndLengthFormatIdentifier, e.g. `0x44` = 4-byte length and 4-byte address.
`max_chunk`	Upper limit of the payload per transfer request (otherwise taken from the response to 0x34).
`data`	List of data files (`file`, `name`); for BIN files `address` is mandatory.
`steps`	Own sequence; if missing, the default sequence applies.

Sequence steps (steps)

Step	Parameters and effect

`session`	`session`: diagnostic session (0x10).
`comm_control`	`control`, `comm_type`: communication control (0x28).
`security`	`level`: request seed, compute and send key (0x27).
`erase`	`block` (`all` or number), `routine` (default FF00): erase memory (0x31).
`download`	`block`: request download, transfer data, transfer exit (0x34, 0x36, 0x37).
`check`, `routine`	`routine`, `data` (hex), `control`, `timeout_ms`: start a routine (0x31).
`request`	`data` (hex), `expect` (expected leading response bytes).
`reset`	`reset`: ECU reset (0x11).
`wait`, `comment`	`ms` wait time or `text` as a comment.

Default sequence

Without `steps` the following is executed in order: session 0x03, disable communication (0x28 03 01), session 0x02, security access (if `security` is set), erase and program all blocks, ECU reset 0x11 01. This sequence is generic and not manufacturer-specific.

Safety notes

Flashing erases and overwrites memory of the ECU. The power supply and the bus connection must not be interrupted meanwhile. Code from packages is never executed; manufacturer-specific seed&key DLLs are not supported. If an error occurs, the sequence stops and the remaining steps are skipped.

Français – Manuel

CAN Tool · Date d'édition : 2026-10-29

1. Généralités et création de projet

2. Paramétrage avant le démarrage

3.1 Rubrique Projet

3.2 Rubrique Données

3.3 Rubrique Trace

3.4 Rubrique Séquence de test

3.5 Rubrique Graphique

3.6 Rubrique Journalisation

3.7 Rubrique Diagnostic

3.8 Barre des rubriques : mode sombre et langue

3.9 Rubrique Aide

3.10 Rubrique XCP

3.11 Rubrique GUI Builder

Annexe A : Référence des commandes de séquence de test

Annexe B : Remarques et limites

Annexe C : pack vFlash et manifeste

1. Description générale et création de projet

1.1 À quoi sert l’outil

Le CAN Tool est un outil Python pour tester et analyser des bus CAN et CAN FD. Il fonctionne au choix avec le **Vector VN1630A / VN7572** ou le **PEAK-System PCAN-USB Pro FD** (tous deux via python-can) ou - sans matériel - avec une **simulation** intégrée.

Fonctions principales :

- Gestion de plusieurs **projets** (chacun avec ses propres fichiers DBC, CDD, séquences et journaux).
- **Conversion DBC/CDD** en JSON ou YAML avant l’utilisation proprement dite (chargement rapide, contrôle facile).
- **Émission cyclique** et réception de messages, accès aux signaux en valeurs physiques.
- **Trace** de toutes les trames émises et reçues, avec décodage des signaux en option.
- **Séquences de test** (JSON) : définir des signaux, attendre des réactions, vérifier, exécuter des diagnostics.
- **Graphique en courbes** des signaux sélectionnés.
- Apparence commutable (**mode sombre**) et **six langues** (allemand, anglais, français, espagnol, chinois/mandarin, thaï).
- **Journalisation Parquet** avec filtre.
- **Diagnostic** via ISO-TP (ISO 15765-2) avec les services UDS de la CDD.

1.2 Flux de données en bref

```
dbc/*.dbc  --(conversion)-->  cache/*.dbc.json|yaml  ---+
cdd/*.cdd  --(conversion)-->  cache/*.cdd.json|yaml  ---+--> Rubrique "Données" : sélection TX / RX
                                                        |        |
                                                        |        +--> Trace   (toutes les trames)
                                                        |        +--> Graphique (signaux choisis)
                                                        |        +--> Journalisation (signaux choisis, filtre)
sequences/*.json  ------------------------------------------+--> Séquence de test (définit/vérifie des signaux, diagnostic)
```

À l’exécution, seul le **cache** (JSON/YAML) est lu. Si un fichier DBC/CDD change, il est automatiquement reconverti à la prochaine ouverture du projet (somme de contrôle du fichier source).

1.3 Installation et démarrage

```
pip install -r requirements.txt
python main.py                      # GUI
python main.py --project demo       # GUI avec un projet donné
python main.py --list               # lister les projets
python main.py --convert demo       # convertir uniquement DBC/CDD (sans GUI)
python main.py --convert demo --format yaml --force
```

Pilotes : Vector nécessite la *Vector XL Driver Library* (incluse dans les pilotes Vector), PEAK le pilote *PCAN-Basic*. Sans matériel, la simulation suffit.

1.4 Structure des dossiers

```
can_tool/
  main.py  gui.py  can_manager.py  dbc_manager.py  cdd_manager.py
  iso_tp_manager.py  sequence_runner.py  parquet_logger.py  project_manager.py
  help_content.py  i18n.py
  lang/               Fichiers de langue (de.yaml, en.yaml, fr.yaml, es.yaml, zh.yaml, th.yaml)
  tools/check_lang.py Vérification des fichiers de langue
  projects/
    <nom du projet>/
      project.json      Paramètres du projet (écrit par l’outil)
      dbc/              Vos fichiers DBC (*.dbc)
      cdd/              Vos descriptions de diagnostic (*.cdd)
      sequences/        Séquences de test (*.json)
      cache/            généré automatiquement : DBC/CDD converties (JSON ou YAML)
      logs/             Fichiers journaux Parquet
```

1.5 Créer un projet

Dans la GUI

1. Ouvrir la rubrique **Projet** et cliquer sur **Nouveau ...**.
2. Saisir le nom du projet. Sont autorisés : lettres, chiffres, trait de soulignement, tiret et espaces (64 caractères max., premier caractère lettre ou chiffre). Si le nom existe déjà, rien n’est écrasé.
3. L’outil crée `projects/<name>/` avec les sous-dossiers `dbc, cdd, sequences, cache, logs` et un fichier `project.json` avec les valeurs par défaut (simulation, 500 kbit/s, cache JSON) et bascule sur le nouveau projet.
4. **Importer DBC/CDD ...** : les fichiers sont copiés dans `dbc/` ou `cdd/` et convertis immédiatement. Sinon, placer les fichiers à la main dans les dossiers et cliquer sur **Reconvertir** (ou rouvrir le projet).
5. Régler le matériel (voir chapitre 2.3) et **Connecter**.

Par fichier

Créer un dossier de projet avec la structure ci-dessus et y placer un fichier `project.json` contenant au minimum `{}` - les paramètres manquants sont complétés par les valeurs par défaut. Le projet de démonstration `projects/demo` sert de modèle.

Changer de projet

Sélection dans la liste des projets. Lors du changement (et à la fermeture), la connexion est coupée, une journalisation en cours est arrêtée et le projet précédent est enregistré. Sont enregistrés : matériel, liste d’émission avec valeurs de signaux et cycles, sélection de réception, filtres de journalisation, sélection du graphique, paramètres de diagnostic et la dernière séquence chargée. L’émission cyclique est toujours **désactivée** après l’ouverture d’un projet (sécurité sur un bus réel).

1.6 Déroulement typique

1. Créer ou choisir un projet, importer la DBC (et la CDD).
2. Régler le matériel, **Connecter**.
3. Rubrique **Données** : choisir dans la DBC les messages à émettre et à recevoir, définir valeurs de signaux et cycles.
4. Activer l'émission cyclique ; observer le résultat dans **Trace** et **Graphique**.
5. Si besoin, démarrer la **Journalisation** et/ou créer et exécuter une **séquence de test**.
6. **Diagnostic** : envoyer des services, vérifier les réponses.

2. Paramétrage des fichiers avant le démarrage

Avant de vous connecter, les fichiers du projet doivent être préparés. Ce chapitre décrit ce que chaque fichier doit contenir et quelles indications l'outil exploite.

2.1 Fichiers DBC (`dbc/*.dbc`)

Fichiers DBC standard issus de CANdb++, CANdb Editor, SavvyCAN, etc. Plusieurs fichiers DBC par projet sont possibles (leur somme forme la base de données). Jeu de caractères UTF-8 ou Windows-1252.

Sont exploités :

Élément DBC	Signification dans l'outil
`BO_`	Message : ID (bit 31 positionné = Extended-ID), nom, longueur (DLC > 8 = CAN FD), émetteur.
`SG_`	Signal : bit de départ, longueur, ordre des octets (Intel/Motorola), signe, facteur, offset, min/max, unité, récepteurs.
`SG_ ... M / m0, m1`	Multiplexage simple (un multiplexeur, plusieurs groupes). Le multiplexage étendu n'est **pas** pris en charge (avertissement).
`VAL_`	Table de valeurs : texte par valeur brute. Dans l'outil, liste déroulante à l'émission et texte à la réception.
`SIG_VALTYPE_`	Signaux à virgule flottante (32/64 bits).
`CM_`	Commentaires sur les messages/signaux.
`BA_ "GenMsgCycleTime"`	Temps de cycle par défaut (ms) d'un message ; 100 ms si non indiqué.
`BA_ "VFrameFormat"`	Format de trame ; les valeurs contenant *FD* désignent des messages CAN FD.

Exemple :

```
VERSION "1.0"
NS_ :
BS_:
BU_: Tester ECU

BO_ 512 EngineData: 8 ECU
 SG_ RPM : 0|16@1+ (0.25,0) [0|8000] "rpm" Tester
 SG_ CoolantTemp : 16|8@1+ (1,-40) [-40|130] "degC" Tester

VAL_ 257 StartButton 0 "Aus" 1 "Ein" ;
BA_DEF_ BO_ "GenMsgCycleTime" INT 0 65535;
BA_ "GenMsgCycleTime" BO_ 512 10;
```

Remarques sur le paramétrage

- Les noms de messages et de signaux sont utilisés dans les séquences de test - choisir des noms parlants et uniques.
- Noms de messages en double dans plusieurs fichiers DBC : la première définition est conservée, la seconde est ignorée avec un avertissement.
- Même ID avec des noms différents : à la réception, le premier nom est attribué (avertissement).
- Les nœuds émetteurs (`BU_`) déterminent, dans la simulation, quels messages le calculateur virtuel émet cycliquement (par défaut : émetteur `ECU`, réglable dans `sim.traffic_senders`). Des valeurs min/max pertinentes dans la DBC donnent des valeurs de simulation réalistes.
- Les avertissements de conversion figurent dans la rubrique **Projet**, dans le champ *Fichiers convertis / Avertissements*.

Le cache (`cache/*.dbc.json` ou `.yaml`)

À l'ouverture du projet, chaque DBC est automatiquement écrite dans le cache (format au choix dans la rubrique **Projet**). La reconversion n'a lieu que si le fichier source a changé ; **Reconvertir** la force. Les modifications manuelles du cache sont écrasées lors d'une nouvelle conversion - les corrections doivent être faites dans la DBC.

```
{
  "format_version": 1,
  "nodes": ["Tester", "ECU"],
  "messages": {
    "EngineData": {
      "id": 512, "extended": false, "dlc": 8, "fd": false, "sender": "ECU",
      "cycle_time_ms": 10, "comment": "", "multiplexor": null,
      "signals": {
        "RPM": {"start": 0, "length": 16, "byte_order": "little", "signed": false,
                "factor": 0.25, "offset": 0, "min": 0, "max": 8000, "unit": "rpm",
                "receivers": ["Tester"], "choices": {}, "comment": ""}
      }
    }
  },
  "warnings": [], "source": "engine.dbc", "source_sha1": "..."
}
```

2.2 Description de diagnostic (`cdd/*.cdd`)

La CDD décrit les services de diagnostic. Comme la DBC, elle est convertie vers `cache/*.cdd.json|yaml`.

Restriction importante : le format XML simple `<DIAGDATA>` est entièrement pris en charge (voir l'exemple et `projects/demo/cdd/diag.cdd`). Les fichiers CDD CANdelaStudio d'origine ne sont lus que de façon **heuristique** (noms de services et, si trouvés, octets de request). Dans ce cas, vérifiez le fichier de cache généré (rubrique **Projet**, avertissements) et complétez-y les adresses, requests et paramètres de réponse, ou utilisez le format simple.

```
<?xml version="1.0" encoding="UTF-8"?>
<DIAGDATA>
  <ECU name="Engine_ECU" request_id="0x7E0" response_id="0x7E8" extended="false" fd="false">
    <SERVICE name="ExtendedSession" request="10 {session}" description="Select diagnostic session">
      <PARAM name="session" bytes="1" default="03"/>
    </SERVICE>
    <SERVICE name="ReadVIN" request="22 F1 90" description="Read VIN">
```

```
      <RESPONSE prefix_len="3">
        <PARAM name="VIN" offset="0" length="17" type="ascii"/>
      </RESPONSE>
    </SERVICE>
    <SERVICE name="ReadSpeed" request="22 F4 0D" description="Read speed">
      <RESPONSE prefix_len="3">
        <PARAM name="Speed" offset="0" length="1" type="uint" factor="1" unit="km/h"/>
      </RESPONSE>
    </SERVICE>
  </ECU>
</DIAGDATA>
```

Élément / Attribut	Signification
`ECU name, request_id, response_id`	Nom du calculateur et IDs CAN (testeur -> ECU, ECU -> testeur), p. ex. `0x7E0`/`0x7E8`.
`ECU extended, fd`	`true` : IDs 29 bits ou CAN FD (ISO-TP jusqu'à 64 octets par trame).
`SERVICE name`	Nom du service (utilisé dans la rubrique Diagnostic et dans l'étape de séquence `diag`).
`SERVICE request`	Octets de la request en hex. Paramètres fictifs entre accolades, p. ex. `10 {session}`.
`SERVICE description`	Texte libre, affiché dans la liste des services.
`PARAM`	Directement dans SERVICE : paramètre d'entrée pour un paramètre fictif : `name`, `bytes` (largeur), `default` (hex).
`RESPONSE prefix_len`	Nombre d'octets au début de la réponse à ignorer (ID de service + DID, p. ex. 3 pour `62 F1 90`).
`RESPONSE/PARAM`	Paramètres de réponse : `name`, `offset` et `length` en octets (à partir de la fin du préfixe), `type` = `ascii | uint | int | hex`, en option `factor`, `offset_val`, `unit`.

L'entrée de cache correspondante (pour retouche manuelle) :

```
{
  "format_version": 1,
  "ecus": {
    "Engine_ECU": {
      "request_id": "0x7E0", "response_id": "0x7E8", "extended": false, "fd": false,
      "services": {
        "ReadVIN": {
          "request": "22 F1 90", "description": "Read VIN", "params": [],
          "response": {"prefix_len": 3,
                       "params": [{"name": "VIN", "offset": 0, "length": 17, "type": "ascii"}]}
        }
      }
    }
  }
}
```

2.3 Matériel et connexion

Réglages dans la rubrique **Projet** (enregistrés dans `project.json` sous `hardware`) :

Réglage	Vector VN1630A / VN7572	PEAK PCAN-USB Pro FD	Simulation
Interface	`vector` (pilote XL)	`pcan` (PCAN-Basic)	`simulation`
Canal	Index du canal, base 0 (p. ex. `0`). Affectation via la *Vector Hardware Config* ; si un nom d'application est défini, le numéro de canal qui y est affecté s'applique.	`PCAN_USBBUS1`, `PCAN_USBBUS2` ...	non nécessaire
Débit	Débit nominal (kbit/s)	Débit nominal (kbit/s) ; en FD fixé à 500k/2M (horloge 80 MHz)	sans importance
CAN FD / Débit des données	Cocher la case et saisir le débit des données	Cocher la case ; timing différent dans `hardware.extra`	Case pour les trames FD
Nom d'application	Nom issu de la Vector Hardware Config (vide = valeur par défaut de python-can)	-	-

Vector VN7572 (carte PCIe) : la carte d'interface Vector VN7572 est elle aussi utilisée comme type `vector` via le pilote Vector XL (Windows uniquement), exactement comme le VN1630A. Avec *Vector Driver Setup* installé, la carte doit être visible dans *Vector Hardware Config*. Le numéro de canal commence à 0 (0 = CH1, 1 = CH2 ...) ; **Détecter le matériel** liste les canaux trouvés avec leur numéro de série. Aucun autre programme (CANalyzer, CANoe, vFlash) ne doit occuper le canal.

- Tous les participants du bus doivent utiliser le même débit ; en CAN FD, également le même débit des données.
- Sur un montage de table avec deux appareils, les extrémités du bus doivent être terminées par 120 ohms.
- `hardware.extra` (uniquement dans `project.json`) transmet des paramètres quelconques à python-can, p. ex. `{"nom_brp": 5}` pour PEAK.
- **Simulation** : *Écho* renvoie les propres messages émis comme réception ; le *calculateur virtuel* répond aux requests UDS (entre autres 10, 11, 14, 19, 22, 27, 3E) sur les adresses du réglage de diagnostic et émet cycliquement les messages des émetteurs ECU.

2.4 Fichiers de séquence (`sequences/*.json`)

Une séquence se compose d'un nom, d'une description et d'une liste d'étapes. Vous pouvez la créer dans la rubrique **Séquence de test** ou modifier directement le fichier. Référence des commandes : annexe A.

```
{
  "name": "startup",
  "description": "Simulate engine start and check run-up",
  "steps": [
    {"type": "enable_tx", "message": "DriverCmd", "enabled": true},
    {"type": "set_signal", "message": "DriverCmd", "signal": "StartButton", "value": 1},
    {"type": "wait_for", "direction": "RX", "message": "EngineData", "signal": "RPM",
     "op": ">", "value": 100, "timeout_ms": 3000},
    {"type": "diag", "service": "ReadVIN", "expect": "62 F1 90"}
```

```
  ]
}
```

2.5 Le fichier de projet (`project.json`)

Écrit par l'outil ; modifiable manuellement si besoin (fermer l'outil auparavant). Les clés manquantes sont complétées par les valeurs par défaut.

```
{
  "hardware": {"type": "vector", "channel": "0", "bitrate": 500000, "fd": true,
               "data_bitrate": 2000000, "app_name": "", "extra": {}},
  "cache_format": "json",
  "tx": [{"message": "DriverCmd", "cycle_ms": 50, "values": {"ThrottleCmd": 0}, "signals": null}],
  "rx": {"EngineData": null, "VehicleSpeed": ["Speed"]},
  "logging": {"selection": {"TX": {}, "RX": {"EngineData": null}},
              "on_change_only": false, "min_interval_ms": 0, "log_raw": false},
  "graph": ["RX:EngineData.RPM"],
  "diag": {"cdd": "diag.cdd.json", "ecu": "Engine_ECU", "request_id": "", "response_id": "",
           "extended": false, "fd": false, "padding": 170},
  "sequence": "startup.json",
  "sim": {"ecu": true, "traffic_senders": ["ECU"]}
}
```

Clé	Signification
`hardware`	Paramètres de connexion (chapitre 2.3) ; débits en bit/s.
`cache_format`	`json` ou `yaml` pour les fichiers convertis.
`tx`	Liste d'émission : message, cycle (ms), valeurs de signaux, `signals` = sélection pour graphique/journalisation (`null` = tous).
`rx`	Sélection de réception : message -> liste des signaux choisis ou `null` (tous).
`logging`	Filtres et options de journalisation (rubrique Journalisation).
`graph`	Clés des signaux cochés dans le graphique, p. ex. `RX:Msg.Signal`.
`diag`	Fichier CDD, ECU, surcharge facultative des adresses (`request_id`, `response_id`), `extended`, `fd`, `padding`.
`sequence`	Dernière séquence chargée (nom de fichier dans `sequences/`).
`sim`	Simulation : `ecu` (calculateur virtuel), `traffic_senders` (nœuds émetteurs simulés cycliquement).

2.6 Liste de contrôle avant la première connexion

1. Fichier(s) DBC dans le dossier `dbc/` ; dans la rubrique Projet apparaissent les messages attendus (nombre) et aucun avertissement inattendu.
2. Éventuellement CDD dans le dossier `cdd/` ; dans la rubrique Diagnostic apparaissent l'ECU et les services.
3. Pilote matériel installé, appareil raccordé, canal et débits adaptés au bus.
4. En CAN FD : case *CAN FD* et débit des données ; les autres participants doivent également gérer le FD.
5. Séquences présentes sous `sequences/` ou créées dans la rubrique Séquence de test.

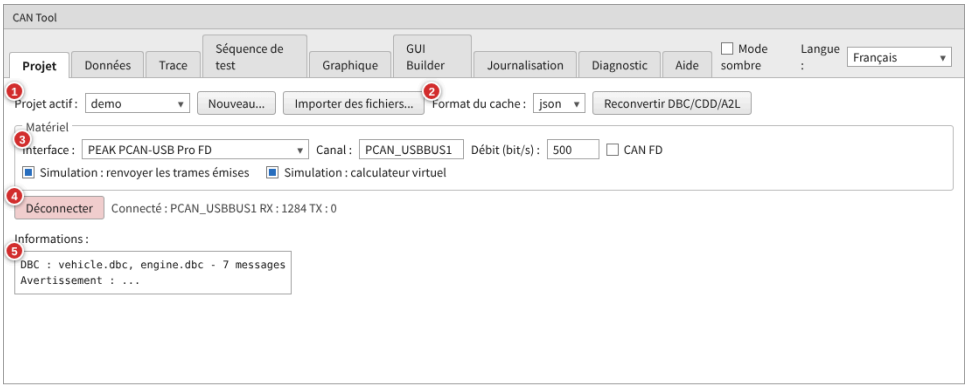
Problèmes fréquents

Symptôme	Cause possible / Remède
Aucun message dans la rubrique Données	DBC absente du dossier `dbc/` ou extension différente de `.dbc` ; lire les avertissements dans la rubrique Projet.
Échec de la connexion	python-can/pilote manquant, mauvais canal, appareil occupé par une autre application (p. ex. CANalyzer/PCAN-View).
Trames visibles mais non décodées	L'ID/le marquage Extended de la DBC ne correspond pas au bus ; message absent de la DBC.
Valeurs de signaux erronées	Vérifier dans la DBC l'ordre des octets (Intel/Motorola), le bit de départ, le facteur/offset.
Diagnostic : timeout	Vérifier Request-/Response-ID, Extended/FD et Padding dans la rubrique Diagnostic ; calculateur dans la bonne session ?
Journalisation indisponible	Paquet Python `pyarrow` manquant (`pip install pyarrow`).

3.1 Rubrique « Projet »

Système d'exploitation : au démarrage, l'outil détecte lui-même s'il tourne sous **Windows** ou **Linux** (affiché dans la barre de titre). La sélection du matériel s'adapte : *Vector* (pilote XL) uniquement sous Windows, *SocketCAN* uniquement sous Linux, *PEAK* et *Simulation* sur les deux. Les entrées inutilisables sont marquées et désactivées. Le bouton **Détecter le matériel** liste les interfaces connectées. Toutes les autres fonctions (trace, séquences, enregistrement, diagnostic, vFlash, aide/PDF) sont identiques sur les deux systèmes ; les dossiers de projet sont échangeables entre Windows et Linux.

Capture annotée



Représentation schématique de l'interface ; les numéros renvoient aux explications ci-dessous.

1. Choix du projet, création d'un nouveau projet et import de fichiers DBC/CDD.
2. Format du cache (json/yaml) et nouvelle conversion des DBC/CDD.
3. Matériel : interface, canal (p. ex. PCAN_USBBUS1), débit et CAN FD.
4. Connexion ou déconnexion avec affichage de l'état (compteurs de réception et d'émission).
5. Zone d'information : fichiers DBC chargés, nombre de messages et avertissements.

Réinit. matériel : à côté de *Connecter*, ce bouton ferme l'interface et la rouvre avec les mêmes paramètres. Il aide lorsque le matériel connecté se bloque. La liste d'émission est conservée (les messages cycliques actifs continuent ensuite) ; un enregistrement en cours est arrêté.

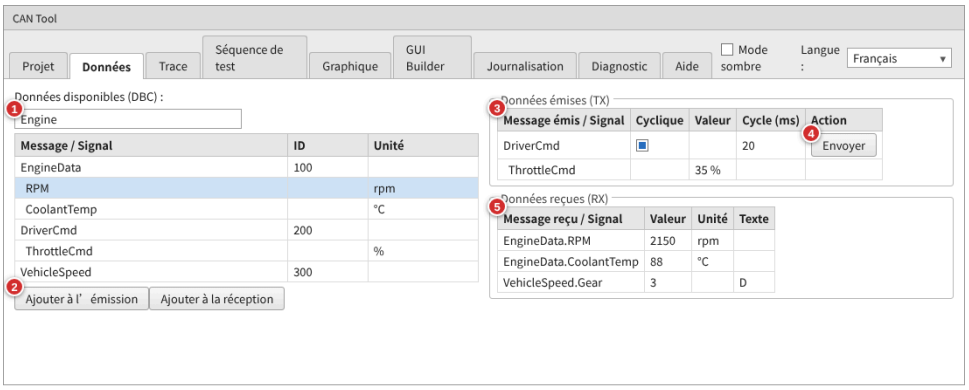
Gérer les projets, intégrer DBC/CDD, régler le matériel et établir la connexion.

Élément de commande	Fonction
Projet actif	Liste de tous les projets (`projects/*/project.json`). Un changement coupe la connexion, arrête la journalisation, enregistre l'ancien projet et charge le nouveau.
Nouveau ...	Crée un projet avec sa structure de dossiers et les paramètres par défaut (chapitre 1.5).
Importer DBC/CDD ...	Copie les fichiers `.dbc`/`.cdd` choisis dans le projet et les convertit.
Format du cache	`json` ou `yaml` ; en cas de changement, reconversion.
Reconvertir	Force la conversion de toutes les DBC/CDD et recharge le projet.
Interface / Canal / Débit / CAN FD / Débit des données / Nom d'application	Paramètres matériels (chapitre 2.3). Verrouillés pendant la connexion.
Simulation : écho	Les propres messages émis apparaissent en plus comme réception.
Simulation : calculateur virtuel	Démarre à la connexion un calculateur logiciel compatible UDS avec trafic de test.
Connecter / Déconnecter	Ouvre ou ferme l'interface. À la déconnexion, une journalisation en cours est arrêtée.
Ligne d'état	État de la connexion, nombre de trames reçues (RX) et émises (TX), dernière erreur.
Fichiers convertis / Avertissements	Liste des fichiers de cache, nombre de messages, avertissements de la DBC/CDD.

3.2 Rubrique « Données »

On définit ici **quels** messages sont émis et reçus et **quels signaux** valent pour le graphique, la journalisation et l'affichage - toujours à partir de l'ensemble des données DBC du projet.

Capture annotée



Représentation schématique de l'interface ; les numéros renvoient aux explications ci-dessous.

1. Filtre et liste de tous les messages et signaux de la DBC.
2. Ajouter la sélection aux données d'émission (TX) ou de réception (RX).
3. Données d'émission : envoi cyclique activé/désactivé, valeurs des signaux et temps de cycle.
4. Envoi unique : envoie immédiatement le message une fois.
5. Données de réception avec les valeurs en direct des signaux.

Côté gauche : données disponibles

- Arborescence de tous les messages avec leurs signaux (ID, émetteur, unité). Le **champ de filtre** cherche dans le nom du message, l'ID et le nom du signal.
- **-> Émettre (TX)** reprend les messages sélectionnés dans la liste d'émission (un double-clic sur un message fait de même). La sélection multiple est possible ; si l'on sélectionne un signal, c'est son message qui est repris.
- **-> Réception (RX)** reprend les messages sélectionnés dans la liste de réception.

En haut à droite : émission

Colonne	Signification
Message émis / Signal	Message (avec ID) et, en dessous, tous ses signaux.
Cyclique	La case sur le message active/désactive l'émission cyclique (l'émission n'a réellement lieu que si la connexion est établie).
Valeur	Valeur physique du signal (champ de saisie ou liste de sélection pour les signaux avec table de valeurs). Les modifications prennent effet dès la trame suivante.
Cycle [ms]	Temps de cycle du message ; prérempli depuis la DBC (`GenMsgCycleTime`), sinon 100 ms. 0 = non cyclique.
Sélection	Case sur le signal : le signal fait partie de la sélection pour le graphique et la journalisation.
Envoyer (bouton)	Émet le message exactement une fois avec les valeurs actuelles.

Supprimer retire de la liste d'émission le message de la ligne sélectionnée.

En bas à droite : réception

Colonne	Signification
Message reçu / Signal	Message (avec ID) et ses signaux.
Valeur / Unité / Texte	Dernière valeur reçue (actualisation toutes les 200 ms), unité et - si présent - texte de la table de valeurs.
Sélection	Case sur le signal : le signal fait partie de la sélection pour le graphique et la journalisation. Les signaux non cochés ne sont pas enregistrés.

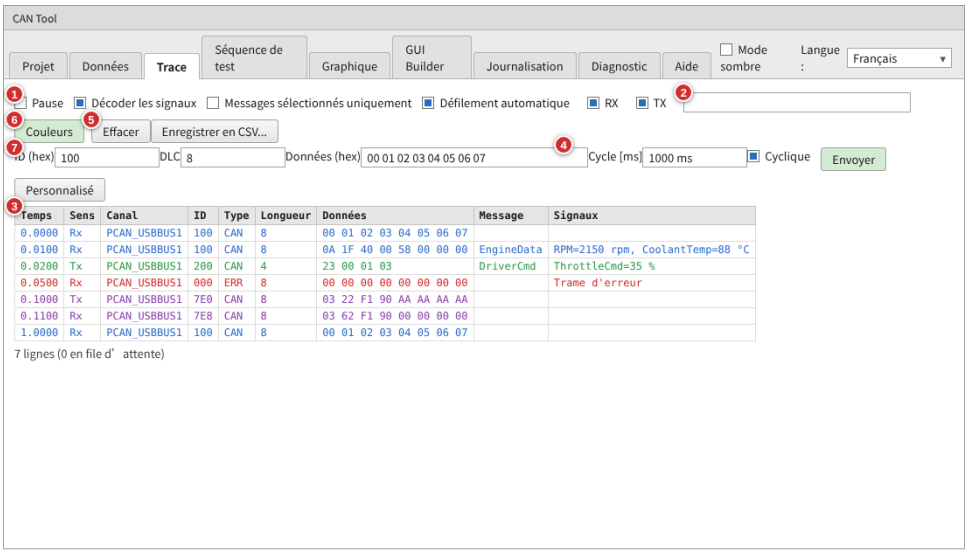
Remarques

- La sélection (cases) détermine ce que voient le **graphique** et la **journalisation**. La **trace**, elle, affiche toutes les trames (en option uniquement les messages choisis).
- La sélection et toutes les valeurs d'émission sont enregistrées avec le projet ; l'émission cyclique est désactivée après le chargement.
- Un message peut figurer en même temps dans la liste TX et RX (p. ex. avec l'écho dans la simulation).

3.3 Rubrique « Trace »

Déroulement chronologique de toutes les trames émises (Tx, mises en évidence en couleur) et reçues (Rx).

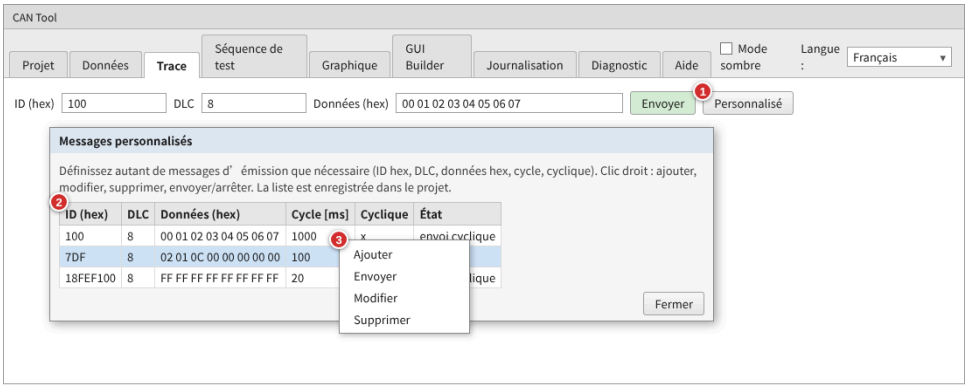
Capture annotée



Représentation schématique de l'interface ; les numéros renvoient aux explications ci-dessous.

1. Options : pause, décoder les signaux, messages choisis seulement, défilement automatique.
2. Filtre par ID ou nom de message.
3. Tableau de trace : temps, sens (Rx/Tx), canal, ID, type, longueur et données brutes. Fonctionne aussi sans DBC/CDD.
4. Noms de messages et valeurs de signaux décodés (si une DBC est chargée).
5. Effacer la trace ou l'enregistrer en fichier CSV.
6. Mode couleur : Rx bleu, Tx vert, erreurs rouge, ISO-TP violet.
7. Ligne d'envoi : ID (hex), DLC, données, durée de cycle, case pour l'envoi cyclique et bouton Envoyer.

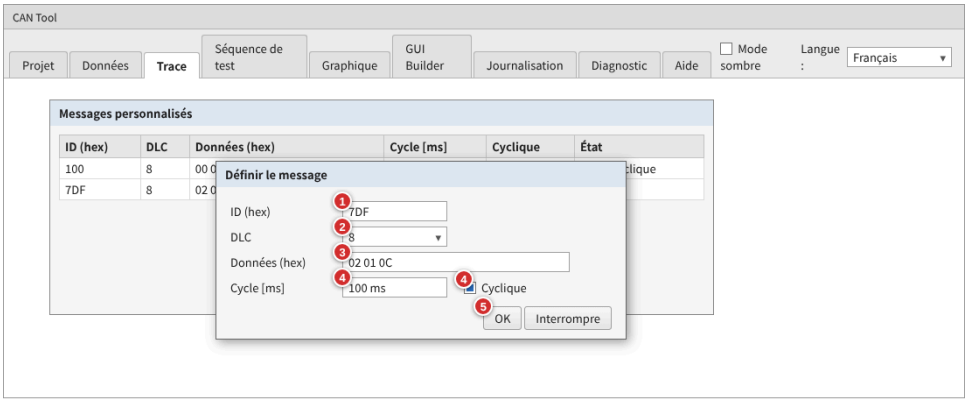
Capture annotée



Représentation schématique de l'interface ; les numéros renvoient aux explications ci-dessous.

1. Le bouton Personnalisé ouvre la fenêtre des messages d'émission personnalisés.
2. Tableau des messages d'émission : ID (hex), DLC, données (hex), cycle [ms], cyclique et état.
3. Menu du clic droit : ajouter, envoyer/arrêter, modifier, supprimer.

Capture annotée



Représentation schématique de l'interface ; les numéros renvoient aux explications ci-dessous.

1. ID (hex) : ID CAN, aussi étendu (> 7FF).
2. DLC : longueur des données, aussi valeurs CAN FD.
3. Données (hex) : octets avec ou sans espaces ; les octets manquants sont complétés par 00.
4. Cycle [ms] et case Cyclique : envoi unique ou cyclique.
5. OK ajoute le message à la liste, Annuler abandonne la saisie.

Colonne	Contenu
Temps [s]	Temps relatif à la première trame affichée (depuis la connexion ou *Effacer*).
Rx/Tx, Canal	Sens et interface.
ID, Type, Len	ID CAN (hex, 8 chiffres pour Extended), type (CAN, FD, FD/BRS) et longueur des données.
Données	Données utiles en hex.
Message, Signaux	Nom issu de la DBC et signaux décodés avec unité (si *Décoder* est actif).
Mode couleur	Bouton à bascule. Activé : messages Rx en bleu moyen, messages Tx en vert moyen, messages erronés (trames d'erreur du bus, type ERR) en rouge et messages ISO-TP (ID de requête/réponse du diagnostic, aussi 0x7DF) en violet moyen. Désactivé : affichage comme avant.

Élément de commande	Fonction
Pause	Fige l'affichage. Les trames continuent d'être mises en tampon (jusqu'à 50 000) et sont restituées à la reprise.
Décoder	Active/désactive le décodage des signaux (désactivé = plus rapide en cas de forte charge du bus).
Messages sélectionnés uniquement	N'affiche que les messages figurant dans la liste TX ou RX de la rubrique Données.
Défilement auto	Suit automatiquement l'entrée la plus récente.
Rx / Tx	Afficher/masquer les sens.
Champ de filtre	IDs en hex séparés par des virgules (p. ex. `100, 200`) et/ou une partie d'un nom de message (p. ex. `engine`).
Effacer	Efface l'affichage et le tampon.
Enregistrer en CSV ...	Exporte les lignes actuellement visibles (séparateur `;`).

Mettre à jour : avec la case *Mettre à jour* cochée, chaque message (direction, canal, ID) n'apparaît qu'une fois dans le tableau, dans l'ordre de la première apparition. À chaque nouvelle occurrence, l'horodatage, les données et les signaux décodés de cette ligne sont mis à jour et la colonne *Nombre* augmente. Décochée, toutes les trames sont écrites les unes après les autres avec défilement. Le changement vide le tableau ; le défilement automatique ne s'applique qu'en mode défilant.

Écart : en plus de *Mettre à jour*, la case *Écart* fait afficher dans la colonne de temps l'écart depuis l'occurrence précédente du même message (direction, canal, ID), c'est-à-dire la durée de cycle, au lieu du temps absolu ; 0 à la première occurrence. L'en-tête de colonne devient *Écart [s]*. Le changement vide le tableau.

Ligne d'envoi : sous la barre d'options, une ligne envoie des trames brutes (par défaut : ID 100 hex, DLC 8, données 00 01 02 03 04 05 06 07, 1000 ms). **Envoyer** transmet la trame une fois ; avec **Cyclique** coché, l'envoi cyclique démarre et un nouveau clic (**Arrêt**) l'arrête. Les ID supérieurs à 7FF sont envoyés en trames étendues, un DLC supérieur à 8 en CAN FD (FD doit être activé dans les paramètres de connexion). La trame apparaît dans le trace comme Tx.

Messages personnalisés (envoi multiple)

La ligne d'envoi n'envoie qu'un seul message. Pour **plusieurs messages**, utilisez le bouton **Personnalisé** : il ouvre une fenêtre séparée qui reste ouverte à côté du trace. On y définit autant de messages d'émission que souhaité et on les envoie individuellement. La liste est enregistrée dans le projet (`custom_tx`) et disponible à la prochaine ouverture du projet.

Définir et envoyer un message

1. Cliquer sur le bouton **Personnalisé** de la ligne d'envoi : la fenêtre « Messages personnalisés » s'ouvre.
2. Faire un **clic droit** dans le tableau et choisir **Ajouter**.
3. Dans la boîte « Définir le message », saisir ID (hex), DLC, données (hex), cycle [ms] et cocher si besoin **Cyclique** ; valider avec **OK**. Les saisies invalides (p. ex. DLC non autorisé ou trop d'octets) sont refusées avec un message, rien n'est enregistré.
4. Clic droit sur la ligne et choisir **Envoyer** : sans la case Cyclique, le message est envoyé une fois ; avec la case, l'envoi cyclique démarre (état : « envoi cyclique »). Le même élément de menu s'appelle alors **Arrêt**.

Champ	Signification
ID (hex)	ID CAN en hexadécimal sans préfixe (p. ex. `7DF`). Les ID supérieurs à 7FF sont envoyés automatiquement comme ID étendus (29 bits).
DLC	Longueur des données : 0 à 8 ainsi que 12, 16, 20, 24, 32, 48, 64. Au-delà de 8, l'envoi se fait en CAN FD (matériel et projet doivent le prendre en charge).
Données (hex)	Octets hexadécimaux avec ou sans espaces. Une saisie trop courte est complétée par 00 jusqu'au DLC, une saisie trop longue est une erreur.
Cycle [ms]	Intervalle d'envoi de 1 à 3 600 000 ms ; actif uniquement avec Cyclique.
Cyclique	Décoché : envoi unique ; coché : envoi cyclique jusqu'à l'arrêt.

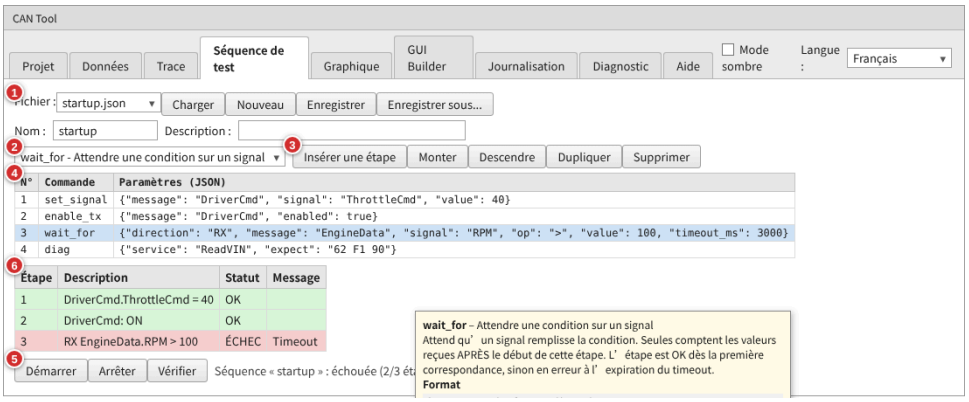
Menu du clic droit : Ajouter (aussi sur la zone libre), Envoyer ou Arrêt, Modifier (aussi par double-clic) et Supprimer. Une entrée en cours est arrêtée avant d'être modifiée ou supprimée. La connexion doit être établie pour envoyer ; à sa fermeture, l'envoi cyclique de toutes les entrées s'arrête. Plusieurs entrées avec le même ID partagent une seule tâche d'envoi. Les messages envoyés apparaissent dans le trace en Tx.

Remarques : 5 000 lignes au maximum sont affichées (les plus anciennes défilent hors de la vue). L'affichage n'est actualisé que tant que la rubrique est visible ; en arrière-plan, l'outil met en tampon jusqu'à 50 000 trames. Pour un enregistrement sans lacune, utilisez la **journalisation**.

3.4 Rubrique « Séquence de test »

Créer, enregistrer, charger et exécuter des séquences de test. **Une seule séquence** est toujours chargée/active ; plusieurs séquences existent sous forme de fichiers dans le dossier `sequences/` du projet.

Capture annotée



Représentation schématique de l'interface ; les numéros renvoient aux explications ci-dessous.

1. Choisir, charger, créer et enregistrer un fichier de séquence de test (une séquence à la fois).
2. Liste des commandes : survoler une entrée affiche l'info-bulle avec format, éléments et exemple.
3. Insérer, déplacer, dupliquer ou supprimer une étape.
4. Liste des étapes : commande et paramètres en JSON, modifiables directement dans le tableau.
5. Démarrer, arrêter et vérifier statiquement la séquence ; le résultat global s'affiche à côté.
6. Tableau des résultats par étape : vert = réussi, rouge = échoué, avec message.

Fichier

Élément de commande	Fonction
Liste de fichiers + Charger	Choisit une séquence dans `sequences/` et la charge dans l'éditeur.
Nouveau	Séquence vide (non enregistrée tant que vous n'avez pas cliqué sur *Enregistrer*).
Enregistrer / Enregistrer sous ...	Écrit la séquence en JSON. Le dernier nom de fichier enregistré/chargé est noté dans le projet et rechargé à la prochaine ouverture.
Nom / Description	Métadonnées de la séquence.

Modifier les étapes

Les étapes figurent dans un tableau (N°, type, paramètres en JSON). Le **texte des paramètres est directement modifiable** (double-clic). Un JSON invalide est rejeté et l'ancienne valeur est restaurée ; le type d'une étape est fixe.

Élément de commande	Fonction
Liste déroulante des commandes	Choisit le type d'étape. **Lorsque la souris survole une entrée, une infobulle apparaît** avec le format, la description de tous les éléments et un exemple ; en dessous figure en plus la description courte de la commande sélectionnée.
Ajouter une étape	Insère la commande avec un modèle après la ligne sélectionnée (ou à la fin).
Monter / Descendre / Dupliquer / Supprimer	Déplacer, copier ou supprimer l'étape sélectionnée.
Sélection du signal + Appliquer à l'étape	Liste tous les signaux de la DBC sous la forme `Message.Signal` (avec complétion à la saisie) et inscrit le message (et, pour `set_signal`, `wait_for`, `check`, aussi le signal) dans l'étape sélectionnée.
Infobulle dans le tableau	L'infobulle de la commande apparaît aussi au-dessus de la colonne Type d'une étape.

Vérifier et exécuter

Élément de commande	Fonction
Vérifier	Contrôle statique : types d'étapes, opérateurs, messages et signaux inconnus (par rapport à la DBC).
Démarrer	Exécute la séquence (uniquement si la connexion est établie). Une vérification est effectuée automatiquement avant le démarrage. Pendant l'exécution, l'éditeur est verrouillé.
Arrêter	Interrompt la séquence (y compris un `wait` en cours) ; statut *aborted*.
Tableau des résultats	Par étape : numéro (boucles : p. ex. `3.2.1`), description, statut (vert = ok, rouge = fail/error) et message (p. ex. valeur mesurée, octets de la réponse).

Règles d'exécution : les étapes s'exécutent les unes après les autres. La **première étape en échec met fin à la séquence** (statut *failed*). À la fin, la ligne d'état affiche p. ex. *startup : passed (10/10 étapes ok)*. Les messages démarrés cycliquement continuent après la fin, sauf s'ils sont arrêtés par `enable_tx`.

Aperçu des commandes

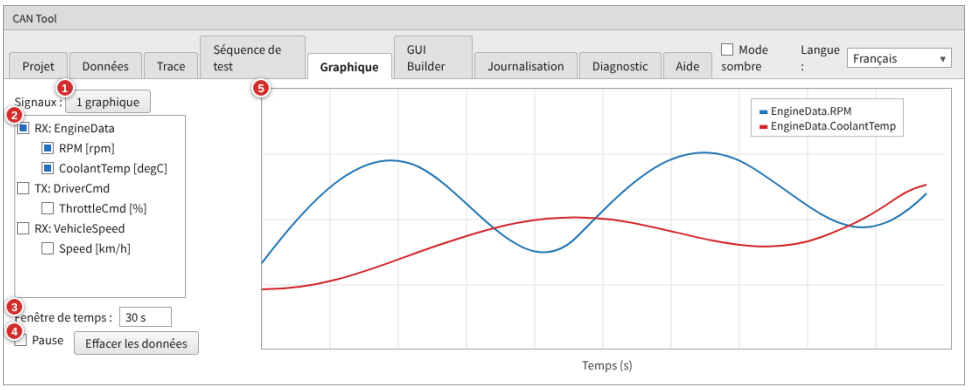
Commande	Titre	Description courte
`set_signal`	Définir la valeur d'un signal	Définit la valeur physique d'un signal dans un message émis. La nouvelle valeur part sur le bus avec la prochaine trame (cyclique ou unique).
`enable_tx`	Activer/désactiver l'émission cyclique	Démarre ou arrête l'émission cyclique d'un message avec son cycle configuré.
`set_cycle`	Modifier le temps de cycle	Modifie le temps de cycle d'un message émis (effet immédiat, même pendant l'émission cyclique).
`send_once`	Émettre un message une fois	Émet exactement une trame avec les valeurs de signaux actuelles, indépendamment de l'émission cyclique.
`send_raw`	Envoyer une trame brute	Envoie une trame brute (ID + données) sur le bus CAN, une fois ou cycliquement. Avec cyclic=true et enabled=false, l'envoi cyclique de cet ID est arrêté.
`wait`	Attendre une durée fixe	Met la séquence en pause pour une durée fixe. L'émission cyclique et la réception continuent.
`wait_for`	Attendre une condition sur un signal	Attend qu'un signal remplisse la condition. Seules comptent les valeurs reçues APRÈS le début de cette étape. L'étape est OK dès la première correspondance, sinon en erreur à l'expiration du timeout.
`check`	Vérifier la valeur d'un signal	Vérifie immédiatement la dernière valeur reçue d'un signal (sans attente). L'étape échoue si la condition n'est pas remplie ou si aucune valeur n'a été reçue depuis le démarrage de la séquence.
`diag`	Service de diagnostic issu de la CDD	Envoie par ISO-TP un service de diagnostic défini dans la CDD et évalue la réponse.
`diag_raw`	Request de diagnostic en hex	Envoie par ISO-TP une request UDS quelconque (octets hex) et évalue la réponse.
`loop`	Boucle	Exécute plusieurs fois de suite les étapes qu'elle contient.
`comment`	Commentaire	Ne fait rien ; sert uniquement à documenter la séquence.

La description complète de chaque commande, avec format, éléments et exemple, se trouve dans l'annexe A.

3.5 Rubrique « Graphique »

Cette rubrique affiche en direct l'évolution des signaux sous forme de courbes (axe du temps : secondes relatives à maintenant, gauche = passé, droite = maintenant). Le commutateur au-dessus de la sélection bascule entre **1 graphique** et **n graphiques**.

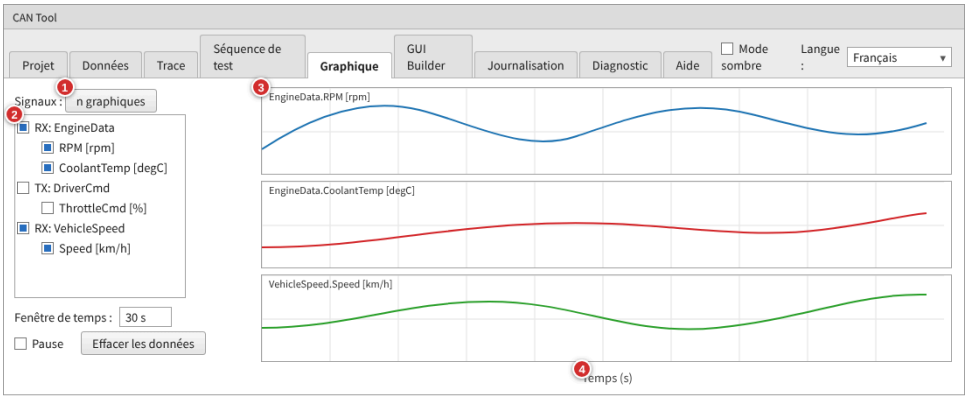
Capture annotée



Représentation schématique de l'interface ; les numéros renvoient aux explications ci-dessous.

1. Commutateur 1 graphique / n graphiques (ici : 1 graphique).
2. Sélection des signaux groupée par message ; la case du message sélectionne tous ses signaux, unité entre crochets.
3. Fenêtre de temps en secondes.
4. Pause fige l'affichage, Effacer les données supprime les courbes.
5. Diagramme commun avec légende ; axe du temps en secondes relatives à maintenant.

Capture annotée



Représentation schématique de l'interface ; les numéros renvoient aux explications ci-dessous.

1. Le commutateur est sur n graphiques.
2. La sélection reste groupée par message ; ici trois signaux sont choisis.
3. Chaque signal a son propre graphique avec son axe de valeurs, intitulé Message.Signal et unité.
4. Axe du temps commun : zoom et déplacement agissent sur tous les graphiques ; graduations seulement en bas.

Sélection groupée par message

La sélection liste tous les signaux choisis dans la rubrique Données, groupés par sens et par message (`RX` = reçu, `TX` = envoyé). Sous chaque message figurent ses signaux avec leur unité entre crochets. La case du message a trois états : tous les signaux sélectionnés, sélection partielle (état intermédiaire) ou aucun. Un clic sélectionne tous les signaux du message ou les retire.

Affichage « 1 graphique »

Tous les signaux sélectionnés sont tracés dans un seul diagramme avec légende (couleurs). Avantage : comparaison directe des courbes. Si les plages de valeurs sont très différentes, les petits signaux paraissent plats - « n graphiques » convient alors mieux.

Affichage « n graphiques »

Chaque signal sélectionné reçoit son propre diagramme avec son propre axe de valeurs, intitulé Message.Signal avec unité ; les diagrammes sont empilés. Tous partagent la même **base de temps** : zoomer ou déplacer l'axe du temps dans un diagramme agit sur tous, et les graduations du temps n'apparaissent qu'en bas.

Éléments de commande

Élément	Fonction
Commutateur 1 graphique / n graphiques	Change le mode d'affichage ; la vue est reconstruite immédiatement. Le réglage est enregistré dans le projet (`graph_multi`).

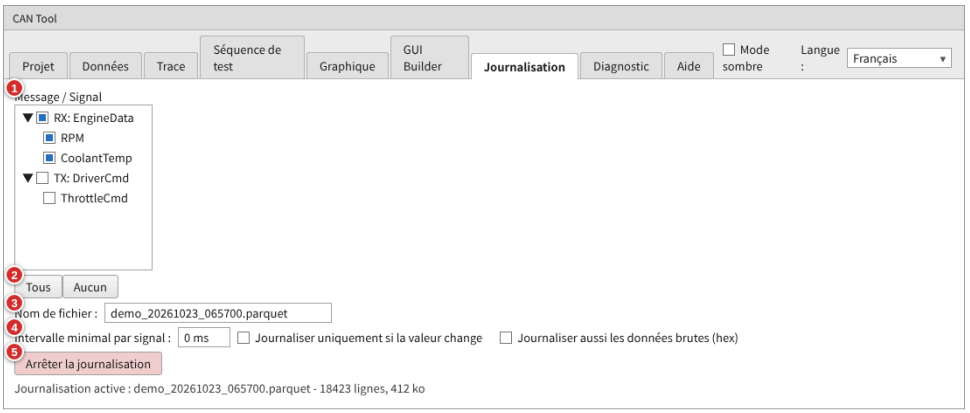
Sélection des signaux	Cocher = afficher la courbe. La case d'un message sélectionne ou retire tous ses signaux. La sélection est enregistrée dans le projet (`graph`).
Fenêtre de temps	Durée visible en secondes (1 à 3600).
Pause	Fige l'affichage ; les données continuent d'être collectées en arrière-plan.
Effacer les données	Supprime toutes les courbes collectées.
Diagramme	Souris : zoom avec la molette, déplacement en maintenant le bouton. En « n graphiques », cela s'applique à tous les diagrammes ensemble.

Remarques : jusqu'à 50 000 points sont conservés par signal. Seuls les signaux cochés dans la rubrique Données sont enregistrés. Les signaux n'apparaissent qu'une fois des données reçues/émises. L'affichage n'est actualisé que si la rubrique est visible.

3.6 Rubrique « Journalisation »

Signaux résolus : lorsqu'une DBC, A2L ou CDD est chargée, le fichier Parquet contient non seulement le message en hexadécimal mais aussi les signaux avec **valeur brute, hexadécimal brut, valeur physique, texte (table de valeurs) et unité**. Colonnes du fichier de signaux : `timestamp, direction, message, signal, value, source, can_id, raw, raw_hex, text, unit, message_hex`. `source` vaut `CAN` (messages DBC), `XCP` (valeurs A2L lues dans la rubrique XCP) ou `UDS` (paramètres de réponse décodés de la rubrique Diagnostic, issus de la CDD) pendant l'enregistrement. Le fichier brut (`_raw.parquet`) contient en plus la colonne `signals` avec tous les signaux de la trame en texte, p. ex. `RPM=0x1F40/2000 rpm`.

Capture annotée



Représentation schématique de l'interface ; les numéros renvoient aux explications ci-dessous.

1. Choix des messages et signaux à enregistrer (filtre parmi les données sélectionnées).
2. Tout sélectionner ou tout désélectionner.
3. Nom du fichier Parquet (dans le dossier logs/ du projet).
4. Options : seulement en cas de changement, intervalle minimal par signal, données brutes en plus.
5. Démarrer ou arrêter l'enregistrement ; la ligne d'état affiche le nombre de lignes et la taille du fichier.

Enregistre les signaux sélectionnés au format **Parquet** (nécessite `pyarrow`). Les fichiers se trouvent dans le dossier `logs/` du projet.

Élément de commande	Fonction
Arborescence des données	Affiche les messages/signaux choisis dans la rubrique Données. **Filtre :** seuls les signaux cochés sont enregistrés (la case sur le message coche/décoche tous ses signaux). La sélection est enregistrée dans le projet.
Tous / Aucun	Sélectionner ou désélectionner tous les signaux.
Nom de fichier	Prérempli avec *Projet_Date_Heure.parquet* ; l'extension est ajoutée si nécessaire.
Intervalle minimal par signal	Au plus une entrée par signal et par période (ms) - réduit le volume de données pour les messages rapides. 0 = chaque trame.
Uniquement si la valeur change	Un signal n'est écrit que si sa valeur diffère de la dernière valeur écrite.
Trames brutes	Écrit en plus *<nom>_raw.parquet* avec **toutes** les trames du bus (indépendamment de la DBC et de la sélection de signaux, y compris les trames d'erreur). Colonnes : temps, direction, canal, ID, étendu, FD, BRS, erreur, DLC, données (binaire), données (texte hex), nom du message. L'enregistrement est donc possible sans sélection de signaux. **ID des données brutes** limite les trames, p. ex. `100, 7E0-7EF` (hex, vide = toutes).
Démarrer / arrêter la journalisation	Démarre ou termine l'enregistrement (uniquement si connecté). À la déconnexion ou au changement de projet, l'enregistrement s'arrête automatiquement et le fichier est correctement refermé.
Ligne d'état	Nom du fichier, lignes écrites, taille du fichier.

Format de fichier

Colonne	Type	Contenu
timestamp	float64	Horodatage (secondes epoch)
direction	string	`RX` ou `TX`
message	string	Nom du message
signal	string	Nom du signal
value	float64	Valeur physique

Le format long (une ligne par valeur de signal) évite les lacunes en cas de temps de cycle différents. Les données sont mises en tampon et écrites sur le disque toutes les 5 secondes ou à partir de 20 000 lignes (compression zstd).

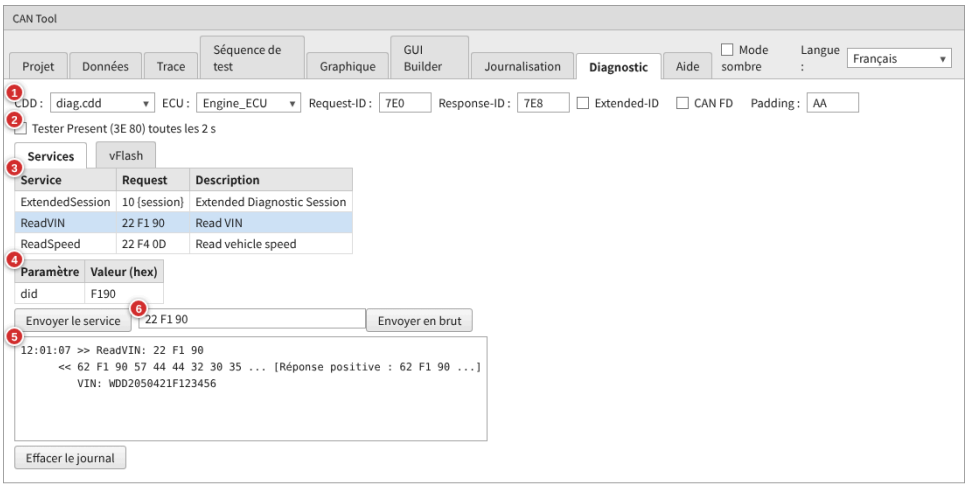
Exploitation en Python (voir aussi l'annexe B) :

```
import pandas as pd
df = pd.read_parquet("projects/demo/logs/demo_20261020_071500.parquet")
wide = df.pivot_table(index="timestamp", columns=["direction", "message", "signal"], values="value")
```

3.7 Rubrique « Diagnostic »

Services de diagnostic (UDS) via **ISO-TP** (ISO 15765-2) sur CAN ou CAN FD. Les services proviennent de la CDD convertie ; des requests quelconques en hex peuvent en outre être envoyées.

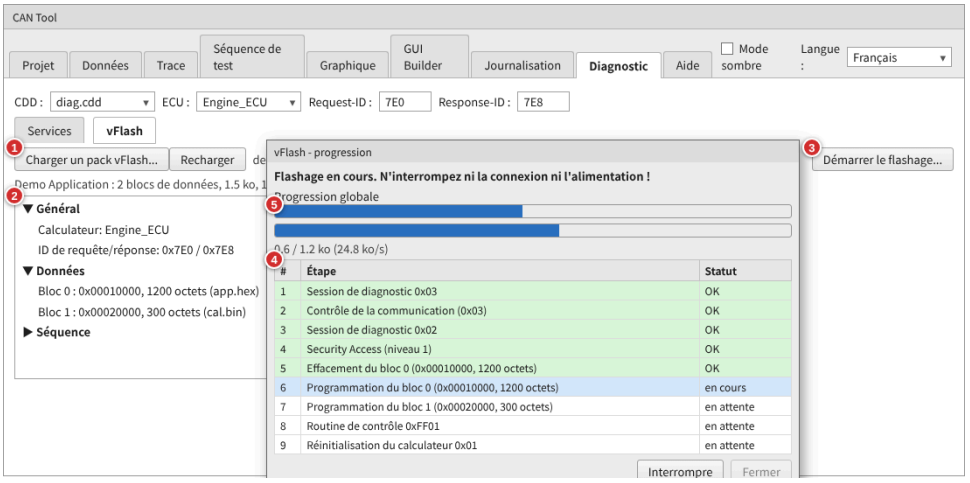
Capture annotée



Représentation schématique de l'interface ; les numéros renvoient aux explications ci-dessous.

1. Configuration du diagnostic : CDD, ECU, ID de requête/réponse, ID étendu, CAN FD, padding.
2. Tester Present (3E 80) est envoyé automatiquement toutes les 2 s.
3. Services de la CDD avec modèle de requête et description.
4. Paramètres du service choisi (valeurs hexadécimales modifiables), puis « Envoyer le service ».
5. Journal des réponses : requêtes envoyées, réponses (positives/négatives) et valeurs décodées.
6. Envoyer une requête brute en hexadécimal (p. ex. 22 F1 90), sans définition CDD.

Capture annotée



Représentation schématique de l'interface ; les numéros renvoient aux explications ci-dessous.

1. Charger un pack vFlash (.vflashpack/.zip, manifeste, HEX/S19/BIN) ; « Recharger » relit le fichier.
2. Interprétation du pack : Général (calculateur, ID, Seed&Key), Données (blocs avec adresse et longueur), Séquence (étapes) et Remarques.
3. Démarrer le flashage : après une demande de confirmation, la séquence s'exécute sur le matériel sélectionné.
4. Fenêtre de progression : statut par étape (en attente, en cours, OK, ÉCHEC, ignorée) avec détails.
5. Progression globale, progression du bloc de données en cours avec débit, et journal ; « Interrompre » arrête entre deux blocs.

Configuration (ligne du haut)

Champ	Signification
CDD	Description de diagnostic convertie, issue du cache du projet.
ECU	Calculateur issu de la CDD ; un changement reprend ses adresses.
Request-ID / Response-ID	IDs CAN en hex (testeur -> ECU / ECU -> testeur). Préremplis depuis la CDD, modifiables ici.
Extended-ID	Adressage 29 bits.
CAN FD	ISO-TP avec trames FD (jusqu'à 64 octets par trame, avec commutation de débit). Doit correspondre au partenaire.
Padding	Octet de remplissage en hex (par défaut AA) ; vide = sans remplissage en CAN classique.
Tester Present	Envoie `3E 80` toutes les 2 s (sans réponse) pour maintenir une session de diagnostic ouverte.

Onglet vFlash	Charge un pack vFlash (ou manifeste/HEX/S19/BIN), affiche l'interprétation et exécute la séquence de flashage sur le matériel sélectionné, avec une fenêtre de progression (Annexe C).

Envoyer des services

1. Sélectionner le service dans la liste (nom, request, description).
2. Si le service a des paramètres, saisir la valeur dans le tableau *Paramètre / Valeur (hex)* (paramètres fictifs de la request).
3. **Envoyer le service**. L'envoi se fait en arrière-plan ; l'interface reste utilisable.
4. Le **journal** affiche la request (>>), la réponse (<<) avec son évaluation (positive/négative, pour les réponses négatives NRC avec texte explicite) ainsi que les paramètres de réponse décodés de la CDD (p. ex. *VIN : WDD...*).

Envoyer en brut : saisir la request sous forme d'octets hex (p. ex. `22 F1 86`) et appuyer sur Entrée ou utiliser le bouton.

Détails techniques

- Segmentation : Single Frame, First Frame, Consecutive Frames et Flow Control, jusqu'à 4095 octets (plus long via le First Frame étendu).
- À la réception de réponses plus longues, l'outil envoie un Flow Control avec Blocksize 0 et STmin 0.
- Timeout de réponse 2 s ; avec *ResponsePending* (NRC 0x78), l'attente se prolonge jusqu'à 10 s.
- Les requests sans réponse (sous-fonction avec bit 7, p. ex. `3E 80`) sont journalisées comme envoyées.
- Dans les séquences de test, `diag` et `diag_raw` utilisent la même configuration d'adresses.

3.8 Barre des rubriques : mode sombre et langue

Sur le bord droit de la barre des rubriques (à côté des onglets) se trouvent deux éléments de commande valables pour toutes les rubriques.

Capture annotée



Représentation schématique de l'interface ; les numéros renvoient aux explications ci-dessous.

1. Mode sombre : bascule entre l'apparence claire et sombre (mémorisé).
2. Langue : liste déroulante de tous les fichiers de langue disponibles ; le changement est immédiat, aide comprise.

Élément de commande	Fonction
Mode sombre	La case bascule entre l’apparence claire et sombre. Sont concernés l’interface, la trace, le graphique, le tableau des résultats de la séquence et cette aide. Le réglage est enregistré et utilisé au prochain démarrage.
Langue	Menu déroulant avec Deutsch (langue principale), English, Français, Español, chinois (mandarin, simplifié) et thaï. Le changement prend effet immédiatement, sans redémarrage. Le choix est enregistré.

Qu’est-ce qui est traduit ?

- Tous les libellés, boutons, en-têtes de tableaux, infobulles, boîtes de dialogue et messages de l’interface.
- La référence des commandes de la séquence de test (infobulles de la liste déroulante, annexe A) et tous les chapitres de cette aide.
- Les messages d’erreur et d’avertissement des composants du programme (p. ex. conversion, connexion, exécution de séquence).
- Ne sont **pas** traduites les données de vos fichiers : noms de messages et de signaux, unités et tables de valeurs de la DBC/CDD, noms de fichiers, noms de projets et contenu des fichiers de séquence.
- Les lignes de journal déjà écrites (journal de diagnostic, tableau des résultats d’une séquence) conservent la langue dans laquelle elles ont été créées.

Pour le chinois et le thaï, une police contenant les caractères correspondants doit être présente sur l’ordinateur (c’est le cas sur les installations Windows, macOS et Linux courantes).

Fichiers de langue

Chaque langue se trouve dans un fichier distinct du dossier `lang/` (YAML ; JSON est également possible) :

Fichier	Langue
`lang/de.yaml`	Deutsch - langue principale et langue de repli : si un texte manque dans une langue, le texte allemand est utilisé.
`lang/en.yaml`	English
`lang/fr.yaml`	Français
`lang/es.yaml`	Español
`lang/zh.yaml`	Chinois (mandarin, caractères simplifiés)
`lang/th.yaml`	Thaï

Structure de chaque fichier : sections `meta` (nom affiché, ordre dans la liste déroulante), `ui` (interface), `core` (messages), `steps` (référence des commandes) et `help` (chapitres de cette aide). Les paramètres fictifs entre accolades (p. ex. `{name}`) et les paramètres fictifs entre doubles crochets dans les textes d’aide ne doivent pas être modifiés.

Ajouter ou modifier une langue

1. Copier `lang/de.yaml` et l’enregistrer sous `lang/<code>.yaml` (p. ex. `it.yaml`).
2. Dans la section `meta`, définir le nom affiché (`name`) et la position (`order`) ; traduire tous les textes.
3. Vérifier avec `python tools/check_lang.py it` : l’outil signale les clés manquantes, les paramètres fictifs divergents et une structure HTML endommagée.
4. Redémarrer le programme - la langue apparaît automatiquement dans la liste déroulante.

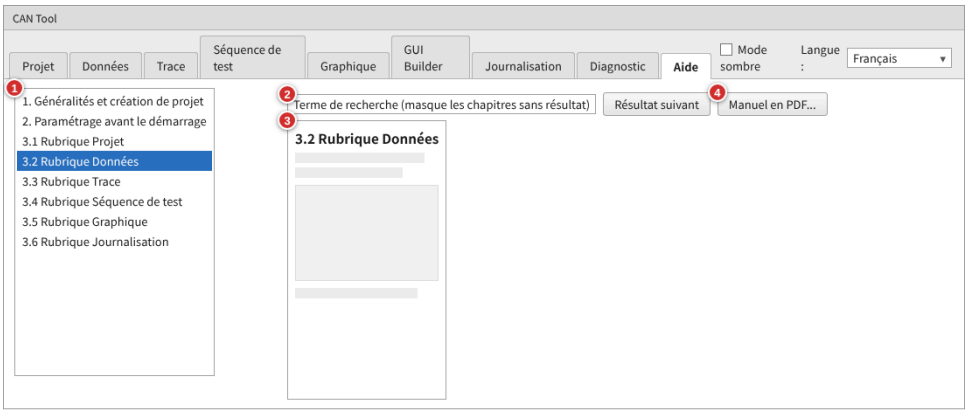
Ligne de commande

Les sorties de `main.py` sont elles aussi traduites : `python main.py --lang en --list`. Sans `--lang`, la dernière langue choisie dans la GUI est utilisée.

3.9 Rubrique « Aide »

Cette rubrique. À gauche les chapitres, à droite le texte.

Capture annotée



Représentation schématique de l'interface ; les numéros renvoient aux explications ci-dessous.

1. Liste des chapitres de l'aide.
2. Champ de recherche : masque les chapitres sans résultat ; « Résultat suivant » passe au suivant.
3. Texte du chapitre avec figures annotées.
4. Manuel en PDF : exporte l'aide dans toutes les langues disponibles (avec page de garde et date d'édition) et propose de l'enregistrer.

- **Champ de recherche** : masque les chapitres où le terme n’apparaît pas et surligne la première occurrence dans le chapitre sélectionné. **Résultat suivant** passe à l’occurrence suivante. Un champ vide affiche de nouveau tous les chapitres.
- Les renvois dans le texte (p. ex. vers l’annexe) sont cliquables.
- Les descriptions des commandes correspondent aux infobulles de la liste déroulante des commandes de la rubrique Séquence de test.
- L’aide s’affiche dans la langue choisie (chapitre 3.8) et est remplacée immédiatement lors du changement de langue ; le basculement clair/sombre s’applique aussi ici.

3.10 Rubrique « XCP »

Mesure et réglage de variables du calculateur via **XCP on CAN** à partir d'un **fichier A2L** (ASAP2). L'A2L est converti comme DBC et CDD : les sources se trouvent dans le dossier `a2l/` du projet ; à l'exécution, seul le cache `cache/<name>.a2l.json|yaml` est lu.

Mise en place

1. Copier l'A2L dans le projet avec **Importer** (onglet Projet) ; il est converti automatiquement (`python main.py --convert demo` fonctionne aussi sans GUI).
2. Connecter le CAN (onglet Projet). XCP utilise la même interface.
3. Onglet XCP : choisir l'A2L et vérifier l'**ID maître** (commandes vers le calculateur) et l'**ID esclave** (réponses). Ils sont préremplis depuis la partie XCP de l'A2L (`IF_DATA XCP / XCP_ON_CAN`) et modifiables.
4. **Connecter XCP**. Le journal affiche toutes les commandes (->) et réponses (<-) en hexadécimal.

Élément	Fonction
Liste de gauche	Toutes les mesures (MEASUREMENT) et tous les paramètres (CHARACTERISTIC de type VALUE) de l'A2L ; filtre par nom ; double-clic ou *Ajouter au suivi* pour les reprendre.
Suivi	Affiche pour chaque entrée la valeur brute et la valeur physique avec unité. La sélection est enregistrée avec le projet.
Lecture cyclique	Lit toutes les entrées suivies à l'intervalle réglé (ms). Interrogation par SHORT_UPLOAD ; les listes DAQ ne sont pas utilisées.
Lire une fois	Lit le suivi une seule fois.
Écrire la valeur	Écrit la valeur saisie (physique ; pour les conversions textuelles aussi le texte, p. ex. *RUN*) dans la ligne choisie (SET_MTA + DOWNLOAD) puis la relit.

Interprétation de l'A2L

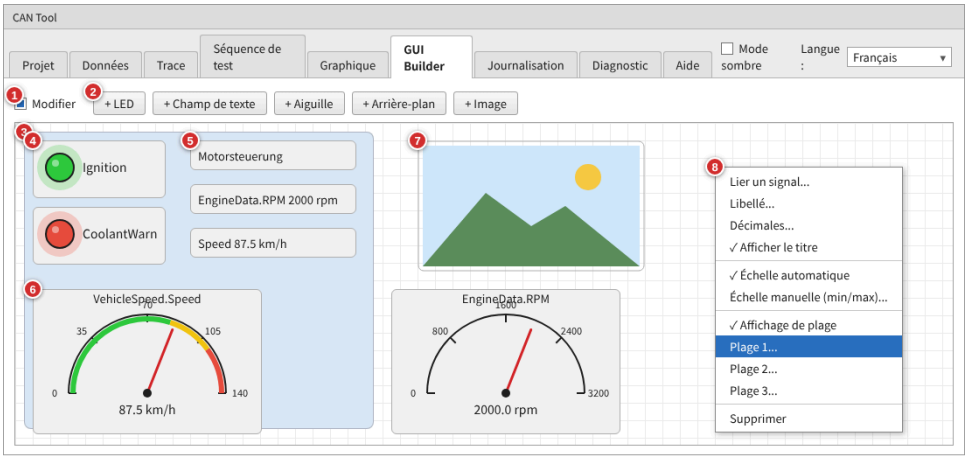
- Types UBYTE, SBYTE, UWORD, SWORD, ULONG, SLONG, A_UINT64, A_INT64, FLOAT32_IEEE, FLOAT64_IEEE ; ordre des octets selon la réponse CONNECT du calculateur.
- Conversion IDENTICAL, LINEAR, RAT_FUNC (sans termes quadratiques) et TAB_VERB (valeurs textuelles) ; BIT_MASK est masqué à la lecture.
- Les objets non pris en charge (courbes, cartographies, tableaux, mesures sans adresse) apparaissent grisés.

Avec le matériel *Simulation*, l'outil démarre un esclave XCP simulé reprenant l'A2L du projet (démo : `projects/demo/a2l/demo.a2l`). Non inclus : DAQ/STIM, seed&key, pages de calibration et programmation flash. Qu'un vrai calculateur autorise tous les accès dépend de ses niveaux de protection.

3.11 Rubrique « GUI Builder »

Créez ici votre propre affichage à partir d'éléments simples pour des signaux existants. Les éléments servent uniquement à l'**affichage** ; il n'y a pas de saisie de valeur. On peut lier les signaux sélectionnés dans la rubrique Données (reçus ou envoyés). La disposition et les liaisons sont enregistrées dans le projet (`builder`).

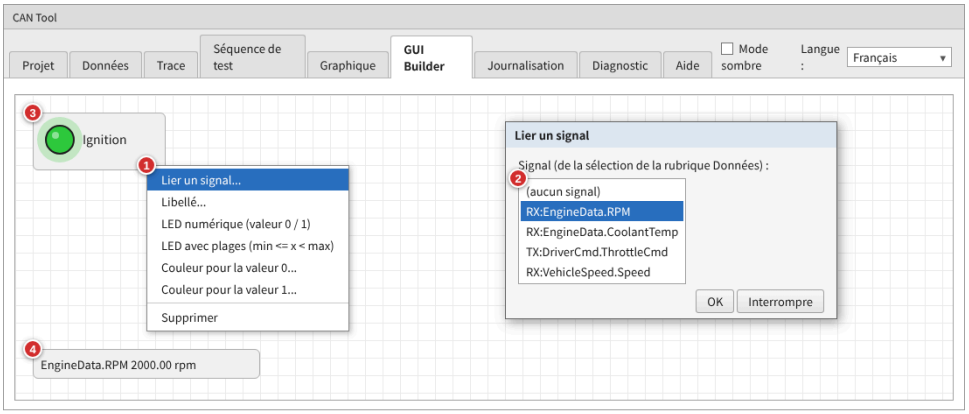
Capture annotée



Représentation schématique de l'interface ; les numéros renvoient aux explications ci-dessous.

1. Case Modifier : grille, ajout, déplacement et redimensionnement possibles.
2. Boutons pour ajouter LED, champ de texte, aiguille, arrière-plan et image.
3. Élément d'arrière-plan à couleur au choix : regroupe visuellement les éléments et se place derrière eux.
4. LED : numériques (valeur 0 / 1) et avec plages (couleur selon la plage min <= x < max).
5. Champs de texte : à gauche sans signal (texte simple), à droite avec signal (valeur et unité en direct).
6. Aiguille à échelle de 180 degrés, automatique ou manuelle, avec affichage de plage en option.
7. Cadre d'image : l'image choisie est adaptée automatiquement au cadre.
8. Menu du clic droit de l'aiguille : signal, titre, échelle, affichage de plage et supprimer.

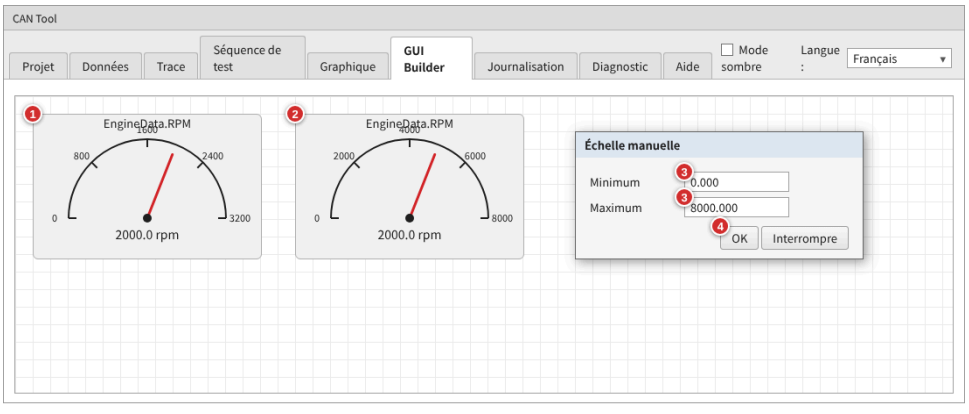
Capture annotée



Représentation schématique de l'interface ; les numéros renvoient aux explications ci-dessous.

1. Clic droit sur la LED : lier un signal, libellé, mode numérique/plages, couleurs, supprimer.
2. Sélection : signaux de la rubrique Données en RX:/TX:Message.Signal, ou (aucun signal).
3. LED : couleur selon la valeur du signal (numérique 0 / 1 ou selon la plage 1 à 3).
4. Champ de texte avec signal : texte, valeur et unité ; décimales réglables.

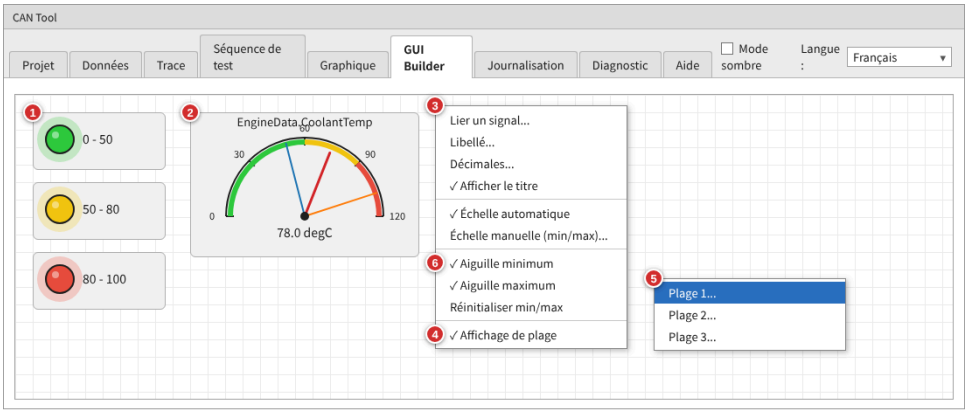
Capture annotée



Représentation schématique de l'interface ; les numéros renvoient aux explications ci-dessous.

1. Aiguille à échelle automatique : bornes arrondies, ici 0 à 3200.
2. Aiguille à échelle manuelle, ici 0 à 8000.
3. Boîte Échelle manuelle : saisir minimum et maximum.
4. OK applique l'échelle (seulement si minimum < maximum), Annuler abandonne.

Capture annotée



Représentation schématique de l'interface ; les numéros renvoient aux explications ci-dessous.

1. Trois LED avec plages : chaque plage de valeurs (min <= x < max) détermine la couleur.
2. Aiguille avec affichage de plage : trois plages colorées sur l'échelle (plage 1 à 3).
3. Menu : Afficher le titre active ou désactive le libellé.
4. Menu : activer ou désactiver l'affichage de plage.
5. Plage 1 à 3 : chacune avec minimum, maximum (règle minimum <= x < maximum), couleur et activation.
6. Menu : deuxième et troisième aiguilles optionnelles pour le minimum (bleu) et le maximum (orange), et réinitialisation.

Éléments

Élément	Fonction
LED	Montre l'état d'un signal par une couleur. **Numérique** : la valeur 0 et la valeur 1 ont chacune une couleur au choix. **Plages** : comme pour l'échelle de l'aiguille, trois plages (min <= x < max) avec une couleur chacune ; si la valeur est dans aucune plage ou absente, la LED est grise.
Champ de texte sans signal	Champ de texte simple au contenu libre, p. ex. titre ou libellé.
Champ de texte avec signal	Affiche le texte et la valeur actuelle du signal avec son unité ; mis à jour en continu. Les décimales sont réglables.
Aiguille	Instrument à aiguille avec échelle sur 180 degrés (demi-cercle) : minimum à gauche, maximum à droite ; la valeur numérique avec unité s'affiche en dessous.
Arrière-plan	Simple rectangle coloré (couleur au choix) sans autre fonction. Il regroupe visuellement d'autres éléments et se trouve toujours derrière les éléments fonctionnels, mais au-dessus de l'arrière-plan standard.
Image	Cadre d'image avec une image au choix (PNG, JPG, BMP, GIF, SVG). L'image est adaptée automatiquement au cadre (le rapport des côtés est conservé) et suit chaque changement de taille.

Utilisation

Élément de commande	Fonction
Modifier	Coché : grille visible ; les éléments peuvent être ajoutés, déplacés, redimensionnés et supprimés. Décoché : l'affichage est verrouillé (consultation seule).
+ LED / + Champ de texte / + Aiguille	Ajoute un nouvel élément (aussi par clic droit sur la zone libre, à la position de la souris).
Glisser	Déplace un élément ; le coin inférieur droit modifie sa taille.
Clic droit sur un élément	Menu : lier un signal (ou « aucun signal »), libellé ou texte, seuil et couleur de la LED, décimales, échelle (aiguille), supprimer.

Aiguille et échelle

L'échelle a deux modes. **Automatique** (par défaut) : la plage s'élargit avec les valeurs observées et est arrondie à des nombres ronds (p. ex. 0 à 100) ; graduations et nombres s'adaptent. **Manuel** : par **clic droit** sur l'aiguille puis **Échelle manuelle (min/max)**, on fixe minimum et maximum (le minimum doit être inférieur au maximum). La case **Échelle automatique** du même menu rétablit le mode automatique. Les valeurs hors échelle placent l'aiguille en butée.

Démarrage rapide

1. Laisser la case **Modifier** cochée (grille visible).
2. Cliquer sur **+ LED**, **+ Champ de texte** ou **+ Aiguille** ; l'élément apparaît sur la zone de dessin (aussi par clic droit sur la zone libre, à la position de la souris).
3. **Clic droit** sur l'élément, choisir **Lier un signal** et sélectionner un signal dans la liste.
4. Régler libellé, seuil de la LED, décimales ou échelle et positionner l'élément par glisser ; le coin inférieur droit modifie sa taille.
5. Décocher **Modifier** : l'affichage est verrouillé et sert uniquement à la consultation.

Lier un signal

La liste montre les signaux sélectionnés dans la rubrique Données sous la forme `RX:Msg.Signal` ou `TX:Msg.Signal`, plus l'entrée « (aucun signal) » qui supprime la liaison. Si un signal manque, le sélectionner d'abord dans la rubrique Données. Les valeurs apparaissent dès que des données sont reçues ou envoyées ; l'unité provient de la DBC. Lors d'un changement de signal, l'échelle automatique est réinitialisée.

Régler la LED et le champ de texte

Réglage	Effet
Libellé / texte	LED : libellé à côté de la LED (nom du signal si vide). Champ de texte : texte libre ; avec un signal, il précède la valeur.
Mode de la LED	Numérique (valeur 0 / valeur 1) ou plages (plage 1 à 3). Numérique : la valeur 0 donne la couleur 0, toute autre valeur la couleur 1.
Couleurs / plages	Numérique : couleur pour la valeur 0 et la valeur 1. Plages : pour chaque plage activation, minimum, maximum et couleur selon la règle **minimum <= x < maximum** ; la première plage correspondante détermine la couleur. Même réglage que l'affichage de plage de l'aiguille.
Décimales	Nombre de décimales de 0 à 9 pour le champ de texte et l'aiguille.
Titre (aiguille)	La case **Afficher le titre** affiche ou masque le libellé au-dessus de l'aiguille ; la place sert alors à l'échelle.
Affichage de plage (aiguille)	La case **Affichage de plage** colore l'échelle en trois plages au maximum (plage 1 à 3), chacune avec minimum, maximum, couleur et activation selon la règle **minimum <= x < maximum** ; les trois plages apparaissent alors dans le menu.
Aiguille min/max (aiguille)	Avec les cases **Aiguille minimum** et **Aiguille maximum**, l'échelle affiche en plus une fine deuxième aiguille (bleue) et une troisième (orange) pour la plus petite et la plus grande valeur depuis le début de l'observation. **Réinitialiser min/max** recommence ; un changement de signal ou de case réinitialise aussi. Les valeurs ne sont pas enregistrées dans le projet, seulement les cases.

Échelle de l'aiguille en détail

Automatique : tant qu'aucune valeur n'est arrivée, 0 à 100 s'applique. Ensuite l'aiguille mémorise la plus petite et la plus grande valeur observées et arrondit les bornes à des nombres ronds (1, 2, 2,5, 5 fois une puissance de dix) ; si toutes les valeurs sont positives et proches de 0, l'échelle commence à 0. L'échelle grandit mais ne rétrécit jamais. La case **Échelle automatique** du menu du clic droit (décocher puis recocher) relance l'observation. L'échelle affiche cinq graduations avec nombres.

Manuelle : clic droit sur l'aiguille, **Échelle manuelle (min/max)** : saisir minimum et maximum dans la boîte (décimales autorisées). Si le minimum n'est pas inférieur au maximum, un message apparaît et rien n'est appliqué. L'échelle manuelle agit immédiatement, reste enregistrée dans le projet et désactive le mode automatique. Les valeurs hors échelle placent l'aiguille en butée.

Arrière-plan et image

Les éléments sont répartis en couches : arrière-plan standard (zone de dessin), au-dessus les éléments d'arrière-plan, puis les éléments fonctionnels (LED, champ de texte, aiguille, image). Les éléments d'arrière-plan sont automatiquement placés derrière les autres lors de l'ajout, du déplacement et du chargement ; un cadre coloré peut ainsi regrouper plusieurs affichages. L'image choisie est copiée dans le dossier du projet (`images`) pour que le projet reste transférable ; sans image, « Aucune image » s'affiche.

Couleurs de la LED et affichage de plage

Exemple LED numérique : valeur 0 = rouge, valeur 1 = vert. Exemple LED avec plages (température) : plage 1 : 0 à 80 vert, plage 2 : 80 à 100 jaune, plage 3 : 100 à 120 rouge ; en dehors, la LED est grise.

Affichage de plage de l'aiguille : il y a trois plages, chacune avec minimum, maximum et couleur. Une valeur x appartient à une plage si **minimum <= x < maximum** (le minimum est inclus, le maximum non). Exemple : plage 1 : 0 à 80 vert, plage 2 : 80 à 100 jaune, plage 3 : 100 à 120 rouge. Chaque plage peut être activée ou désactivée séparément ; les parties hors de l'échelle sont coupées et le minimum doit être inférieur au maximum. Avec l'échelle automatique, les bandes suivent l'échelle actuelle.

Enregistrement et limites

Éléments, positions, tailles, liaisons et réglages sont enregistrés dans le projet (`builder`). La zone de dessin défile. Le GUI Builder n'affiche que des valeurs ; il n'y a ni éléments de saisie ni de commande. Seuls les signaux de la sélection de la rubrique Données peuvent être affichés.

Remarque : les signaux n'apparaissent qu'après réception ou envoi de données ; jusque-là, le champ de texte et l'aiguille affichent « -- » ou la position zéro et la LED est éteinte.

Annexe A : Référence des commandes de séquence de test

Chaque étape est un objet JSON avec le champ `type` et les éléments indiqués ci-dessous. Les valeurs physiques sont les valeurs après facteur et offset de la DBC.

set_signal - Définir la valeur d'un signal

Définit la valeur physique d'un signal dans un message émis. La nouvelle valeur part sur le bus avec la prochaine trame (cyclique ou unique).

Format

```
{"type": "set_signal", "message": "<message>", "signal": "<signal>", "value": <nombre ou texte>}
```

Éléments

Élément	Type	Description
`message`	texte, obligatoire	Nom du message DBC. S'il est absent de la liste d'émission, il y est ajouté (d'abord sans émission cyclique active).
`signal`	texte, obligatoire	Nom du signal dans le message.
`value`	nombre ou texte, obligatoire	Valeur physique (après facteur/offset, p. ex. 40 pour 40 %). Pour les signaux avec table de valeurs, le texte correspondant est aussi accepté, p. ex. "R". Les valeurs hors de la plage du signal sont limitées.

Exemple

```
{
  "type": "set_signal",
  "message": "DriverCmd",
  "signal": "ThrottleCmd",
  "value": 40
}
```

Remarque : Les signaux non indiqués du message restent inchangés (valeur initiale 0).

enable_tx - Activer/désactiver l'émission cyclique

Démarre ou arrête l'émission cyclique d'un message avec son cycle configuré.

Format

```
{"type": "enable_tx", "message": "<message>", "enabled": <true|false>}
```

Éléments

Élément	Type	Description
`message`	texte, obligatoire	Nom du message DBC (ajouté si nécessaire à la liste d'émission ; cycle issu de la DBC, sinon 100 ms).
`enabled`	true/false, obligatoire	true = émission cyclique, false = arrêter l'émission.

Exemple

```
{
  "type": "enable_tx",
  "message": "DriverCmd",
  "enabled": true
}
```

Remarque : La première trame part immédiatement, puis selon le cycle configuré.

set_cycle - Modifier le temps de cycle

Modifie le temps de cycle d'un message émis (effet immédiat, même pendant l'émission cyclique).

Format

```
{"type": "set_cycle", "message": "<message>", "cycle_ms": <nombre>}
```

Éléments

Élément	Type	Description
`message`	texte, obligatoire	Nom du message DBC.
`cycle_ms`	nombre, obligatoire	Temps de cycle en millisecondes. 0 met l'émission cyclique en pause.

Exemple

```
{
  "type": "set_cycle",
  "message": "DriverCmd",
  "cycle_ms": 20
}
```

send_once - Émettre un message une fois

Émet exactement une trame avec les valeurs de signaux actuelles, indépendamment de l'émission cyclique.

Format

```
{"type": "send_once", "message": "<message>"}
```

Éléments

Élément	Type	Description
`message`	texte, obligatoire	Nom du message DBC.

Exemple

```
{
  "type": "send_once",
  "message": "DriverCmd"
}
```

send_raw - Envoyer une trame brute

Envoie une trame brute (ID + données) sur le bus CAN, une fois ou cycliquement. Avec cyclic=true et enabled=false, l'envoi cyclique de cet ID est arrêté.

Format

```
{"type": "send_raw", "id": "<hex>", "data": "<hex>", "dlc": <n>, "cyclic": <true|false>, "cycle_ms": <n>}
```

Éléments

Élément	Type	Description
`id`	nombre ou texte, obligatoire	ID CAN, hexadécimal (p. ex. "0x100" ou "100").
`data`	texte hex, obligatoire	Données utiles en octets hexadécimaux, p. ex. "00 01 02 03 04 05 06 07".
`dlc`	nombre, optionnel	Longueur des données (0-8, 12, 16, 20, 24, 32, 48, 64). Par défaut : nombre d'octets.
`cyclic`	true/false, optionnel	true = envoi cyclique, false = envoi unique (par défaut).
`cycle_ms`	nombre, optionnel	Durée de cycle en ms avec cyclic=true (par défaut 100).
`enabled`	true/false, optionnel	false arrête l'envoi cyclique de cet ID (seulement avec cyclic=true, par défaut true).
`extended`	true/false, optionnel	true force un ID étendu (par défaut : automatique au-delà de 0x7FF).
`fd`	true/false, optionnel	true force CAN FD (par défaut : automatique si DLC supérieur à 8).

Exemple

```
{
  "type": "send_raw",
  "id": "0x100",
  "data": "00 01 02 03 04 05 06 07",
  "dlc": 8,
  "cyclic": true,
  "cycle_ms": 1000
}
```

Remarque : Les octets manquants sont complétés par 00. ID supérieur à 0x7FF = étendu, DLC supérieur à 8 = CAN FD. Les trames envoyées cycliquement continuent après l'étape.

wait - Attendre une durée fixe

Met la séquence en pause pour une durée fixe. L'émission cyclique et la réception continuent.

Format

```
{"type": "wait", "ms": <nombre>}
```

Éléments

Élément	Type	Description
`ms`	nombre, obligatoire	Durée d'attente en millisecondes.

Exemple

```
{
  "type": "wait",
  "ms": 500
}
```

Remarque : Un clic sur Arrêter interrompt immédiatement une attente en cours.

wait_for - Attendre une condition sur un signal

Attend qu'un signal remplisse la condition. Seules comptent les valeurs reçues APRÈS le début de cette étape. L'étape est OK dès la première correspondance, sinon en erreur à l'expiration du timeout.

Format

```
{"type": "wait_for", "direction": "RX", "message": "<message>", "signal": "<signal>", "op": "<opérateur>", "value": <nombre>, "timeout_ms":
```

Éléments

Élément	Type	Description
`direction`	"RX" ou "TX", obligatoire	Évaluer le message reçu ou émis.
`message`	texte, obligatoire	Nom du message DBC.
`signal`	texte, obligatoire	Nom du signal.
`op`	texte, obligatoire	Comparaison : == != < <= > >=
`value`	nombre, obligatoire	Valeur de comparaison (physique).
`timeout_ms`	nombre, optionnel	Durée d'attente maximale en ms (par défaut 3000).

Exemple

```
{
  "type": "wait_for",
  "direction": "RX",
  "message": "EngineData",
  "signal": "RPM",
  "op": ">",
  "value": 100,
  "timeout_ms": 3000
}
```

Remarque : L'évaluation est indépendante de la sélection de signaux dans la rubrique « Données ».

check - Vérifier la valeur d'un signal

Vérifie immédiatement la dernière valeur reçue d'un signal (sans attente). L'étape échoue si la condition n'est pas remplie ou si aucune valeur n'a été reçue depuis le démarrage de la séquence.

Format

```
{"type": "check", "direction": "RX", "message": "<message>", "signal": "<signal>", "op": "<opérateur>", "value": <nombre>}
```

Éléments

Élément	Type	Description
`direction`	"RX" ou "TX", obligatoire	Évaluer le message reçu ou émis.
`message`	texte, obligatoire	Nom du message DBC.
`signal`	texte, obligatoire	Nom du signal.
`op`	texte, obligatoire	Comparaison : == != < <= > >=
`value`	nombre, obligatoire	Valeur de comparaison (physique).

Exemple

```
{
  "type": "check",
  "direction": "RX",
  "message": "EngineData",
  "signal": "CoolantTemp",
  "op": "<",
  "value": 150
}
```

Remarque : Juste après le démarrage, il n'y a souvent pas encore de valeur - synchroniser auparavant avec wait_for.

diag - Service de diagnostic issu de la CDD

Envoie par ISO-TP un service de diagnostic défini dans la CDD et évalue la réponse.

Format

```
{"type": "diag", "service": "<nom du service>", "params": {"<paramètre>": <valeur>}, "expect": "<préfixe hex>", "timeout_ms": <nombre>}
```

Éléments

Élément	Type	Description
`service`	texte, obligatoire	Nom du service dans la CDD (rubrique Diagnostic), p. ex. ReadVIN.
`params`	objet, optionnel	Valeurs pour les {Platzhalter} de la request. Nombre = décimal, texte = hexadécimal ("F186").
`expect`	texte hex, optionnel	Début attendu de la réponse, p. ex. "62 F1 90". Vide = toute réponse positive convient.
`timeout_ms`	nombre, optionnel	Timeout de réponse en ms (par défaut 1000). Avec 'ResponsePending' (NRC 0x78), l'attente est prolongée.

Exemple

```
{
  "type": "diag",
  "service": "ExtendedSession",
  "params": {
    "session": 3
  },
  "expect": "50 03"
}
```

Remarque : L'adressage (Request-/Response-ID, FD) provient de la rubrique « Diagnostic ».

diag_raw - Request de diagnostic en hex

Envoie par ISO-TP une request UDS quelconque (octets hex) et évalue la réponse.

Format

```
{"type": "diag_raw", "request": "<octets hex>", "expect": "<préfixe hex>", "timeout_ms": <nombre>}
```

Éléments

Élément	Type	Description
`request`	texte hex, obligatoire	Octets de la request, p. ex. "22 F1 86".
`expect`	texte hex, optionnel	Début attendu de la réponse, p. ex. "62 F1 86". Vide = toute réponse positive convient.
`timeout_ms`	nombre, optionnel	Timeout de réponse en ms (par défaut 1000).

Exemple

```
{
  "type": "diag_raw",
  "request": "3E 00",
  "expect": "7E 00"
}
```

Remarque : Les requests sans réponse (sous-fonction avec bit 7 positionné, p. ex. "3E 80") sont considérées comme OK.

loop - Boucle

Exécute plusieurs fois de suite les étapes qu'elle contient.

Format

```
{"type": "loop", "count": <nombre>, "steps": [ <étape>, <étape>, ... ]}
```

Éléments

Élément	Type	Description
`count`	nombre, obligatoire	Nombre d'itérations.
`steps`	liste, obligatoire	Étapes imbriquées (de tout type, y compris d'autres boucles).

Exemple

```
{
  "type": "loop",
  "count": 3,
  "steps": [
    {
      "type": "set_signal",
      "message": "DriverCmd",
      "signal": "ThrottleCmd",
      "value": 20
    },
    {
      "type": "wait",
      "ms": 200
    }
  ]
}
```

Remarque : Dans le résultat, les étapes internes apparaissent p. ex. sous la forme 3.2.1 (étape 3, itération 2, étape 1).

comment - Commentaire

Ne fait rien ; sert uniquement à documenter la séquence.

Format

```
{"type": "comment", "text": "<texte>"}
```

Éléments

Élément	Type	Description
`text`	texte, optionnel	Commentaire libre.

Exemple

```
{
  "type": "comment",
  "text": "Hier beginnt der Hochlauftest"
}
```

Annexe B : Remarques et limites

- **Précision temporelle** : l'émission cyclique repose sur un timer logiciel (précision de l'ordre de quelques millisecondes), sans déclenchement matériel.
- **DBC** : le multiplexage étendu et les variables d'environnement ne sont pas pris en charge.
- **CDD** : fichiers CANdelaStudio d'origine lus uniquement de façon heuristique (chapitre 2.2).
- **PEAK + CAN FD** : bit timing prédéfini (80 MHz, 500k/2M) ; écarts via `hardware.extra`.
- **Sécurité** : l'émission cyclique ne démarre jamais automatiquement au chargement d'un projet. Sur des bus de véhicule ou de banc d'essai, vérifier la liste d'émission et les valeurs avant de vous connecter.
- **Clavier** : dans les tableaux, modifier par double-clic ; dans le champ de diagnostic brut, Entrée envoie la request.

Tester sans matériel

Le projet de démonstration `demo` utilise la simulation avec calculateur virtuel. Il permet d'essayer toutes les rubriques (émission/réception, trace, graphique, journalisation, séquences *startup* et *driving*, diagnostic) sans matériel. Les tests essentiels s'exécutent avec `python -m unittest discover -s tests -v`.

Annexe C : pack vFlash et manifeste

L'onglet vFlash de la section Diagnostic charge un paquet de flashage, l'interprète et l'exécute avec la séquence de flashage UDS intégrée sur le matériel sélectionné (Vector, PEAK, SocketCAN, simulation). Le logiciel Vector vFlash n'est pas nécessaire.

Sources prises en charge

- `*.vflashpack` ou `*.zip` avec `flashpack.json` (ou `flashpack.yaml`) et les fichiers de données : entièrement pris en charge.
- Pack vFlash authentique sans manifeste : le format interne n'est pas documenté publiquement. Les fichiers HEX, S-record et BIN sont détectés et le projet est lu de façon heuristique. La séquence, les adresses et le Seed&Key doivent généralement être ajoutés via un manifeste.
- Fichier manifeste unique ou image (`.hex`, `.s19`, `.srec`, `.mot`, `.bin`) avec la séquence générique par défaut.

Manifeste flashpack.json

```
{
  "format": "canTool-flashpack-1",
  "name": "Demo Application",
  "ecu": "Engine_ECU",
  "request_id": "0x7E0",
  "response_id": "0x7E8",
  "extended": false,
  "fd": false,
  "security": {
    "type": "xor",
    "level": 1,
    "value": "0xFFFFFFFF"
  },
  "address_format": "0x44",
  "data": [
    {
      "file": "app.hex",
      "name": "Application"
    },
    {
      "file": "cal.bin",
      "address": "0x00020000",
      "name": "Calibration"
    }
  ],
  "steps": [
    {
      "type": "session",
      "session": 3
    },
    {
      "type": "comm_control",
      "control": 3,
      "comm_type": 1
    },
    {
      "type": "session",
      "session": 2
    },
    {
      "type": "security",
      "level": 1
    },
    {
      "type": "erase",
      "block": "all"
    },
    {
      "type": "download",
      "block": "all"
    },
    {
      "type": "check",
      "routine": "FF01"
    },
    {
      "type": "reset",
      "reset": 1
    }
  ]
}
```

Élément	Signification
`name`, `ecu`	Nom affiché et calculateur.
`request_id`, `response_id`, `extended`, `fd`	Adressage ISO-TP ; en son absence, la configuration de diagnostic de la section Diagnostic s'applique.
`security`	Seed&Key : `type` = `none`, `xor`, `add` ou `constant`, plus `level` (niveau du seed, key = niveau + 1) et `value`.
`address_format`	addressAndLengthFormatIdentifier, p. ex. `0x44` = longueur sur 4 octets et adresse sur 4 octets.
`max_chunk`	Limite supérieure des données utiles par requête de transfert (sinon tirée de la réponse à 0x34).
`data`	Liste des fichiers de données (`file`, `name`) ; pour les fichiers BIN, `address` est obligatoire.
`steps`	Séquence propre ; en son absence, la séquence par défaut s'applique.

Étapes de la séquence (steps)

Étape	Paramètres et effet

`session`	`session` : session de diagnostic (0x10).
`comm_control`	`control`, `comm_type` : contrôle de la communication (0x28).
`security`	`level` : demander le seed, calculer et envoyer la key (0x27).
`erase`	`block` (`all` ou numéro), `routine` (par défaut FF00) : effacer la mémoire (0x31).
`download`	`block` : Request Download, Transfer Data, Transfer Exit (0x34, 0x36, 0x37).
`check`, `routine`	`routine`, `data` (hex), `control`, `timeout_ms` : démarrer une routine (0x31).
`request`	`data` (hex), `expect` (premiers octets de réponse attendus).
`reset`	`reset` : réinitialisation du calculateur (0x11).
`wait`, `comment`	`ms` durée d'attente ou `text` comme commentaire.

Séquence par défaut

Sans `steps`, les opérations suivantes sont exécutées dans l'ordre : session 0x03, désactivation de la communication (0x28 03 01), session 0x02, Security Access (si `security` est défini), effacement et programmation de tous les blocs, réinitialisation du calculateur 0x11 01. Cette séquence est générique et non spécifique au constructeur.

Consignes de sécurité

Le flashage efface et écrase la mémoire du calculateur. L'alimentation et la connexion au bus ne doivent pas être interrompues pendant ce temps. Le code contenu dans les paquets n'est jamais exécuté ; les DLL Seed&Key spécifiques au constructeur ne sont pas prises en charge. En cas d'erreur, la séquence s'arrête et les étapes restantes sont ignorées.

Español – Manual

CAN Tool · Fecha de emisión: 2026-10-29

1. Información general y creación de proyectos

2. Preparación de datos antes del inicio

3.1 Pestaña Proyecto

3.2 Pestaña Datos

3.3 Pestaña Trace

3.4 Pestaña Secuencia de prueba

3.5 Pestaña Gráfico

3.6 Pestaña Logging

3.7 Pestaña Diagnóstico

3.8 Barra de pestañas: modo oscuro e idioma

3.9 Pestaña Ayuda

3.10 Pestaña XCP

3.11 Pestaña GUI Builder

Anexo A: Referencia de comandos de la secuencia de prueba

Anexo B: Notas y límites

Anexo C: paquete vFlash y manifiesto

1. Descripción general y creación de proyectos

1.1 Para qué sirve la herramienta

CAN Tool es una herramienta en Python para probar y analizar buses CAN y CAN FD. Funciona, a elección, con el **Vector VN1630A / VN7572** o con el **PEAK-System PCAN-USB Pro FD** (ambos mediante python-can) o, sin hardware, con una **simulación** integrada.

Funciones principales:

- Gestión de varios **proyectos** (cada uno con sus propios archivos DBC, CDD, de secuencias y de log).
- **Conversión de DBC/CDD** a JSON o YAML antes del uso (carga rápida, revisión sencilla).
- **Envío cíclico** y recepción de mensajes, acceso a señales en valores físicos.
- **Trace** de todas las tramas enviadas y recibidas, opcionalmente con decodificación de señales.
- **Secuencias de prueba** (JSON): establecer señales, esperar reacciones, comprobar, ejecutar diagnóstico.
- **Gráfico de líneas** de las señales seleccionadas.
- Apariencia conmutable (**modo oscuro**) y **seis idiomas** (alemán, inglés, francés, español, chino mandarín, tailandés).
- **Logging Parquet** con filtro.
- **Diagnóstico** mediante ISO-TP (ISO 15765-2) con servicios UDS del CDD.

1.2 Flujo de datos de un vistazo

```
dbc/*.dbc  --(conversión)-->  cache/*.dbc.json|yaml  ---+
cdd/*.cdd  --(conversión)-->  cache/*.cdd.json|yaml  ---+--> pestaña "Datos": selección TX / RX
                                                        |        |
                                                        |        +--> Trace   (todas las tramas)
                                                        |        +--> Gráfico (señales elegidas)
                                                        |        +--> Logging (señales elegidas, filtro)
sequences/*.json  ------------------------------------------+--> Secuencia de prueba (establece/comprueba señales, diagnóstico)
```

En tiempo de ejecución solo se lee la **caché** (JSON/YAML). Si cambia un archivo DBC/CDD, se reconvierte automáticamente la próxima vez que se abra el proyecto (suma de verificación del archivo de origen).

1.3 Instalación e inicio

```
pip install -r requirements.txt
python main.py                      # GUI
python main.py --project demo       # GUI con un proyecto concreto
python main.py --list               # listar proyectos
python main.py --convert demo       # solo convertir DBC/CDD (sin GUI)
python main.py --convert demo --format yaml --force
```

Controladores: Vector necesita la *Vector XL Driver Library* (parte de los controladores de Vector), PEAK el controlador *PCAN-Basic*. Sin hardware basta con la simulación.

1.4 Estructura de carpetas

```
can_tool/
  main.py  gui.py  can_manager.py  dbc_manager.py  cdd_manager.py
  iso_tp_manager.py  sequence_runner.py  parquet_logger.py  project_manager.py
  help_content.py  i18n.py
  lang/               archivos de idioma (de.yaml, en.yaml, fr.yaml, es.yaml, zh.yaml, th.yaml)
  tools/check_lang.py comprobación de los archivos de idioma
  projects/
    <nombre_proyecto>/
      project.json      ajustes del proyecto (lo escribe la herramienta)
      dbc/              sus archivos DBC (*.dbc)
      cdd/              sus descripciones de diagnóstico (*.cdd)
      sequences/        secuencias de prueba (*.json)
      cache/            generada automáticamente: DBC/CDD convertidos (JSON o YAML)
      logs/             archivos de log Parquet
```

1.5 Crear un proyecto

En la GUI

1. Abra la pestaña **Proyecto** y haga clic en **Nuevo ...**.
2. Introduzca el nombre del proyecto. Se permiten letras, cifras, guion bajo, guion y espacios (máx. 64 caracteres, el primero debe ser letra o cifra). Si el nombre ya existe, no se sobrescribe nada.
3. La herramienta crea `projects/<name>/` con las subcarpetas `dbc, cdd, sequences, cache, logs` y un `project.json` con valores por defecto (simulación, 500 kbit/s, caché JSON) y cambia al nuevo proyecto.
4. **Importar DBC/CDD ...**: los archivos se copian a `dbc/` o `cdd/` y se convierten de inmediato. Como alternativa, copie los archivos a mano en las carpetas y haga clic en **Reconvertir** (o vuelva a abrir el proyecto).
5. Configure el hardware (véase el capítulo 2.3) y pulse **Conectar**.

Mediante archivo

Cree una carpeta de proyecto con la estructura anterior y coloque en ella un `project.json` con al menos `{}`; los ajustes que falten se completan con valores por defecto. El proyecto de demostración `projects/demo` sirve de plantilla.

Cambiar de proyecto

Selección en la lista de proyectos. Al cambiar (y al salir) se desconecta, se detiene el logging en curso y se guarda el proyecto anterior. Se guardan: hardware, lista de envío con valores de señal y ciclos, selección de recepción, filtros de logging, selección del gráfico, ajustes de diagnóstico y la última secuencia cargada. Tras abrir un proyecto, el envío cíclico está siempre **desactivado** (seguridad en el bus real).

1.6 Flujo de trabajo típico

1. Cree o elija un proyecto, importe el DBC (y el CDD).
2. Configure el hardware y pulse **Conectar**.
3. Pestaña **Datos**: elija mensajes de envío y recepción del DBC, establezca valores de señal y ciclos.
4. Active el envío cíclico; observe el resultado en **Trace** y **Gráfico**.
5. Si es necesario, inicie el **Logging** y/o cree y ejecute una **secuencia de prueba**.
6. **Diagnóstico**: envíe servicios y compruebe las respuestas.

2. Preparación de los archivos antes del inicio

Antes de conectar, deben estar preparados los archivos del proyecto. Este capítulo describe qué debe contener cada archivo y qué datos evalúa la herramienta.

2.1 Archivos DBC (`dbc/*.dbc`)

Archivos DBC estándar de CANdb++, CANdb Editor, SavvyCAN y otros. Se admiten varios archivos DBC por proyecto (su suma forma la base de datos). Codificación UTF-8 o Windows-1252.

Se evalúan:

Elemento DBC	Significado en la herramienta
`BO_`	Mensaje: ID (bit 31 activado = Extended-ID), nombre, longitud (DLC > 8 = CAN FD), emisor.
`SG_`	Señal: bit de inicio, longitud, orden de bytes (Intel/Motorola), signo, factor, offset, mín/máx, unidad, receptores.
`SG_ ... M / m0, m1`	Multiplexado simple (un multiplexor, varios grupos). El multiplexado extendido **no** se admite (advertencia).
`VAL_`	Tabla de valores: texto por valor sin procesar. En la herramienta, como desplegable al enviar y como texto al recibir.
`SIG_VALTYPE_`	Señales de coma flotante (32/64 bits).
`CM_`	Comentarios de mensajes/señales.
`BA_ "GenMsgCycleTime"`	Tiempo de ciclo estándar (ms) de un mensaje; sin indicación, 100 ms.
`BA_ "VFrameFormat"`	Formato de trama; los valores con *FD* identifican mensajes CAN FD.

Ejemplo:

```
VERSION "1.0"
NS_ :
BS_:
BU_: Tester ECU

BO_ 512 EngineData: 8 ECU
 SG_ RPM : 0|16@1+ (0.25,0) [0|8000] "rpm" Tester
 SG_ CoolantTemp : 16|8@1+ (1,-40) [-40|130] "degC" Tester

VAL_ 257 StartButton 0 "Aus" 1 "Ein" ;
BA_DEF_ BO_ "GenMsgCycleTime" INT 0 65535;
BA_ "GenMsgCycleTime" BO_ 512 10;
```

Notas sobre la preparación de datos

- Los nombres de mensajes y señales se usan en las secuencias de prueba: elija nombres descriptivos e inequívocos.
- Nombres de mensaje duplicados en varios archivos DBC: se mantiene la primera definición y la segunda se ignora con una advertencia.
- Mismo ID con nombres distintos: en la recepción se asigna al primer nombre (advertencia).
- Los nodos emisores (`BU_`) controlan en la simulación qué mensajes envía cíclicamente la unidad de control virtual (por defecto: emisor `ECU`, configurable en `sim.traffic_senders`). Valores mín/máx razonables en el DBC dan valores de simulación realistas.
- Las advertencias de la conversión aparecen en la pestaña **Proyecto**, en el campo *Archivos convertidos / Advertencias*.

La caché (`cache/*.dbc.json` o `.yaml`)

Al abrir el proyecto, cada DBC se escribe automáticamente en la caché (formato seleccionable en la pestaña **Proyecto**). Solo se reconvierte si ha cambiado el archivo de origen; **Reconvertir** lo fuerza. Los cambios propios en la caché se sobrescriben en una nueva conversión: las correcciones pertenecen al DBC.

```
{
  "format_version": 1,
  "nodes": ["Tester", "ECU"],
  "messages": {
    "EngineData": {
      "id": 512, "extended": false, "dlc": 8, "fd": false, "sender": "ECU",
      "cycle_time_ms": 10, "comment": "", "multiplexor": null,
      "signals": {
        "RPM": {"start": 0, "length": 16, "byte_order": "little", "signed": false,
                "factor": 0.25, "offset": 0, "min": 0, "max": 8000, "unit": "rpm",
                "receivers": ["Tester"], "choices": {}, "comment": ""}
      }
    }
  },
  "warnings": [], "source": "engine.dbc", "source_sha1": "..."
}
```

2.2 Descripción de diagnóstico (`cdd/*.cdd`)

El CDD describe servicios de diagnóstico. Se convierte, como el DBC, a `cache/*.cdd.json|yaml`.

Limitación importante: solo se admite por completo el formato XML sencillo `<DIAGDATA>` (véase el ejemplo y `projects/demo/cdd/diag.cdd`). Los archivos CDD originales de CANdelaStudio se leen únicamente de forma **heurística** (nombres de servicio y, si se encuentran, bytes del request). En ese caso, revise el archivo de caché generado (pestaña **Proyecto**, advertencias) y complete allí las direcciones, requests y parámetros de respuesta, o hágalo en el formato sencillo.

```
<?xml version="1.0" encoding="UTF-8"?>
<DIAGDATA>
  <ECU name="Engine_ECU" request_id="0x7E0" response_id="0x7E8" extended="false" fd="false">
    <SERVICE name="ExtendedSession" request="10 {session}" description="Select diagnostic session">
      <PARAM name="session" bytes="1" default="03"/>
    </SERVICE>
```

```
    <SERVICE name="ReadVIN" request="22 F1 90" description="Read VIN">
      <RESPONSE prefix_len="3">
        <PARAM name="VIN" offset="0" length="17" type="ascii"/>
      </RESPONSE>
    </SERVICE>
    <SERVICE name="ReadSpeed" request="22 F4 0D" description="Read speed">
      <RESPONSE prefix_len="3">
        <PARAM name="Speed" offset="0" length="1" type="uint" factor="1" unit="km/h"/>
      </RESPONSE>
    </SERVICE>
  </ECU>
</DIAGDATA>
```

Elemento / Atributo	Significado
`ECU name, request_id, response_id`	Nombre de la unidad de control e IDs CAN (tester -> ECU, ECU -> tester), p. ej. `0x7E0`/`0x7E8`.
`ECU extended, fd`	`true`: IDs de 29 bits o CAN FD (ISO-TP con hasta 64 bytes por trama).
`SERVICE name`	Nombre del servicio (se usa en la pestaña Diagnóstico y en el paso de secuencia `diag`).
`SERVICE request`	Bytes del request en hex. Marcadores entre llaves, p. ej. `10 {session}`.
`SERVICE description`	Texto libre, se muestra en la lista de servicios.
`PARAM`	Directamente en SERVICE: parámetro de entrada para un marcador: `name`, `bytes` (ancho), `default` (hex).
`RESPONSE prefix_len`	Número de bytes al inicio de la respuesta que se omiten (ID de servicio + DID, p. ej. 3 en `62 F1 90`).
`RESPONSE/PARAM`	Parámetros de respuesta: `name`, `offset` y `length` en bytes (desde el final del prefijo), `type` = `ascii | uint | int | hex`, opcionalmente `factor`, `offset_val`, `unit`.

La entrada de caché correspondiente (para retocar manualmente):

```
{
  "format_version": 1,
  "ecus": {
    "Engine_ECU": {
      "request_id": "0x7E0", "response_id": "0x7E8", "extended": false, "fd": false,
      "services": {
        "ReadVIN": {
          "request": "22 F1 90", "description": "Read VIN", "params": [],
          "response": {"prefix_len": 3,
                       "params": [{"name": "VIN", "offset": 0, "length": 17, "type": "ascii"}]}
        }
      }
    }
  }
}
```

2.3 Hardware y conexión

Ajustes en la pestaña **Proyecto** (se guardan en `project.json` bajo `hardware`):

Ajuste	Vector VN1630A / VN7572	PEAK PCAN-USB Pro FD	Simulación
Interfaz	`vector` (controlador XL)	`pcan` (PCAN-Basic)	`simulation`
Canal	Índice de canal, base 0 (p. ej. `0`). Asignación mediante *Vector Hardware Config*; si se indica el nombre de aplicación, rige el número de canal asignado allí.	`PCAN_USBBUS1`, `PCAN_USBBUS2` ...	no necesario
Velocidad	Velocidad nominal (kbit/s)	Velocidad nominal (kbit/s); con FD fija 500k/2M (reloj de 80 MHz)	sin importancia
CAN FD / Velocidad de datos	Marque la casilla e introduzca la velocidad de datos	Marque la casilla; temporización distinta en `hardware.extra`	Casilla para tramas FD
Nombre de app	Nombre de Vector Hardware Config (vacío = valor por defecto de python-can)	-	-

Vector VN7572 (tarjeta PCIe): la tarjeta de interfaz Vector VN7572 también se usa como tipo `vector` mediante el controlador Vector XL (solo Windows), igual que la VN1630A. Con *Vector Driver Setup* instalado, la tarjeta debe ser visible en *Vector Hardware Config*. El número de canal empieza en 0 (0 = CH1, 1 = CH2 ...); **Detectar hardware** lista los canales encontrados con su número de serie. Ningún otro programa (CANalyzer, CANoe, vFlash) debe ocupar el canal.

- Todos los nodos del bus deben usar la misma velocidad; en CAN FD, además, la misma velocidad de datos.
- En un montaje de sobremesa con dos dispositivos, los extremos del bus deben terminarse con 120 ohmios.
- `hardware.extra` (solo en `project.json`) pasa parámetros arbitrarios a python-can, p. ej. `{"nom_brp": 5}` para PEAK.
- **Simulación**: *Eco* devuelve como recepción los propios mensajes de envío; la *unidad de control virtual* responde a requests UDS (entre otros 10, 11, 14, 19, 22, 27, 3E) en las direcciones del ajuste de diagnóstico y envía cíclicamente los mensajes de los emisores ECU.

2.4 Archivos de secuencia (`sequences/*.json`)

Una secuencia consta de nombre, descripción y una lista de pasos. Puede crearla en la pestaña **Secuencia de prueba** o editar directamente el archivo. Referencia de comandos: Anexo A.

```
{
  "name": "startup",
  "description": "Simulate engine start and check run-up",
  "steps": [
    {"type": "enable_tx", "message": "DriverCmd", "enabled": true},
    {"type": "set_signal", "message": "DriverCmd", "signal": "StartButton", "value": 1},
    {"type": "wait_for", "direction": "RX", "message": "EngineData", "signal": "RPM",
     "op": ">", "value": 100, "timeout_ms": 3000},
    {"type": "diag", "service": "ReadVIN", "expect": "62 F1 90"}
```

```
  ]
}
```

2.5 El archivo de proyecto (`project.json`)

Lo escribe la herramienta; se puede editar manualmente si es necesario (cerrando antes la herramienta). Las claves que falten se completan con valores por defecto.

```
{
  "hardware": {"type": "vector", "channel": "0", "bitrate": 500000, "fd": true,
               "data_bitrate": 2000000, "app_name": "", "extra": {}},
  "cache_format": "json",
  "tx": [{"message": "DriverCmd", "cycle_ms": 50, "values": {"ThrottleCmd": 0}, "signals": null}],
  "rx": {"EngineData": null, "VehicleSpeed": ["Speed"]},
  "logging": {"selection": {"TX": {}, "RX": {"EngineData": null}},
              "on_change_only": false, "min_interval_ms": 0, "log_raw": false},
  "graph": ["RX:EngineData.RPM"],
  "diag": {"cdd": "diag.cdd.json", "ecu": "Engine_ECU", "request_id": "", "response_id": "",
           "extended": false, "fd": false, "padding": 170},
  "sequence": "startup.json",
  "sim": {"ecu": true, "traffic_senders": ["ECU"]}
}
```

Clave	Significado
`hardware`	Parámetros de conexión (capítulo 2.3); velocidades en bit/s.
`cache_format`	`json` o `yaml` para los archivos convertidos.
`tx`	Lista de envío: mensaje, ciclo (ms), valores de señal, `signals` = selección para gráfico/logging (`null` = todas).
`rx`	Selección de recepción: mensaje -> lista de señales elegidas o `null` (todas).
`logging`	Filtros y opciones de logging (pestaña Logging).
`graph`	Claves de las señales marcadas en el gráfico, p. ej. `RX:Msg.Signal`.
`diag`	Archivo CDD, ECU, sustitución opcional de direcciones (`request_id`, `response_id`), `extended`, `fd`, `padding`.
`sequence`	Última secuencia cargada (nombre de archivo en `sequences/`).
`sim`	Simulación: `ecu` (unidad de control virtual), `traffic_senders` (nodos emisores que se simulan cíclicamente).

2.6 Lista de comprobación antes de la primera conexión

1. Archivo(s) DBC en la carpeta `dbc/`; en la pestaña Proyecto aparecen los mensajes esperados (cantidad) y ninguna advertencia inesperada.
2. Opcionalmente, CDD en la carpeta `cdd/`; en la pestaña Diagnóstico aparecen ECU y servicios.
3. Controlador de hardware instalado, dispositivo conectado, canal y velocidades acordes al bus.
4. En CAN FD: casilla *CAN FD* y velocidad de datos; los otros nodos también deben admitir FD.
5. Secuencias presentes en `sequences/` o creadas en la pestaña Secuencia de prueba.

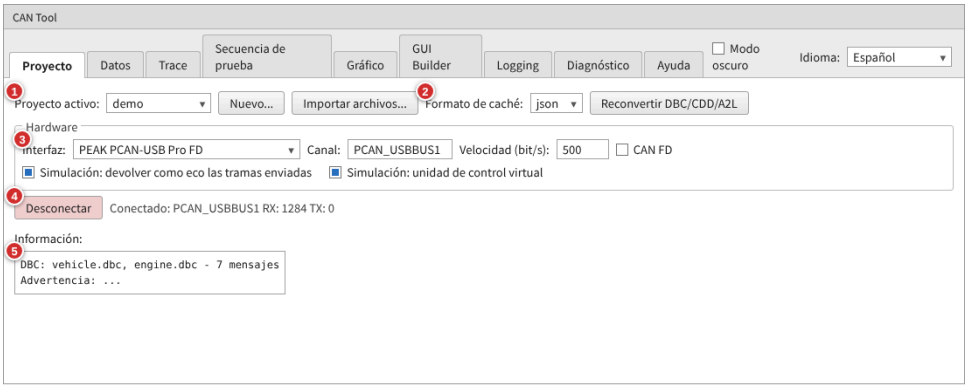
Problemas frecuentes

Síntoma	Posible causa / solución
No hay mensajes en la pestaña Datos	El DBC no está en la carpeta `dbc/` o la extensión no es `.dbc`; lea las advertencias en la pestaña Proyecto.
Falla la conexión	Falta python-can/el controlador, canal incorrecto, dispositivo ocupado por otra aplicación (p. ej. CANalyzer/PCAN-View).
Se ven tramas, pero no se decodifican	El ID/la marca Extended del DBC no coincide con el bus; el mensaje falta en el DBC.
Valores de señal incorrectos	Compruebe en el DBC el orden de bytes (Intel/Motorola), el bit de inicio y el factor/offset.
Diagnóstico: timeout	Compruebe Request-/Response-ID, Extended/FD y padding en la pestaña Diagnóstico; ¿está la unidad de control en la sesión adecuada?
Logging no disponible	Falta el paquete de Python `pyarrow` (`pip install pyarrow`).

3.1 Pestaña «Proyecto»

Sistema operativo: al iniciar, la herramienta detecta por sí misma si se ejecuta en **Windows** o **Linux** (se muestra en la barra de título). La selección de hardware se adapta: *Vector* (controlador XL) solo en Windows, *SocketCAN* solo en Linux, *PEAK* y *Simulación* en ambos. Las entradas no utilizables están marcadas y desactivadas. El botón **Detectar hardware** lista las interfaces conectadas. Todas las demás funciones (trace, secuencias, registro, diagnóstico, vFlash, ayuda/PDF) son idénticas en ambos sistemas; las carpetas de proyecto son intercambiables entre Windows y Linux.

Captura comentada



Representación esquemática de la interfaz; los números remiten a las explicaciones de abajo.

1. Selección de proyecto, creación de un proyecto nuevo e importación de DBC/CDD.
2. Formato de caché (json/yaml) y nueva conversión de DBC/CDD.
3. Hardware: interfaz, canal (p. ej. PCAN_USBBUS1), velocidad y CAN FD.
4. Conectar o desconectar, con indicación de estado (contadores de recepción y envío).
5. Campo de información: archivos DBC cargados, número de mensajes y advertencias.

Reinic. HW: junto a *Conectar*, este botón cierra la interfaz y la vuelve a abrir con los mismos ajustes. Ayuda cuando el hardware conectado se bloquea. La lista de envío se conserva (los mensajes cíclicos activos continúan después); el registro en curso se detiene.

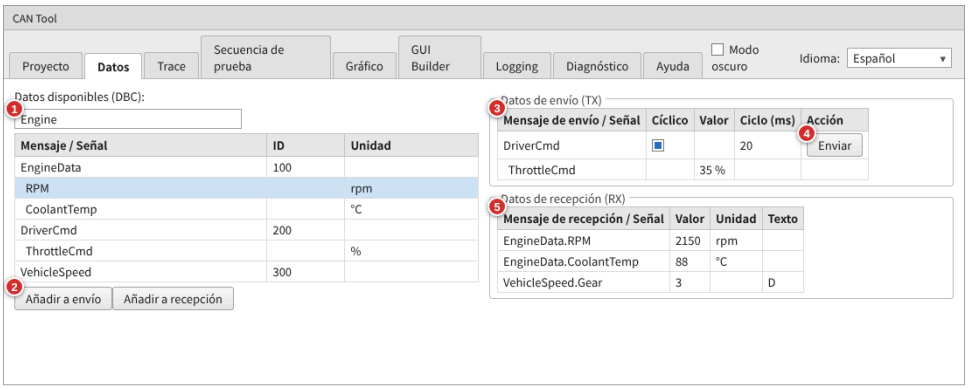
Gestionar proyectos, incorporar DBC/CDD, configurar el hardware y establecer la conexión.

Elemento de control	Función
Proyecto activo	Lista de selección de todos los proyectos (`projects/*/project.json`). Un cambio desconecta, detiene el logging, guarda el proyecto anterior y carga el nuevo.
Nuevo ...	Crea un proyecto con estructura de carpetas y ajustes por defecto (capítulo 1.5).
Importar DBC/CDD ...	Copia los archivos `.dbc`/`.cdd` elegidos al proyecto y los convierte.
Formato de caché	`json` o `yaml`; al cambiarlo se reconvierte.
Reconvertir	Fuerza la conversión de todos los DBC/CDD y recarga el proyecto.
Interfaz / Canal / Velocidad / CAN FD / Velocidad de datos / Nombre de app	Parámetros de hardware (capítulo 2.3). Bloqueados durante la conexión.
Simulación: eco	Los propios mensajes de envío aparecen además como recepción.
Simulación: unidad de control virtual	Al conectar, inicia una unidad de control por software compatible con UDS con tráfico de prueba.
Conectar / Desconectar	Abre o cierra la interfaz. Al desconectar se detiene un logging en curso.
Línea de estado	Estado de la conexión, número de tramas recibidas (RX) y enviadas (TX), último error.
Archivos convertidos / Advertencias	Lista de archivos de caché, número de mensajes, advertencias de DBC/CDD.

3.2 Pestaña «Datos»

Aquí se define **qué** mensajes se envían y reciben y **qué señales** se aplican al gráfico, al logging y a la visualización, siempre a partir de la suma de todos los datos DBC del proyecto.

Captura comentada



Representación esquemática de la interfaz; los números remiten a las explicaciones de abajo.

1. Filtro y lista de todos los mensajes y señales de la DBC.
2. Añadir la selección a los datos de envío (TX) o de recepción (RX).
3. Datos de envío: cíclico activado/desactivado, valores de señal y tiempo de ciclo.
4. Envío único: envía el mensaje una vez de inmediato.
5. Datos de recepción con los valores en vivo de las señales.

Lado izquierdo: Datos disponibles

- Árbol de todos los mensajes con sus señales (ID, emisor, unidad). El **campo de filtro** busca en el nombre del mensaje, el ID y el nombre de la señal.
- **-> Enviar (TX)** añade los mensajes marcados a la lista de envío (doble clic en un mensaje hace lo mismo). Se admite la selección múltiple; si se marca una señal, se añade su mensaje.
- **-> Recibir (RX)** añade los mensajes marcados a la lista de recepción.

Arriba a la derecha: Enviar

Columna	Significado
Mensaje de envío / Señal	Mensaje (con ID) y debajo todas sus señales.
Cíclico	La casilla del mensaje activa/desactiva el envío cíclico (solo se envía realmente si hay conexión).
Valor	Valor físico de la señal (campo de entrada o lista de selección en señales con tabla de valores). Los cambios surten efecto desde la siguiente trama.
Ciclo [ms]	Tiempo de ciclo del mensaje; valor inicial del DBC (`GenMsgCycleTime`), si no 100 ms. 0 = no cíclico.
Selección	Casilla de la señal: la señal pertenece a la selección para gráfico y logging.
Enviar (botón)	Envía el mensaje exactamente una vez con los valores actuales.

Quitar elimina de la lista de envío el mensaje de la fila marcada.

Abajo a la derecha: Recibir

Columna	Significado
Mensaje de recepción / Señal	Mensaje (con ID) y sus señales.
Valor / Unidad / Texto	Último valor recibido (actualización cada 200 ms), unidad y, si existe, el texto de la tabla de valores.
Selección	Casilla de la señal: la señal pertenece a la selección para gráfico y logging. Las señales no marcadas no se guardan.

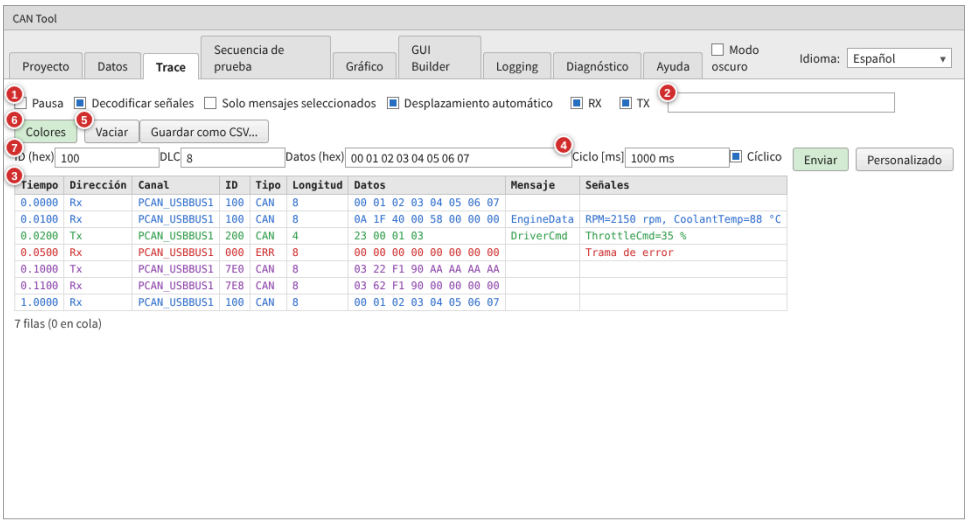
Notas

- La selección (casillas) determina lo que ven el **Gráfico** y el **Logging**. El **Trace**, en cambio, muestra todas las tramas (opcionalmente solo los mensajes elegidos).
- La selección y todos los valores de envío se guardan con el proyecto; tras cargarlo, el envío cíclico está desactivado.
- Un mensaje puede estar a la vez en la lista TX y en la RX (p. ej. con eco en la simulación).

3.3 Pestaña «Trace»

Desarrollo temporal de todas las tramas enviadas (Tx, resaltadas en color) y recibidas (Rx).

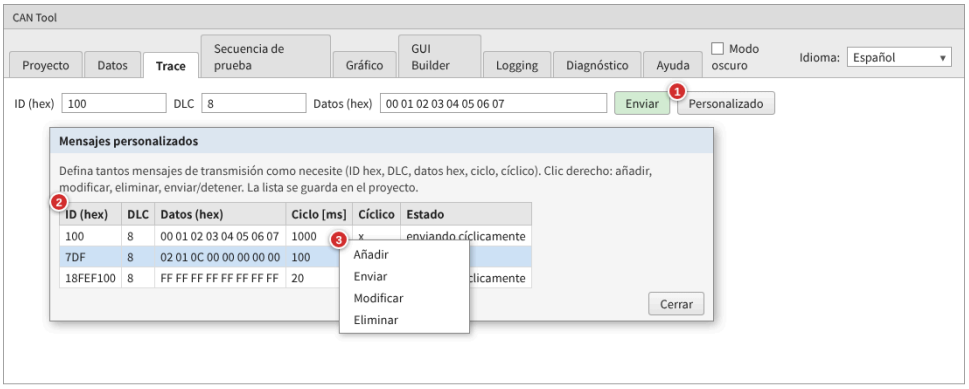
Captura comentada



Representación esquemática de la interfaz; los números remiten a las explicaciones de abajo.

1. Opciones: pausa, decodificar señales, solo mensajes elegidos, desplazamiento automático.
2. Filtro por ID o nombre de mensaje.
3. Tabla de traza: tiempo, dirección (Rx/Tx), canal, ID, tipo, longitud y datos en bruto. Funciona también sin DBC/CDD.
4. Nombres de mensajes y valores de señal decodificados (si hay una DBC cargada).
5. Vaciar la traza o guardarla como archivo CSV.
6. Modo de color: Rx azul, Tx verde, errores rojo, ISO-TP violeta.
7. Fila de envío: ID (hex), DLC, datos, tiempo de ciclo, casilla de envío cíclico y botón Enviar.

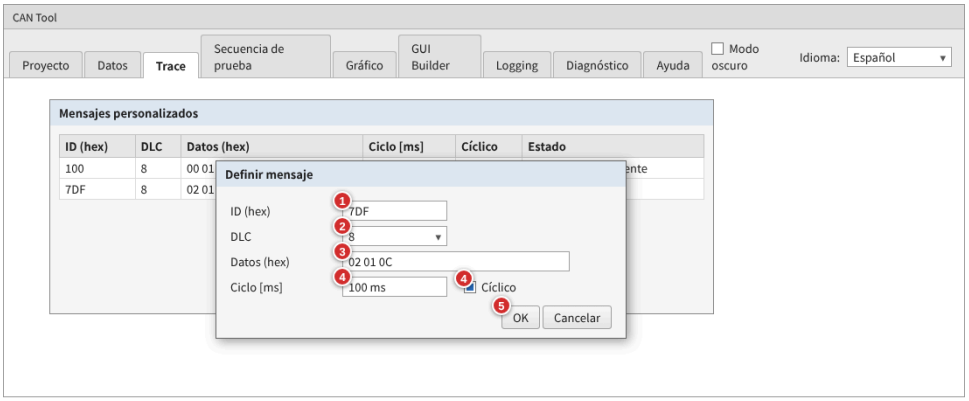
Captura comentada



Representación esquemática de la interfaz; los números remiten a las explicaciones de abajo.

1. El botón Personalizado abre la ventana de mensajes de transmisión propios.
2. Tabla de mensajes de transmisión: ID (hex), DLC, datos (hex), ciclo [ms], cíclico y estado.
3. Menú del clic derecho: añadir, enviar/detener, modificar, eliminar.

Captura comentada



Representación esquemática de la interfaz; los números remiten a las explicaciones de abajo.

1. ID (hex): ID CAN, también extendido (> 7FF).
2. DLC: longitud de datos, también valores CAN FD.
3. Datos (hex): bytes con o sin espacios; los bytes que faltan se completan con 00.
4. Ciclo [ms] y casilla Cíclico: enviar una vez o cíclicamente.
5. OK añade el mensaje a la lista, Cancelar descarta la entrada.

Columna	Contenido
Tiempo [s]	Tiempo relativo a la primera trama mostrada (desde la conexión o desde *Vaciar*).
Rx/Tx, Canal	Dirección e interfaz.
ID, Tipo, Len	ID CAN (hex, 8 dígitos en Extended), tipo (CAN, FD, FD/BRS) y longitud de datos.
Datos	Datos útiles en hex.
Mensaje, Señales	Nombre del DBC y señales decodificadas con unidad (si *Decodificar* está activo).
Modo de color	Botón conmutable. Activado: mensajes Rx en azul medio, mensajes Tx en verde medio, mensajes erróneos (tramas de error del bus, tipo ERR) en rojo y mensajes ISO-TP (ID de petición/respuesta de diagnóstico, también 0x7DF) en violeta medio. Desactivado: visualización como antes.

Elemento de control	Función
Pausa	Detiene la visualización. Las tramas se siguen almacenando en búfer (hasta 50 000) y se entregan al reanudar.
Decodificar	Activa/desactiva la decodificación de señales (desactivada = más rápido con alta carga del bus).
Solo mensajes seleccionados	Muestra solo los mensajes que están en la lista TX o RX de la pestaña Datos.
Autoscroll	Sigue automáticamente la entrada más reciente.
Rx / Tx	Mostrar/ocultar direcciones.
Campo de filtro	IDs en hex separados por comas (p. ej. `100, 200`) y/o parte de un nombre de mensaje (p. ej. `engine`).
Vaciar	Borra la visualización y el búfer.
Guardar como CSV ...	Exporta las filas visibles actualmente (separador `;`).

Actualizar: con la casilla *Actualizar* marcada, cada mensaje (dirección, canal, ID) aparece una sola vez en la tabla, en el orden de la primera aparición. Cuando vuelve a aparecer, se actualizan la marca de tiempo, los datos y las señales decodificadas de esa fila y aumenta la columna *Cantidad*. Sin marcar, todas las tramas se escriben una tras otra con desplazamiento. Al cambiar se vacía la tabla; el desplazamiento automático solo actúa en el modo de desplazamiento.

Tiempo dif.: además de *Actualizar*, la casilla *Tiempo dif.* hace que la columna de tiempo muestre el intervalo desde la aparición anterior del mismo mensaje (dirección, canal, ID), es decir, el tiempo de ciclo, en lugar del tiempo absoluto; 0 en la primera aparición. El encabezado pasa a *Tiempo dif. [s]*. Al cambiar se vacía la tabla.

Fila de envío: bajo la barra de opciones, una fila envía tramas sin procesar (por defecto: ID 100 hex, DLC 8, datos 00 01 02 03 04 05 06 07, 1000 ms). **Enviar** transmite la trama una vez; con **Cíclico** marcado inicia el envío cíclico y otro clic (**Parar**) lo detiene. Los ID superiores a 7FF se envían como tramas extendidas, un DLC superior a 8 como CAN FD (FD debe estar activado en la configuración de conexión). La trama aparece en el trace como Tx.

Mensajes personalizados (envío múltiple)

La fila de envío envía exactamente un mensaje. Para **varios mensajes** sirve el botón **Personalizado**: abre una ventana aparte que permanece abierta junto al trace. Allí se definen tantos mensajes de transmisión como se quiera y se envían individualmente. La lista se guarda en el proyecto (`custom_tx`) y está disponible de nuevo al abrir el proyecto.

Definir y enviar un mensaje

1. Pulsar el botón **Personalizado** de la fila de envío: se abre la ventana «Mensajes personalizados».
2. Hacer **clic derecho** en la tabla y elegir **Añadir**.
3. En el diálogo «Definir mensaje» introducir ID (hex), DLC, datos (hex), ciclo [ms] y marcar si hace falta **Cíclico**; confirmar con **OK**. Las entradas no válidas (p. ej. un DLC no permitido o demasiados bytes) se rechazan con un aviso y no se guarda nada.
4. Clic derecho en la fila y elegir **Enviar**: sin la casilla Cíclico el mensaje se envía una vez; con ella comienza el envío cíclico (estado: «enviando cíclicamente»). La misma opción de menú pasa a llamarse **Detener**.

Campo	Significado
ID (hex)	ID CAN en hexadecimal sin prefijo (p. ej. `7DF`). Los ID mayores que 7FF se envían automáticamente como ID extendido (29 bits).
DLC	Longitud de datos: 0 a 8 y 12, 16, 20, 24, 32, 48, 64. Los valores mayores que 8 se envían como CAN FD (el hardware y el proyecto deben admitirlo).
Datos (hex)	Bytes hexadecimales con o sin espacios. Una entrada demasiado corta se completa con 00 hasta el DLC; una demasiado larga es un error.
Ciclo [ms]	Intervalo de envío de 1 a 3 600 000 ms; solo actúa con Cíclico.
Cíclico	Sin marcar: se envía una vez; marcado: se envía cíclicamente hasta detenerlo.

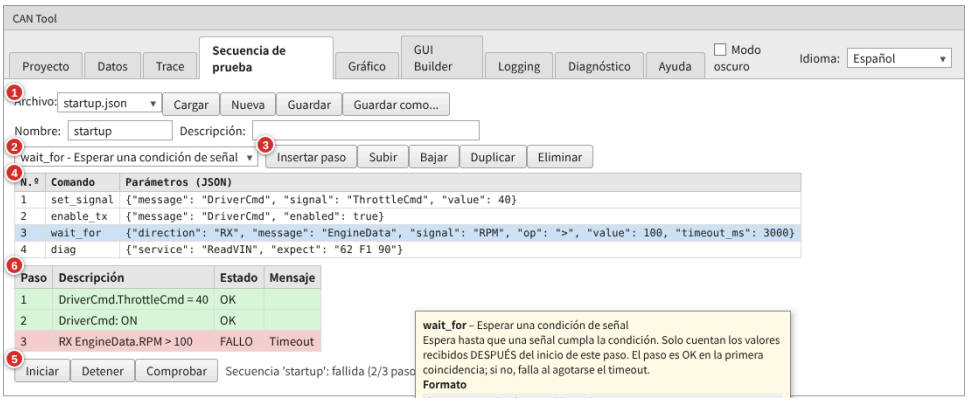
Menú de clic derecho: Añadir (también en zona libre), Enviar o Detener, Modificar (también con doble clic) y Eliminar. Una entrada en ejecución se detiene antes de modificarla o eliminarla. Para enviar debe haber conexión; al cerrarla se detiene el envío cíclico de todas las entradas. Varias entradas con el mismo ID comparten una única tarea de envío. Los mensajes enviados aparecen en el trace como Tx.

Notas: se muestran como máximo 5 000 filas (las más antiguas desaparecen). La visualización solo se actualiza mientras la pestaña está visible; en segundo plano la herramienta almacena hasta 50 000 tramas. Para un registro sin huecos sirve el **Logging**.

3.4 Pestaña «Secuencia de prueba»

Crear, guardar, cargar y ejecutar secuencias de prueba. Siempre hay **exactamente una secuencia** cargada/activa; varias secuencias se encuentran como archivos en la carpeta `sequences/` del proyecto.

Captura comentada



Representación esquemática de la interfaz; los números remiten a las explicaciones de abajo.

1. Elegir, cargar, crear y guardar un archivo de secuencia de prueba (una a la vez).
2. Desplegable de comandos: al pasar el ratón sobre una entrada aparece la ayuda con formato, elementos y ejemplo.
3. Insertar, mover, duplicar o eliminar un paso.
4. Lista de pasos: comando y parámetros en JSON, editables directamente en la tabla.
5. Iniciar, detener y comprobar estáticamente la secuencia; el resultado global se muestra al lado.
6. Tabla de resultados por paso: verde = superado, rojo = fallido, con mensaje.

Archivo

Elemento de control	Función
Lista de archivos + Cargar	Elige una secuencia de `sequences/` y la carga en el editor.
Nueva	Secuencia vacía (no guardada hasta *Guardar*).
Guardar / Guardar como ...	Escribe la secuencia como JSON. El último nombre de archivo guardado/cargado se anota en el proyecto y se vuelve a cargar al abrirlo la próxima vez.
Nombre / Descripción	Metadatos de la secuencia.

Editar pasos

Los pasos están en una tabla (N.º, tipo, parámetros como JSON). El **texto de parámetros se puede editar directamente** (doble clic). Un JSON no válido se rechaza y se restaura el valor anterior; el tipo de un paso es fijo.

Elemento de control	Función
Desplegable de comandos	Elige el tipo de paso. **Al pasar el ratón sobre una entrada aparece un tooltip** con el formato, la descripción de todos los elementos y un ejemplo; debajo se muestra además la descripción breve del comando marcado.
Añadir paso	Inserta el comando con una plantilla tras la fila marcada (o al final).
Subir / Bajar / Duplicar / Eliminar	Mover, copiar o quitar el paso marcado.
Selección de señal + Aplicar señal al paso	Lista todas las señales del DBC como `Message.Signal` (con autocompletado al escribir) e inserta el mensaje (y, en `set_signal`, `wait_for`, `check`, también la señal) en el paso marcado.
Tooltip en la tabla	El tooltip del comando también aparece sobre la columna de tipo de un paso.

Comprobar y ejecutar

Elemento de control	Función
Comprobar	Comprobación estática: tipos de paso, operadores, mensajes y señales desconocidos (frente al DBC).
Iniciar	Ejecuta la secuencia (solo con conexión). Antes del inicio se comprueba automáticamente. Durante la ejecución el editor está bloqueado.
Detener	Cancela la secuencia (también un `wait` en curso); estado *aborted*.
Tabla de resultados	Por paso: número (bucles: p. ej. `3.2.1`), descripción, estado (verde = ok, rojo = fail/error) y mensaje (p. ej. valor medido, bytes de respuesta).

Reglas de ejecución: los pasos se ejecutan uno tras otro. El **primer paso fallido finaliza la secuencia** (estado *failed*). Al terminar, la línea de estado muestra p. ej. *startup: passed (10/10 pasos ok)*. Los mensajes iniciados de forma cíclica siguen enviándose tras el final, salvo que se detengan con `enable_tx`.

Resumen de comandos

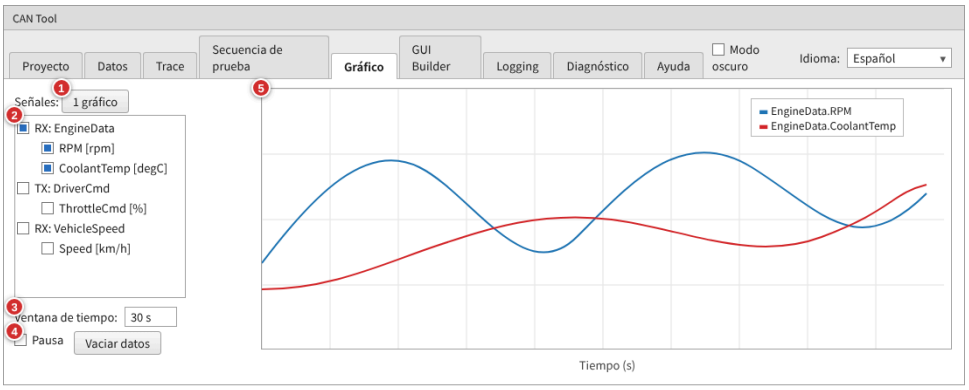
Comando	Título	Descripción breve
`set_signal`	Establecer valor de señal	Establece el valor físico de una señal en un mensaje de envío. El nuevo valor sale al bus con la siguiente trama (cíclica o única).
`enable_tx`	Activar/desactivar envío cíclico	Inicia o detiene el envío cíclico de un mensaje con su ciclo configurado.
`set_cycle`	Cambiar tiempo de ciclo	Cambia el tiempo de ciclo de un mensaje de envío (surte efecto de inmediato, incluso mientras envía cíclicamente).
`send_once`	Enviar mensaje una vez	Envía exactamente una trama con los valores de señal actuales, independientemente del envío cíclico.
`send_raw`	Enviar trama sin procesar	Envía una trama sin procesar (ID + datos) al bus CAN, una vez o cíclicamente. Con cyclic=true y enabled=false se detiene el envío cíclico de este ID.
`wait`	Esperar un tiempo fijo	Pausa la secuencia durante un tiempo fijo. El envío cíclico y la recepción continúan.
`wait_for`	Esperar una condición de señal	Espera hasta que una señal cumpla la condición. Solo cuentan los valores recibidos DESPUÉS del inicio de este paso. El paso es OK en la primera coincidencia; si no, falla al agotarse el timeout.
`check`	Comprobar valor de señal	Comprueba de inmediato el último valor recibido de una señal (sin esperar). El paso falla si no se cumple la condición o si aún no se ha recibido ningún valor desde el inicio de la secuencia.
`diag`	Servicio de diagnóstico del CDD	Envía por ISO-TP un servicio de diagnóstico definido en el CDD y evalúa la respuesta.
`diag_raw`	Request de diagnóstico en hex	Envía por ISO-TP cualquier request UDS (bytes hex) y evalúa la respuesta.
`loop`	Bucle	Ejecuta varias veces seguidas los pasos contenidos.
`comment`	Comentario	No hace nada; solo sirve para documentar la secuencia.

La descripción completa de cada comando con formato, elementos y ejemplo se encuentra en el Anexo A.

3.5 Pestaña «Gráfico»

Esta pestaña muestra en directo la evolución de las señales como diagrama de líneas (eje de tiempo: segundos relativos a ahora, izquierda = pasado, derecha = ahora). El interruptor sobre la selección alterna entre **1 gráfico** y **n gráficos**.

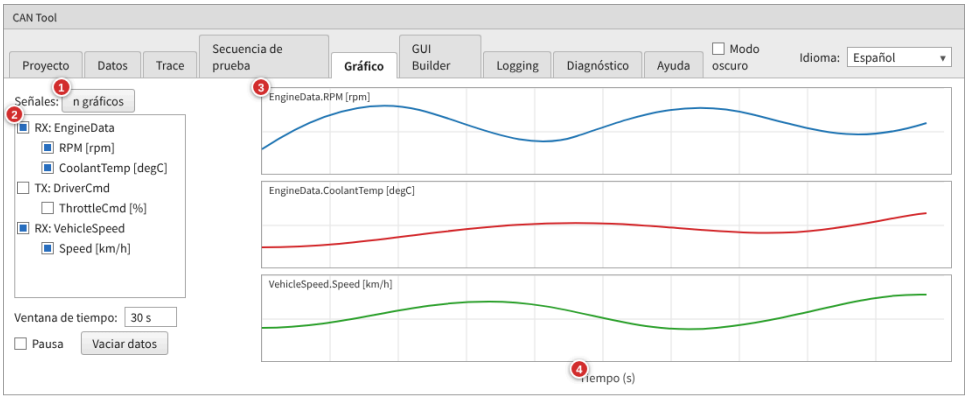
Captura comentada



Representación esquemática de la interfaz; los números remiten a las explicaciones de abajo.

1. Interruptor 1 gráfico / n gráficos (aquí: 1 gráfico).
2. Selección de señales agrupada por mensaje; la casilla del mensaje selecciona todas sus señales, unidad entre corchetes.
3. Ventana de tiempo en segundos.
4. Pausa congela la vista, Borrar datos descarta las curvas.
5. Diagrama común con leyenda; eje de tiempo en segundos relativos a ahora.

Captura comentada



Representación esquemática de la interfaz; los números remiten a las explicaciones de abajo.

1. El interruptor está en n gráficos.
2. La selección sigue agrupada por mensaje; aquí hay tres señales elegidas.
3. Cada señal tiene su propio gráfico con su eje de valores, rotulado Mensaje.Señal y unidad.
4. Eje de tiempo común: zoom y desplazamiento afectan a todos los gráficos; etiquetas solo abajo.

Selección agrupada por mensaje

La selección muestra todas las señales elegidas en la pestaña Datos, agrupadas por dirección y mensaje (`RX` = recibido, `TX` = enviado). Bajo cada mensaje aparecen sus señales con la unidad entre corchetes. La casilla del mensaje tiene tres estados: todas las señales seleccionadas, selección parcial (estado intermedio) o ninguna. Un clic selecciona todas las señales del mensaje o las quita.

Vista «1 gráfico»

Todas las señales seleccionadas se dibujan en un único diagrama con leyenda (colores). Ventaja: comparación directa de las curvas. Con rangos de valores muy distintos las señales pequeñas se ven planas; entonces es mejor «n gráficos».

Vista «n gráficos»

Cada señal seleccionada tiene su propio diagrama con su propio eje de valores, rotulado Mensaje.Señal con unidad; los diagramas se apilan. Todos comparten la misma **base de tiempo**: al hacer zoom o desplazar el eje de tiempo en uno, se aplica a todos, y las etiquetas de tiempo solo aparecen en el inferior.

Elementos de control

Elemento	Función
Interruptor 1 gráfico / n gráficos	Cambia el modo de vista; se reconstruye al instante. El ajuste se guarda en el proyecto (`graph_multi`).

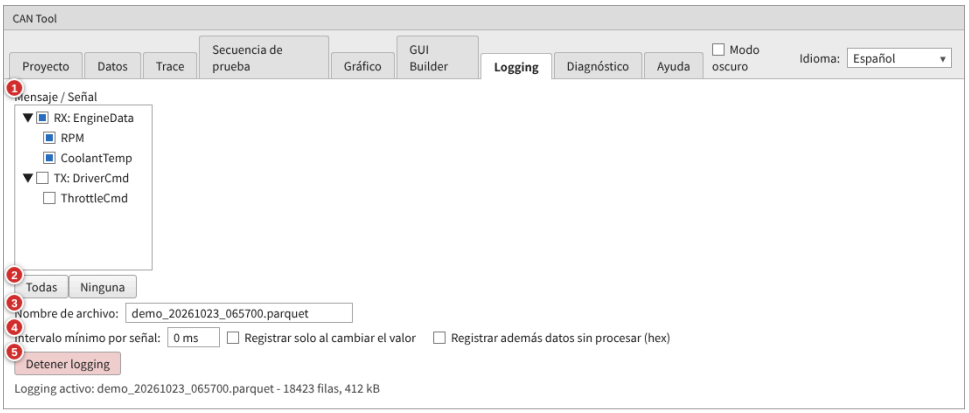
Selección de señales	Marcar = mostrar la curva. La casilla de un mensaje selecciona o quita todas sus señales. La selección se guarda en el proyecto (`graph`).
Ventana de tiempo	Periodo visible en segundos (1 a 3600).
Pausa	Congela la vista; los datos se siguen recopilando en segundo plano.
Borrar datos	Descarta todas las curvas recopiladas.
Diagrama	Ratón: zoom con la rueda, desplazar con el botón pulsado. En «n gráficos» se aplica a todos los diagramas a la vez.

Notas: por señal se conservan hasta 50 000 puntos. Solo se registran las señales marcadas en la pestaña Datos. Las señales aparecen únicamente cuando se han recibido/enviado datos. La visualización se actualiza solo con la pestaña visible.

3.6 Pestaña «Logging»

Señales resueltas: cuando hay una DBC, A2L o CDD cargada, el archivo Parquet contiene no solo el mensaje en hexadecimal sino las señales con **valor crudo, hex crudo, valor físico, texto (tabla de valores) y unidad**. Columnas del archivo de señales: `timestamp, direction, message, signal, value, source, can_id, raw, raw_hex, text, unit, message_hex`. `source` es `CAN` (mensajes DBC), `XCP` (valores A2L leídos en la pestaña XCP) o `UDS` (parámetros de respuesta decodificados de la pestaña Diagnóstico, de la CDD) mientras el registro está activo. El archivo crudo (`_raw.parquet`) tiene además la columna `signals` con todas las señales de la trama como texto, p. ej. `RPM=0x1F40/2000 rpm`.

Captura comentada



Representación esquemática de la interfaz; los números remiten a las explicaciones de abajo.

1. Selección de los mensajes y señales a registrar (filtro sobre los datos elegidos).
2. Marcar todas las señales o ninguna.
3. Nombre del archivo Parquet (en la carpeta logs/ del proyecto).
4. Opciones: solo al cambiar, intervalo mínimo por señal, datos en bruto adicionales.
5. Iniciar o detener el registro; la línea de estado muestra el número de filas y el tamaño del archivo.

Registra las señales seleccionadas en formato **Parquet** (requiere `pyarrow`). Los archivos se guardan en la carpeta `logs/` del proyecto.

Elemento de control	Función
Árbol de datos	Muestra los mensajes/señales elegidos en la pestaña Datos. **Filtro:** solo se registran las señales marcadas (la casilla del mensaje marca/desmarca todas sus señales). La selección se guarda en el proyecto.
Todas / Ninguna	Marcar o desmarcar todas las señales.
Nombre de archivo	Predefinido como *Proyecto_Fecha_Hora.parquet*; la extensión se añade si hace falta.
Intervalo mínimo por señal	Por señal, como máximo una entrada por intervalo (ms); reduce el volumen de datos en mensajes rápidos. 0 = cada trama.
Solo al cambiar el valor	Una señal solo se escribe si su valor cambia respecto al último escrito.
Tramas sin procesar	Escribe además *<nombre>_raw.parquet* con **todas** las tramas del bus (independientemente de la DBC y de la selección de señales, incluidas las tramas de error). Columnas: tiempo, dirección, canal, ID, extendido, FD, BRS, error, DLC, datos (binario), datos (texto hex), nombre del mensaje. Así se puede registrar sin selección de señales. **ID de datos brutos** limita las tramas, p. ej. `100, 7E0-7EF` (hex, vacío = todas).
Iniciar / detener logging	Inicia o finaliza el registro (solo con conexión). Al desconectar o cambiar de proyecto se detiene automáticamente y el archivo se cierra correctamente.
Línea de estado	Nombre de archivo, filas escritas, tamaño del archivo.

Formato de archivo

Columna	Tipo	Contenido
timestamp	float64	Marca de tiempo (segundos Epoch)
direction	string	`RX` o `TX`
message	string	Nombre del mensaje
signal	string	Nombre de la señal
value	float64	Valor físico

El formato largo (una fila por valor de señal) evita huecos con distintos tiempos de ciclo. Los datos se almacenan en búfer y se escriben en disco cada 5 segundos o a partir de 20 000 filas (compresión zstd).

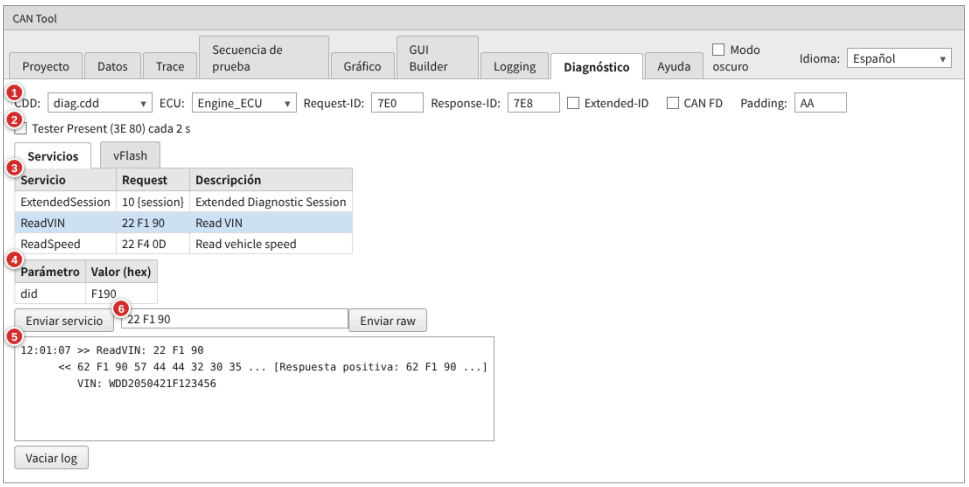
Evaluación en Python (véase también el Anexo B):

```
import pandas as pd
df = pd.read_parquet("projects/demo/logs/demo_20261020_071500.parquet")
wide = df.pivot_table(index="timestamp", columns=["direction", "message", "signal"], values="value")
```

3.7 Pestaña «Diagnóstico»

Servicios de diagnóstico (UDS) mediante **ISO-TP** (ISO 15765-2) sobre CAN o CAN FD. Los servicios proceden del CDD convertido; además se pueden enviar requests arbitrarios en hex.

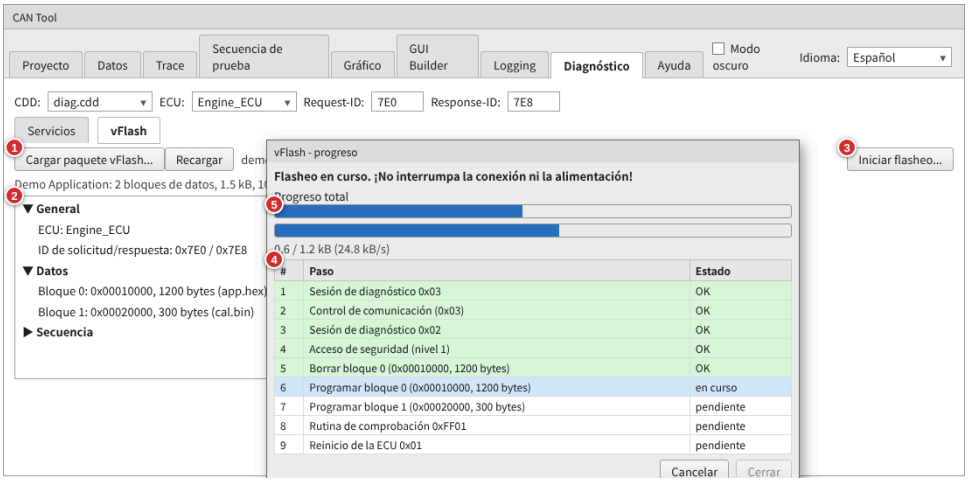
Captura comentada



Representación esquemática de la interfaz; los números remiten a las explicaciones de abajo.

1. Configuración de diagnóstico: CDD, ECU, ID de petición/respuesta, ID extendido, CAN FD, relleno.
2. Tester Present (3E 80) se envía automáticamente cada 2 s.
3. Servicios de la CDD con plantilla de petición y descripción.
4. Parámetros del servicio elegido (valores hex editables) y luego «Enviar servicio».
5. Registro de respuestas: peticiones enviadas, respuestas (positivas/negativas) y valores decodificados.
6. Enviar una petición en bruto en hex (p. ej. 22 F1 90), sin definición CDD.

Captura comentada



Representación esquemática de la interfaz; los números remiten a las explicaciones de abajo.

1. Cargue un paquete vFlash (.vflashpack/.zip, manifiesto, HEX/S19/BIN); "Recargar" vuelve a leer el archivo.
2. Interpretación del paquete: General (ECU, ID, seed&key), Datos (bloques con dirección y longitud), Secuencia (pasos) y Notas.
3. Iniciar el flasheo: tras un aviso de seguridad, la secuencia se ejecuta en el hardware seleccionado.
4. Ventana de progreso: estado por paso (pendiente, en curso, OK, ERROR, omitido) con detalles.
5. Progreso total, progreso del bloque de datos actual con velocidad de datos y registro; "Cancelar" detiene entre dos bloques.

Configuración (fila superior)

Campo	Significado
CDD	Descripción de diagnóstico convertida de la caché del proyecto.
ECU	Unidad de control del CDD; al cambiar se adoptan sus direcciones.
Request-ID / Response-ID	IDs CAN en hex (tester -> ECU / ECU -> tester). Predefinidos desde el CDD, se pueden sobrescribir aquí.
Extended-ID	Direccionamiento de 29 bits.
CAN FD	ISO-TP con tramas FD (hasta 64 bytes por trama, con cambio de velocidad). Debe coincidir con el otro extremo.
Padding	Byte de relleno en hex (por defecto AA); vacío = sin relleno en CAN clásico.
Tester Present	Envía `3E 80` cada 2 s (sin respuesta) para mantener abierta una sesión de diagnóstico.

Pestaña vFlash	Carga un paquete vFlash (o manifiesto/HEX/S19/BIN), muestra la interpretación y ejecuta la secuencia de flasheo en el hardware seleccionado, con una ventana de progreso (Anexo C).

Enviar servicios

1. Marque el servicio en la lista (nombre, request, descripción).
2. Si el servicio tiene parámetros, introduzca el valor en la tabla *Parámetro / Valor (hex)* (marcadores en el request).
3. **Enviar servicio**. El envío se realiza en segundo plano; la interfaz sigue siendo utilizable.
4. El **log** muestra el request (>>), la respuesta (<<) con su evaluación (positiva/negativa, en respuestas negativas NRC con texto descriptivo) y los parámetros de respuesta decodificados del CDD (p. ej. *VIN: WDD...*).

Enviar raw: introduzca el request como bytes hex (p. ej. `22 F1 86`) y pulse Intro o use el botón.

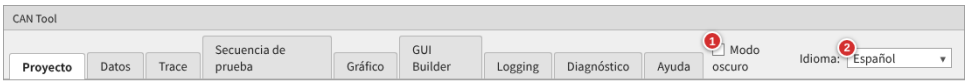
Detalles técnicos

- Segmentación: Single Frame, First Frame, Consecutive Frames y Flow Control, hasta 4095 bytes (más largos mediante el First Frame extendido).
- Al recibir respuestas largas, la herramienta envía Flow Control con Blocksize 0 y STmin 0.
- Timeout de respuesta 2 s; con *ResponsePending* (NRC 0x78) se sigue esperando hasta 10 s.
- Los requests sin respuesta (sub-función con bit 7, p. ej. `3E 80`) se registran como enviados.
- En las secuencias de prueba, `diag` y `diag_raw` usan la misma configuración de direcciones.

3.8 Barra de pestañas: modo oscuro e idioma

En el borde derecho de la barra de pestañas (junto a las fichas) hay dos elementos de control que valen para todas las pestañas.

Captura comentada



Representación esquemática de la interfaz; los números remiten a las explicaciones de abajo.

1. Modo oscuro: cambia entre apariencia clara y oscura (se recuerda).
2. Idioma: desplegable con todos los archivos de idioma disponibles; el cambio es inmediato, también en la ayuda.

Elemento de control	Función
Modo oscuro	La casilla alterna entre apariencia clara y oscura. Afecta a la interfaz, el trace, el gráfico, la tabla de resultados de la secuencia y esta ayuda. El ajuste se guarda y se usa en el siguiente inicio.
Idioma	Menú desplegable con Deutsch (idioma principal), English, Français, Español, chino (mandarín, simplificado) y tailandés. El cambio surte efecto de inmediato, sin reiniciar. La selección se guarda.

¿Qué se traduce?

- Todas las etiquetas, botones, cabeceras de tabla, tooltips, diálogos y mensajes de la interfaz.
- La referencia de comandos de la secuencia de prueba (tooltips del desplegable, Anexo A) y todos los capítulos de esta ayuda.
- Mensajes de error y advertencia de los componentes del programa (p. ej. conversión, conexión, ejecución de secuencias).
- **No** se traducen los datos de sus archivos: nombres de mensajes y señales, unidades y tablas de valores de DBC/CDD, nombres de archivo, nombres de proyecto y el contenido de los archivos de secuencia.
- Las líneas de registro ya escritas (log de diagnóstico, tabla de resultados de una secuencia) conservan el idioma en que se generaron.

Para chino y tailandés debe haber en el equipo una fuente con los caracteres correspondientes (algo habitual en las instalaciones comunes de Windows, macOS y Linux).

Archivos de idioma

Cada idioma está en su propio archivo en la carpeta `lang/` (YAML; también es posible JSON):

Archivo	Idioma
`lang/de.yaml`	Alemán - idioma principal y de reserva: si en un idioma falta un texto, se usa el alemán.
`lang/en.yaml`	Inglés
`lang/fr.yaml`	Francés
`lang/es.yaml`	Español
`lang/zh.yaml`	Chino (mandarín, caracteres simplificados)
`lang/th.yaml`	Tailandés

Estructura de cada archivo: secciones `meta` (nombre visible, orden en el desplegable), `ui` (interfaz), `core` (mensajes), `steps` (referencia de comandos) y `help` (capítulos de esta ayuda). Los marcadores entre llaves (p. ej. `{name}`) y los marcadores entre dobles corchetes en los textos de ayuda no deben modificarse.

Añadir o cambiar un idioma

1. Copie `lang/de.yaml` y guárdelo como `lang/<code>.yaml` (p. ej. `it.yaml`).
2. En la sección `meta` establezca el nombre visible (`name`) y la posición (`order`); traduzca todos los textos.
3. Compruebe con `python tools/check_lang.py it`: la herramienta informa de claves que faltan, marcadores distintos y estructura HTML dañada.
4. Reinicie el programa: el idioma aparece automáticamente en el desplegable.

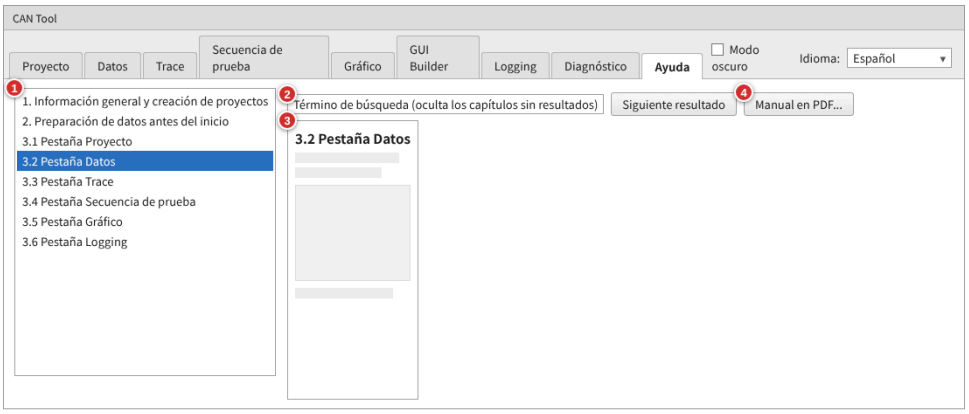
Línea de comandos

También las salidas de `main.py` están traducidas: `python main.py --lang en --list`. Sin `--lang` se usa el último idioma elegido en la GUI.

3.9 Pestaña «Ayuda»

Esta pestaña. A la izquierda están los capítulos, a la derecha el texto.

Captura comentada



Representación esquemática de la interfaz; los números remiten a las explicaciones de abajo.

1. Lista de capítulos de la ayuda.
2. Cuadro de búsqueda: oculta los capítulos sin coincidencias; «Siguiente coincidencia» avanza.
3. Texto del capítulo con figuras comentadas.
4. Manual en PDF: exporta la ayuda en todos los idiomas disponibles (con portada y fecha de emisión) y ofrece guardarla.

- **Campo de búsqueda**: oculta los capítulos en los que no aparece el término y marca la primera coincidencia en el capítulo elegido. **Siguiente resultado** salta a la siguiente coincidencia. Con el campo vacío se muestran de nuevo todos los capítulos.
- Los enlaces del texto (p. ej. al anexo) se pueden pulsar.
- Las descripciones de comandos coinciden con los tooltips del desplegable de comandos de la pestaña Secuencia de prueba.
- La ayuda aparece en el idioma elegido (capítulo 3.8) y se sustituye de inmediato al cambiar de idioma; el cambio claro/oscuro también se aplica aquí.

3.10 Pestaña «XCP»

Medir y ajustar variables de la ECU mediante **XCP on CAN** a partir de un **archivo A2L** (ASAP2). El A2L se convierte como DBC y CDD: los archivos fuente están en la carpeta `a2l/` del proyecto; en ejecución solo se lee la caché `cache/<name>.a2l.json|yaml`.

Puesta en marcha

1. Copie el A2L al proyecto con **Importar** (pestaña Proyecto); se convierte automáticamente (`python main.py --convert demo` también funciona sin GUI).
2. Conecte el CAN (pestaña Proyecto). XCP usa la misma interfaz.
3. Pestaña XCP: elija el A2L y compruebe el **ID maestro** (comandos a la ECU) y el **ID esclavo** (respuestas). Se rellenan desde la parte XCP del A2L (`IF_DATA XCP / XCP_ON_CAN`) y pueden cambiarse.
4. **Conectar XCP**. El registro muestra todos los comandos (->) y respuestas (<-) en hexadecimal.

Elemento	Función
Lista izquierda	Todas las mediciones (MEASUREMENT) y parámetros (CHARACTERISTIC de tipo VALUE) del A2L; filtro por nombre; doble clic o *Añadir al seguimiento* los incorpora.
Seguimiento	Muestra de cada entrada el valor crudo y el valor físico con unidad. La selección se guarda con el proyecto.
Lectura cíclica	Lee todas las entradas seguidas con el intervalo ajustado (ms). Se consulta con SHORT_UPLOAD; no se usan listas DAQ.
Leer una vez	Lee el seguimiento una sola vez.
Escribir valor	Escribe el valor introducido (físico; en conversiones de texto también el texto, p. ej. *RUN*) en la fila elegida (SET_MTA + DOWNLOAD) y lo vuelve a leer.

Interpretación del A2L

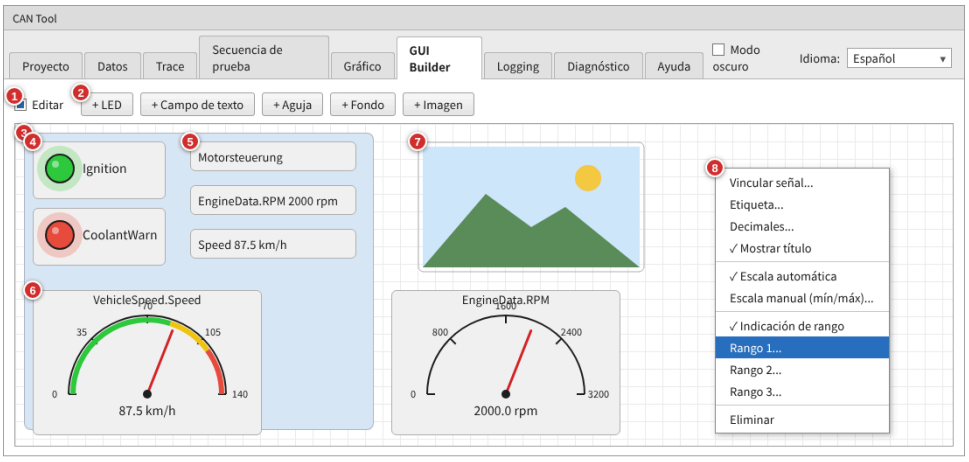
- Tipos UBYTE, SBYTE, UWORD, SWORD, ULONG, SLONG, A_UINT64, A_INT64, FLOAT32_IEEE, FLOAT64_IEEE; orden de bytes según la respuesta CONNECT de la ECU.
- Conversión IDENTICAL, LINEAR, RAT_FUNC (sin términos cuadráticos) y TAB_VERB (valores de texto); BIT_MASK se enmascara al leer.
- Los objetos no compatibles (curvas, mapas, matrices, mediciones sin dirección) aparecen en gris.

Con hardware *Simulación*, la herramienta inicia un esclavo XCP simulado que refleja el A2L del proyecto (demo: `projects/demo/a2l/demo.a2l`). No incluido: DAQ/STIM, seed&key, páginas de calibración y programación flash. Que una ECU real permita todos los accesos depende de sus niveles de protección.

3.11 Pestaña «GUI Builder»

Aquí se crea una vista propia con elementos sencillos para señales existentes. Los elementos son solo de **visualización**; no hay entrada de valores. Se pueden vincular las señales seleccionadas en la pestaña Datos (recibidas o enviadas). El diseño y las vinculaciones se guardan en el proyecto (`builder`).

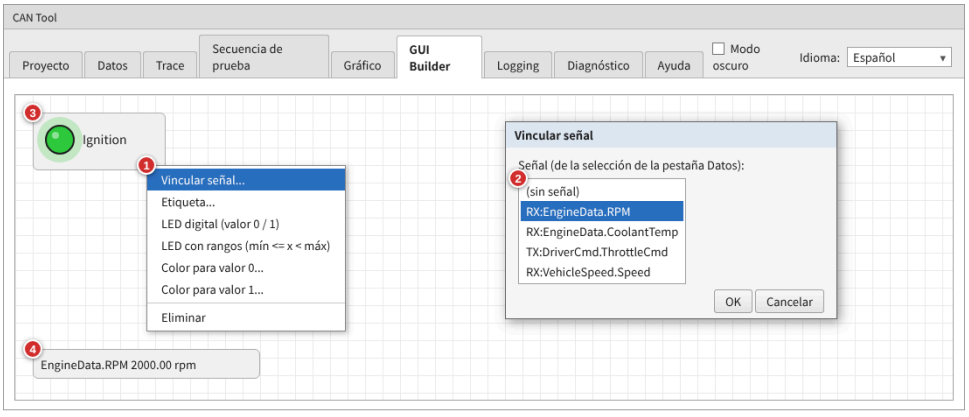
Captura comentada



Representación esquemática de la interfaz; los números remiten a las explicaciones de abajo.

1. Casilla Editar: cuadrícula, añadir, mover y redimensionar posibles.
2. Botones para añadir LED, campo de texto, aguja, fondo e imagen.
3. Elemento de fondo con color a elegir: agrupa visualmente los elementos y queda detrás de ellos.
4. LED: digitales (valor 0 / 1) y con rangos (color según el rango mín <= x < máx).
5. Campos de texto: a la izquierda sin señal (texto simple), a la derecha con señal (valor y unidad en vivo).
6. Aguja con escala de 180 grados, automática o manual, opcionalmente con indicación de rango.
7. Marco de imagen: la imagen elegida se escala automáticamente al marco.
8. Menú de clic derecho de la aguja: señal, título, escala, indicación de rango y eliminar.

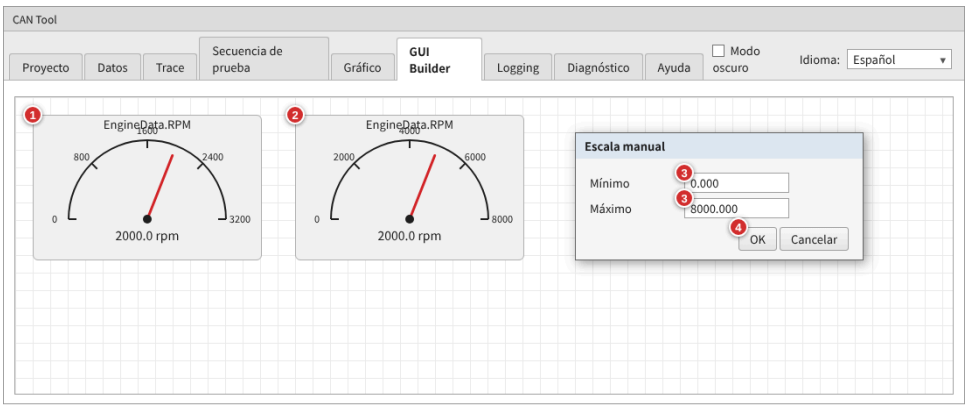
Captura comentada



Representación esquemática de la interfaz; los números remiten a las explicaciones de abajo.

1. Clic derecho en el LED: vincular señal, etiqueta, modo digital/rangos, colores, eliminar.
2. Selección: señales de la pestaña Datos como RX:/TX:Mensaje.Señal, o (sin señal).
3. LED: color según el valor de la señal (digital 0 / 1 o por rango 1 a 3).
4. Campo de texto con señal: texto, valor y unidad; decimales ajustables.

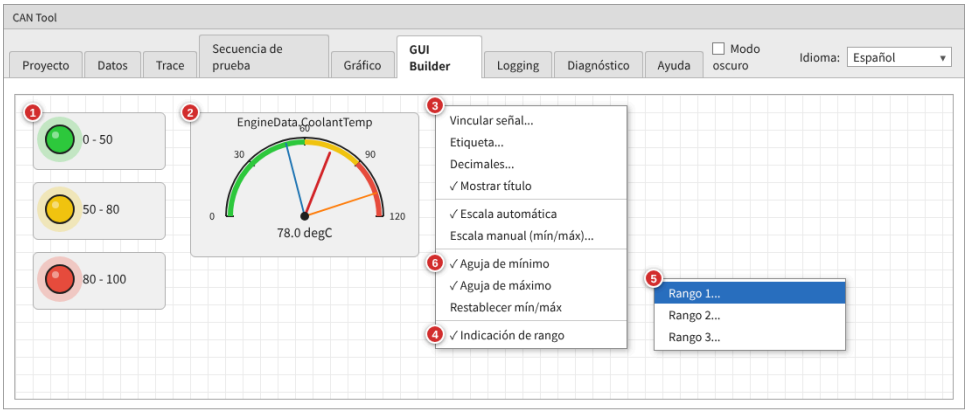
Captura comentada



Representación esquemática de la interfaz; los números remiten a las explicaciones de abajo.

1. Aguja con escala automática: límites redondeados, aquí 0 a 3200.
2. Aguja con escala manual, aquí 0 a 8000.
3. Diálogo Escala manual: introducir mínimo y máximo.
4. OK aplica la escala (solo si mínimo < máximo), Cancelar la descarta.

Captura comentada



Representación esquemática de la interfaz; los números remiten a las explicaciones de abajo.

1. Tres LED con rangos: cada rango de valores (mín <= x < máx) determina el color.
2. Aguja con indicación de rango: tres rangos de color en la escala (rango 1 a 3).
3. Menú: Mostrar título activa o desactiva la etiqueta.
4. Menú: activar o desactivar la indicación de rango.
5. Rango 1 a 3: cada uno con mínimo, máximo (regla mínimo <= x < máximo), color y activación.
6. Menú: segunda y tercera aguja opcionales para mínimo (azul) y máximo (naranja), y restablecer.

Elementos

Elemento	Función
LED	Muestra el estado de una señal como color. **Digital**: el valor 0 y el valor 1 tienen cada uno un color a elegir. **Rangos**: como en la escala de la aguja, tres rangos (mín <= x < máx) con un color cada uno; si el valor no está en ningún rango o falta, el LED es gris.
Campo de texto sin señal	Campo de texto simple con texto libre, p. ej. como título o etiqueta.
Campo de texto con señal	Muestra el texto y el valor actual de la señal con su unidad; se actualiza de forma continua. Los decimales son ajustables.
Aguja	Instrumento de aguja con escala de 180 grados (semicírculo): mínimo a la izquierda, máximo a la derecha; debajo se muestra el valor numérico con su unidad.
Fondo	Rectángulo de color sencillo (color a elegir) sin más función. Agrupa visualmente otros elementos y siempre queda detrás de los elementos funcionales, pero sobre el fondo estándar.
Imagen	Marco de imagen con una imagen a elegir (PNG, JPG, BMP, GIF, SVG). La imagen se adapta automáticamente al marco (se mantiene la proporción) y sigue cada cambio de tamaño.

Manejo

Elemento de control	Función
Editar	Marcado: cuadrícula visible; se pueden añadir, mover, redimensionar y eliminar elementos. Sin marcar, la vista queda bloqueada (solo visualización).
+ LED / + Campo de texto / + Aguja	Añade un elemento nuevo (también con clic derecho en la zona libre, en la posición del ratón).
Arrastrar	Mueve un elemento; la esquina inferior derecha cambia su tamaño.
Clic derecho en un elemento	Menú: vincular señal (o «sin señal»), etiqueta o texto, umbral y color del LED, decimales, escala (aguja), eliminar.

Aguja y escala

La escala tiene dos modos. **Automático** (por defecto): el rango crece con los valores observados y se ajusta a números redondos (p. ej. 0 a 100); marcas y números se adaptan. **Manual**: con **clic derecho** en la aguja y **Escala manual (mín/máx)** se indican mínimo y máximo (el mínimo debe ser menor que el máximo). La casilla **Escala automática** del mismo menú vuelve al modo automático. Los valores fuera de la escala dejan la aguja en el tope.

Inicio rápido

1. Dejar marcada la casilla **Editar** (cuadrícula visible).
2. Pulsar **+ LED**, **+ Campo de texto** o **+ Aguja**; el elemento aparece en el área de dibujo (también con clic derecho en la zona libre, en la posición del ratón).
3. **Clic derecho** en el elemento, elegir **Vincular señal** y seleccionar una señal de la lista.
4. Ajustar etiqueta, umbral del LED, decimales o escala y colocar el elemento arrastrando; la esquina inferior derecha cambia su tamaño.
5. Desmarcar **Editar**: la vista queda bloqueada y solo sirve para visualizar.

Vincular una señal

La lista muestra las señales seleccionadas en la pestaña Datos como `RX:Msg.Signal` o `TX:Msg.Signal`, además de la entrada «(sin señal)», que elimina la vinculación. Si falta una señal, selecciónela primero en la pestaña Datos. Los valores aparecen en cuanto se reciben o envían datos; la unidad procede de la DBC. Al cambiar la señal se reinicia la escala automática.

Ajustar LED y campo de texto

Ajuste	Efecto
Etiqueta / texto	LED: etiqueta junto al LED (nombre de la señal si está vacío). Campo de texto: texto libre; con señal precede al valor.
Modo del LED	Digital (valor 0 / valor 1) o rangos (rango 1 a 3). Digital: el valor 0 da el color 0, cualquier otro valor el color 1.
Colores / rangos	Digital: color para el valor 0 y el valor 1. Rangos: por rango activación, mínimo, máximo y color según la regla **mínimo <= x < máximo**; el primer rango que coincide determina el color. Es el mismo ajuste que la indicación de rango de la aguja.
Decimales	Número de decimales de 0 a 9 para campo de texto y aguja.
Título (aguja)	La casilla **Mostrar título** muestra u oculta la etiqueta sobre la aguja; el espacio se usa entonces para la escala.
Indicación de rango (aguja)	La casilla **Indicación de rango** colorea la escala en hasta tres rangos (rango 1 a 3), cada uno con mínimo, máximo, color y activación según la regla **mínimo <= x < máximo**; los tres rangos aparecen entonces en el menú.
Aguja mín/máx (aguja)	Con las casillas **Aguja de mínimo** y **Aguja de máximo** la escala muestra además una segunda aguja fina (azul) y una tercera (naranja) para el menor y el mayor valor desde el inicio de la observación. **Restablecer mín/máx** empieza de nuevo; cambiar la señal o las casillas también restablece. Los valores no se guardan en el proyecto, solo las casillas.

Escala de la aguja en detalle

Automática: hasta que llega el primer valor rige 0 a 100. Después la aguja recuerda el valor observado más pequeño y el mayor y redondea los límites a números redondos (1, 2, 2,5, 5 por una potencia de diez); si todos los valores son no negativos y cercanos a 0, la escala empieza en 0. La escala crece, pero nunca se reduce. La casilla **Escala automática** del menú de clic derecho (desmarcar y volver a marcar) reinicia la observación. La escala muestra cinco marcas con números.

Manual: clic derecho en la aguja, **Escala manual (mín/máx)**: introducir mínimo y máximo en el diálogo (se admiten decimales). Si el mínimo no es menor que el máximo, aparece un aviso y no se aplica nada. La escala manual actúa de inmediato, queda guardada en el proyecto y desactiva el modo automático. Los valores fuera de la escala dejan la aguja en el tope.

Fondo e imagen

Los elementos están en capas: fondo estándar (área de dibujo), encima los elementos de fondo y encima los elementos funcionales (LED, campo de texto, aguja, imagen). Los elementos de fondo se colocan automáticamente detrás de los demás al añadirlos, moverlos y cargarlos; así, un marco de color puede agrupar varias indicaciones. La imagen elegida se copia en la carpeta del proyecto (`images`) para que el proyecto siga siendo portable; sin imagen aparece «Sin imagen».

Colores del LED e indicación de rango

Ejemplo de LED digital: valor 0 = rojo, valor 1 = verde. Ejemplo de LED con rangos (temperatura): rango 1: 0 a 80 verde, rango 2: 80 a 100 amarillo, rango 3: 100 a 120 rojo; fuera, el LED es gris.

Indicación de rango de la aguja: hay tres rangos, cada uno con mínimo, máximo y color. Un valor x pertenece a un rango si se cumple **mínimo <= x < máximo** (el mínimo se incluye, el máximo no). Ejemplo: rango 1: 0 a 80 verde, rango 2: 80 a 100 amarillo, rango 3: 100 a 120 rojo. Cada rango puede activarse o desactivarse por separado; las partes fuera de la escala se recortan y el mínimo debe ser menor que el máximo. Con escala automática, las bandas siguen la escala actual.

Guardado y límites

Elementos, posiciones, tamaños, vinculaciones y ajustes se guardan en el proyecto (`builder`). El área de dibujo tiene desplazamiento. El GUI Builder solo muestra valores; no hay elementos de entrada ni de mando. Solo pueden mostrarse señales de la selección de la pestaña Datos.

Nota: las señales solo aparecen cuando se han recibido o enviado datos; hasta entonces el campo de texto y la aguja muestran «--» o la posición cero y el LED está apagado.

Anexo A: Referencia de comandos de la secuencia de prueba

Cada paso es un objeto JSON con el campo `type` y los elementos indicados a continuación. Los valores físicos son valores tras aplicar el factor y el offset del DBC.

set_signal - Establecer valor de señal

Establece el valor físico de una señal en un mensaje de envío. El nuevo valor sale al bus con la siguiente trama (cíclica o única).

Formato

```
{"type": "set_signal", "message": "<mensaje>", "signal": "<señal>", "value": <número o texto>}
```

Elementos

Elemento	Tipo	Descripción
`message`	Texto, obligatorio	Nombre del mensaje DBC. Si no está en la lista de envío, se añade (inicialmente sin envío cíclico activo).
`signal`	Texto, obligatorio	Nombre de la señal dentro del mensaje.
`value`	Número o texto, obligatorio	Valor físico (tras factor/offset, p. ej. 40 para 40 %). En señales con tabla de valores también su texto, p. ej. "R". Los valores fuera del rango de la señal se limitan.

Ejemplo

```
{
  "type": "set_signal",
  "message": "DriverCmd",
  "signal": "ThrottleCmd",
  "value": 40
}
```

Nota: Las señales del mensaje no indicadas permanecen sin cambios (valor inicial 0).

enable_tx - Activar/desactivar envío cíclico

Inicia o detiene el envío cíclico de un mensaje con su ciclo configurado.

Formato

```
{"type": "enable_tx", "message": "<mensaje>", "enabled": <true|false>}
```

Elementos

Elemento	Tipo	Descripción
`message`	Texto, obligatorio	Nombre del mensaje DBC (se añade a la lista de envío si hace falta; ciclo del DBC, si no 100 ms).
`enabled`	true/false, obligatorio	true = enviar cíclicamente, false = detener el envío.

Ejemplo

```
{
  "type": "enable_tx",
  "message": "DriverCmd",
  "enabled": true
}
```

Nota: El primer telegrama sale de inmediato y después según el ciclo configurado.

set_cycle - Cambiar tiempo de ciclo

Cambia el tiempo de ciclo de un mensaje de envío (surte efecto de inmediato, incluso mientras envía cíclicamente).

Formato

```
{"type": "set_cycle", "message": "<mensaje>", "cycle_ms": <número>}
```

Elementos

Elemento	Tipo	Descripción
`message`	Texto, obligatorio	Nombre del mensaje DBC.
`cycle_ms`	Número, obligatorio	Tiempo de ciclo en milisegundos. 0 pausa el envío cíclico.

Ejemplo

```
{
  "type": "set_cycle",
  "message": "DriverCmd",
  "cycle_ms": 20
}
```

send_once - Enviar mensaje una vez

Envía exactamente una trama con los valores de señal actuales, independientemente del envío cíclico.

Formato

```
{"type": "send_once", "message": "<mensaje>"}
```

Elementos

Elemento	Tipo	Descripción
`message`	Texto, obligatorio	Nombre del mensaje DBC.

Ejemplo

```
{
  "type": "send_once",
  "message": "DriverCmd"
}
```

send_raw - Enviar trama sin procesar

Envía una trama sin procesar (ID + datos) al bus CAN, una vez o cíclicamente. Con cyclic=true y enabled=false se detiene el envío cíclico de este ID.

Formato

```
{"type": "send_raw", "id": "<hex>", "data": "<hex>", "dlc": <n>, "cyclic": <true|false>, "cycle_ms": <n>}
```

Elementos

Elemento	Tipo	Descripción
`id`	Número o texto, obligatorio	ID CAN, hexadecimal (p. ej. "0x100" o "100").
`data`	Texto hex, obligatorio	Datos como bytes hexadecimales, p. ej. "00 01 02 03 04 05 06 07".
`dlc`	Número, opcional	Longitud de datos (0-8, 12, 16, 20, 24, 32, 48, 64). Por defecto: número de bytes.
`cyclic`	true/false, opcional	true = envío cíclico, false = envío único (por defecto).
`cycle_ms`	Número, opcional	Tiempo de ciclo en ms con cyclic=true (por defecto 100).
`enabled`	true/false, opcional	false detiene el envío cíclico de este ID (solo con cyclic=true, por defecto true).
`extended`	true/false, opcional	true fuerza un ID extendido (por defecto: automático por encima de 0x7FF).
`fd`	true/false, opcional	true fuerza CAN FD (por defecto: automático si el DLC es superior a 8).

Ejemplo

```
{
  "type": "send_raw",
  "id": "0x100",
  "data": "00 01 02 03 04 05 06 07",
  "dlc": 8,
  "cyclic": true,
  "cycle_ms": 1000
}
```

Nota: Los bytes que faltan se rellenan con 00. ID superior a 0x7FF = extendido, DLC superior a 8 = CAN FD. Las tramas enviadas cíclicamente siguen activas después del paso.

wait - Esperar un tiempo fijo

Pausa la secuencia durante un tiempo fijo. El envío cíclico y la recepción continúan.

Formato

```
{"type": "wait", "ms": <número>}
```

Elementos

Elemento	Tipo	Descripción
`ms`	Número, obligatorio	Tiempo de espera en milisegundos.

Ejemplo

```
{
  "type": "wait",
  "ms": 500
}
```

Nota: Un clic en Detener también finaliza de inmediato una espera en curso.

wait_for - Esperar una condición de señal

Espera hasta que una señal cumpla la condición. Solo cuentan los valores recibidos DESPUÉS del inicio de este paso. El paso es OK en la primera coincidencia; si no, falla al agotarse el timeout.

Formato

```
{"type": "wait_for", "direction": "RX", "message": "<mensaje>", "signal": "<señal>", "op": "<operador>", "value": <número>, "timeout_ms": <n
```

Elementos

Elemento	Tipo	Descripción
direction	"RX" o "TX", obligatorio	Evaluar el mensaje recibido o el enviado.
message	Texto, obligatorio	Nombre del mensaje DBC.
signal	Texto, obligatorio	Nombre de la señal.
op	Texto, obligatorio	Comparación: == != < <= > >=
value	Número, obligatorio	Valor de comparación (físico).
timeout_ms	Número, opcional	Tiempo máximo de espera en ms (por defecto 3000).

Ejemplo

```
{
  "type": "wait_for",
  "direction": "RX",
  "message": "EngineData",
  "signal": "RPM",
  "op": ">",
  "value": 100,
  "timeout_ms": 3000
}
```

Nota: La evaluación es independiente de la selección de señales de la pestaña 'Datos'.

check - Comprobar valor de señal

Comprueba de inmediato el último valor recibido de una señal (sin esperar). El paso falla si no se cumple la condición o si aún no se ha recibido ningún valor desde el inicio de la secuencia.

Formato

```
{"type": "check", "direction": "RX", "message": "<mensaje>", "signal": "<señal>", "op": "<operador>", "value": <número>}
```

Elementos

Elemento	Tipo	Descripción
direction	"RX" o "TX", obligatorio	Evaluar el mensaje recibido o el enviado.
message	Texto, obligatorio	Nombre del mensaje DBC.
signal	Texto, obligatorio	Nombre de la señal.
op	Texto, obligatorio	Comparación: == != < <= > >=
value	Número, obligatorio	Valor de comparación (físico).

Ejemplo

```
{
  "type": "check",
  "direction": "RX",
  "message": "EngineData",
  "signal": "CoolantTemp",
  "op": "<",
  "value": 150
}
```

Nota: Justo tras el inicio suele no haber aún ningún valor - sincronice antes con wait_for.

diag - Servicio de diagnóstico del CDD

Envía por ISO-TP un servicio de diagnóstico definido en el CDD y evalúa la respuesta.

Formato

```
{"type": "diag", "service": "<nombre_servicio>", "params": {"<parámetro>": <valor>}, "expect": "<prefijo_hex>", "timeout_ms": <número>}
```

Elementos

Elemento	Tipo	Descripción
service	Texto, obligatorio	Nombre del servicio en el CDD (pestaña Diagnóstico), p. ej. ReadVIN.
params	Objeto, opcional	Valores para {Platzhalter} en el request. Número = decimal, texto = hexadecimal ("F186").
expect	Texto hex, opcional	Inicio esperado de la respuesta, p. ej. "62 F1 90". Vacío = basta cualquier respuesta positiva.
timeout_ms	Número, opcional	Timeout de respuesta en ms (por defecto 1000). Con 'ResponsePending' (NRC 0x78) se espera más tiempo.

Ejemplo

```
{
  "type": "diag",
  "service": "ExtendedSession",
  "params": {
    "session": 3
  },
  "expect": "50 03"
}
```

Nota: El direccionamiento (Request-/Response-ID, FD) procede de la pestaña 'Diagnóstico'.

diag_raw - Request de diagnóstico en hex

Envía por ISO-TP cualquier request UDS (bytes hex) y evalúa la respuesta.

Formato

```
{"type": "diag_raw", "request": "<bytes_hex>", "expect": "<prefijo_hex>", "timeout_ms": <número>}
```

Elementos

Elemento	Tipo	Descripción
`request`	Texto hex, obligatorio	Bytes del request, p. ej. "22 F1 86".
`expect`	Texto hex, opcional	Inicio esperado de la respuesta, p. ej. "62 F1 86". Vacío = basta cualquier respuesta positiva.
`timeout_ms`	Número, opcional	Timeout de respuesta en ms (por defecto 1000).

Ejemplo

```
{
  "type": "diag_raw",
  "request": "3E 00",
  "expect": "7E 00"
}
```

Nota: Los requests sin respuesta (sub-función con el bit 7 activado, p. ej. "3E 80") se consideran OK.

loop - Bucle

Ejecuta varias veces seguidas los pasos contenidos.

Formato

```
{"type": "loop", "count": <número>, "steps": [ <paso>, <paso>, ... ]}
```

Elementos

Elemento	Tipo	Descripción
`count`	Número, obligatorio	Número de iteraciones.
`steps`	Lista, obligatorio	Pasos anidados (de cualquier tipo, también otros bucles).

Ejemplo

```
{
  "type": "loop",
  "count": 3,
  "steps": [
    {
      "type": "set_signal",
      "message": "DriverCmd",
      "signal": "ThrottleCmd",
      "value": 20
    },
    {
      "type": "wait",
      "ms": 200
    }
  ]
}
```

Nota: En el resultado, los pasos internos aparecen p. ej. como 3.2.1 (paso 3, iteración 2, paso 1).

comment - Comentario

No hace nada; solo sirve para documentar la secuencia.

Formato

```
{"type": "comment", "text": "<texto>"}
```

Elementos

Elemento	Tipo	Descripción
`text`	Texto, opcional	Cualquier comentario.

Ejemplo

```
{
  "type": "comment",
  "text": "Hier beginnt der Hochlauftest"
}
```

Anexo B: Notas y límites

- **Precisión temporal**: el envío cíclico se realiza mediante un temporizador de software (precisión del orden de pocos milisegundos), sin disparo por hardware.
- **DBC**: no se admiten el multiplexado extendido ni las variables de entorno.
- **CDD**: los archivos originales de CANdelaStudio solo de forma heurística (capítulo 2.2).
- **PEAK + CAN FD**: temporización de bits fija (80 MHz, 500k/2M); para desviaciones, use `hardware.extra`.
- **Seguridad**: el envío cíclico nunca se inicia automáticamente al cargar un proyecto. En buses de vehículo o de banco de pruebas, revise la lista de envío y los valores antes de conectar.
- **Teclado**: en las tablas se edita con doble clic; en el campo de diagnóstico raw, Intro envía el request.

Pruebas sin hardware

El proyecto de demostración `demo` usa la simulación con unidad de control virtual. Con ella se pueden probar todas las pestañas (enviar/recibir, trace, gráfico, logging, secuencias *startup* y *driving*, diagnóstico) sin hardware. Las pruebas básicas se ejecutan con `python -m unittest discover -s tests -v`.

Anexo C: paquete vFlash y manifiesto

La pestaña vFlash de la sección Diagnóstico carga un paquete de flasheo, lo interpreta y lo ejecuta con la secuencia de flasheo UDS integrada en el hardware seleccionado (Vector, PEAK, SocketCAN, simulación). El software Vector vFlash no es necesario.

Fuentes admitidas

- `*.vflashpack` o `*.zip` con `flashpack.json` (o `flashpack.yaml`) y los archivos de datos: totalmente compatible.
- Paquete vFlash original sin manifiesto: el formato interno no está documentado públicamente. Se encuentran los archivos HEX, S-record y BIN y el proyecto se lee de forma heurística. La secuencia, las direcciones y Seed&Key normalmente deben añadirse mediante un manifiesto.
- Archivo de manifiesto único o imagen (`.hex`, `.s19`, `.srec`, `.mot`, `.bin`) con la secuencia predeterminada genérica.

Manifiesto flashpack.json

```
{
  "format": "canTool-flashpack-1",
  "name": "Demo Application",
  "ecu": "Engine_ECU",
  "request_id": "0x7E0",
  "response_id": "0x7E8",
  "extended": false,
  "fd": false,
  "security": {
    "type": "xor",
    "level": 1,
    "value": "0xFFFFFFFF"
  },
  "address_format": "0x44",
  "data": [
    {
      "file": "app.hex",
      "name": "Application"
    },
    {
      "file": "cal.bin",
      "address": "0x00020000",
      "name": "Calibration"
    }
  ],
  "steps": [
    {
      "type": "session",
      "session": 3
    },
    {
      "type": "comm_control",
      "control": 3,
      "comm_type": 1
    },
    {
      "type": "session",
      "session": 2
    },
    {
      "type": "security",
      "level": 1
    },
    {
      "type": "erase",
      "block": "all"
    },
    {
      "type": "download",
      "block": "all"
    },
    {
      "type": "check",
      "routine": "FF01"
    },
    {
      "type": "reset",
      "reset": 1
    }
  ]
}
```

Elemento	Significado
`name`, `ecu`	Nombre para mostrar y ECU.
`request_id`, `response_id`, `extended`, `fd`	Direccionamiento ISO-TP; si faltan, se aplica la configuración de diagnóstico de la sección Diagnóstico.
`security`	Seed&Key: `type` = `none`, `xor`, `add` o `constant`, más `level` (nivel de seed, key = level + 1) y `value`.
`address_format`	addressAndLengthFormatIdentifier, p. ej. `0x44` = longitud de 4 bytes y dirección de 4 bytes.
`max_chunk`	Límite superior de la carga útil por solicitud de transferencia (de lo contrario se toma de la respuesta a 0x34).
`data`	Lista de archivos de datos (`file`, `name`); para archivos BIN, `address` es obligatorio.
`steps`	Secuencia propia; si falta, se aplica la secuencia predeterminada.

Pasos de la secuencia (steps)

Paso	Parámetros y efecto

`session`	`session`: sesión de diagnóstico (0x10).
`comm_control`	`control`, `comm_type`: control de comunicación (0x28).
`security`	`level`: solicitar seed, calcular y enviar key (0x27).
`erase`	`block` (`all` o número), `routine` (por defecto FF00): borrar memoria (0x31).
`download`	`block`: solicitar descarga, transferir datos, finalizar transferencia (0x34, 0x36, 0x37).
`check`, `routine`	`routine`, `data` (hex), `control`, `timeout_ms`: iniciar una rutina (0x31).
`request`	`data` (hex), `expect` (primeros bytes esperados de la respuesta).
`reset`	`reset`: reinicio de la ECU (0x11).
`wait`, `comment`	`ms` tiempo de espera o `text` como comentario.

Secuencia predeterminada

Sin `steps` se ejecuta lo siguiente en orden: sesión 0x03, desactivar la comunicación (0x28 03 01), sesión 0x02, acceso de seguridad (si `security` está definido), borrar y programar todos los bloques, reinicio de la ECU 0x11 01. Esta secuencia es genérica y no es específica de ningún fabricante.

Notas de seguridad

El flasheo borra y sobrescribe la memoria de la ECU. La alimentación y la conexión al bus no deben interrumpirse mientras tanto. El código de los paquetes nunca se ejecuta; las DLL de Seed&Key específicas del fabricante no son compatibles. Si se produce un error, la secuencia se detiene y los pasos restantes se omiten.

中文(简体) – 用户手册

CAN Tool · 发布日期:2026-10-29

1. 概述与项目创建

2. 启动前的文件配置

3.1 “项目”页

3.2 “数据”页

3.3 “Trace”页

3.4 “测试序列”页

3.5 “图表”页

3.6 “记录”页

3.7 “诊断”页

3.8 标签栏：深色模式与语言

3.9 “帮助”页

3.10 XCP 页

3.11 GUI Builder 页

附录 A：测试序列命令参考

附录 B：提示与限制

附录 C:vFlash 包与清单文件

1. 概述与项目创建

1.1 工具用途

CAN Tool 是用于测试和分析 CAN 与 CAN FD 总线的 Python 工具。可使用 **Vector VN1630A / VN7572** 或 **PEAK-System PCAN-USB Pro FD**（均通过 python-can），也可在无硬件时使用内置的**仿真**。

核心功能：

- 管理多个**项目**（每个项目有各自的 DBC、CDD、序列和日志文件）。
- 使用前将 **DBC/CDD 转换**为 JSON 或 YAML（加载快，便于检查）。
- 报文的**周期发送**与接收，以物理值访问信号。
- 所有发送和接收帧的 **Trace**，可选信号解码。
- **测试序列** (JSON)：设置信号、等待响应、检查、执行诊断。
- 所选信号的**折线图**。
- 可切换外观（**深色模式**）和**六种语言**（德语、英语、法语、西班牙语、中文/普通话、泰语）。
- 带过滤的 **Parquet 记录**。
- 基于 ISO-TP (ISO 15765-2) 的**诊断**，使用 CDD 中的 UDS 服务。

1.2 数据流概览

```
dbc/*.dbc  --(转换)-->  cache/*.dbc.json|yaml  ---+
cdd/*.cdd  --(转换)-->  cache/*.cdd.json|yaml  ---+--> “数据”页：选择 TX / RX
                                                   |        |
                                                   |        +--> Trace   (所有帧)
                                                   |        +--> 图表   (所选信号)
                                                   |        +--> 记录 (所选信号，过滤)
sequences/*.json  ------------------------------------------+--> 测试序列 (设置/检查信号，诊断)
```

运行时始终只读取**缓存** (JSON/YAML)。DBC/CDD 文件发生变化后， 下次打开项目时会自动重新转换（依据源文件的校验和）。

1.3 安装与启动

```
pip install -r requirements.txt
python main.py                       # GUI
python main.py --project demo        # 使用指定项目启动 GUI
python main.py --list                # 列出项目
python main.py --convert demo        # 仅转换 DBC/CDD（无 GUI）
python main.py --convert demo --format yaml --force
```

驱动：Vector 需要 *Vector XL Driver Library*（Vector 驱动的一部分），PEAK 需要 *PCAN-Basic* 驱动。无硬件时使用仿真即可。

1.4 文件夹结构

```
can_tool/
  main.py  gui.py  can_manager.py  dbc_manager.py  cdd_manager.py
  iso_tp_manager.py  sequence_runner.py  parquet_logger.py  project_manager.py
  help_content.py  i18n.py
  lang/               语言文件 (de.yaml, en.yaml, fr.yaml, es.yaml, zh.yaml, th.yaml)
  tools/check_lang.py 语言文件检查
  projects/
    <项目名称>/
      project.json      项目设置（由工具写入）
      dbc/              您的 DBC 文件 (*.dbc)
      cdd/              您的诊断描述 (*.cdd)
      sequences/        测试序列 (*.json)
      cache/            自动生成：转换后的 DBC/CDD (JSON 或 YAML)
      logs/             Parquet 日志文件
```

1.5 创建项目

在 GUI 中

1. 打开**项目**页并点击**新建 ...**。
2. 输入项目名称。允许使用字母、数字、下划线、连字符和空格（最多 64 个字符，首字符须为字母或数字）。若名称已存在，不会覆盖任何内容。
3. 工具会创建 `projects/<name>/`，包含子文件夹 `dbc`, `cdd`, `sequences`, `cache`, `logs` 以及带默认值（仿真、500 kbit/s、JSON 缓存）的 `project.json`，并切换到新项目。
4. **导入 DBC/CDD ...**：文件会被复制到 `dbc/` 或 `cdd/` 并立即转换。也可以手动把文件放入文件夹，然后点击**重新转换**（或重新打开项目）。
5. 设置硬件（见 2.3 节）并点击**连接**。

通过文件

按上述结构创建项目文件夹，并放入 `project.json`，其内容至少为 `{}` - 缺失的设置会用默认值补全。演示项目 `projects/demo` 可作为模板。

切换项目

在项目列表中选择。切换（以及退出）时会断开连接、停止正在进行的 记录并保存当前项目。保存内容包括：硬件、发送列表（含信号值和 周期）、接收选择、记录过滤、图表选择、诊断设置以及最近加载的序列。 打开项目后，周期发送始终处于**关闭**状态（保证真实总线的安全）。

1.6 典型工作流程

1. 创建或选择项目，导入 DBC（及 CDD）。
2. 设置硬件，**连接**。
3. **数据**页：从 DBC 中选择发送和接收报文，设置信号值和周期。
4. 开启周期发送；在 **Trace** 和**图表**中观察结果。
5. 需要时启动**记录**和/或创建并执行**测试序列**。
6. **诊断**：发送服务，检查响应。

2. 启动前的文件配置

连接之前，必须先准备好项目文件。本章说明各文件中应包含什么内容， 以及工具会评估哪些信息。

2.1 DBC 文件 (`dbc/*.dbc`)

来自 CANdb++、CANdb Editor、SavvyCAN 等的标准 DBC 文件。每个项目可有多个 DBC 文件 （它们的总和构成数据基础）。字符集为 UTF-8 或 Windows-1252。

被评估的内容：

DBC 元素	在工具中的含义
`BO_`	报文：ID（bit 31 置位 = 扩展 ID）、名称、长度（DLC > 8 = CAN FD）、发送方。
`SG_`	信号：起始位、长度、字节顺序 (Intel/Motorola)、符号、系数、偏移、最小/最大值、单位、接收方。
`SG_ ... M / m0, m1`	简单复用（一个复用器，多个分组）。**不**支持扩展复用（给出警告）。
`VAL_`	值表：每个原始值对应的文本。在工具中发送时为下拉列表，接收时显示为文本。
`SIG_VALTYPE_`	浮点信号（32/64 位）。
`CM_`	报文/信号的注释。
`BA_ "GenMsgCycleTime"`	报文的默认周期时间 (ms)；未指定时为 100 ms。
`BA_ "VFrameFormat"`	帧格式；含 *FD* 的值表示 CAN FD 报文。

示例：

```
VERSION "1.0"
NS_ :
BS_:
BU_: Tester ECU

BO_ 512 EngineData: 8 ECU
 SG_ RPM : 0|16@1+ (0.25,0) [0|8000] "rpm" Tester
 SG_ CoolantTemp : 16|8@1+ (1,-40) [-40|130] "degC" Tester

VAL_ 257 StartButton 0 "Aus" 1 "Ein" ;
BA_DEF_ BO_ "GenMsgCycleTime" INT 0 65535;
BA_ "GenMsgCycleTime" BO_ 512 10;
```

配置提示

- 报文名和信号名会用于测试序列 - 请选用有意义且唯一的名称。
- 多个 DBC 文件中报文名重复：保留第一个定义，第二个被忽略并给出警告。
- 相同 ID 但名称不同：接收时归属到第一个名称（警告）。
- 发送节点 (`BU_`) 决定仿真中虚拟控制器周期发送哪些报文（默认：发送方 `ECU`，可在 `sim.traffic_senders` 中设置）。DBC 中合理的最小/最大值可产生真实的仿真值。
- 转换产生的警告显示在**项目**页的*已转换文件 / 警告*字段中。

缓存 (`cache/*.dbc.json` 或 `.yaml`)

打开项目时，每个 DBC 都会自动写入缓存（格式可在**项目**页 选择）。仅当源文件发生变化时才重新转换；**重新转换**可强制执行。 对缓存的手动修改会在重新转换时被覆盖 - 修正应在 DBC 中进行。

```
{
  "format_version": 1,
  "nodes": ["Tester", "ECU"],
  "messages": {
    "EngineData": {
      "id": 512, "extended": false, "dlc": 8, "fd": false, "sender": "ECU",
      "cycle_time_ms": 10, "comment": "", "multiplexor": null,
      "signals": {
        "RPM": {"start": 0, "length": 16, "byte_order": "little", "signed": false,
                "factor": 0.25, "offset": 0, "min": 0, "max": 8000, "unit": "rpm",
                "receivers": ["Tester"], "choices": {}, "comment": ""}
      }
    }
  },
  "warnings": [], "source": "engine.dbc", "source_sha1": "..."
}
```

2.2 诊断描述 (`cdd/*.cdd`)

CDD 描述诊断服务。与 DBC 一样，它会被转换为 `cache/*.cdd.json|yaml`。

重要限制：完全支持简单的 XML 格式 `<DIAGDATA>` （见示例和 `projects/demo/cdd/diag.cdd`）。原始 CANdelaStudio CDD 文件仅以 **启发式**方式读取（服务名称，以及能找到时的请求字节）。这种情况下请检查生成的 缓存文件（**项目**页，警告），并在其中或使用 简单格式补充地址、请求和响应参数。

```
<?xml version="1.0" encoding="UTF-8"?>
<DIAGDATA>
  <ECU name="Engine_ECU" request_id="0x7E0" response_id="0x7E8" extended="false" fd="false">
    <SERVICE name="ExtendedSession" request="10 {session}" description="Select diagnostic session">
      <PARAM name="session" bytes="1" default="03"/>
    </SERVICE>
```

```
    <SERVICE name="ReadVIN" request="22 F1 90" description="Read VIN">
      <RESPONSE prefix_len="3">
        <PARAM name="VIN" offset="0" length="17" type="ascii"/>
      </RESPONSE>
    </SERVICE>
    <SERVICE name="ReadSpeed" request="22 F4 0D" description="Read speed">
      <RESPONSE prefix_len="3">
        <PARAM name="Speed" offset="0" length="1" type="uint" factor="1" unit="km/h"/>
      </RESPONSE>
    </SERVICE>
  </ECU>
</DIAGDATA>
```

元素 / 属性	含义
`ECU name, request_id, response_id`	控制器名称和 CAN ID（测试仪 -> ECU，ECU -> 测试仪），如 `0x7E0`/`0x7E8`。
`ECU extended, fd`	`true`：29 位 ID 或 CAN FD（ISO-TP，每帧最多 64 字节）。
`SERVICE name`	服务名称（用于诊断页和序列步骤 `diag`）。
`SERVICE request`	十六进制请求字节。占位符用花括号，如 `10 {session}`。
`SERVICE description`	自由文本，显示在服务列表中。
`PARAM`	直接位于 SERVICE 内：占位符的输入参数：`name`、`bytes`（宽度）、`default` (Hex)。
`RESPONSE prefix_len`	响应开头要跳过的字节数（服务 ID + DID，如 `62 F1 90` 时为 3）。
`RESPONSE/PARAM`	响应参数：`name`、`offset` 和 `length`（字节，从前缀结尾算起），`type` = `ascii | uint | int | hex`，可选 `factor`、`offset_val`、`unit`。

对应的缓存条目（供手动修改）：

```
{
  "format_version": 1,
  "ecus": {
    "Engine_ECU": {
      "request_id": "0x7E0", "response_id": "0x7E8", "extended": false, "fd": false,
      "services": {
        "ReadVIN": {
          "request": "22 F1 90", "description": "Read VIN", "params": [],
          "response": {"prefix_len": 3,
                       "params": [{"name": "VIN", "offset": 0, "length": 17, "type": "ascii"}]}
        }
      }
    }
  }
}
```

2.3 硬件与连接

在**项目**页中设置（保存在 `project.json` 的 `hardware` 下）：

设置	Vector VN1630A / VN7572	PEAK PCAN-USB Pro FD	仿真
接口	`vector` (XL 驱动)	`pcan` (PCAN-Basic)	`simulation`
通道	通道索引，从 0 开始（如 `0`）。通过 *Vector Hardware Config* 分配；设置了应用程序名称时，以其中分配的通道号为准。	`PCAN_USBBUS1`, `PCAN_USBBUS2` …	不需要
波特率	标称波特率 (kbit/s)	标称波特率 (kbit/s)；FD 时固定为 500k/2M（80 MHz 时钟）	无意义
CAN FD / 数据波特率	勾选并填写数据波特率	勾选；不同的时序在 `hardware.extra` 中设置	勾选以使用 FD 帧
应用程序名称	Vector Hardware Config 中的名称（留空 = python-can 默认值）	-	-

Vector VN7572(PCIe 卡): Vector 接口卡 VN7572 与 VN1630A 一样,也以 `vector` 类型通过 Vector XL 驱动使用(仅限 Windows)。安装 *Vector Driver Setup* 后,该卡必须在 *Vector Hardware Config* 中可见。通道号从 0 开始(0 = CH1,1 = CH2 ...);**检测硬件**会列出找到的通道及其序列号。不得有其他程序(CANalyzer、CANoe、vFlash)占用该通道。

- 总线上的所有节点必须使用相同的波特率；CAN FD 时还须使用相同的数据波特率。
- 在两台设备的桌面搭建中，总线两端必须接 120 欧姆终端电阻。
- `hardware.extra`（仅在 `project.json` 中）可将任意参数传递给 python-can，如 PEAK 的 `{"nom_brp": 5}`。
- **仿真**：*回显*将自己的发送报文作为接收返回；*虚拟控制器*在诊断设置的地址上响应 UDS 请求（包括 10、11、14、19、22、27、3E），并周期发送 ECU 发送方的报文。

2.4 序列文件 (`sequences/*.json`)

序列由名称、描述和步骤列表组成。可在**测试序列**页中创建， 也可直接编辑文件。命令参考：附录 A。

```
{
  "name": "startup",
  "description": "Simulate engine start and check run-up",
  "steps": [
    {"type": "enable_tx", "message": "DriverCmd", "enabled": true},
    {"type": "set_signal", "message": "DriverCmd", "signal": "StartButton", "value": 1},
    {"type": "wait_for", "direction": "RX", "message": "EngineData", "signal": "RPM",
     "op": ">", "value": 100, "timeout_ms": 3000},
    {"type": "diag", "service": "ReadVIN", "expect": "62 F1 90"}
```

```
  ]
}
```

2.5 项目文件 (`project.json`)

由工具写入；需要时可手动编辑（编辑时请先关闭工具）。缺失的键会用默认值补全。

```
{
  "hardware": {"type": "vector", "channel": "0", "bitrate": 500000, "fd": true,
               "data_bitrate": 2000000, "app_name": "", "extra": {}},
  "cache_format": "json",
  "tx": [{"message": "DriverCmd", "cycle_ms": 50, "values": {"ThrottleCmd": 0}, "signals": null}],
  "rx": {"EngineData": null, "VehicleSpeed": ["Speed"]},
  "logging": {"selection": {"TX": {}, "RX": {"EngineData": null}},
              "on_change_only": false, "min_interval_ms": 0, "log_raw": false},
  "graph": ["RX:EngineData.RPM"],
  "diag": {"cdd": "diag.cdd.json", "ecu": "Engine_ECU", "request_id": "", "response_id": "",
           "extended": false, "fd": false, "padding": 170},
  "sequence": "startup.json",
  "sim": {"ecu": true, "traffic_senders": ["ECU"]}
}
```

键	含义
`hardware`	连接参数（2.3 节）；波特率单位为 bit/s。
`cache_format`	转换文件使用 `json` 或 `yaml`。
`tx`	发送列表：报文、周期 (ms)、信号值，`signals` = 图表/记录的选择（`null` = 全部）。
`rx`	接收选择：报文 -> 所选信号列表或 `null`（全部）。
`logging`	记录过滤器和选项（记录页）。
`graph`	图表中已勾选信号的键，如 `RX:Msg.Signal`。
`diag`	CDD 文件、ECU、可选的地址覆盖（`request_id`、`response_id`）、`extended`、`fd`、`padding`。
`sequence`	最近加载的序列（`sequences/` 中的文件名）。
`sim`	仿真：`ecu`（虚拟控制器）、`traffic_senders`（被周期仿真的发送节点）。

2.6 首次连接前的检查清单

1. DBC 文件位于 `dbc/` 文件夹；项目页中显示预期的报文（数量），且没有意外的警告。
2. 可选的 CDD 位于 `cdd/` 文件夹；诊断页中显示 ECU 和服务。
3. 已安装硬件驱动、已连接设备，通道和波特率与总线匹配。
4. 使用 CAN FD 时：勾选 *CAN FD* 并设置数据波特率；对端也须支持 FD。
5. `sequences/` 下已有序列，或已在测试序列页中创建。

常见问题

现象	可能原因 / 解决办法
数据页中没有报文	DBC 不在 `dbc/` 文件夹中，或扩展名不是 `.dbc`；请查看项目页中的警告。
连接失败	缺少 python-can/驱动、通道错误、设备被其他应用占用（如 CANalyzer/PCAN-View）。
能看到帧但未解码	DBC 中的 ID/扩展标识与总线不一致；DBC 中缺少该报文。
信号值错误	检查 DBC 中的字节顺序 (Intel/Motorola)、起始位、系数/偏移。
诊断：超时	检查诊断页中的请求/响应 ID、扩展/FD 和填充；控制器是否处于合适的会话？
记录不可用	缺少 Python 包 `pyarrow` (`pip install pyarrow`)。

3.1 “项目”页

操作系统：工具启动时会自动识别运行在 **Windows** 还是 **Linux**（显示在标题栏）。硬件选项随之调整：*Vector*（XL 驱动）仅限 Windows，*SocketCAN* 仅限 Linux，*PEAK* 和*仿真*两者皆可。不可用的条目会被标记并禁用。**检测硬件**按钮会列出已连接的接口。其余所有功能（Trace、序列、日志、诊断、vFlash、帮助/PDF）在两个系统上完全一致；项目文件夹可在 Windows 与 Linux 之间互换。

带注释的界面图



界面示意图;编号对应下方说明。

1. 选择项目、新建项目并导入 DBC/CDD 文件。
2. 缓存格式(json/yaml)及重新转换 DBC/CDD。
3. 硬件:接口、通道(如 PCAN_USBBUS1)、波特率和 CAN FD。
4. 连接或断开,并显示状态(接收和发送计数)。
5. 信息栏:已加载的 DBC 文件、报文数量和警告。

重新初始化硬件：位于*连接*旁边的按钮，会关闭接口并以相同设置重新打开。当所连接的硬件卡死时可使用。发送列表保持不变（处于循环发送状态的报文之后继续发送）；正在进行的日志记录会停止。

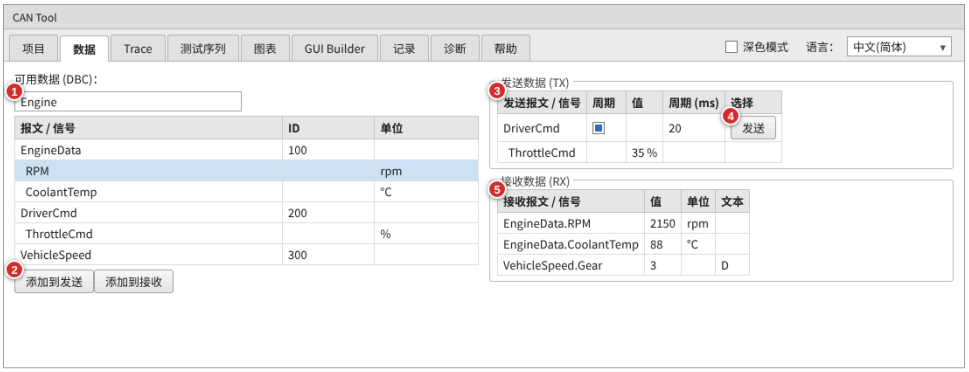
管理项目、导入 DBC/CDD、设置硬件并建立连接。

控件	功能
当前项目	所有项目的选择列表 (`projects/*/project.json`)。切换会断开连接、停止记录、保存旧项目并加载新项目。
新建 ...	创建带文件夹结构和默认设置的项目（1.5 节）。
导入 DBC/CDD ...	将所选 `.dbc`/`.cdd` 文件复制到项目中并转换。
缓存格式	`json` 或 `yaml`；更改后会重新转换。
重新转换	强制转换所有 DBC/CDD 并重新加载项目。
接口 / 通道 / 波特率 / CAN FD / 数据波特率 / 应用程序名称	硬件参数（2.3 节）。连接期间锁定。
仿真：回显	自己的发送报文同时作为接收显示。
仿真：虚拟控制器	连接时启动支持 UDS 的软件控制器，并产生测试流量。
连接 / 断开	打开或关闭接口。断开时会停止正在进行的记录。
状态栏	连接状态、已接收 (RX) 和已发送 (TX) 的帧数、最近一次错误。
已转换文件 / 警告	缓存文件列表、报文数量、来自 DBC/CDD 的警告。

3.2 “数据”页

在此确定发送和接收**哪些**报文，以及图表、 记录和显示使用**哪些信号** - 均来自项目中所有 DBC 数据的总和。

带注释的界面图



界面示意图;编号对应下方说明。

1. 过滤框及 DBC 中所有报文和信号的列表。
2. 将所选项添加到发送数据 (TX) 或接收数据 (RX)。
3. 发送数据:周期发送开/关,设置信号值和周期时间。
4. 单次发送:立即发送一次该报文。
5. 接收数据,显示信号的实时值。

左侧：可用数据

- 包含所有报文及其信号的树（ID、发送方、单位）。**过滤框**在报文名称、ID 和信号名称中搜索。
- **-> 发送 (TX)** 将选中的报文加入发送列表（双击报文效果相同）。可多选；选中某个信号时，会添加其所属报文。
- **-> 接收 (RX)** 将选中的报文加入接收列表。

右上：发送

列	含义
发送报文 / 信号	报文（含 ID）及其下方的所有信号。
周期	报文上的勾选框用于开启/关闭周期发送（仅在已连接时才真正发送）。
值	信号的物理值（输入框；带值表的信号为选择列表）。更改从下一帧起生效。
周期 [ms]	报文的周期时间；默认取自 DBC (`GenMsgCycleTime`)，否则为 100 ms。0 = 非周期。
选择	信号上的勾选框：该信号属于图表和记录的选择。
发送（按钮）	以当前值恰好发送一次该报文。

移除会将选中行所属的报文从发送列表中删除。

右下：接收

列	含义
接收报文 / 信号	报文（含 ID）及其信号。
值 / 单位 / 文本	最近收到的值（每 200 ms 刷新）、单位，以及（如有）值表中的文本。
选择	信号上的勾选框：该信号属于图表和记录的选择。未勾选的信号不会被保存。

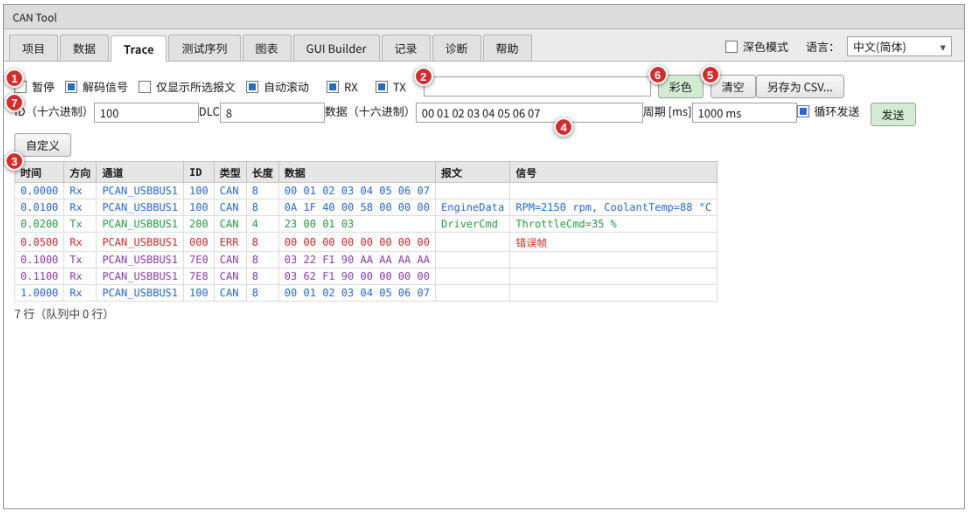
提示

- 选择（勾选）决定**图表**和**记录**能看到什么。而 **Trace** 显示所有帧（可选仅显示所选报文）。
- 选择和所有发送值会随项目保存；加载后周期发送为关闭状态。
- 同一报文可同时出现在 TX 和 RX 列表中（如仿真中的回显）。

3.3 “Trace”页

按时间顺序显示所有发送（Tx，彩色高亮）和接收 (Rx) 的帧。

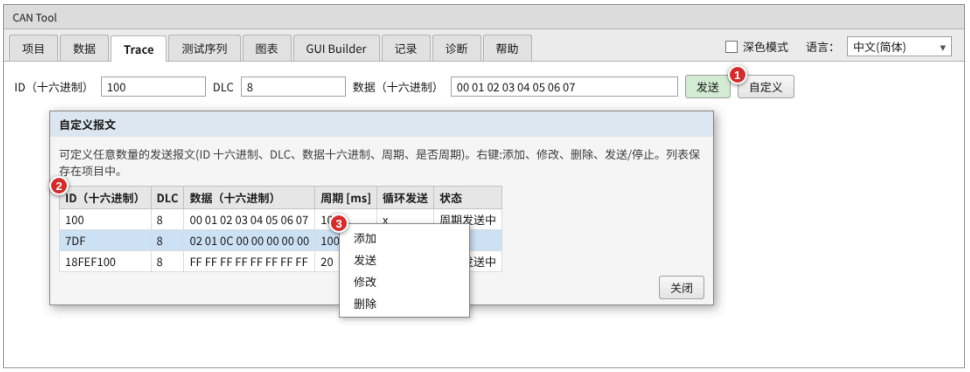
带注释的界面图



界面示意图;编号对应下方说明。

1. 选项:暂停、解码信号、仅显示所选报文、自动滚动。
2. 按 ID 或报文名称过滤。
3. 跟踪表:时间、方向 (Rx/Tx)、通道、ID、类型、长度和原始数据。没有 DBC/CDD 也可使用。
4. 解码后的报文名称和信号值(已加载 DBC 时)。
5. 清空跟踪或保存为 CSV 文件。
6. 彩色模式:Rx 蓝色,Tx 绿色,错误红色,ISO-TP 紫色。
7. 发送行：ID（十六进制）、DLC、数据、周期时间、循环发送复选框和发送按钮。

带注释的界面图



界面示意图;编号对应下方说明。

1. “自定义”按钮打开自定义发送报文窗口。
2. 发送报文表:ID(十六进制)、DLC、数据(十六进制)、周期 [ms]、是否周期和状态。
3. 右键菜单:添加、发送/停止、修改、删除。

带注释的界面图



界面示意图;编号对应下方说明。

1. ID(十六进制):CAN ID,也可为扩展 ID(> 7FF)。
2. DLC:数据长度,也可为 CAN FD 数值。
3. 数据(十六进制):字节可带或不带空格;缺少的字节用 00 补足。
4. 周期 [ms] 和周期发送复选框:发送一次或周期发送。
5. 确定将报文加入列表,取消则放弃输入。

列	内容
时间 [s]	相对于第一个显示帧的时间（自连接或*清空*起）。
Rx/Tx、通道	方向和接口。
ID、类型、Len	CAN ID（十六进制，扩展 ID 为 8 位）、类型（CAN、FD、FD/BRS）和数据长度。
数据	十六进制有效数据。
报文、信号	来自 DBC 的名称，以及带单位的解码信号（启用*解码*时）。
彩色模式	切换按钮。开启:Rx 报文为中蓝色,Tx 报文为中绿色,错误报文(总线错误帧,类型 ERR)为红色,ISO-TP 报文(诊断请求/响应 ID,含 0x7DF)为中紫色。关闭:与以前相同的显示。

控件	功能
暂停	停止显示。帧继续缓冲（最多 50 000 个），取消暂停后补充显示。
解码	开启/关闭信号解码（关闭 = 总线负载高时更快）。
仅显示所选报文	仅显示数据页 TX 或 RX 列表中的报文。
自动滚动	自动跟随最新条目。
Rx / Tx	显示/隐藏相应方向。
过滤框	以逗号分隔的十六进制 ID（如 `100, 200`）和/或报文名称的一部分（如 `engine`）。
清空	清除显示和缓冲区。
另存为 CSV ...	导出当前可见的行（分隔符 `;`）。

原地更新：勾选*原地更新*后，每条报文（方向、通道、ID）在表中只出现一次，按首次出现的顺序排列。再次出现时，更新该行的时间戳、数据和解码后的信号，并增加*次数*列。不勾选时，所有帧按顺序滚动写入。切换时会清空表格；自动滚动仅在滚动模式下有效。

时间差：在*原地更新*的基础上，勾选*时间差*后，时间列显示与同一报文（方向、通道、ID）上次出现的间隔（即周期时间），而不是绝对时间；首次出现时为 0。列标题变为*时间差 [s]*。切换时会清空表格。

发送行：选项栏下方的一行用于发送原始帧（默认：ID 100 十六进制，DLC 8，数据 00 01 02 03 04 05 06 07，1000 ms）。**发送**只发送一次；勾选**循环发送**后开始循环发送，再次点击（**停止**）即可结束。大于 7FF 的 ID 以扩展帧发送，大于 8 的 DLC 以 CAN FD 发送（需在连接设置中启用 FD）。该帧会在 Trace 中显示为 Tx。

自定义报文(多条发送)

发送行一次只发送一条报文。需要**多条报文**时使用**自定义**按钮:它会打开一个独立窗口,并保持在 Trace 旁边。在其中可定义任意数量的发送报文并逐条发送。列表保存在项目中(`custom_tx`),下次打开项目时仍然可用。

定义并发送报文

1. 点击发送行中的**自定义**按钮:打开“自定义报文”窗口。
2. 在表格中**单击右键**,选择**添加**。
3. 在“定义报文”对话框中输入 ID(十六进制)、DLC、数据(十六进制)、周期 [ms],必要时勾选**周期发送**,点击**确定**。无效输入(如不允许的 DLC 或数据字节过多)会提示并被拒绝,不会保存任何内容。
4. 在该行上单击右键并选择**发送**:未勾选周期发送时只发送一次;勾选后开始周期发送(状态:“周期发送中”)。此时该菜单项变为**停止**。

字段	含义
ID(十六进制)	CAN ID,十六进制,无前缀(如 `7DF`)。大于 7FF 的 ID 自动按扩展 ID(29 位)发送。
DLC	数据长度:0 到 8,以及 12、16、20、24、32、48、64。大于 8 的值按 CAN FD 发送(硬件和项目须支持 CAN FD)。
数据(十六进制)	十六进制字节,可带或不带空格。输入过短时用 00 补足到 DLC,过长则报错。
周期 [ms]	发送间隔,1 到 3 600 000 ms;仅在周期发送时有效。
周期发送	不勾选:发送一次;勾选:周期发送直到停止。

右键菜单:添加(也可在空白处)、发送或停止、修改(也可双击)和删除。正在发送的条目在修改或删除前会先停止。发送需要已建立连接;断开连接时所有条目的周期发送都会停止。ID 相同的多个条目共用同一个发送任务。已发送的报文在 Trace 中显示为 Tx。

提示：最多显示 5 000 行（较旧的行会滚出）。仅当该页 可见时才更新显示；在后台工具最多缓冲 50 000 帧。如需无遗漏的记录，请使用**记录**。

3.4 “测试序列”页

创建、保存、加载和执行测试序列。始终**仅有一个序列**处于加载/活动状态； 多个序列以文件形式存放在项目的 `sequences/` 文件夹中。

带注释的界面图



界面示意图;编号对应下方说明。

1. 选择、加载、新建和保存测试序列文件(一次一个序列)。
2. 命令下拉列表:鼠标悬停在条目上时显示包含格式、元素和示例的提示。
3. 插入、移动、复制或删除步骤。
4. 步骤列表:命令和参数为 JSON,可直接在表格中编辑。
5. 启动、停止并静态检查序列;旁边显示总体结果。
6. 各步骤结果表:绿色 = 通过,红色 = 失败,附带信息。

文件

控件	功能
文件列表 + 加载	从 `sequences/` 中选择序列并加载到编辑器。
新建	空序列（在点击*保存*之前不会保存）。
保存 / 另存为 ...	将序列写为 JSON。最近保存/加载的文件名会记录在项目中，下次打开时自动重新加载。
名称 / 描述	序列的元数据。

编辑步骤

步骤显示在表格中（序号、类型、JSON 参数）。**参数文本可直接编辑**（双击）。无效的 JSON 会被拒绝并恢复旧值；步骤的类型是固定的。

控件	功能
命令下拉列表	选择步骤类型。**鼠标悬停在条目上时会出现工具提示**，包含格式、所有元素的说明和一个示例；下方另外显示所选命令的简介。
添加步骤	以模板形式在选中行之后（或末尾）插入该命令。
上移 / 下移 / 复制 / 删除	移动、复制或删除选中的步骤。
信号选择 + 应用到步骤	以 `Message.Signal` 形式列出 DBC 中的所有信号（支持输入补全），并将报文（对于 `set_signal`、`wait_for`、`check` 还包括信号）填入选中的步骤。
表格中的工具提示	鼠标悬停在步骤的类型列上也会显示命令工具提示。

检查与执行

控件	功能
检查	静态检查：未知的步骤类型、运算符、报文和信号（对照 DBC）。
开始	执行序列（仅在已连接时）。启动前会自动检查。运行期间编辑器被锁定。
停止	中止序列（包括正在进行的 `wait`）；状态为 *aborted*。
结果表	每个步骤：序号（循环中如 `3.2.1`）、描述、状态（绿色 = ok，红色 = fail/error）和信息（如测得的值、响应字节）。

运行规则：步骤依次执行。**第一个失败的步骤会结束序列**（状态 *failed*）。 结束后状态栏显示，如 *startup: passed (10/10 个步骤 ok)*。以周期方式启动的报文在序列结束后继续发送， 除非通过 `enable_tx` 停止。

命令概览

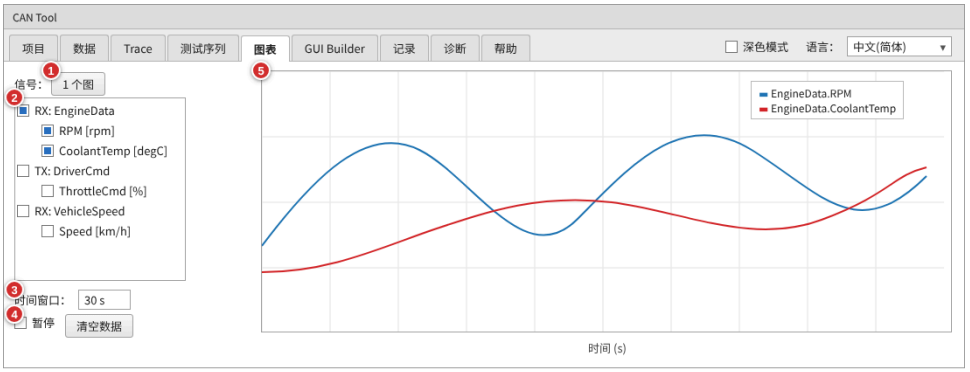
命令	标题	简介
set_signal	设置信号值	设置发送报文中某信号的物理值。新值随下一帧（周期或单次）发送到总线。
enable_tx	开启/关闭周期发送	按设定周期启动或停止报文的周期发送。
set_cycle	修改周期时间	修改发送报文的周期时间（立即生效，周期发送过程中同样有效）。
send_once	发送一次报文	以当前信号值恰好发送一帧，与周期发送无关。
send_raw	发送原始帧	在 CAN 总线上发送原始帧（ID + 数据），单次或循环。cyclic=true 且 enabled=false 时，停止该 ID 的循环发送。
wait	等待固定时间	使序列暂停固定时间。周期发送和接收继续进行。
wait_for	等待信号条件	等待某信号满足条件。仅统计在本步骤开始之后收到的值。首次满足即为 ok，超时则为错误。
check	检查信号值	立即检查某信号最近收到的值（不等待）。若条件不满足，或序列启动后尚未收到值，则该步骤失败。
diag	来自 CDD 的诊断服务	通过 ISO-TP 发送 CDD 中定义的诊断服务并评估响应。
diag_raw	十六进制诊断请求	通过 ISO-TP 发送任意 UDS 请求（十六进制字节）并评估响应。
loop	循环	依次多次执行其中包含的步骤。
comment	注释	不执行任何操作；仅用于记录序列说明。

每条命令的完整说明（含格式、元素和示例）见附录 A。

3.5 “图表”页

本页以折线图实时显示信号曲线(时间轴:相对于现在的秒数,左 = 过去,右 = 现在)。选择区上方的开关用于在**1 个图**和**n 个图**之间切换。

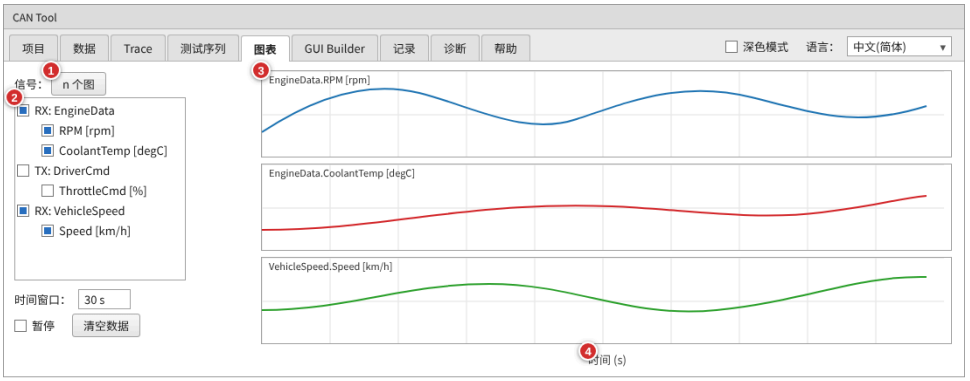
带注释的界面图



界面示意图;编号对应下方说明。

1. 开关 1 个图 / n 个图(此处:1 个图)。
2. 按报文分组的信号选择;报文复选框可选中其所有信号,单位在方括号中。
3. 时间窗口(秒)。
4. 暂停冻结显示,清除数据丢弃曲线。
5. 带图例的共用图表;时间轴为相对于现在的秒数。

带注释的界面图



界面示意图;编号对应下方说明。

1. 开关设为 n 个图。
2. 选择仍按报文分组;此处选中了三个信号。
3. 每个信号有独立的图和数值轴,标注为 报文.信号 及单位。
4. 共用时间轴:缩放和平移作用于所有图;刻度仅显示在底部。

按报文分组的选择

选择区列出在“数据”页中选定的所有信号,按方向和报文分组(RX = 接收,TX = 发送)。每个报文下方是其信号,单位显示在方括号中。报文前的复选框有三种状态:全部信号已选、部分已选(半选状态)、全部未选。单击可选中该报文的所有信号或全部取消。

显示方式“1 个图”

所有选定信号绘制在同一个带图例(颜色对应)的图中,便于直接比较曲线。若数值范围相差很大,较小的信号会显得很平,此时更适合使用“n 个图”。

显示方式“n 个图”

每个选定信号都有独立的图和独立的数值轴,标注为 报文.信号 及单位;各图上下排列。所有图共用同一**时间基准**:在任一图中缩放或平移时间轴,会同时作用于所有图,时间刻度只显示在最下方的图。

操作元素

操作元素	功能
开关 1 个图 / n 个图	切换显示方式,界面立即重建。该设置保存在项目中(`graph_multi`)。
信号选择	勾选 = 显示曲线。报文前的复选框可选中或取消其所有信号。选择内容保存在项目中(`graph`)。

时间窗口	可见时间范围,单位秒(1 到 3600)。
暂停	冻结显示;数据仍在后台继续收集。
清除数据	丢弃所有已收集的信号曲线。
图表	鼠标:滚轮缩放,按住按键平移。在“n 个图”中对所有图同时生效。

提示：每个信号最多保留 50 000 个点。仅记录数据页中已勾选的信号。 信号只有在收到/发送数据后才会出现。仅当该页可见时才更新显示。

3.6 “记录”页

解析后的信号：加载了 DBC、A2L 或 CDD 后，Parquet 文件不仅包含十六进制报文，还包含带有**原始值、原始十六进制、物理值、文本（值表）和单位**的信号。信号文件的列：`timestamp, direction, message, signal, value, source, can_id, raw, raw_hex, text, unit, message_hex`。记录期间，`source` 为 `CAN`（DBC 报文）、`XCP`（XCP 页读取的 A2L 值）或 `UDS`（诊断页中来自 CDD 的已解码响应参数）。原始文件（`_raw.parquet`）另有 `signals` 列，以文本形式给出该帧的所有信号，例如 `RPM=0x1F40/2000 rpm`。

带注释的界面图



界面示意图;编号对应下方说明。

1. 选择要记录的报文和信号(从已选数据中过滤)。
2. 全选或全不选信号。
3. Parquet 文件名(保存在项目的 logs/ 文件夹中)。
4. 选项:仅在值变化时、每个信号的最小间隔、另外记录原始数据。
5. 开始或停止记录;状态行显示行数和文件大小。

以 **Parquet** 格式记录所选信号（需要 `pyarrow`）。文件位于项目的 `logs/` 文件夹中。

控件	功能
数据树	显示数据页中选择的报文/信号。**过滤：**仅记录已勾选的信号（勾选报文会选中/取消其所有信号）。选择会保存在项目中。
全选 / 全不选	选中或取消选中所有信号。
文件名	默认为 *项目_日期_时间.parquet*；需要时自动补全扩展名。
每个信号最小间隔	每个信号在该时间段 (ms) 内最多记录一条 - 可减少高速报文的数据量。0 = 每一帧。
仅在值变化时	仅当信号值与上次写入的值不同时才写入。
原始帧	另外写入 *<名称>_raw.parquet*，包含总线上的**所有**帧（与 DBC 和信号选择无关，包括错误帧）。列：时间、方向、通道、ID、扩展、FD、BRS、错误、DLC、数据（二进制）、数据（十六进制文本）、报文名称。因此无需选择信号也可记录。**原始数据 ID** 用于限制帧，例如 `100, 7E0-7EF`（十六进制，留空 = 全部）。
开始 / 停止记录	开始或结束记录（仅在已连接时）。断开连接或切换项目时会自动停止并正确关闭文件。
状态栏	文件名、已写入的行数、文件大小。

文件格式

列	类型	内容
timestamp	float64	时间戳（Epoch 秒）
direction	string	`RX` 或 `TX`
message	string	报文名称
signal	string	信号名称
value	float64	物理值

长格式（每个信号值一行）可避免因周期时间不同而产生的空缺。数据先缓冲， 每 5 秒或达到 20 000 行时写入磁盘（zstd 压缩）。

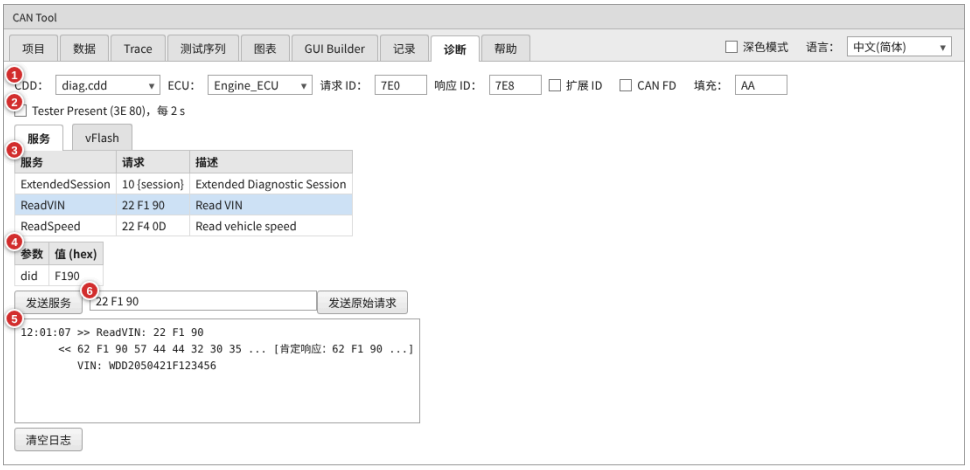
在 Python 中分析（另见附录 B）：

```
import pandas as pd
df = pd.read_parquet("projects/demo/logs/demo_20261020_071500.parquet")
wide = df.pivot_table(index="timestamp", columns=["direction", "message", "signal"], values="value")
```

3.7 “诊断”页

在 CAN 或 CAN FD 上通过 **ISO-TP** (ISO 15765-2) 进行诊断服务 (UDS)。服务来自转换后的 CDD； 此外还可发送任意十六进制请求。

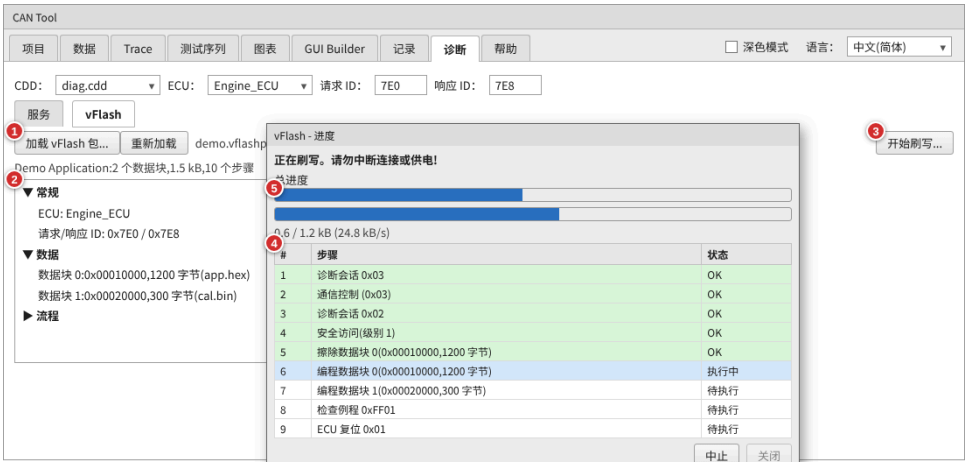
带注释的界面图



界面示意图;编号对应下方说明。

1. 诊断配置:CDD、ECU、请求/响应 ID、扩展 ID、CAN FD、填充。
2. 每 2 秒自动发送 Tester Present (3E 80)。
3. CDD 中的服务,含请求模板和说明。
4. 所选服务的参数(十六进制值可编辑),然后点击“发送服务”。
5. 响应日志:已发送的请求、响应(肯定/否定)及解码值。
6. 发送十六进制原始请求(如 22 F1 90),无需 CDD 定义。

带注释的界面图



界面示意图;编号对应下方说明。

1. 加载 vFlash 包(.vflashpack/.zip、清单文件、HEX/S19/BIN);“重新加载”会再次读取该文件。
2. 包的解析结果:常规(ECU、ID、Seed&Key)、数据(含地址和长度的数据块)、流程(步骤)以及提示。
3. 开始刷写:经安全提示确认后,流程将在所选硬件上运行。
4. 进度窗口:显示每个步骤的状态(待执行、执行中、OK、失败、已跳过)及详情。
5. 总进度、当前数据块的进度(含数据速率)和日志;“中止”会在两个数据块之间停止。

配置（顶部一行）

字段	含义
CDD	来自项目缓存的已转换诊断描述。
ECU	CDD 中的控制器；切换时会采用其地址。
请求 ID / 响应 ID	十六进制 CAN ID（测试仪 -> ECU / ECU -> 测试仪）。默认取自 CDD，可在此覆盖。
扩展 ID	29 位寻址。
CAN FD	使用 FD 帧的 ISO-TP（每帧最多 64 字节，带波特率切换）。须与对端一致。
填充	十六进制填充字节（默认 AA）；留空 = 经典 CAN 下不填充。

Tester Present	每 2 s 发送 `3E 80`（无响应），以保持诊断会话。
vFlash 选项卡	加载 vFlash 包(或清单文件/HEX/S19/BIN),显示解析结果,并在所选硬件上运行刷写流程,附带进度窗口(附录 C)。

发送服务

1. 在列表中选中服务（名称、请求、描述）。
2. 如果服务带有参数，在*参数 / 值 (hex)*表中填入值（请求中的占位符）。
3. **发送服务**。发送在后台进行；界面仍可操作。
4. **日志**显示请求 (>>)、响应 (<<) 及其评估（肯定/否定，否定响应时显示 NRC 及文字说明），以及 CDD 中解码的响应参数（如 *VIN: WDD...*）。

发送原始请求：以十六进制字节输入请求（如 `22 F1 86`），按回车或点击按钮。

技术细节

- 分段：Single Frame、First Frame、Consecutive Frames 和 Flow Control，最多 4095 字节（更长的通过扩展 First Frame）。
- 接收较长响应时，工具发送 Blocksize 0 和 STmin 0 的 Flow Control。
- 响应超时 2 s；遇到 *ResponsePending* (NRC 0x78) 时最多继续等待 10 s。
- 无响应的请求（带 bit 7 的子功能，如 `3E 80`）记录为已发送。
- 在测试序列中，`diag` 和 `diag_raw` 使用相同的地址配置。

3.8 标签栏：深色模式与语言

在标签栏右侧（标签页旁）有两个控件，对所有页面生效。

带注释的界面图



界面示意图;编号对应下方说明。

1. 深色模式:在浅色和深色外观之间切换(会被保存)。
2. 语言:列出所有可用语言文件的下拉列表;切换立即生效,包括帮助。

控件	功能
深色模式	勾选框在浅色和深色外观之间切换。影响界面、Trace、图表、序列结果表以及本帮助。该设置会被保存并在下次启动时使用。
语言	下拉菜单，含 Deutsch（主语言）、English、Francais、Espanol、中文（普通话，简体）和泰语。切换立即生效，无需重启。所选语言会被保存。

哪些内容会被翻译？

- 界面的所有标签、按钮、表头、工具提示、对话框和信息。
- 测试序列的命令参考（下拉列表中的工具提示、附录 A）以及本帮助的所有章节。
- 程序各部分的错误和警告信息（如转换、连接、序列运行）。
- **不**翻译您文件中的数据：报文名和信号名、DBC/CDD 中的单位和值表、文件名、项目名以及序列文件的内容。
- 已写入的日志行（诊断日志、序列结果表）保持其生成时所用的语言。

使用中文和泰语时，计算机上必须安装包含相应字符的字体 （常见的 Windows、macOS 和 Linux 安装通常都具备）。

语言文件

每种语言有各自的文件，位于 `lang/` 文件夹（YAML；也可使用 JSON）：

文件	语言
`lang/de.yaml`	Deutsch - 主语言和回退语言：若某语言缺少某文本，则使用德语文本。
`lang/en.yaml`	English
`lang/fr.yaml`	Francais
`lang/es.yaml`	Espanol
`lang/zh.yaml`	中文（普通话，简体字）
`lang/th.yaml`	泰语

每个文件的结构：`meta`（显示名称、下拉列表中的顺序）、`ui`（界面）、 `core`（信息）、`steps`（命令参考）和 `help`（本帮助的章节）。花括号中的占位符（如 `{name}`）以及帮助文本中双方括号内的占位符 不得更改。

添加或修改语言

1. 复制 `lang/de.yaml` 并保存为 `lang/<code>.yaml`（如 `it.yaml`）。
2. 在 `meta` 部分设置显示名称 (`name`) 和位置 (`order`)；翻译所有文本。
3. 用 `python tools/check_lang.py it` 检查：该工具会报告缺失的键、不一致的占位符和损坏的 HTML 结构。
4. 重启程序 - 该语言会自动出现在下拉列表中。

命令行

`main.py` 的输出同样已翻译：`python main.py --lang en --list`。 未使用 `--lang` 时，采用最近在 GUI 中选择的语言。

3.9 “帮助”页

即本页。左侧为章节，右侧为正文。

带注释的界面图



界面示意图;编号对应下方说明。

1. 帮助章节列表。
2. 搜索框:隐藏没有匹配的章节;“下一个匹配”跳转到下一处。
3. 章节正文及带注释的插图。
4. PDF 手册:导出所有可用语言的帮助(含封面和发布日期)并提供保存。

- **搜索框**：隐藏不含该词的章节，并在所选章节中 标出第一处匹配。**下一个匹配**跳转到下一处。清空输入框后重新显示所有章节。
- 正文中的链接（如指向附录）可以点击。
- 命令说明与测试序列页命令下拉列表中的工具提示一致。
- 帮助以所选语言显示（3.8 节），切换语言时立即替换；浅色/深色切换在此同样有效。

3.10 “XCP”页

基于 **A2L 文件**（ASAP2），通过 **XCP on CAN** 测量和调整 ECU 变量。A2L 的转换方式与 DBC 和 CDD 相同： 源文件位于项目文件夹 `a2l/`，运行时只读取缓存 `cache/<name>.a2l.json|yaml`。

设置

1. 用**导入**（项目页）把 A2L 复制到项目中；它会自动转换（`python main.py --convert demo` 也可在无 GUI 时使用）。
2. 连接 CAN（项目页）。XCP 使用同一接口。
3. XCP 页：选择 A2L，检查**主站 ID**（发给 ECU 的命令）和**从站 ID**（响应）。它们由 A2L 的 XCP 部分（`IF_DATA XCP / XCP_ON_CAN`）预填，可修改。
4. **连接 XCP**。日志以十六进制显示所有命令（->）和响应（<-）。

控件	功能
左侧列表	A2L 中的所有测量量（MEASUREMENT）和参数（VALUE 类型的 CHARACTERISTIC）；按名称过滤；双击或*加入监视*即可采用。
监视	显示每个条目的原始值和带单位的物理值。选择会随项目保存。
循环读取	按设定间隔（ms）读取所有被监视的条目。通过 SHORT_UPLOAD 轮询，不使用 DAQ 列表。
读取一次	读取一次监视列表。
写入值	把输入的值（物理值；文本转换时也可用文本，例如 *RUN*）写入所选行（SET_MTA + DOWNLOAD），然后读回。

A2L 的解释

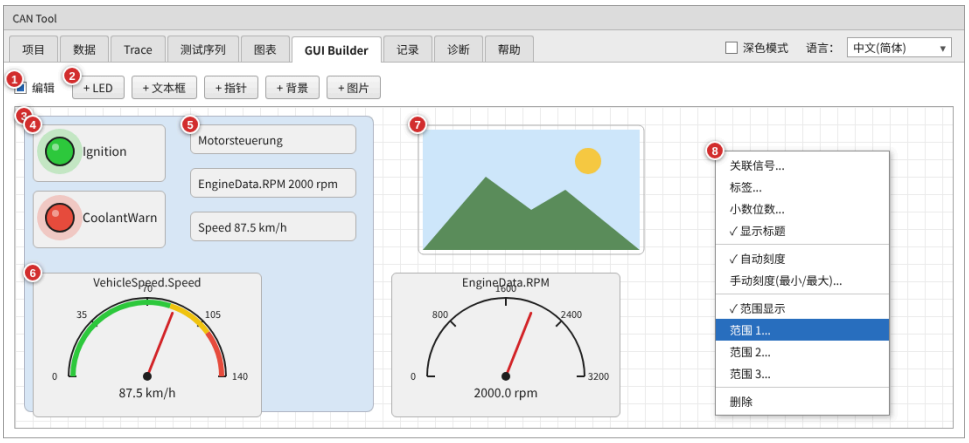
- 数据类型 UBYTE、SBYTE、UWORD、SWORD、ULONG、SLONG、A_UINT64、A_INT64、FLOAT32_IEEE、FLOAT64_IEEE；字节序依据 ECU 的 CONNECT 响应。
- 转换方式 IDENTICAL、LINEAR、RAT_FUNC（不含二次项）和 TAB_VERB（文本值）；读取时会屏蔽 BIT_MASK。
- 不支持的对象（曲线、图、数组、无地址的测量量）显示为灰色。

硬件为*仿真*时，工具会启动一个映射项目 A2L 的模拟 XCP 从站（演示：`projects/demo/a2l/demo.a2l`）。 不包含：DAQ/STIM、Seed&Key、标定页和闪存编程。真实 ECU 是否允许所有访问取决于其保护级别。

3.11 “GUI Builder”页

在此用简单元素为现有信号搭建自己的显示界面。元素仅用于**显示**,不能输入数值。可关联在“数据”页中选定的信号(接收或发送)。布局和关联保存在项目中(`builder`)。

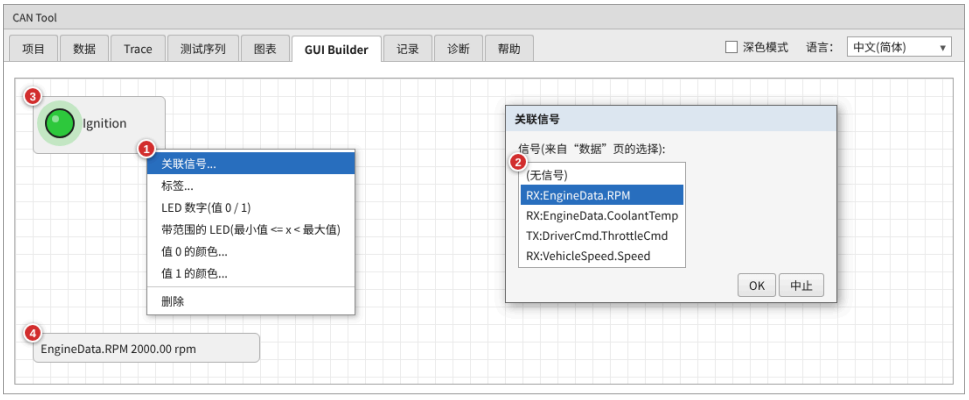
带注释的界面图



界面示意图;编号对应下方说明。

1. “编辑”复选框:可显示网格、添加、移动和调整大小。
2. 用于添加 LED、文本框、指针、背景和图片的按钮。
3. 可选颜色的背景元素:在视觉上归组元素,位于它们之后。
4. LED:数字(值 0 / 1)和带范围(颜色取决于范围 最小值 <= x < 最大值)。
5. 文本框:左侧无信号(纯文本),右侧带信号(实时数值和单位)。
6. 180 度刻度的指针,自动或手动刻度,可选范围显示。
7. 图片框:所选图片自动缩放以适应图片框。
8. 指针的右键菜单:信号、标题、刻度、范围显示和删除。

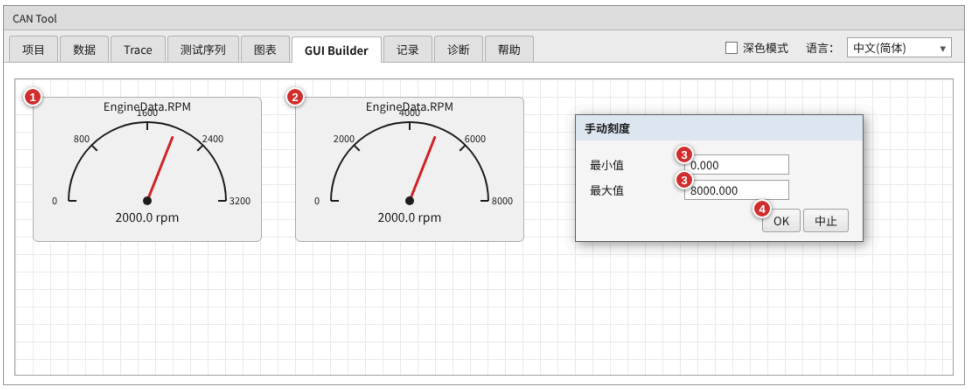
带注释的界面图



界面示意图;编号对应下方说明。

1. 右键单击 LED:关联信号、标签、数字/范围模式、颜色、删除。
2. 选择:“数据”页中的信号,格式 RX:/TX:报文.信号,或(无信号)。
3. LED:颜色取决于信号值(数字 0 / 1 或按范围 1 到 3)。
4. 带信号的文本框:文本、数值和单位;小数位数可调。

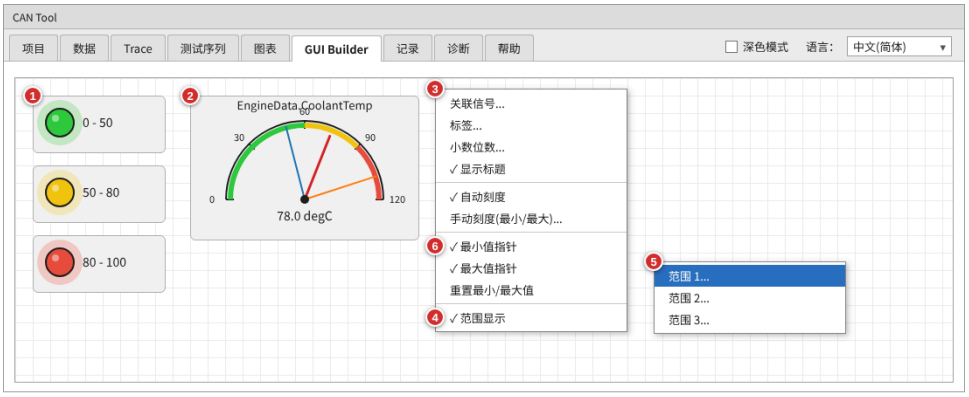
带注释的界面图



界面示意图;编号对应下方说明。

1. 自动刻度的指针:边界取整,此处为 0 到 3200。
2. 手动刻度的指针,此处为 0 到 8000。
3. 手动刻度对话框:输入最小值和最大值。
4. 确定应用刻度(仅当最小值小于最大值),取消则放弃。

带注释的界面图



界面示意图;编号对应下方说明。

1. 三个带范围的 LED:每个数值范围(最小值 <= x < 最大值)决定颜色。
2. 带范围显示的指针:刻度上有三个彩色范围(范围 1 到 3)。
3. 菜单:显示标题用于开关标签。
4. 菜单:开启或关闭范围显示。
5. 范围 1 到 3:各有最小值、最大值(规则:最小值 <= x < 最大值)、颜色和开关。
6. 菜单:可选的第二、第三根指针,分别表示最小值(蓝色)和最大值(橙色),以及重置。

元素

元素	功能
LED	以颜色显示信号状态。**数字**:值 0 和值 1 各有一种可选颜色。**范围**:与指针刻度相同,三个范围(最小值 <= x < 最大值),各有一种颜色;数值不在任何范围内或无数值时,LED 为灰色。
无信号文本框	纯文本框,文本可自由设定,例如用作标题或标签。
带信号文本框	显示文本及信号的当前值和单位,持续更新。小数位数可设置。
指针	刻度为 180 度(半圆)的指针仪表:左侧为最小值,右侧为最大值;下方显示带单位的数值。
背景	简单的彩色矩形(颜色可选),无其他功能。它在视觉上归组其他元素,始终位于功能元素之后、标准背景之上。
图片	带可选图片的图片框(PNG、JPG、BMP、GIF、SVG)。图片会自动适应图片框(保持宽高比),并随大小变化而调整。

操作

操作元素	功能
编辑	勾选:显示网格,可添加、移动、调整大小和删除元素。不勾选:界面锁定(仅显示)。
+ LED / + 文本框 / + 指针	添加新元素(也可在空白区域右键,在鼠标位置添加)。
拖动	移动元素;右下角用于调整大小。
右键单击元素	菜单:关联信号(或“无信号”)、标签或文本、LED 阈值和颜色、小数位数、刻度(指针)、删除。

指针与刻度

刻度有两种模式。**自动**(默认):范围随观测到的数值扩大,并取整为整数(如 0 到 100);刻度线和数字随之调整。**手动**:在指针上**右键**并选择**手动刻度(最小/最大)**,设定最小值和最大值(最小值必须小于最大值)。同一菜单中的**自动刻度**复选项可切回自动模式。超出刻度的数值会使指针停在端点。

快速入门

1. 保持勾选**编辑**(显示网格)。
2. 点击 **+ LED**、**+ 文本框** 或 **+ 指针**;元素出现在画布上(也可在空白区域右键,在鼠标位置添加)。
3. 在元素上**单击右键**,选择**关联信号**并从列表中选取信号。
4. 设置标签、LED 阈值、小数位数或刻度,并通过拖动放置元素;右下角用于调整大小。
5. 取消勾选**编辑**:界面被锁定,仅用于查看。

关联信号

列表显示在“数据”页中选定的信号,格式为 `RX:Msg.Signal` 或 `TX:Msg.Signal`,另有“(无信号)”项用于解除关联。若缺少某信号,请先在“数据”页中选择。收到或发送数据后数值即会显示;单位来自 DBC。更换信号时,自动刻度会被重置。

设置 LED 和文本框

设置	作用
标签 / 文本	LED:LED 旁边的标签(为空时显示信号名)。文本框:自由文本;带信号时位于数值之前。
LED 模式	数字(值 0 / 值 1)或范围(范围 1 到 3)。数字:值 0 对应颜色 0,其他任何值对应颜色 1。
颜色 / 范围	数字:值 0 和值 1 的颜色。范围:每个范围可开关,并按规则**最小值 <= x < 最大值**设置最小值、最大值和颜色;第一个匹配的范围决定颜色。与指针的范围显示设置相同。
小数位数	文本框和指针的小数位数,0 到 9。
标题(指针)	复选项**显示标题**用于显示或隐藏指针上方的标签;空间随后用于刻度。
范围显示(指针)	复选项**范围显示**按规则**最小值 <= x < 最大值**将刻度最多分为三个范围着色(范围 1 到 3),每个范围有最小值、最大值、颜色和开关;三个范围随后出现在菜单中。
最小/最大值指针(指针)	勾选**最小值指针**和**最大值指针**后,刻度上会额外显示细的第二根指针(蓝色)和第三根指针(橙色),分别表示自观测开始以来的最小值和最大值。**重置最小/最大值**重新开始;更换信号或切换复选项也会重置。数值不保存在项目中,只保存复选项。

指针刻度详解

自动:在收到第一个数值之前,范围为 0 到 100。之后指针记住观测到的最小值和最大值,并将边界取整为整数(1、2、2.5、5 乘以 10 的幂);若所有数值非负且接近 0,则刻度从 0 开始。刻度只会扩大,不会缩小。右键菜单中的**自动刻度**复选项(关闭再开启)会重新开始观测。刻度显示五个刻度线及数字。

手动:在指针上**单击右键**,选择**手动刻度(最小/最大)**:在对话框中输入最小值和最大值(可带小数)。若最小值不小于最大值,会提示且不会应用。手动刻度立即生效,保存在项目中,并关闭自动模式。超出范围的数值使指针停在端点。

背景与图片

元素分层放置:标准背景(画布),其上是背景元素,再上是功能元素(LED、文本框、指针、图片)。背景元素在添加、移动和加载时自动置于其他元素之后,因此彩色底框可以把多个显示归为一组。所选图片会复制到项目文件夹(`images`),使项目便于迁移;无图片时显示“无图片”。

LED 颜色与范围显示

数字 LED 示例:值 0 = 红色,值 1 = 绿色。范围 LED 示例(温度):范围 1:0 到 80 绿色,范围 2:80 到 100 黄色,范围 3:100 到 120 红色;范围之外 LED 为灰色。

指针的范围显示:共有三个范围,每个范围有最小值、最大值和颜色。当 **最小值 <= x < 最大值** 成立时,数值 x 属于该范围(包含最小值,不包含最大值)。示例:范围 1:0 到 80 绿色,范围 2:80 到 100 黄色,范围 3:100 到 120 红色。每个范围可单独开关;超出刻度的部分会被截去,最小值必须小于最大值。使用自动刻度时,色带随当前刻度调整。

保存与限制

元素、位置、大小、关联和设置都保存在项目中(`builder`)。画布可滚动。GUI Builder 仅显示数值,没有输入或开关元素。只能显示“数据”页所选信号。

提示:信号只有在收到或发送数据后才会出现;在此之前文本框和指针显示“--”或零位,LED 熄灭。

附录 A：测试序列命令参考

每个步骤都是一个 JSON 对象，包含字段 `type` 和下列元素。物理值是指经 DBC 的系数和偏移换算后的值。

set_signal - 设置信号值

设置发送报文中某信号的物理值。新值随下一帧（周期或单次）发送到总线。

格式

```
{"type": "set_signal", "message": "<消息>", "signal": "<信号>", "value": <数字或文本>}
```

元素

元素	类型	描述
`message`	文本，必填	DBC 报文名称。若不在发送列表中，则自动添加（初始不周期发送）。
`signal`	文本，必填	报文内的信号名称。
`value`	数字或文本，必填	物理值（经系数/偏移换算后，如 40 表示 40 %）。带值表的信号也可用其文本，如 "R"。超出信号范围的值会被限幅。

示例

```
{
  "type": "set_signal",
  "message": "DriverCmd",
  "signal": "ThrottleCmd",
  "value": 40
}
```

提示： 报文中未指定的信号保持不变（初始值 0）。

enable_tx - 开启/关闭周期发送

按设定周期启动或停止报文的周期发送。

格式

```
{"type": "enable_tx", "message": "<消息>", "enabled": <true|false>}
```

元素

元素	类型	描述
`message`	文本，必填	DBC 报文名称（必要时添加到发送列表；周期取自 DBC，否则为 100 ms）。
`enabled`	true/false，必填	true = 周期发送，false = 停止发送。

示例

```
{
  "type": "enable_tx",
  "message": "DriverCmd",
  "enabled": true
}
```

提示： 第一帧立即发出，之后按设定周期发送。

set_cycle - 修改周期时间

修改发送报文的周期时间（立即生效，周期发送过程中同样有效）。

格式

```
{"type": "set_cycle", "message": "<消息>", "cycle_ms": <数字>}
```

元素

元素	类型	描述
`message`	文本，必填	DBC 报文名称。
`cycle_ms`	数字，必填	周期时间，单位毫秒。0 表示暂停周期发送。

示例

```
{
  "type": "set_cycle",
  "message": "DriverCmd",
  "cycle_ms": 20
}
```

send_once - 发送一次报文

以当前信号值恰好发送一帧，与周期发送无关。

格式

```
{"type": "send_once", "message": "<消息>"}
```

元素

元素	类型	描述
message	文本，必填	DBC 报文名称。

示例

```
{
  "type": "send_once",
  "message": "DriverCmd"
}
```

send_raw - 发送原始帧

在 CAN 总线上发送原始帧（ID + 数据），单次或循环。cyclic=true 且 enabled=false 时，停止该 ID 的循环发送。

格式

```
{"type": "send_raw", "id": "<hex>", "data": "<hex>", "dlc": <n>, "cyclic": <true|false>, "cycle_ms": <n>}
```

元素

元素	类型	描述
id	数字或文本，必填	CAN ID，十六进制（例如 "0x100" 或 "100"）。
data	十六进制文本，必填	十六进制字节形式的数据，例如 "00 01 02 03 04 05 06 07"。
dlc	数字，可选	数据长度（0-8、12、16、20、24、32、48、64）。默认：数据字节数。
cyclic	true/false，可选	true = 循环发送，false = 单次发送（默认）。
cycle_ms	数字，可选	cyclic=true 时的周期时间（毫秒，默认 100）。
enabled	true/false，可选	false 停止该 ID 的循环发送（仅与 cyclic=true 一起使用，默认 true）。
extended	true/false，可选	true 强制使用扩展 ID（默认：大于 0x7FF 时自动）。
fd	true/false，可选	true 强制使用 CAN FD（默认：DLC 大于 8 时自动）。

示例

```
{
  "type": "send_raw",
  "id": "0x100",
  "data": "00 01 02 03 04 05 06 07",
  "dlc": 8,
  "cyclic": true,
  "cycle_ms": 1000
}
```

提示： 缺少的数据字节用 00 补齐。ID 大于 0x7FF 为扩展帧，DLC 大于 8 为 CAN FD。循环发送的帧在该步骤结束后继续发送。

wait - 等待固定时间

使序列暂停固定时间。周期发送和接收继续进行。

格式

```
{"type": "wait", "ms": <数字>}
```

元素

元素	类型	描述
ms	数字，必填	等待时间，单位毫秒。

示例

```
{
  "type": "wait",
  "ms": 500
}
```

提示： 点击停止会立即结束正在进行的等待。

wait_for - 等待信号条件

等待某信号满足条件。仅统计在本步骤开始之后收到的值。首次满足即为 ok，超时则为错误。

格式

```
{"type": "wait_for", "direction": "RX", "message": "<消息>", "signal": "<信号>", "op": "<运算符>", "value": <数字>, "timeout_ms": <数字>}
```

元素

元素	类型	描述
direction	"RX" 或 "TX"，必填	评估接收或发送的报文。
message	文本，必填	DBC 报文名称。
signal	文本，必填	信号名称。
op	文本，必填	比较：== != < <= > >=
value	数字，必填	比较值（物理值）。
timeout_ms	数字，可选	最长等待时间，单位 ms（默认 3000）。

示例

```
{
  "type": "wait_for",
  "direction": "RX",
  "message": "EngineData",
  "signal": "RPM",
  "op": ">",
  "value": 100,
  "timeout_ms": 3000
}
```

提示： 评估与“数据”页中的信号选择无关。

check - 检查信号值

立即检查某信号最近收到的值（不等待）。若条件不满足，或序列启动后尚未收到值，则该步骤失败。

格式

```
{"type": "check", "direction": "RX", "message": "<消息>", "signal": "<信号>", "op": "<运算符>", "value": <数字>}
```

元素

元素	类型	描述
direction	"RX" 或 "TX"，必填	评估接收或发送的报文。
message	文本，必填	DBC 报文名称。
signal	文本，必填	信号名称。
op	文本，必填	比较：== != < <= > >=
value	数字，必填	比较值（物理值）。

示例

```
{
  "type": "check",
  "direction": "RX",
  "message": "EngineData",
  "signal": "CoolantTemp",
  "op": "<",
  "value": 150
}
```

提示： 刚启动时通常还没有值 - 请先用 wait_for 同步。

diag - 来自 CDD 的诊断服务

通过 ISO-TP 发送 CDD 中定义的诊断服务并评估响应。

格式

```
{"type": "diag", "service": "<服务名>", "params": {"<参数>": <值>}, "expect": "<十六进制前缀>", "timeout_ms": <数字>}
```

元素

元素	类型	描述
service	文本，必填	CDD 中的服务名称（诊断页），如 ReadVIN。
params	对象，可选	请求中 {Platzhalter} 的值。数字 = 十进制，文本 = 十六进制 ("F186")。
expect	十六进制文本，可选	期望的响应开头，如 "62 F1 90"。留空 = 任何肯定响应均可。
timeout_ms	数字，可选	响应超时，单位 ms（默认 1000）。遇到 'ResponsePending' (NRC 0x78) 时会等待更久。

示例

```
{
  "type": "diag",
  "service": "ExtendedSession",
  "params": {
    "session": 3
  },
  "expect": "50 03"
}
```

提示： 寻址（请求/响应 ID、FD）取自“诊断”页。

diag_raw - 十六进制诊断请求

通过 ISO-TP 发送任意 UDS 请求（十六进制字节）并评估响应。

格式

```
{"type": "diag_raw", "request": "<十六进制字节>", "expect": "<十六进制前缀>", "timeout_ms": <数字>}
```

元素

元素	类型	描述
request	十六进制文本，必填	请求字节，如 "22 F1 86"。
expect	十六进制文本，可选	期望的响应开头，如 "62 F1 86"。留空 = 任何肯定响应均可。
timeout_ms	数字，可选	响应超时，单位 ms（默认 1000）。

示例

```
{
  "type": "diag_raw",
  "request": "3E 00",
  "expect": "7E 00"
}
```

提示： 无响应的请求（子功能置位 bit 7，如 "3E 80"）视为 ok。

loop - 循环

依次多次执行其中包含的步骤。

格式

```
{"type": "loop", "count": <数字>, "steps": [ <步骤>, <步骤>, ... ]}
```

元素

元素	类型	描述
count	数字，必填	循环次数。
steps	列表，必填	嵌套步骤（任意类型，也可包含其他循环）。

示例

```
{
  "type": "loop",
  "count": 3,
  "steps": [
    {
      "type": "set_signal",
      "message": "DriverCmd",
      "signal": "ThrottleCmd",
      "value": 20
    },
    {
      "type": "wait",
      "ms": 200
    }
  ]
}
```

提示： 结果中内部步骤显示为如 3.2.1（第 3 步，第 2 次循环，第 1 步）。

comment - 注释

不执行任何操作；仅用于记录序列说明。

格式

```
{"type": "comment", "text": "<文本>"}
```

元素

元素	类型	描述

text	文本，可选	任意注释。

示例

```
{
  "type": "comment",
  "text": "Hier beginnt der Hochlauftest"
}
```

附录 B：提示与限制

- **时间精度**：周期发送通过软件定时器实现（精度在几毫秒范围内），并非硬件触发。
- **DBC**：不支持扩展复用和环境变量。
- **CDD**：原始 CANdelaStudio 文件仅以启发式方式读取（2.2 节）。
- **PEAK + CAN FD**：位时序固定预设（80 MHz，500k/2M）；如需不同设置，请通过 `hardware.extra`。
- **安全**：加载项目时周期发送绝不会自动启动。在车辆或试验台总线上，连接前请检查发送列表和数值。
- **键盘**：表格中双击进行编辑；在原始诊断输入框中，按回车发送请求。

无硬件测试

演示项目 `demo` 使用带虚拟控制器的仿真。借此可在无硬件的情况下试用 所有页面（发送/接收、Trace、图表、记录、序列 *startup* 和 *driving*、诊断）。 核心测试通过 `python -m unittest discover -s tests -v` 运行。

附录 C:vFlash 包与清单文件

“诊断”部分的 vFlash 选项卡用于加载刷写包、对其进行解析,并使用内置的 UDS 刷写流程在所选硬件(Vector、PEAK、SocketCAN、仿真)上运行。无需 Vector 软件 vFlash。

支持的来源

- `*.vflashpack` 或 `*.zip`,包含 `flashpack.json`(或 `flashpack.yaml`)及数据文件:完全支持。
- 不含清单文件的原生 vFlash 包:其内部格式未公开。程序会找出 HEX、S-record 和 BIN 文件,并以启发式方式读取工程。流程、地址和 Seed&Key 通常需要通过清单文件补充。
- 单个清单文件或镜像(`.hex`、`.s19`、`.srec`、`.mot`、`.bin`),将使用通用默认流程。

清单文件 flashpack.json

```
{
  "format": "canTool-flashpack-1",
  "name": "Demo Application",
  "ecu": "Engine_ECU",
  "request_id": "0x7E0",
  "response_id": "0x7E8",
  "extended": false,
  "fd": false,
  "security": {
    "type": "xor",
    "level": 1,
    "value": "0xFFFFFFFF"
  },
  "address_format": "0x44",
  "data": [
    {
      "file": "app.hex",
      "name": "Application"
    },
    {
      "file": "cal.bin",
      "address": "0x00020000",
      "name": "Calibration"
    }
  ],
  "steps": [
    {
      "type": "session",
      "session": 3
    },
    {
      "type": "comm_control",
      "control": 3,
      "comm_type": 1
    },
    {
      "type": "session",
      "session": 2
    },
    {
      "type": "security",
      "level": 1
    },
    {
      "type": "erase",
      "block": "all"
    },
    {
      "type": "download",
      "block": "all"
    },
    {
      "type": "check",
      "routine": "FF01"
    },
    {
      "type": "reset",
      "reset": 1
    }
  ]
}
```

元素	含义
`name`, `ecu`	显示名称和 ECU。
`request_id`, `response_id`, `extended`, `fd`	ISO-TP 寻址;若缺省,则采用“诊断”部分的诊断配置。
`security`	Seed&Key:`type` = `none`、`xor`、`add` 或 `constant`,以及 `level`(seed 级别,key = level + 1)和 `value`。
`address_format`	addressAndLengthFormatIdentifier,例如 `0x44` = 4 字节长度和 4 字节地址。
`max_chunk`	每次传输请求的有效载荷上限(否则取自 0x34 的响应)。
`data`	数据文件列表(`file`、`name`);对于 BIN 文件,`address` 为必填项。
`steps`	自定义流程;若缺省,则采用默认流程。

流程步骤 (steps)

步骤	参数与作用
session	session:诊断会话 (0x10)。
comm_control	control、comm_type:通信控制 (0x28)。
security	level:请求 seed,计算并发送 key (0x27)。
erase	block(all 或编号)、routine(默认 FF00):擦除存储器 (0x31)。
download	block:请求下载、传输数据、退出传输 (0x34, 0x36, 0x37)。
check, routine	routine、data(十六进制)、control、timeout_ms:启动例程 (0x31)。
request	data(十六进制)、expect(期望的响应起始字节)。
reset	reset:ECU 复位 (0x11)。
wait, comment	ms 为等待时间,或 text 作为注释。

默认流程

未设置 steps 时,将按顺序执行:会话 0x03、禁用通信 (0x28 03 01)、会话 0x02、安全访问(若设置了 security)、擦除并编程所有数据块、ECU 复位 0x11 01。该流程为通用流程,非整车厂专用。

安全注意事项

刷写会擦除并覆盖 ECU 的存储器。在此期间不得中断供电和总线连接。包中的代码绝不会被执行;不支持整车厂专用的 Seed&Key DLL。若发生错误,流程将停止,其余步骤将被跳过。

ไทย – คู่มือ

CAN Tool · วันที่เผยแพร่: 2026-10-29

1. ภาพรวมและการสร้างโปรเจกต์

2. การเตรียมไฟล์ก่อนเริ่ม

3.1 แท็บโปรเจกต์

3.2 แท็บข้อมูล

3.3 แท็บ Trace

3.4 แท็บลำดับทดสอบ

3.5 แท็บกราฟ

3.6 แท็บ Logging

3.7 แท็บวินิจฉัย

3.8 แถบแท็บ: โหมดมืดและภาษา

3.9 แท็บช่วยเหลือ

3.10 แท็บ XCP

3.11 แท็บ GUI Builder

ภาคผนวก A: อ้างอิงคำสั่งลำดับทดสอบ

ภาคผนวก B: หมายเหตุและข้อจำกัด

ภาคผนวก C: แพ็ก vFlash และ Manifest

1. คำอธิบายทั่วไปและการสร้างโปรเจกต์

1.1 เครื่องมือนี้ใช้ทำอะไร

CAN Tool เป็นเครื่องมือ Python สำหรับทดสอบและวิเคราะห์บัส CAN และ CAN FD ทำงานได้กับ **Vector VN1630A / VN7572** หรือ **PEAK-System PCAN-USB Pro FD** (ทั้งสองผ่าน python-can) หรือใช้ **การจำลอง** ในตัวโดยไม่ต้องมีฮาร์ดแวร์

ฟังก์ชันหลัก:

- จัดการ **โปรเจกต์** หลายโปรเจกต์ (แต่ละโปรเจกต์มีไฟล์ DBC, CDD, ลำดับทดสอบ และ log ของตัวเอง)
- **แปลง DBC/CDD** เป็น JSON หรือ YAML ก่อนใช้งานจริง (โหลดเร็ว ตรวจสอบง่าย)
- **ส่งเป็นรอบ** และรับข้อความ เข้าถึงสัญญาณเป็นค่าทางกายภาพ
- **Trace** ของเฟรมที่ส่งและรับทั้งหมด พร้อมถอดรหัสสัญญาณได้
- **ลำดับทดสอบ** (JSON): ตั้งค่าสัญญาณ รอการตอบสนอง ตรวจสอบ รันการวินิจฉัย
- **กราฟเส้น** ของสัญญาณที่เลือก
- สลับธีมได้ (**โหมดมืด**) และรองรับ **หกภาษา** (เยอรมัน อังกฤษ ฝรั่งเศส สเปน จีนกลาง ไทย)
- **Parquet logging** พร้อมตัวกรอง
- **การวินิจฉัย** ผ่าน ISO-TP (ISO 15765-2) ด้วย UDS Service จาก CDD

1.2 ภาพรวมการไหลของข้อมูล

```
dbc/*.dbc  --(แปลง)-->  cache/*.dbc.json|yaml  ---+
cdd/*.cdd  --(แปลง)-->  cache/*.cdd.json|yaml  ---+--> แท็บ "ข้อมูล": เลือก TX / RX
                                                   |        |
                                                   |        +--> Trace   (ทุกเฟรม)
                                                   |        +--> กราฟ    (สัญญาณที่เลือก)
                                                   |        +--> Logging (สัญญาณที่เลือก, ตัวกรอง)
sequences/*.json  ----------------------------------------+--> ลำดับทดสอบ (ตั้งค่า/ตรวจสอบสัญญาณ, วินิจฉัย)
```

ขณะรันจะอ่านเฉพาะ **แคช** (JSON/YAML) เสมอ หากไฟล์ DBC/CDD เปลี่ยนแปลง จะถูกแปลงใหม่อัตโนมัติเมื่อเปิดโปรเจกต์ครั้งถัดไป (ตรวจจาก checksum ของไฟล์ต้นฉบับ)

1.3 การติดตั้งและการเริ่มต้น

```
pip install -r requirements.txt
python main.py                       # GUI
python main.py --project demo        # GUI พร้อมโปรเจกต์ที่ระบุ
python main.py --list                # แสดงรายการโปรเจกต์
python main.py --convert demo        # แปลง DBC/CDD อย่างเดียว (ไม่ใช้ GUI)
python main.py --convert demo --format yaml --force
```

ไดรเวอร์: Vector ต้องใช้ *Vector XL Driver Library* (เป็นส่วนหนึ่งของไดรเวอร์ Vector) ส่วน PEAK ต้องใช้ ไดรเวอร์ *PCAN-Basic* หากไม่มีฮาร์ดแวร์ ใช้การจำลองได้

1.4 โครงสร้างโฟลเดอร์

```
can_tool/
  main.py  gui.py  can_manager.py  dbc_manager.py  cdd_manager.py
  iso_tp_manager.py  sequence_runner.py  parquet_logger.py  project_manager.py
  help_content.py  i18n.py
  lang/              ไฟล์ภาษา (de.yaml, en.yaml, fr.yaml, es.yaml, zh.yaml, th.yaml)
  tools/check_lang.py ตรวจสอบไฟล์ภาษา
  projects/
    <ชื่อโปรเจกต์>/
      project.json      การตั้งค่าของโปรเจกต์ (เครื่องมือเขียนให้)
      dbc/              ไฟล์ DBC ของคุณ (*.dbc)
      cdd/              คำอธิบายการวินิจฉัยของคุณ (*.cdd)
      sequences/        ลำดับทดสอบ (*.json)
      cache/            สร้างอัตโนมัติ: DBC/CDD ที่แปลงแล้ว (JSON หรือ YAML)
      logs/             ไฟล์ log แบบ Parquet
```

1.5 การสร้างโปรเจกต์

ใน GUI

1. เปิดแท็บ **โปรเจกต์** แล้วคลิก **ใหม่ ...**
2. ป้อนชื่อโปรเจกต์ ใช้ได้เฉพาะตัวอักษร ตัวเลข ขีดล่าง ขีดกลาง และช่องว่าง (สูงสุด 64 ตัวอักษร ตัวแรกต้องเป็นตัวอักษรหรือตัวเลข) หากมีชื่อนี้อยู่แล้วจะไม่เขียนทับ

3. เครื่องมือจะสร้าง `projects/<name>/` พร้อมโฟลเดอร์ย่อย `dbc, cdd, sequences, cache, logs` และ `project.json` ที่มีค่าเริ่มต้น (จำลอง, 500 kbit/s, แคช JSON) แล้วสลับไปยังโปรเจกต์ใหม่
4. **นำเข้า DBC/CDD ...**: ไฟล์จะถูกคัดลอกไปที่ `dbc/` หรือ `cdd/` และแปลงทันที หรือจะวางไฟล์ลงโฟลเดอร์เองแล้วคลิก **แปลงใหม่** (หรือเปิดโปรเจกต์ใหม่) ก็ได้
5. ตั้งค่าฮาร์ดแวร์ (ดูบทที่ 2.3) แล้ว **เชื่อมต่อ**

ผ่านไฟล์

สร้างโฟลเดอร์โปรเจกต์ตามโครงสร้างข้างต้น และวาง `project.json` ที่มีอย่างน้อย `{}` - ค่าที่ไม่ได้ตั้งจะถูกเติมด้วยค่าเริ่มต้น โปรเจกต์ตัวอย่าง `projects/demo` ใช้เป็นแม่แบบได้

การสลับโปรเจกต์

เลือกจากรายการโปรเจกต์ เมื่อสลับ (และเมื่อออกจากโปรแกรม) การเชื่อมต่อจะถูกตัด logging ที่กำลังทำงานจะหยุด และโปรเจกต์เดิมจะถูกบันทึก สิ่งที่บันทึก: ฮาร์ดแวร์ รายการส่งพร้อมค่าสัญญาณและ คาบ รายการรับที่เลือก ตัวกรอง logging การเลือกกราฟ การตั้งค่าวินิจฉัย และลำดับที่โหลดล่าสุด การส่งเป็นรอบจะ **ปิด**เสมอหลังเปิดโปรเจกต์ (เพื่อความปลอดภัยบนบัสจริง)

1.6 ขั้นตอนการทำงานทั่วไป

1. สร้างหรือเลือกโปรเจกต์ แล้วนำเข้า DBC (และ CDD)
2. ตั้งค่าฮาร์ดแวร์ แล้ว **เชื่อมต่อ**
3. แท็บ **ข้อมูล**: เลือกข้อความส่งและรับจาก DBC ตั้งค่าสัญญาณและคาบ
4. เปิดการส่งเป็นรอบ ดูผลใน **Trace** และ **กราฟ**
5. หากต้องการ ให้เริ่ม **Logging** และ/หรือสร้างและรัน **ลำดับทดสอบ**
6. **วินิจฉัย**: ส่ง Service ตรวจสอบการตอบกลับ

2. การเตรียมไฟล์ก่อนเริ่มใช้งาน

ก่อนเชื่อมต่อ ต้องเตรียมไฟล์ของโปรเจกต์ให้พร้อม บทนี้อธิบายว่าแต่ละไฟล์ต้องมีอะไร และเครื่องมือใช้ข้อมูลส่วนใดบ้าง

2.1 ไฟล์ DBC (`dbc/*.dbc`)

ไฟล์ DBC มาตรฐานจาก CANdb++, CANdb Editor, SavvyCAN และอื่น ๆ ใช้ DBC ได้หลายไฟล์ต่อโปรเจกต์ (รวมกันเป็นฐานข้อมูล) ชุดอักขระ UTF-8 หรือ Windows-1252

ส่วนที่เครื่องมือใช้:

องค์ประกอบ DBC	ความหมายในเครื่องมือ
`BO_`	ข้อความ: ID (ตั้งบิต 31 = Extended-ID), ชื่อ, ความยาว (DLC > 8 = CAN FD), ผู้ส่ง
`SG_`	สัญญาณ: start bit, ความยาว, ลำดับไบต์ (Intel/Motorola), เครื่องหมาย, factor, offset, min/max, หน่วย, ผู้รับ
`SG_ ... M / m0, m1`	Multiplexing แบบง่าย (หนึ่ง multiplexor หลายกลุ่ม) **ไม่รองรับ** Extended Multiplexing (จะมีคำเตือน)
`VAL_`	ตารางค่า: ข้อความของแต่ละค่าดิบ ในเครื่องมือแสดงเป็นดรอปดาวน์ตอนส่งและเป็นข้อความตอนรับ
`SIG_VALTYPE_`	สัญญาณทศนิยมลอยตัว (32/64 บิต)
`CM_`	คอมเมนต์ของข้อความ/สัญญาณ
`BA_ "GenMsgCycleTime"`	คาบมาตรฐาน (ms) ของข้อความ หากไม่ระบุใช้ 100 ms
`BA_ "VFrameFormat"`	รูปแบบเฟรม ค่าที่มี *FD* หมายถึงข้อความ CAN FD

ตัวอย่าง:

```
VERSION "1.0"
NS_ :
BS_:
BU_: Tester ECU

BO_ 512 EngineData: 8 ECU
 SG_ RPM : 0|16@1+ (0.25,0) [0|8000] "rpm" Tester
 SG_ CoolantTemp : 16|8@1+ (1,-40) [-40|130] "degC" Tester

VAL_ 257 StartButton 0 "Aus" 1 "Ein" ;
BA_DEF_ BO_ "GenMsgCycleTime" INT 0 65535;
BA_ "GenMsgCycleTime" BO_ 512 10;
```

ข้อแนะนำในการเตรียมข้อมูล

- ชื่อข้อความและชื่อสัญญาณถูกใช้ในลำดับทดสอบ - ควรตั้งชื่อให้สื่อความหมายและไม่ซ้ำกัน
- ชื่อข้อความซ้ำข้ามหลายไฟล์ DBC: คงนิยามแรกไว้ นิยามที่สองจะถูกละเว้นพร้อมคำเตือน
- ID เดียวกันแต่ชื่อต่างกัน: ตอนรับจะจับคู่กับชื่อแรก (มีคำเตือน)
- โหนดผู้ส่ง (`BU_`) กำหนดว่า ECU เสมือนในโหมดจำลองจะส่งข้อความใดเป็นรอบ (ค่าเริ่มต้น: ผู้ส่ง `ECU` ตั้งค่าได้ใน `sim.traffic_senders`) การกำหนดค่า min/max ที่เหมาะสมใน DBC จะให้ค่าจำลองที่สมจริง
- คำเตือนจากการแปลงอยู่ในแท็บ **โปรเจกต์** ในช่อง *ไฟล์ที่แปลงแล้ว / คำเตือน*

แคช (`cache/*.dbc.json` หรือ `.yaml`)

เมื่อเปิดโปรเจกต์ DBC ทุกไฟล์จะถูกเขียนลงแคชอัตโนมัติ (เลือกรูปแบบได้ในแท็บ **โปรเจกต์**) จะแปลงใหม่เฉพาะเมื่อไฟล์ต้นฉบับเปลี่ยน **แปลงใหม่** ใช้บังคับให้แปลง การแก้ไขในแคชด้วยตนเองจะถูกเขียนทับเมื่อแปลงใหม่ - การแก้ไขต้องทำใน DBC

```
{
  "format_version": 1,
  "nodes": ["Tester", "ECU"],
  "messages": {
    "EngineData": {
      "id": 512, "extended": false, "dlc": 8, "fd": false, "sender": "ECU",
      "cycle_time_ms": 10, "comment": "", "multiplexor": null,
      "signals": {
        "RPM": {"start": 0, "length": 16, "byte_order": "little", "signed": false,
                "factor": 0.25, "offset": 0, "min": 0, "max": 8000, "unit": "rpm",
                "receivers": ["Tester"], "choices": {}, "comment": ""}
      }
    }
  },
  "warnings": [], "source": "engine.dbc", "source_sha1": "..."
}
```

2.2 คำอธิบายการวินิจฉัย (`cdd/*.cdd`)

CDD อธิบาย Service วินิจฉัย จะถูกแปลงเป็น `cache/*.cdd.json|yaml` เช่นเดียวกับ DBC

ข้อจำกัดสำคัญ: รองรับเต็มรูปแบบเฉพาะรูปแบบ XML ง่าย ๆ <DIAGDATA> (ดูตัวอย่างและ `projects/demo/cdd/diag.cdd`) ไฟล์ CDD ต้นฉบับของ CANdelaStudio จะอ่านแบบ **heuristic** เท่านั้น (ชื่อ Service และไบต์ของ Request หากหาพบ) ในกรณีนั้นให้ตรวจสอบไฟล์ แคชที่สร้างขึ้น (แท็บ **โปรเจกต์** คำเตือน) และเติมที่อยู่ Request และพารามิเตอร์การตอบกลับที่นั่น หรือในรูปแบบง่าย

```
<?xml version="1.0" encoding="UTF-8"?>
<DIAGDATA>
  <ECU name="Engine_ECU" request_id="0x7E0" response_id="0x7E8" extended="false" fd="false">
    <SERVICE name="ExtendedSession" request="10 {session}" description="Select diagnostic session">
      <PARAM name="session" bytes="1" default="03"/>
    </SERVICE>
    <SERVICE name="ReadVIN" request="22 F1 90" description="Read VIN">
      <RESPONSE prefix_len="3">
        <PARAM name="VIN" offset="0" length="17" type="ascii"/>
      </RESPONSE>
    </SERVICE>
    <SERVICE name="ReadSpeed" request="22 F4 0D" description="Read speed">
      <RESPONSE prefix_len="3">
        <PARAM name="Speed" offset="0" length="1" type="uint" factor="1" unit="km/h"/>
      </RESPONSE>
    </SERVICE>
  </ECU>
</DIAGDATA>
```

องค์ประกอบ / แอตทริบิวต์	ความหมาย
`ECU name, request_id, response_id`	ชื่อ ECU และ CAN ID (Tester -> ECU, ECU -> Tester) เช่น `0x7E0`/`0x7E8`
`ECU extended, fd`	`true`: ID 29 บิต หรือ CAN FD (ISO-TP สูงสุด 64 ไบต์ต่อเฟรม)
`SERVICE name`	ชื่อ Service (ใช้ในแท็บวินิจฉัยและในขั้นตอนลำดับ `diag`)
`SERVICE request`	ไบต์ของ Request เป็น Hex ตัวแทนค่าอยู่ในวงเล็บปีกกา เช่น `10 {session}`
`SERVICE description`	ข้อความอิสระ แสดงในรายการ Service
`PARAM`	อยู่โดยตรงใน SERVICE: พารามิเตอร์ป้อนเข้าสำหรับตัวแทนค่า: `name`, `bytes` (ความกว้าง), `default` (Hex)
`RESPONSE prefix_len`	จำนวนไบต์ต้นการตอบกลับที่ข้ามไป (Service-ID + DID เช่น 3 สำหรับ `62 F1 90`)
`RESPONSE/PARAM`	พารามิเตอร์การตอบกลับ: `name`, `offset` และ `length` เป็นไบต์ (นับจากท้าย prefix), `type` = `ascii | uint | int | hex`, เพิ่มเติมได้ `factor`, `offset_val`, `unit`

รายการแคชที่สอดคล้องกัน (สำหรับแก้ไขด้วยตนเอง):

```
{
  "format_version": 1,
  "ecus": {
    "Engine_ECU": {
      "request_id": "0x7E0", "response_id": "0x7E8", "extended": false, "fd": false,
      "services": {
        "ReadVIN": {
          "request": "22 F1 90", "description": "Read VIN", "params": [],
          "response": {"prefix_len": 3,
                       "params": [{"name": "VIN", "offset": 0, "length": 17, "type": "ascii"}]}
        }
      }
    }
  }
}
```

2.3 ฮาร์ดแวร์และการเชื่อมต่อ

การตั้งค่าในแท็บ **โปรเจกต์** (บันทึกใน `project.json` ภายใต้ `hardware`):

การตั้งค่า	Vector VN1630A / VN7572	PEAK PCAN-USB Pro FD	จำลอง
อินเทอร์เฟซ	`vector` (ไดรเวอร์ XL)	`pcan` (PCAN-Basic)	`simulation`
ช่อง	ดัชนีช่อง เริ่มจาก 0 (เช่น `0`) การกำหนดทำผ่าน *Vector Hardware Config* หากตั้งชื่อแอปไว้ จะใช้หมายเลขช่องที่กำหนดไว้ที่นั่น	`PCAN_USBBUS1`, `PCAN_USBBUS2` ...	ไม่ต้องใช้
บิตเรต	Nominal bitrate (kbit/s)	Nominal bitrate (kbit/s) ในโหมด FD ตายตัว 500k/2M (clock 80 MHz)	ไม่มีผล
CAN FD / บิตเรตข้อมูล	ติ๊กเครื่องหมายและกรอกบิตเรตข้อมูล	ติ๊กเครื่องหมาย; timing อื่นตั้งใน `hardware.extra`	ติ๊กเพื่อใช้เฟรม FD
ชื่อแอป	ชื่อจาก Vector Hardware Config (ว่าง = ค่าเริ่มต้นของ python-can)	-	-

Vector VN7572 (การ์ด PCIe): การ์ดอินเทอร์เฟซ Vector VN7572 ก็ใช้เป็นชนิด `vector` ผ่านไดรเวอร์ Vector XL (เฉพาะ Windows) เช่นเดียวกับ VN1630A เมื่อติดตั้ง *Vector Driver Setup* แล้ว การ์ดต้องปรากฏใน *Vector Hardware Config* หมายเลขช่องเริ่มจาก 0 (0 = CH1, 1 = CH2 ...) **ตรวจหาฮาร์ดแวร์** จะแสดงรายการช่องที่พบพร้อมหมายเลขซีเรียล ห้ามมีโปรแกรมอื่น (CANalyzer, CANoe, vFlash) ใช้ช่องนั้นอยู่

- ทุกโหนดบนบัสต้องใช้บิตเรตเดียวกัน และสำหรับ CAN FD ต้องใช้บิตเรตข้อมูลเดียวกันด้วย
- ในชุดทดสอบบนโต๊ะที่มีอุปกรณ์สองตัว ปลายบัสต้องมี terminator 120 โอห์ม

- `hardware.extra` (ตั้งได้เฉพาะใน `project.json`) ส่งพารามิเตอร์ใดก็ได้ต่อไปยัง python-can เช่น `{"nom_brp": 5}` สำหรับ PEAK
- **จำลอง**: *สะท้อน* ส่งข้อความที่ส่งออกกลับมาเป็นข้อความที่รับ *ECU เสมือน* ตอบ UDS Request (เช่น 10, 11, 14, 19, 22, 27, 3E) ตามที่อยู่ในการตั้งค่าวินิจฉัย และส่งข้อความของผู้ส่ง ECU เป็นรอบ

2.4 ไฟล์ลำดับ (`sequences/*.json`)

ลำดับประกอบด้วยชื่อ คำอธิบาย และรายการขั้นตอน สร้างได้ในแท็บ **ลำดับทดสอบ** หรือแก้ไขไฟล์โดยตรง อ้างอิงคำสั่ง: ภาคผนวก A

```
{
  "name": "startup",
  "description": "Simulate engine start and check run-up",
  "steps": [
    {"type": "enable_tx", "message": "DriverCmd", "enabled": true},
    {"type": "set_signal", "message": "DriverCmd", "signal": "StartButton", "value": 1},
    {"type": "wait_for", "direction": "RX", "message": "EngineData", "signal": "RPM",
     "op": ">", "value": 100, "timeout_ms": 3000},
    {"type": "diag", "service": "ReadVIN", "expect": "62 F1 90"}
  ]
}
```

2.5 ไฟล์โปรเจกต์ (`project.json`)

เครื่องมือเขียนให้ แก้ไขด้วยตนเองได้เมื่อจำเป็น (ปิดเครื่องมือก่อน) คีย์ที่ขาดจะถูกเติมด้วยค่าเริ่มต้น

```
{
  "hardware": {"type": "vector", "channel": "0", "bitrate": 500000, "fd": true,
               "data_bitrate": 2000000, "app_name": "", "extra": {}},
  "cache_format": "json",
  "tx": [{"message": "DriverCmd", "cycle_ms": 50, "values": {"ThrottleCmd": 0}, "signals": null}],
  "rx": {"EngineData": null, "VehicleSpeed": ["Speed"]},
  "logging": {"selection": {"TX": {}, "RX": {"EngineData": null}},
              "on_change_only": false, "min_interval_ms": 0, "log_raw": false},
  "graph": ["RX:EngineData.RPM"],
  "diag": {"cdd": "diag.cdd.json", "ecu": "Engine_ECU", "request_id": "", "response_id": "",
           "extended": false, "fd": false, "padding": 170},
  "sequence": "startup.json",
  "sim": {"ecu": true, "traffic_senders": ["ECU"]}
}
```

คีย์	ความหมาย
`hardware`	พารามิเตอร์การเชื่อมต่อ (บทที่ 2.3) บิตเรตเป็น bit/s
`cache_format`	`json` หรือ `yaml` สำหรับไฟล์ที่แปลงแล้ว
`tx`	รายการส่ง: ข้อความ คาบ (ms) ค่าสัญญาณ `signals` = สัญญาณที่เลือกสำหรับกราฟ/logging (`null` = ทั้งหมด)
`rx`	รายการรับที่เลือก: ข้อความ -> รายการสัญญาณที่เลือก หรือ `null` (ทั้งหมด)
`logging`	ตัวกรองและตัวเลือกของ logging (แท็บ Logging)
`graph`	คีย์ของสัญญาณที่ติ๊กในกราฟ เช่น `RX:Msg.Signal`
`diag`	ไฟล์ CDD, ECU, การกำหนดที่อยู่ทับ (ไม่บังคับ: `request_id`, `response_id`), `extended`, `fd`, `padding`
`sequence`	ลำดับที่โหลดล่าสุด (ชื่อไฟล์ใน `sequences/`)
`sim`	จำลอง: `ecu` (ECU เสมือน), `traffic_senders` (โหนดผู้ส่งที่จำลองเป็นรอบ)

2.6 เช็กลิสต์ก่อนเชื่อมต่อครั้งแรก

1. ไฟล์ DBC อยู่ในโฟลเดอร์ `dbc/` ในแท็บโปรเจกต์แสดงข้อความตามที่คาดไว้ (จำนวน) และไม่มีคำเตือนที่ไม่คาดคิด
2. CDD (ถ้ามี) อยู่ในโฟลเดอร์ `cdd/` ในแท็บวินิจฉัยแสดง ECU และ Service
3. ติดตั้งไดรเวอร์ฮาร์ดแวร์แล้ว ต่ออุปกรณ์แล้ว ช่องและบิตเรตตรงกับบัส
4. สำหรับ CAN FD: ติ๊ก *CAN FD* และตั้งบิตเรตข้อมูล โหนดอีกฝั่งต้องรองรับ FD ด้วย
5. มีลำดับอยู่ใน `sequences/` หรือสร้างไว้ในแท็บลำดับทดสอบแล้ว

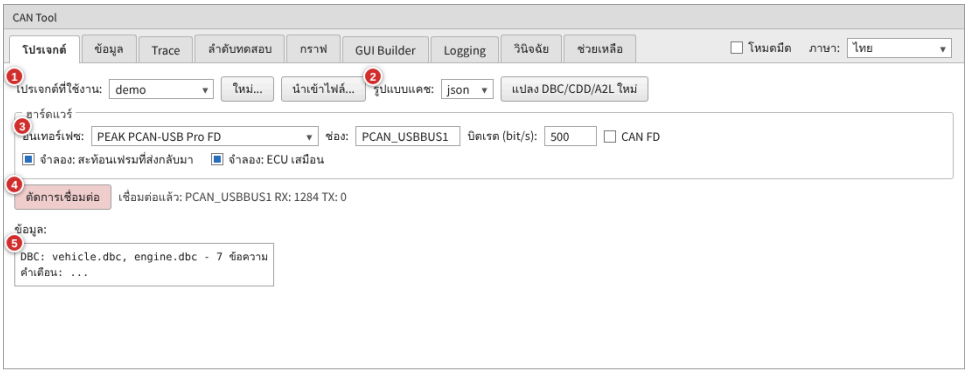
ปัญหาที่พบบ่อย

อาการ	สาเหตุที่เป็นไปได้ / วิธีแก้
ไม่มีข้อความในแท็บข้อมูล	DBC ไม่อยู่ในโฟลเดอร์ `dbc/` หรือนามสกุลไม่ใช่ `.dbc`; อ่านคำเตือนในแท็บโปรเจกต์
เชื่อมต่อไม่สำเร็จ	ไม่มี python-can/ไดรเวอร์ ช่องผิด อุปกรณ์ถูกโปรแกรมอื่นใช้อยู่ (เช่น CANalyzer/PCAN-View)
เห็นเฟรมแต่ไม่ถอดรหัส	ID/การระบุ Extended ใน DBC ไม่ตรงกับบัส; ไม่มีข้อความนี้ใน DBC
ค่าสัญญาณผิด	ตรวจลำดับไบต์ (Intel/Motorola), start bit, factor/offset ใน DBC
วินิจฉัย: Timeout	ตรวจ Request/Response-ID, Extended/FD และ Padding ในแท็บวินิจฉัย; ECU อยู่ใน session ที่เหมาะสมหรือไม่?
ใช้ Logging ไม่ได้	ไม่มีแพ็กเกจ Python `pyarrow` (`pip install pyarrow`)

3.1 แท็บ "โปรเจกต์"

ระบบปฏิบัติการ: เมื่อเริ่มต้น เครื่องมือจะตรวจจับเองว่ากำลังทำงานบน **Windows** หรือ **Linux** (แสดงในแถบชื่อเรื่อง) ตัวเลือกฮาร์ดแวร์จะปรับตาม: *Vector* (ไดรเวอร์ XL) เฉพาะ Windows, *SocketCAN* เฉพาะ Linux, *PEAK* และ *การจำลอง* ใช้ได้ทั้งสองระบบ รายการที่ใช้ไม่ได้จะถูกทำเครื่องหมายและปิดใช้งาน ปุ่ม **ตรวจหาฮาร์ดแวร์** จะแสดงอินเทอร์เฟซที่เชื่อมต่อ ฟังก์ชันอื่นทั้งหมด (Trace, ลำดับ, การบันทึก, การวินิจฉัย, vFlash, วิธีใช้/PDF) เหมือนกันทั้งสองระบบ โฟลเดอร์โปรเจกต์ใช้แลกเปลี่ยนระหว่าง Windows และ Linux ได้

ภาพหน้าจอพร้อมคำอธิบาย



ภาพแสดงหน้าจอแบบแผนภาพ หมายเลขอ้างอิงถึงคำอธิบายด้านล่าง

1. เลือกโปรเจกต์ สร้างโปรเจกต์ใหม่ และนำเข้าไฟล์ DBC/CDD
2. รูปแบบแคช (json/yaml) และการแปลง DBC/CDD ใหม่
3. ฮาร์ดแวร์: อินเทอร์เฟซ ช่อง (เช่น PCAN_USBBUS1) บิตเรต และ CAN FD
4. เชื่อมต่อหรือตัดการเชื่อมต่อ พร้อมแสดงสถานะ (ตัวนับรับและส่ง)
5. ช่องข้อมูล: ไฟล์ DBC ที่โหลด จำนวนข้อความ และคำเตือน

เริ่มฮาร์ดแวร์ใหม่: ปุ่มนี้อยู่ข้างปุ่ม *เชื่อมต่อ* จะปิดอินเทอร์เฟซแล้วเปิดใหม่ด้วยการตั้งค่าเดิม ช่วยได้เมื่อฮาร์ดแวร์ที่เชื่อมต่อค้าง รายการส่งยังคงอยู่ (ข้อความที่ส่งวนซ้ำจะส่งต่อ) การบันทึกที่กำลังทำงานจะหยุด

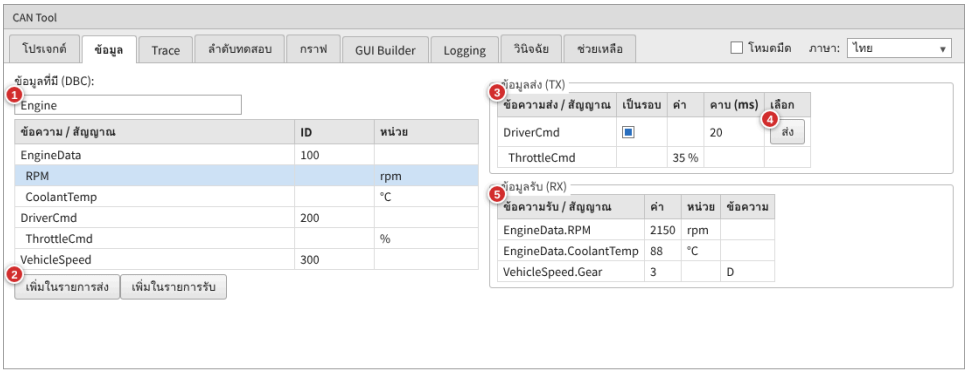
จัดการโปรเจกต์ นำเข้า DBC/CDD ตั้งค่าฮาร์ดแวร์ และเชื่อมต่อ

ส่วนควบคุม	หน้าที่
โปรเจกต์ที่ใช้งาน	รายการเลือกโปรเจกต์ทั้งหมด (`projects/*/project.json`) การสลับจะตัดการเชื่อมต่อ หยุด logging บันทึกโปรเจกต์เดิม และโหลดโปรเจกต์ใหม่
ใหม่ ...	สร้างโปรเจกต์พร้อมโครงสร้างโฟลเดอร์และค่าเริ่มต้น (บทที่ 1.5)
นำเข้า DBC/CDD ...	คัดลอกไฟล์ `.dbc`/`.cdd` ที่เลือกเข้าโปรเจกต์และแปลง
รูปแบบแคช	`json` หรือ `yaml`; เมื่อเปลี่ยนจะแปลงใหม่
แปลงใหม่	บังคับแปลง DBC/CDD ทั้งหมดและโหลดโปรเจกต์ใหม่
อินเทอร์เฟซ / ช่อง / บิตเรต / CAN FD / บิตเรตข้อมูล / ชื่อแอป	พารามิเตอร์ฮาร์ดแวร์ (บทที่ 2.3) ถูกล็อกขณะเชื่อมต่ออยู่
จำลอง: สะท้อน	ข้อความที่ส่งออกจะปรากฏเป็นข้อความที่รับด้วย
จำลอง: ECU เสมือน	เมื่อเชื่อมต่อจะเริ่ม ECU ซอฟต์แวร์ที่รองรับ UDS พร้อมทราฟฟิกทดสอบ
เชื่อมต่อ / ตัดการเชื่อมต่อ	เปิดหรือปิดอินเทอร์เฟซ เมื่อตัดการเชื่อมต่อ logging ที่กำลังทำงานจะหยุด
แถบสถานะ	สถานะการเชื่อมต่อ จำนวนเฟรมที่รับ (RX) และส่ง (TX) ข้อผิดพลาดล่าสุด
ไฟล์ที่แปลงแล้ว / คำเตือน	รายการไฟล์แคช จำนวนข้อความ คำเตือนจาก DBC/CDD

3.2 แท็บ "ข้อมูล"

ที่นี่กำหนดว่า**ข้อความ**ใดถูกส่งและรับ และ**สัญญาณ**ใดใช้สำหรับกราฟ logging และการแสดงผล - เลือกจากข้อมูล DBC ทั้งหมดของโปรเจกต์

ภาพหน้าจอพร้อมคำอธิบาย



ภาพแสดงหน้าจอแบบแผนภาพ หมายเลขอ้างอิงถึงคำอธิบายด้านล่าง

1. ตัวกรองและรายการข้อความและสัญญาณทั้งหมดจาก DBC
2. เพิ่มรายการที่เลือกไปยังข้อมูลส่ง (TX) หรือข้อมูลรับ (RX)
3. ข้อมูลส่ง: เปิด/ปิดการส่งเป็นรอบ ตั้งค่าสัญญาณและเวลาต่อรอบ
4. ส่งครั้งเดียว: ส่งข้อความทันทีหนึ่งครั้ง
5. ข้อมูลรับพร้อมค่าสดปัจจุบันของสัญญาณ

ฝั่งซ้าย: ข้อมูลที่มี

- ต้นไม้ของข้อความทั้งหมดพร้อมสัญญาณ (ID, ผู้ส่ง, หน่วย) **ช่องตัวกรอง**ค้นหาในชื่อข้อความ ID และชื่อสัญญาณ
- **-> ส่ง (TX)** นำข้อความที่เลือกไปใส่ในรายการส่ง (ดับเบิลคลิกที่ข้อความให้ผลเหมือนกัน) เลือกได้หลายรายการ หากเลือกสัญญาณ จะนำข้อความของสัญญาณนั้นไปใช้
- **-> รับ (RX)** นำข้อความที่เลือกไปใส่ในรายการรับ

ขวาบน: ส่ง

คอลัมน์	ความหมาย
ข้อความส่ง / สัญญาณ	ข้อความ (พร้อม ID) และสัญญาณทั้งหมดของข้อความใต้ลงมา
เป็นรอบ	เครื่องหมายติ๊กที่ข้อความเปิด/ปิดการส่งเป็นรอบ (ส่งจริงเมื่อมีการเชื่อมต่อเท่านั้น)
ค่า	ค่าสัญญาณทางกายภาพ (ช่องป้อนค่า หรือรายการเลือกสำหรับสัญญาณที่มีตารางค่า) การเปลี่ยนแปลงมีผลตั้งแต่เฟรมถัดไป
คาบ [ms]	คาบของข้อความ ค่าตั้งต้นจาก DBC (`GenMsgCycleTime`) หรือ 100 ms 0 = ไม่ส่งเป็นรอบ
เลือก	เครื่องหมายติ๊กที่สัญญาณ: สัญญาณเป็นส่วนของตัวเลือกสำหรับกราฟและ logging
ส่ง (ปุ่ม)	ส่งข้อความหนึ่งครั้งด้วยค่าปัจจุบัน

ลบ ลบข้อความของแถวที่เลือกออกจากรายการส่ง

ขวาล่าง: รับ

คอลัมน์	ความหมาย
ข้อความรับ / สัญญาณ	ข้อความ (พร้อม ID) และสัญญาณของข้อความ
ค่า / หน่วย / ข้อความ	ค่าที่ได้รับล่าสุด (อัปเดตทุก 200 ms) หน่วย และข้อความจากตารางค่า (ถ้ามี)
เลือก	เครื่องหมายติ๊กที่สัญญาณ: สัญญาณเป็นส่วนของตัวเลือกสำหรับกราฟและ logging สัญญาณที่ไม่ติ๊กจะไม่ถูกบันทึก

หมายเหตุ

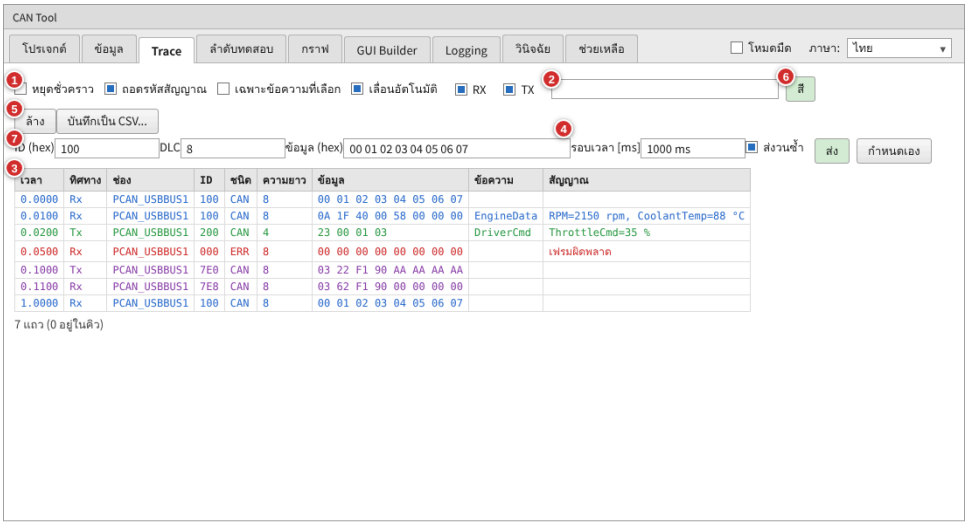
- การเลือก (เครื่องหมายติ๊ก) กำหนดสิ่งที่ **กราฟ** และ **Logging** มองเห็น ส่วน **Trace** แสดงทุกเฟรม (เลือกแสดงเฉพาะข้อความที่เลือกได้)
- การเลือกและค่าที่ส่งทั้งหมดถูกบันทึกกับโปรเจกต์ การส่งเป็นรอบจะปิดหลังโหลด

- ข้อความหนึ่งอยู่ในทั้งรายการ TX และ RX ได้พร้อมกัน (เช่น กับการสะท้อนในโหมดจำลอง)

3.3 แท็บ "Trace"

ลำดับเวลาของเฟรมที่ส่ง (Tx, เน้นสี) และที่รับ (Rx) ทั้งหมด

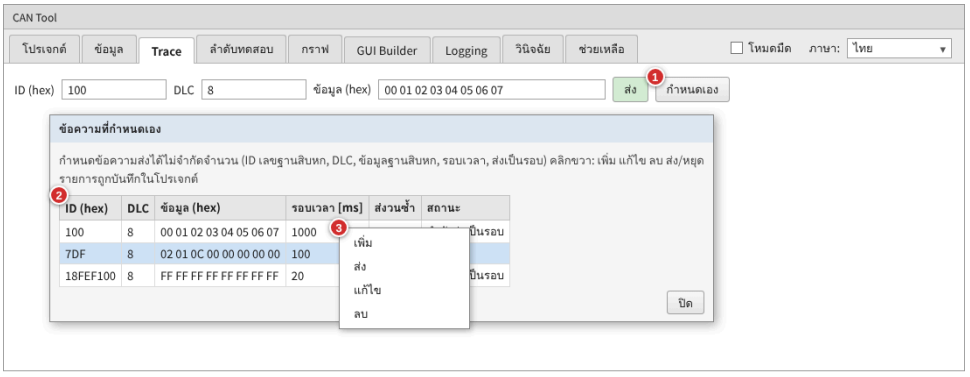
ภาพหน้าจอพร้อมคำอธิบาย



ภาพแสดงหน้าจอแบบแผนภาพ หมายเลขอ้างอิงถึงคำอธิบายด้านล่าง

1. ตัวเลือก: หยุดชั่วคราว ถอดรหัสสัญญาณ เฉพาะข้อความที่เลือก เลื่อนอัตโนมัติ
2. กรองตาม ID หรือชื่อข้อความ
3. ตาราง Trace: เวลา ทิศทาง (Rx/Tx) ช่อง ID ชนิด ความยาว และข้อมูลดิบ ใช้ได้แม้ไม่มี DBC/CDD
4. ชื่อข้อความและค่าสัญญาณที่ถอดรหัสแล้ว (เมื่อโหลด DBC)
5. ล้าง Trace หรือบันทึกเป็นไฟล์ CSV
6. โหมดสี: Rx น้ำเงิน, Tx เขียว, ข้อผิดพลาดแดง, ISO-TP ม่วง
7. แถวส่ง: ID (hex), DLC, ข้อมูล, รอบเวลา, ช่องติ๊กส่งวนซ้ำ และปุ่มส่ง

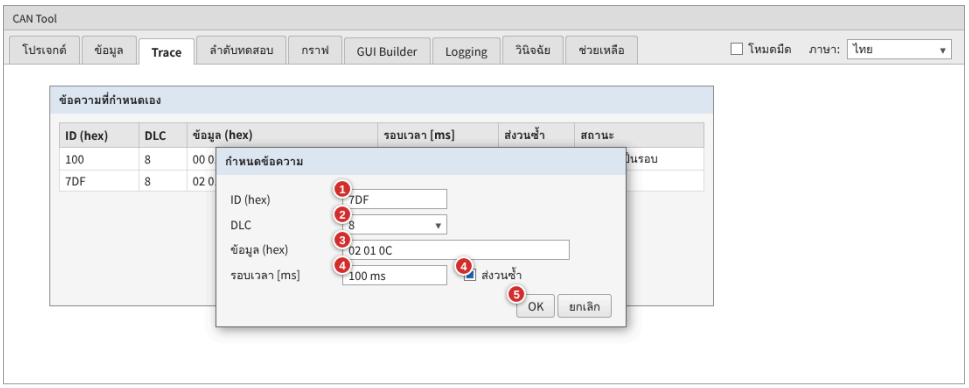
ภาพหน้าจอพร้อมคำอธิบาย



ภาพแสดงหน้าจอแบบแผนภาพ หมายเลขอ้างอิงถึงคำอธิบายด้านล่าง

1. ปุ่ม กำหนดเอง เปิดหน้าต่างข้อความส่งของคุณเอง
2. ตารางข้อความส่ง: ID (hex), DLC, ข้อมูล (hex), รอบเวลา [ms], ส่งเป็นรอบ และสถานะ
3. เมนูคลิกขวา: เพิ่ม ส่ง/หยุด แก้ไข ลบ

ภาพหน้าจอพร้อมคำอธิบาย



ภาพแสดงหน้าจอแบบแผนภาพ หมายเลขอ้างอิงถึงคำอธิบายด้านล่าง

1. ID (hex): CAN ID รวมถึง Extended (> 7FF)
2. DLC: ความยาวข้อมูล รวมถึงค่า CAN FD
3. ข้อมูล (hex): ไบต์มีหรือไม่มีเว้นวรรค ไบต์ที่ขาดจะเติม 00
4. รอบเวลา [ms] และช่องส่งเป็นรอบ: ส่งครั้งเดียวหรือเป็นรอบ
5. OK เพิ่มข้อความลงรายการ ยกเลิกทิ้งข้อมูลที่ป้อน

คอลัมน์	**เนื้อหา**
เวลา [s]	เวลาสัมพัทธ์กับเฟรมแรกที่แสดง (นับจากเชื่อมต่อหรือจาก *ล้าง*)
Rx/Tx, ช่อง	ทิศทางและอินเทอร์เฟซ
ID, ชนิด, Len	CAN ID (hex, 8 หลักสำหรับ Extended) ชนิด (CAN, FD, FD/BRS) และความยาวข้อมูล
ข้อมูล	ข้อมูลเป็น Hex
ข้อความ, สัญญาณ	ชื่อจาก DBC และสัญญาณที่ถอดรหัสพร้อมหน่วย (เมื่อเปิด *ถอดรหัส*)
โหมดสี	ปุ่มสลับ เปิด: ข้อความ Rx สีน้ำเงินกลาง ข้อความ Tx สีเขียวกลาง ข้อความผิดพลาด (เฟรมข้อผิดพลาดของบัส ชนิด ERR) สีแดง และข้อความ ISO-TP (ID คำขอ/ตอบกลับของการวินิจฉัย รวมถึง 0x7DF) สีม่วงกลาง ปิด: แสดงผลเหมือนเดิม

ส่วนควบคุม	**หน้าที่**
หยุดชั่วคราว	หยุดการแสดงผล เฟรมยังถูกเก็บในบัฟเฟอร์ต่อไป (สูงสุด 50 000) และจะแสดงเมื่อยกเลิกการหยุด
ถอดรหัส	เปิด/ปิดการถอดรหัสสัญญาณ (ปิด = เร็วกว่าเมื่อโหลดบัสสูง)
เฉพาะข้อความที่เลือก	แสดงเฉพาะข้อความที่อยู่ในรายการ TX หรือ RX ของแท็บข้อมูล
เลื่อนอัตโนมัติ	ตามรายการล่าสุดโดยอัตโนมัติ
Rx / Tx	แสดง/ซ่อนทิศทาง
ช่องตัวกรอง	ID แบบ Hex คั่นด้วยจุลภาค (เช่น `100, 200`) และ/หรือบางส่วนของชื่อข้อความ (เช่น `engine`)
ล้าง	ลบการแสดงผลและบัฟเฟอร์
บันทึกเป็น CSV ...	ส่งออกแถวที่มองเห็นอยู่ (ตัวคั่น `;`)

อัปเดตแถวเดิม: เมื่อติ๊ก *อัปเดตแถวเดิม* แต่ละข้อความ (ทิศทาง ช่อง ID) จะแสดงเพียงครั้งเดียวในตารางตามลำดับที่ปรากฏครั้งแรก เมื่อปรากฏอีก จะอัปเดตเวลา ข้อมูล และสัญญาณที่ถอดรหัสของแถวนั้น และเพิ่มคอลัมน์ *จำนวน* เมื่อไม่ติ๊ก เฟรมทั้งหมดจะถูกเขียนต่อกันแบบเลื่อน การสลับจะล้างตาราง การเลื่อนอัตโนมัติใช้ได้เฉพาะโหมดเลื่อน

ส่วนต่างเวลา: นอกจาก *อัปเดตแถวเดิม* แล้ว เมื่อติ๊ก *ส่วนต่างเวลา* คอลัมน์เวลาจะแสดงช่วงห่างจากการปรากฏครั้งก่อนของข้อความเดียวกัน (ทิศทาง ช่อง ID) ซึ่งคือรอบเวลา แทนเวลาสัมบูรณ์ ครั้งแรกเป็น 0 หัวคอลัมน์เปลี่ยนเป็น *ส่วนต่างเวลา [s]* การสลับจะล้างตาราง

แถวส่ง: ใต้แถบตัวเลือกมีแถวสำหรับส่งเฟรมดิบ (ค่าเริ่มต้น: ID 100 hex, DLC 8, ข้อมูล 00 01 02 03 04 05 06 07, 1000 ms) **ส่ง** จะส่งเฟรมหนึ่งครั้ง เมื่อติ๊ก **ส่งวนซ้ำ** จะเริ่มส่งวนซ้ำ และคลิกอีกครั้ง (**หยุด**) เพื่อสิ้นสุด ID ที่มากกว่า 7FF จะส่งเป็นเฟรมขยาย DLC ที่มากกว่า 8 จะส่งเป็น CAN FD (ต้องเปิด FD ในการตั้งค่าการเชื่อมต่อ) เฟรมจะแสดงใน Trace เป็น Tx

ข้อความที่กำหนดเอง (ส่งหลายข้อความ)

แถวส่งส่งได้ทีละหนึ่งข้อความ สำหรับ**หลายข้อความ**ใช้ปุ่ม **กำหนดเอง**: จะเปิดหน้าต่างแยกที่เปิดค้างไว้ข้าง Trace ที่นั่นกำหนดข้อความส่งได้ไม่จำกัดจำนวนและส่งแยกทีละข้อความ รายการถูกบันทึกในโปรเจกต์ (`custom_tx`) และใช้ได้อีกเมื่อเปิดโปรเจกต์ครั้งถัดไป

กำหนดและส่งข้อความ

1. คลิกปุ่ม **กำหนดเอง** ในแถวส่ง: หน้าต่าง "ข้อความที่กำหนดเอง" จะเปิดขึ้น
2. **คลิกขวา**ในตารางแล้วเลือก **เพิ่ม**

3. ในกล่องโต้ตอบ "กำหนดข้อความ" ป้อน ID (hex), DLC, ข้อมูล (hex), รอบเวลา [ms] และติ๊ก **ส่งเป็นรอบ** หากต้องการ แล้วยืนยันด้วย **OK** ข้อมูลที่ไม่ถูกต้อง (เช่น DLC ที่ไม่อนุญาตหรือไบต์มากเกินไป) จะถูกปฏิเสธพร้อมข้อความและไม่มีอะไรถูกบันทึก
4. คลิกขวาที่แถวแล้วเลือก **ส่ง**: หากไม่ติ๊กส่งเป็นรอบจะส่งหนึ่งครั้ง หากติ๊กจะเริ่มส่งเป็นรอบ (สถานะ: "กำลังส่งเป็นรอบ") เมนูเดิมจะเปลี่ยนเป็น **หยุด**

ช่อง	ความหมาย
ID (hex)	CAN ID ฐานสิบหกไม่มีคำนำหน้า (เช่น 7DF) ID ที่มากกว่า 7FF จะถูกส่งเป็น Extended ID (29 บิต) อัตโนมัติ
DLC	ความยาวข้อมูล: 0 ถึง 8 และ 12, 16, 20, 24, 32, 48, 64 ค่าที่มากกว่า 8 จะถูกส่งเป็น CAN FD (ฮาร์ดแวร์และโปรเจกต์ต้องรองรับ CAN FD)
ข้อมูล (hex)	ไบต์ฐานสิบหกมีหรือไม่มีเว้นวรรคก็ได้ ข้อมูลที่สั้นเกินไปจะถูกเติม 00 จนครบ DLC ข้อมูลที่ยาวเกินไปถือเป็นข้อผิดพลาด
รอบเวลา [ms]	ช่วงเวลาส่งตั้งแต่ 1 ถึง 3 600 000 ms มีผลเฉพาะเมื่อส่งเป็นรอบ
ส่งเป็นรอบ	ไม่ติ๊ก: ส่งครั้งเดียว ติ๊ก: ส่งเป็นรอบจนกว่าจะหยุด

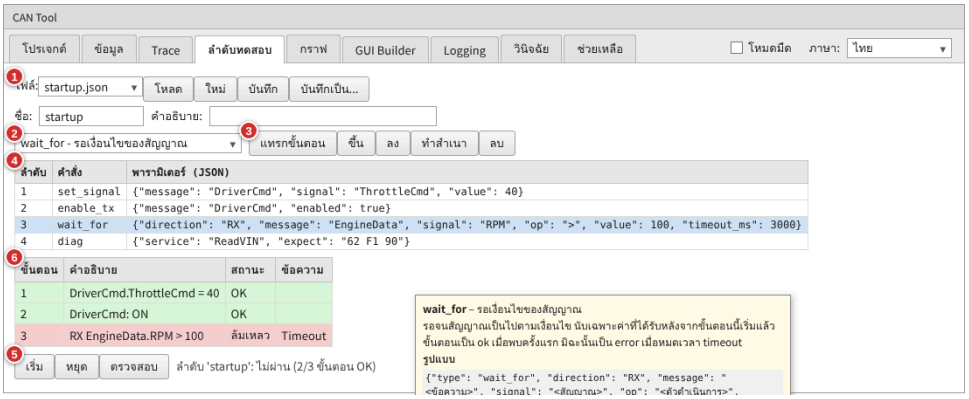
เมนูคลิกขวา: เพิ่ม (คลิกบนพื้นที่ว่างได้ด้วย) ส่งหรือหยุด แก้ไข (ดับเบิลคลิกได้ด้วย) และลบ รายการที่กำลังส่งอยู่จะถูกหยุดก่อนแก้ไขหรือลบ ต้องเชื่อมต่อก่อนจึงจะส่งได้ เมื่อตัดการเชื่อมต่อ การส่งเป็นรอบของทุกรายการจะหยุด หลายรายการที่ ID เดียวกันใช้งานส่งร่วมกัน ข้อความที่ส่งแล้วปรากฏใน Trace เป็น Tx

หมายเหตุ: แสดงสูงสุด 5 000 แถว (แถวเก่าจะเลื่อนหายไป) การแสดงผลอัปเดตเฉพาะขณะที่ แท็บแสดงอยู่ เบื้องหลังเครื่องมือเก็บบัฟเฟอร์ได้สูงสุด 50 000 เฟรม สำหรับการบันทึกที่ไม่ขาดช่วงให้ใช้ **Logging**

3.4 แท็บ "ลำดับทดสอบ"

สร้าง บันทึก โหลด และรันลำดับทดสอบ จะมี**ลำดับเดียวเท่านั้น**ที่โหลด/ใช้งานอยู่เสมอ ลำดับหลายรายการเก็บเป็นไฟล์ในโฟลเดอร์ `sequences/` ของโปรเจกต์

ภาพหน้าจอพร้อมคำอธิบาย



ภาพแสดงหน้าจอแบบแผนภาพ หมายเลขอ้างอิงถึงคำอธิบายด้านล่าง

1. เลือก โหลด สร้าง และบันทึกไฟล์ลำดับการทดสอบ (ครั้งละหนึ่งลำดับ)
2. รายการคำสั่ง: เมื่อวางเมาส์บนรายการจะแสดงคำแนะนำพร้อมรูปแบบ องค์ประกอบ และตัวอย่าง
3. แทรก ย้าย ทำซ้ำ หรือลบขั้นตอน
4. รายการขั้นตอน: คำสั่งและพารามิเตอร์เป็น JSON แก้ไขได้โดยตรงในตาราง
5. เริ่ม หยุด และตรวจสอบลำดับแบบสถิต ผลรวมแสดงอยู่ข้างๆ
6. ตารางผลลัพธ์ต่อขั้นตอน: เขียว = ผ่าน แดง = ไม่ผ่าน พร้อมข้อความ

ไฟล์

ส่วนควบคุม	หน้าที่
รายการไฟล์ + โหลด	เลือกลำดับจาก `sequences/` และโหลดเข้าตัวแก้ไข
ใหม่	ลำดับว่าง (ยังไม่บันทึกจนกว่าจะ *บันทึก*)
บันทึก / บันทึกเป็น ...	เขียนลำดับเป็น JSON ชื่อไฟล์ที่บันทึก/โหลดล่าสุดจะถูกจดไว้ในโปรเจกต์และโหลดอีกครั้งเมื่อเปิดครั้งถัดไป
ชื่อ / คำอธิบาย	เมทาดาทาของลำดับ

การแก้ไขขั้นตอน

ขั้นตอนอยู่ในตาราง (ลำดับ, ชนิด, พารามิเตอร์เป็น JSON) **ข้อความพารามิเตอร์แก้ไขได้โดยตรง** (ดับเบิลคลิก) JSON ที่ไม่ถูกต้องจะถูกปฏิเสธและคืนค่าเดิม ชนิดของขั้นตอนเปลี่ยนไม่ได้

ส่วนควบคุม	หน้าที่
ดรอปดาวน์คำสั่ง	เลือกชนิดขั้นตอน **เมื่อเลื่อนเมาส์ไปบนรายการจะมี tooltip** แสดงรูปแบบ คำอธิบายขององค์ประกอบทั้งหมด และตัวอย่าง ด้านล่างยังแสดงสรุปของคำสั่งที่เลือกอยู่ด้วย
เพิ่มขั้นตอน	แทรกคำสั่งพร้อมแม่แบบหลังแถวที่เลือก (หรือท้ายสุด)
ขึ้น / ลง / ทำสำเนา / ลบ	ย้าย คัดลอก หรือลบขั้นตอนที่เลือก
เลือกสัญญาณ + ใส่สัญญาณในขั้นตอน	แสดงสัญญาณทั้งหมดของ DBC เป็น `Message.Signal` (พร้อมเติมคำอัตโนมัติขณะพิมพ์) และใส่ข้อความ (และสัญญาณ สำหรับ `set_signal`, `wait_for`, `check`) ลงในขั้นตอนที่เลือก
Tooltip ในตาราง	tooltip ของคำสั่งจะแสดงเมื่อชี้ที่คอลัมน์ชนิดของขั้นตอนด้วย

ตรวจสอบและรัน

ส่วนควบคุม	หน้าที่
ตรวจสอบ	ตรวจแบบสถิต: ชนิดขั้นตอน ตัวดำเนินการ ข้อความ และสัญญาณที่ไม่รู้จัก (เทียบกับ DBC)
เริ่ม	รันลำดับ (เมื่อเชื่อมต่ออยู่เท่านั้น) ก่อนเริ่มจะตรวจสอบอัตโนมัติ ตัวแก้ไขถูกล็อกขณะรัน
หยุด	ยกเลิกลำดับ (รวมถึง `wait` ที่กำลังรัน) สถานะ *aborted*

ตารางผลลัพธ์	ต่อขั้นตอน: หมายเลข (ลูป: เช่น `3.2.1`) คำอธิบาย สถานะ (เขียว = ok, แดง = fail/error) และข้อความ (เช่น ค่าที่วัดได้ ไบต์ของการตอบกลับ)

กฎการรัน: ขั้นตอนทำงานต่อเนื่องกัน **ขั้นตอนแรกที่ล้มเหลวจะจบลำดับ** (สถานะ *failed*) เมื่อจบ แถบสถานะแสดงเช่น *startup: passed (10/10 ขั้นตอน ok)* ข้อความที่เริ่มส่งเป็นรอบจะส่งต่อไปหลังจบ เว้นแต่หยุดด้วย `enable_tx`

ภาพรวมคำสั่ง

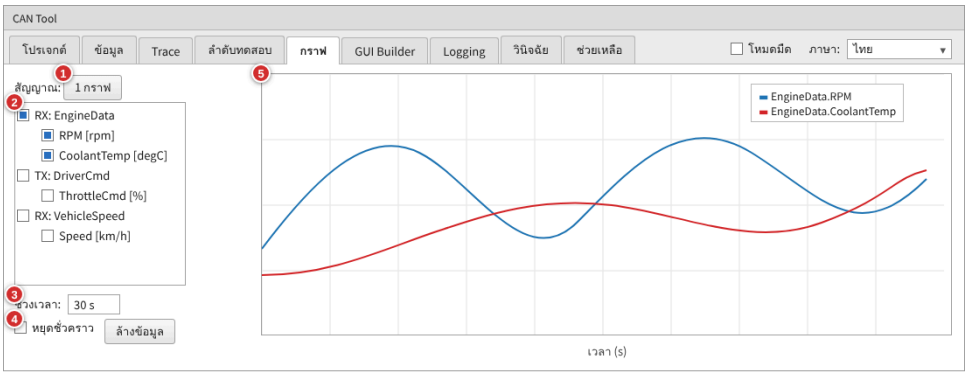
คำสั่ง	ชื่อ	สรุป
`set_signal`	ตั้งค่าสัญญาณ	ตั้งค่าทางกายภาพของสัญญาณในข้อความส่ง ค่าใหม่จะออกบนบัสพร้อมเฟรมถัดไป (แบบเป็นรอบหรือส่งครั้งเดียว)
`enable_tx`	เปิด/ปิดการส่งเป็นรอบ	เริ่มหรือหยุดการส่งข้อความเป็นรอบตามคาบที่ตั้งไว้
`set_cycle`	เปลี่ยนคาบ	เปลี่ยนคาบของข้อความส่ง (มีผลทันที แม้ขณะกำลังส่งเป็นรอบ)
`send_once`	ส่งข้อความหนึ่งครั้ง	ส่งหนึ่งเฟรมด้วยค่าสัญญาณปัจจุบัน โดยไม่ขึ้นกับการส่งเป็นรอบ
`send_raw`	ส่งเฟรมดิบ	ส่งเฟรมดิบ (ID + ข้อมูล) บนบัส CAN ครั้งเดียวหรือวนซ้ำ เมื่อ cyclic=true และ enabled=false จะหยุดการส่งวนซ้ำของ ID นี้
`wait`	รอตามเวลาที่กำหนด	หยุดลำดับชั่วคราวตามเวลาที่กำหนด การส่งเป็นรอบและการรับยังทำงานต่อ
`wait_for`	รอเงื่อนไขของสัญญาณ	รอจนสัญญาณเป็นไปตามเงื่อนไข นับเฉพาะค่าที่ได้รับหลังจากขั้นตอนนี้เริ่มแล้ว ขั้นตอนเป็น ok เมื่อพบครั้งแรก มิฉะนั้นเป็น error เมื่อหมดเวลา timeout
`check`	ตรวจสอบค่าสัญญาณ	ตรวจสอบค่าล่าสุดที่ได้รับของสัญญาณทันที (ไม่รอ) ขั้นตอนจะล้มเหลวหากไม่เป็นไปตามเงื่อนไข หรือยังไม่เคยได้รับค่าตั้งแต่เริ่มลำดับ
`diag`	Service วินิจฉัยจาก CDD	ส่ง Service วินิจฉัยที่นิยามใน CDD ผ่าน ISO-TP และประเมินการตอบกลับ
`diag_raw`	Request วินิจฉัยแบบ Hex	ส่ง UDS Request ใดก็ได้ (ไบต์ Hex) ผ่าน ISO-TP และประเมินการตอบกลับ
`loop`	วนซ้ำ	รันขั้นตอนที่อยู่ภายในซ้ำหลายรอบต่อเนื่องกัน
`comment`	หมายเหตุ	ไม่ทำอะไร ใช้บันทึกเอกสารของลำดับเท่านั้น

คำอธิบายครบถ้วนของแต่ละคำสั่ง พร้อมรูปแบบ องค์ประกอบ และตัวอย่าง อยู่ใน ภาคผนวก A

3.5 แท็บ "กราฟ"

แท็บนี้แสดงเส้นสัญญาณแบบสดเป็นกราฟเส้น (แกนเวลา: วินาทีเทียบกับปัจจุบัน ซ้าย = อดีต ขวา = ปัจจุบัน) สวิตช์เหนือรายการเลือกใช้สลับระหว่าง **1 กราฟ** และ **n กราฟ**

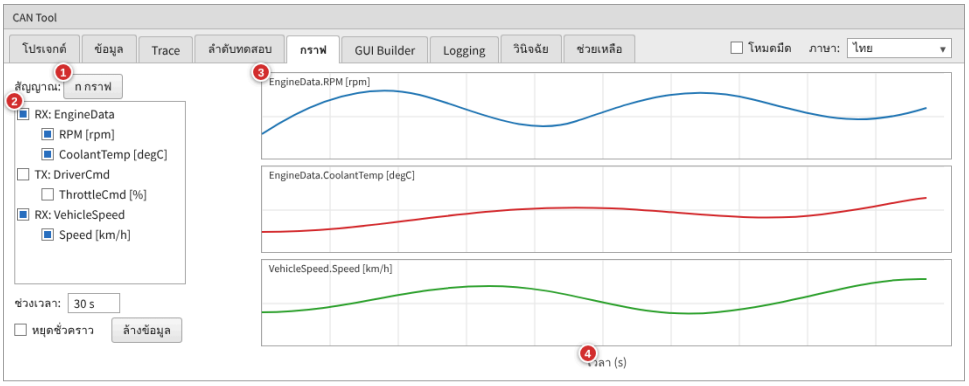
ภาพหน้าจอพร้อมคำอธิบาย



ภาพแสดงหน้าจอแบบแผนภาพ หมายเลขอ้างอิงถึงคำอธิบายด้านล่าง

1. สวิตช์ 1 กราฟ / n กราฟ (ที่นี่: 1 กราฟ)
2. การเลือกสัญญาณจัดกลุ่มตามข้อความ ช่องของข้อความเลือกสัญญาณทั้งหมด หน่วยอยู่ในวงเล็บเหลี่ยม
3. ช่วงเวลาเป็นวินาที
4. หยุดชั่วคราวตรึงหน้าจอ ล้างข้อมูลทิ้งเส้นสัญญาณ
5. แผนภูมิรวมพร้อมคำอธิบาย แกนเวลาเป็นวินาทีเทียบกับปัจจุบัน

ภาพหน้าจอพร้อมคำอธิบาย



ภาพแสดงหน้าจอแบบแผนภาพ หมายเลขอ้างอิงถึงคำอธิบายด้านล่าง

1. สวิตช์ตั้งเป็น n กราฟ
2. การเลือกยังจัดกลุ่มตามข้อความ ที่นี่เลือกสามสัญญาณ
3. แต่ละสัญญาณมีกราฟและแกนค่าของตนเอง ติดป้ายเป็น ข้อความ.สัญญาณ และหน่วย
4. แกนเวลาร่วม: ซูมและเลื่อนมีผลกับทุกกราฟ ป้ายอยู่ด้านล่างเท่านั้น

การเลือกที่จัดกลุ่มตามข้อความ

รายการเลือกแสดงสัญญาณทั้งหมดที่เลือกไว้ในแท็บข้อมูล จัดกลุ่มตามทิศทางและข้อความ (RX = รับ, TX = ส่ง) ใต้แต่ละข้อความจะแสดงสัญญาณพร้อมหน่วยในวงเล็บเหลี่ยม ช่องทำเครื่องหมายของข้อความมีสามสถานะ: เลือกครบทุกสัญญาณ เลือกบางส่วน หรือไม่เลือกเลย การคลิกจะเลือกสัญญาณทั้งหมดของข้อความหรือยกเลิกทั้งหมด

การแสดงผล "1 กราฟ"

สัญญาณที่เลือกทั้งหมดถูกวาดในแผนภูมิเดียวพร้อมคำอธิบายสี ข้อดีคือเปรียบเทียบเส้นได้โดยตรง หากช่วงค่าต่างกันมาก สัญญาณขนาดเล็กจะดูแบน ในกรณีนั้น "n กราฟ" เหมาะกว่า

การแสดงผล "n กราฟ"

แต่ละสัญญาณที่เลือกมีแผนภูมิและแกนค่าของตนเอง ติดป้ายเป็น ข้อความ.สัญญาณ พร้อมหน่วย โดยเรียงซ้อนกันในแนวตั้ง ทุกแผนภูมิใช้ **ฐานเวลา** เดียวกัน: การซูมหรือเลื่อนแกนเวลาในแผนภูมิหนึ่งมีผลกับทุกแผนภูมิ และป้ายเวลาจะแสดงเฉพาะแผนภูมิล่างสุด

ส่วนควบคุม

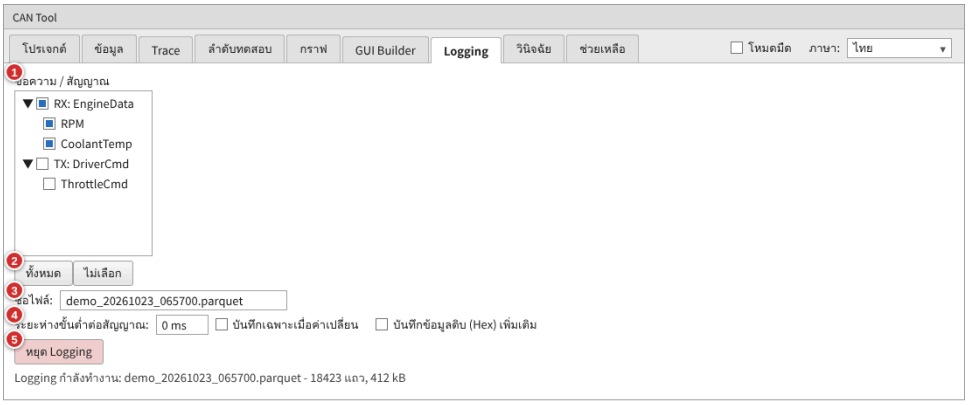
ส่วนควบคุม	ฟังก์ชัน
สวิตช์ 1 กราฟ / n กราฟ	เปลี่ยนรูปแบบการแสดงผล หน้าจอถูกสร้างใหม่ทันที การตั้งค่าถูกบันทึกในโปรเจกต์ (`graph_multi`)
การเลือกสัญญาณ	ติ๊ก = แสดงเส้น ช่องทำเครื่องหมายของข้อความเลือกหรือยกเลิกสัญญาณทั้งหมดของมัน การเลือกถูกบันทึกในโปรเจกต์ (`graph`)
ช่วงเวลา	ช่วงเวลาที่มองเห็นเป็นวินาที (1 ถึง 3600)
หยุดชั่วคราว	ตรึงหน้าจอ ข้อมูลยังถูกเก็บต่อเบื้องหลัง
ล้างข้อมูล	ทิ้งเส้นสัญญาณที่เก็บไว้ทั้งหมด
แผนภูมิ	เมาส์: ซูมด้วยลูกกลิ้ง เลื่อนด้วยการกดปุ่มค้าง ใน "n กราฟ" มีผลกับทุกแผนภูมิพร้อมกัน

หมายเหตุ: เก็บได้สูงสุด 50 000 จุดต่อสัญญาณ บันทึกเฉพาะสัญญาณที่ติ๊กในแท็บข้อมูล สัญญาณจะปรากฏเมื่อมีการรับ/ส่งข้อมูลแล้วเท่านั้น การแสดงผลอัปเดตเฉพาะเมื่อแท็บแสดงอยู่

3.6 แท็บ "Logging"

สัญญาณที่ถอดรหัสแล้ว: เมื่อโหลด DBC, A2L หรือ CDD ไฟล์ Parquet จะมีไม่เพียงข้อความเป็นเลขฐานสิบหก แต่ยังมีสัญญาณพร้อม**ค่าดิบ ค่าดิบฐานสิบหก ค่าทางกายภาพ ข้อความ (ตารางค่า) และหน่วย** คอลัมน์ของไฟล์สัญญาณ: `timestamp, direction, message, signal, value, source, can_id, raw, raw_hex, text, unit, message_hex` ระหว่างบันทึก `source` คือ `CAN` (ข้อความ DBC), `XCP` (ค่า A2L ที่อ่านในแท็บ XCP) หรือ `UDS` (พารามิเตอร์การตอบกลับที่ถอดรหัสจากแท็บวินิจฉัยตาม CDD) ไฟล์ดิบ (`_raw.parquet`) มีคอลัมน์ `signals` เพิ่มเติม แสดงสัญญาณทั้งหมดของเฟรมเป็นข้อความ เช่น `RPM=0x1F40/2000 rpm`

ภาพหน้าจอพร้อมคำอธิบาย



ภาพแสดงหน้าจอแบบแผนภาพ หมายเลขอ้างอิงถึงคำอธิบายด้านล่าง

1. เลือกข้อความและสัญญาณที่จะบันทึก (กรองจากข้อมูลที่เลือกไว้)
2. เลือกสัญญาณทั้งหมดหรือไม่เลือกเลย
3. ชื่อไฟล์ Parquet (เก็บในโฟลเดอร์ logs/ ของโปรเจกต์)
4. ตัวเลือก: เฉพาะเมื่อค่าเปลี่ยน ระยะห่างขั้นต่ำต่อสัญญาณ บันทึกข้อมูลดิบเพิ่ม
5. เริ่มหรือหยุดการบันทึก แถบสถานะแสดงจำนวนแถวและขนาดไฟล์

บันทึกสัญญาณที่เลือกในรูปแบบ **Parquet** (ต้องใช้ `pyarrow`) ไฟล์อยู่ในโฟลเดอร์ `logs/` ของโปรเจกต์

ส่วนควบคุม	หน้าที่
ต้นไม้ข้อมูล	แสดงข้อความ/สัญญาณที่เลือกในแท็บข้อมูล **ตัวกรอง:** บันทึกเฉพาะสัญญาณที่ติ๊ก (การติ๊กที่ข้อความจะเลือก/ยกเลิกสัญญาณทั้งหมดของข้อความ) การเลือกถูกบันทึกในโปรเจกต์
ทั้งหมด / ไม่เลือก	เลือกหรือยกเลิกสัญญาณทั้งหมด
ชื่อไฟล์	ค่าตั้งต้นเป็น *โปรเจกต์_วันที่_เวลา.parquet* จะเติมนามสกุลให้เมื่อจำเป็น
ระยะห่างขั้นต่ำต่อสัญญาณ	ต่อสัญญาณบันทึกได้มากสุดหนึ่งรายการต่อช่วงเวลา (ms) - ลดปริมาณข้อมูลของข้อความที่เร็ว 0 = ทุกเฟรม
เฉพาะเมื่อค่าเปลี่ยน	เขียนสัญญาณเฉพาะเมื่อค่าต่างจากค่าที่เขียนล่าสุด
เฟรมดิบ	เขียน *<ชื่อ>_raw.parquet* เพิ่มเติม ซึ่งมี**ทุก**เฟรมบนบัส (ไม่ขึ้นกับ DBC และการเลือกสัญญาณ รวมถึงเฟรมผิดพลาด) คอลัมน์: เวลา, ทิศทาง, ช่อง, ID, ขยาย, FD, BRS, ผิดพลาด, DLC, ข้อมูล (ไบนารี), ข้อมูล (ข้อความ hex), ชื่อข้อความ จึงบันทึกได้แม้ไม่เลือกสัญญาณ **ID ข้อมูลดิบ** ใช้จำกัดเฟรม เช่น `100, 7E0-7EF` (hex, ว่าง = ทั้งหมด)
เริ่ม / หยุด Logging	เริ่มหรือจบการบันทึก (เมื่อเชื่อมต่อเท่านั้น) เมื่อตัดการเชื่อมต่อหรือสลับโปรเจกต์ จะหยุดอัตโนมัติและปิดไฟล์อย่างถูกต้อง
แถบสถานะ	ชื่อไฟล์ จำนวนแถวที่เขียน ขนาดไฟล์

รูปแบบไฟล์

คอลัมน์	ชนิด	เนื้อหา
timestamp	float64	Timestamp (วินาทีแบบ Epoch)
direction	string	`RX` หรือ `TX`
message	string	ชื่อข้อความ
signal	string	ชื่อสัญญาณ
value	float64	ค่าทางกายภาพ

รูปแบบยาว (หนึ่งแถวต่อค่าสัญญาณหนึ่งค่า) หลีกเลี่ยงช่องว่างเมื่อคาบต่างกัน ข้อมูลถูกเก็บในบัฟเฟอร์และ เขียนลงดิสก์ทุก 5 วินาทีหรือเมื่อครบ 20 000 แถว (บีบอัดด้วย zstd)

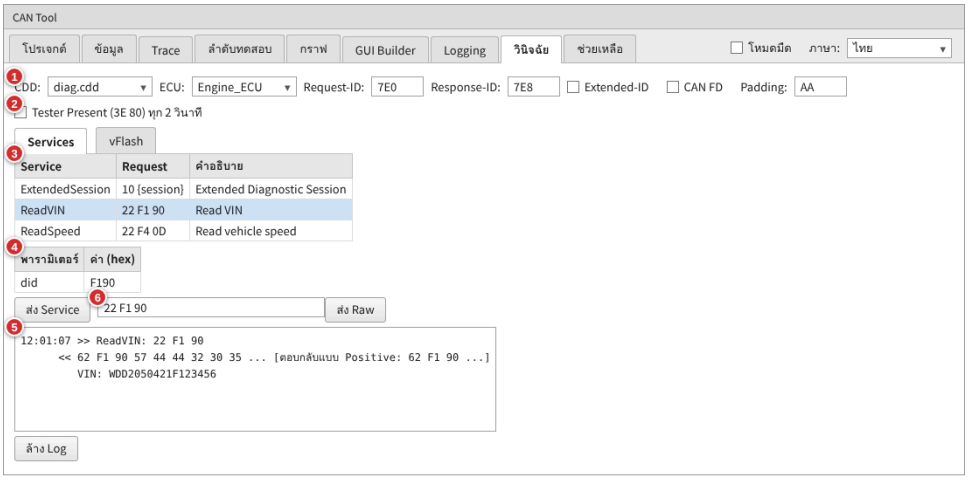
การวิเคราะห์ใน Python (ดูภาคผนวก B ด้วย):

```
import pandas as pd
df = pd.read_parquet("projects/demo/logs/demo_20261020_071500.parquet")
wide = df.pivot_table(index="timestamp", columns=["direction", "message", "signal"], values="value")
```

3.7 แท็บ "วินิจฉัย"

Service วินิจฉัย (UDS) ผ่าน **ISO-TP** (ISO 15765-2) บน CAN หรือ CAN FD Service มาจาก CDD ที่แปลงแล้ว และยังส่ง Request แบบ Hex ใดก็ได้เพิ่มเติมได้

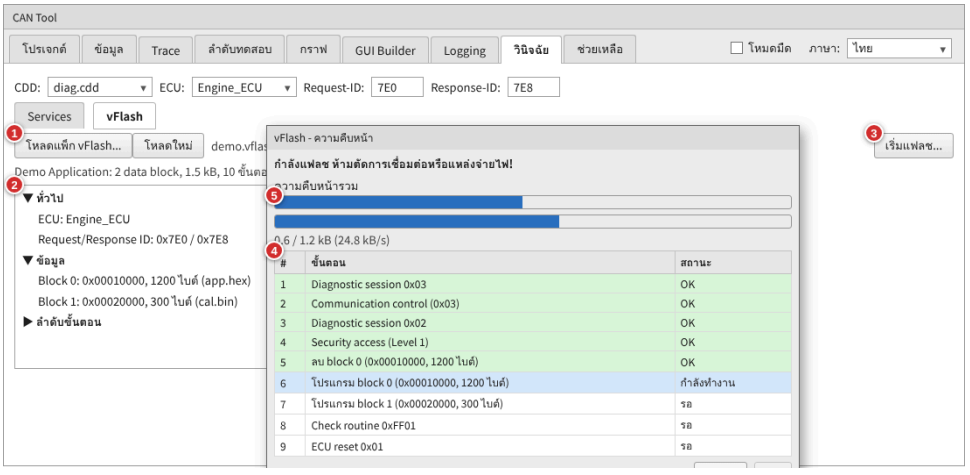
ภาพหน้าจอพร้อมคำอธิบาย



ภาพแสดงหน้าจอแบบแผนภาพ หมายเลขอ้างอิงถึงคำอธิบายด้านล่าง

1. การตั้งค่าวินิจฉัย: CDD, ECU, ID คำขอ/ตอบกลับ, Extended ID, CAN FD, Padding
2. ส่ง Tester Present (3E 80) อัตโนมัติทุก 2 วินาที
3. เซอร์วิสจาก CDD พร้อมเทมเพลตคำขอและคำอธิบาย
4. พารามิเตอร์ของเซอร์วิสที่เลือก (แก้ไขค่าเลขฐานสิบหกได้) จากนั้นกด “ส่งเซอร์วิส”
5. บันทึกการตอบกลับ: คำขอที่ส่ง การตอบกลับ (บวก/ลบ) และค่าที่ถอดรหัสแล้ว
6. ส่งคำขอดิบแบบเลขฐานสิบหก (เช่น 22 F1 90) โดยไม่ต้องมีคำจำกัดความ CDD

ภาพหน้าจอพร้อมคำอธิบาย



ภาพแสดงหน้าจอแบบแผนภาพ หมายเลขอ้างอิงถึงคำอธิบายด้านล่าง

1. โหลดแพ็ก vFlash (.vflashpack/.zip, manifest, HEX/S19/BIN) "โหลดใหม่" จะอ่านไฟล์อีกครั้ง
2. การตีความแพ็ก: ทั่วไป (ECU, ID, Seed&Key), ข้อมูล (block พร้อม address และ length), ลำดับขั้นตอน (steps) และหมายเหตุ
3. เริ่มแฟลช: หลังการยืนยันความปลอดภัย ลำดับขั้นตอนจะรันบนฮาร์ดแวร์ที่เลือก
4. หน้าต่างความคืบหน้า: สถานะของแต่ละขั้นตอน (รอ, กำลังทำงาน, OK, ล้มเหลว, ข้าม) พร้อมรายละเอียด
5. ความคืบหน้ารวม ความคืบหน้าของ data block ปัจจุบันพร้อมอัตราข้อมูล และบันทึก "ยกเลิก" จะหยุดระหว่างสอง block

การตั้งค่า (แถวบน)

ช่อง	ความหมาย
CDD	คำอธิบายการวินิจฉัยที่แปลงแล้วจากแคชของโปรเจกต์
ECU	ECU จาก CDD เมื่อเปลี่ยนจะใช้ที่อยู่ของ ECU นั้น
Request-ID / Response-ID	CAN ID เป็น Hex (Tester -> ECU / ECU -> Tester) ตั้งต้นจาก CDD แก้ทับได้ที่นี่

Extended-ID	การระบุที่อยู่ 29 บิต
CAN FD	ISO-TP กับเฟรม FD (สูงสุด 64 ไบต์ต่อเฟรม พร้อมสลับบิตเรต) ต้องตรงกับอีกฝั่ง
Padding	ไบต์เติมเป็น Hex (ค่าเริ่มต้น AA) ว่าง = ไม่เติมสำหรับ Classic CAN
Tester Present	ส่ง `3E 80` ทุก 2 วินาที (ไม่มีการตอบกลับ) เพื่อให้ diagnostic session เปิดอยู่
แท็บ vFlash	โหลดแพ็ก vFlash (หรือ manifest/HEX/S19/BIN) แสดงการตีความ และรันลำดับการแฟลชบนฮาร์ดแวร์ที่เลือก พร้อมหน้าต่างความคืบหน้า (ภาคผนวก C)

การส่ง Service

1. เลือก Service ในรายการ (ชื่อ, Request, คำอธิบาย)
2. หาก Service มีพารามิเตอร์ ให้ป้อนค่าในตาราง *พารามิเตอร์ / ค่า (hex)* (ตัวแทนค่าใน Request)
3. **ส่ง Service** การส่งทำงานเบื้องหลัง หน้าจอยังใช้งานได้
4. **Log** แสดง Request (>>) การตอบกลับ (<<) พร้อมการประเมิน (positive/negative หากเป็น negative จะแสดง NRC พร้อมคำอธิบาย) และพารามิเตอร์การตอบกลับที่ถอดรหัสจาก CDD (เช่น *VIN: WDD...*)

ส่ง Raw: ป้อน Request เป็นไบต์ Hex (เช่น `22 F1 86`) แล้วกด Enter หรือใช้ปุ่ม

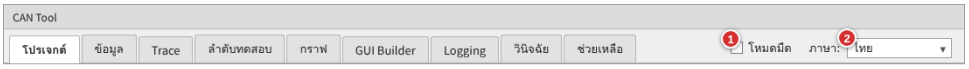
รายละเอียดทางเทคนิค

- การแบ่งส่วน: Single Frame, First Frame, Consecutive Frame และ Flow Control สูงสุด 4095 ไบต์ (ยาวกว่านั้นใช้ extended First Frame)
- เมื่อรับการตอบกลับที่ยาว เครื่องมือส่ง Flow Control ด้วย Blocksize 0 และ STmin 0
- Timeout การตอบกลับ 2 วินาที เมื่อเป็น *ResponsePending* (NRC 0x78) จะรอต่อได้สูงสุด 10 วินาที
- Request ที่ไม่มีการตอบกลับ (Sub-Function ที่มีบิต 7 เช่น `3E 80`) จะถูกบันทึกว่าส่งแล้ว
- ในลำดับทดสอบ `diag` และ `diag_raw` ใช้การตั้งค่าที่อยู่เดียวกัน

3.8 แถบแท็บ: โหมดมืดและภาษา

ที่ขอบขวาของแถบแท็บ (ข้างแท็บต่าง ๆ) มีส่วนควบคุมสองอย่างที่ใช้กับทุกแท็บ

ภาพหน้าจอพร้อมคำอธิบาย



ภาพแสดงหน้าจอแบบแผนภาพ หมายเลขอ้างอิงถึงคำอธิบายด้านล่าง

1. โหมดมืด: สลับระหว่างธีมสว่างและมืด (จดจำไว้)
2. ภาษา: รายการแบบเลื่อนลงของไฟล์ภาษาทั้งหมด การสลับมีผลทันที รวมถึงวิธีใช้

ส่วนควบคุม	หน้าที่
โหมดมืด	เครื่องหมายติ๊กสลับระหว่างธีมสว่างและธีมมืด มีผลกับหน้าจอ Trace กราฟ ตารางผลลัพธ์ของลำดับ และวิธีใช้นี้ การตั้งค่าจะถูกบันทึกและใช้ในการเริ่มครั้งถัดไป
ภาษา	เมนูดรอปดาวน์: Deutsch (ภาษาหลัก), English, Francais, Espanol, จีน (กลาง, ตัวย่อ) และไทย การสลับมีผลทันทีโดยไม่ต้องรีสตาร์ท การเลือกจะถูกบันทึก

อะไรบ้างที่ถูกแปล?

- ป้ายกำกับ ปุ่ม หัวตาราง tooltip กล่องโต้ตอบ และข้อความทั้งหมดของหน้าจอ
- อ้างอิงคำสั่งของลำดับทดสอบ (tooltip ในดรอปดาวน์ ภาคผนวก A) และทุกบทของวิธีใช้นี้
- ข้อความข้อผิดพลาดและคำเตือนของส่วนโปรแกรม (เช่น การแปลง การเชื่อมต่อ การรันลำดับ)
- **ไม่**แปลงข้อมูลจากไฟล์ของคุณ: ชื่อข้อความและสัญญาณ หน่วย และตารางค่าจาก DBC/CDD ชื่อไฟล์ ชื่อโปรเจกต์ และเนื้อหาของไฟล์ลำดับ
- บรรทัดบันทึกที่เขียนไปแล้ว (Log วินิจฉัย ตารางผลลัพธ์ของลำดับ) จะคงภาษาที่เกิดขึ้น

สำหรับภาษาจีนและไทย ในเครื่องต้องมีฟอนต์ที่มีอักขระที่เกี่ยวข้อง (การติดตั้ง Windows, macOS และ Linux ทั่วไปมีอยู่แล้ว)

ไฟล์ภาษา

แต่ละภาษาอยู่ในไฟล์ของตัวเองในโฟลเดอร์ `lang/` (YAML; ใช้ JSON ได้เช่นกัน):

ไฟล์	ภาษา
`lang/de.yaml`	Deutsch - ภาษาหลักและภาษาสำรอง: หากภาษาใดขาดข้อความ จะใช้ข้อความภาษาเยอรมันแทน
`lang/en.yaml`	English
`lang/fr.yaml`	Francais
`lang/es.yaml`	Espanol
`lang/zh.yaml`	จีนกลาง (อักษรจีนตัวย่อ)
`lang/th.yaml`	ไทย

โครงสร้างของแต่ละไฟล์: ส่วน `meta` (ชื่อที่แสดง ลำดับในดรอปดาวน์), `ui` (หน้าจอ), `core` (ข้อความแจ้ง), `steps` (อ้างอิงคำสั่ง) และ `help` (บทของวิธีใช้นี้) ตัวแทนค่าใน วงเล็บปีกกา (เช่น `{name}`) และตัวแทนค่าในวงเล็บเหลี่ยมคู่ในข้อความวิธีใช้ ห้ามแก้ไข

การเพิ่มหรือแก้ไขภาษา

1. คัดลอก `lang/de.yaml` แล้วบันทึกเป็น `lang/<code>.yaml` (เช่น `it.yaml`)
2. ในส่วน `meta` ตั้งชื่อที่แสดง (`name`) และตำแหน่ง (`order`) แล้วแปลข้อความทั้งหมด
3. ตรวจสอบด้วย `python tools/check_lang.py it`: เครื่องมือจะรายงานคีย์ที่ขาด ตัวแทนค่าที่ไม่ตรง และโครงสร้าง HTML ที่เสียหาย
4. รีสตาร์ทโปรแกรม - ภาษาจะปรากฏในดรอปดาวน์เองโดยอัตโนมัติ

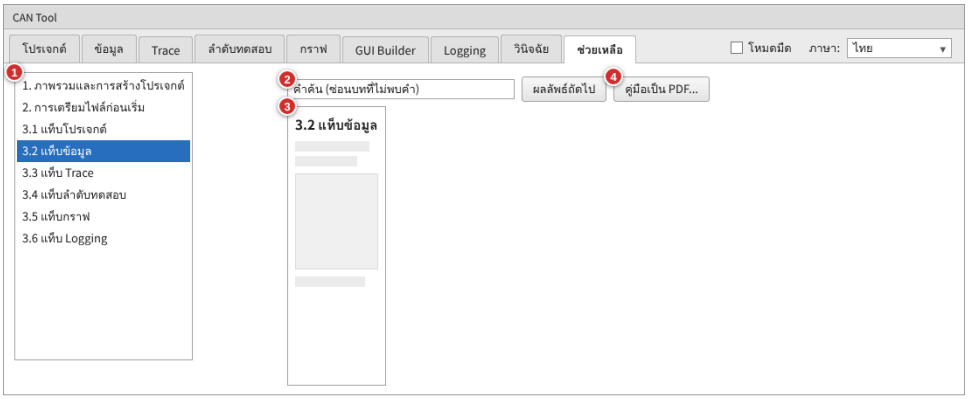
บรรทัดคำสั่ง

เอาต์พุตของ `main.py` ก็ถูกแปลเช่นกัน: `python main.py --lang en --list` หากไม่ระบุ `--lang` จะใช้ภาษาที่เลือกล่าสุดใน GUI

3.9 แท็บ "ช่วยเหลือ"

แท็บนี้ ด้านซ้ายคือบท ด้านขวาคือเนื้อหา

ภาพหน้าจอพร้อมคำอธิบาย



ภาพแสดงหน้าจอแบบแผนภาพ หมายเลขอ้างอิงถึงคำอธิบายด้านล่าง

1. รายการบทของวิธีใช้
2. ช่องค้นหา: ซ่อนบทที่ไม่พบ “ผลถัดไป” ข้ามไปยังรายการถัดไป
3. เนื้อหาบทพร้อมภาพประกอบที่มีคำอธิบาย
4. คู่มือ PDF: ส่งออกวิธีใช้ทุกภาษาที่มี (มีหน้าปกและวันที่เผยแพร่) และให้บันทึกไฟล์

- **ช่องค้นหา**: ซ่อนบทที่ไม่มีคำนั้น และไฮไลต์ตำแหน่งแรกที่พบใน บทที่เลือก **ผลลัพธ์ถัดไป** ข้ามไปตำแหน่งถัดไป เมื่อล้างช่อง จะแสดงทุกบทอีกครั้ง
- ลิงก์ในเนื้อหา (เช่น ไปยังภาคผนวก) คลิกได้
- คำอธิบายคำสั่งตรงกับ tooltip ในดรอปดาวน์คำสั่งของแท็บลำดับทดสอบ
- วิธีใช้แสดงเป็นภาษาที่เลือก (บทที่ 3.8) และจะเปลี่ยนทันทีเมื่อสลับภาษา การสลับสว่าง/มืดก็มีผลที่นี่ด้วย

3.10 แท็บ "XCP"

วัดและปรับค่าตัวแปรของ ECU ผ่าน **XCP on CAN** โดยอิงจาก**ไฟล์ A2L** (ASAP2) A2L ถูกแปลงเหมือน DBC และ CDD: ไฟล์ต้นฉบับอยู่ในโฟลเดอร์ `a2l/` ของโปรเจกต์ ขณะทำงานจะอ่านเฉพาะแคช `cache/<name>.a2l.json|yaml`

การตั้งค่า

1. คัดลอก A2L เข้าโปรเจกต์ด้วย **นำเข้า** (แท็บโปรเจกต์) จะแปลงอัตโนมัติ (`python main.py --convert demo` ใช้ได้โดยไม่ต้องมี GUI)
2. เชื่อมต่อ CAN (แท็บโปรเจกต์) XCP ใช้อินเทอร์เฟซเดียวกัน
3. แท็บ XCP: เลือก A2L และตรวจสอบ **ID มาสเตอร์** (คำสั่งไปยัง ECU) และ **ID สเลฟ** (การตอบกลับ) ค่าจะถูกกรอกให้จากส่วน XCP ของ A2L (`IF_DATA XCP / XCP_ON_CAN`) และแก้ไขได้
4. **เชื่อมต่อ XCP** บันทึกแสดงคำสั่งทั้งหมด (->) และการตอบกลับ (<-) เป็นเลขฐานสิบหก

ส่วนควบคุม	หน้าที่
รายการด้านซ้าย	ค่าที่วัด (MEASUREMENT) และพารามิเตอร์ (CHARACTERISTIC ชนิด VALUE) ทั้งหมดของ A2L กรองตามชื่อ ดับเบิลคลิกหรือ *เพิ่มในรายการเฝ้าดู* เพื่อนำมาใช้
เฝ้าดู	แสดงค่าดิบและค่าทางกายภาพพร้อมหน่วยของแต่ละรายการ การเลือกจะถูกบันทึกกับโปรเจกต์
อ่านวนซ้ำ	อ่านทุกรายการที่เฝ้าดูตามช่วงเวลาที่ตั้ง (ms) ใช้ SHORT_UPLOAD ในการโพลล์ ไม่ใช้รายการ DAQ
อ่านหนึ่งครั้ง	อ่านรายการเฝ้าดูหนึ่งครั้ง
เขียนค่า	เขียนค่าที่ป้อน (ค่าทางกายภาพ หากเป็นการแปลงข้อความใช้ข้อความได้ เช่น *RUN*) ลงแถวที่เลือก (SET_MTA + DOWNLOAD) แล้วอ่านกลับ

การตีความ A2L

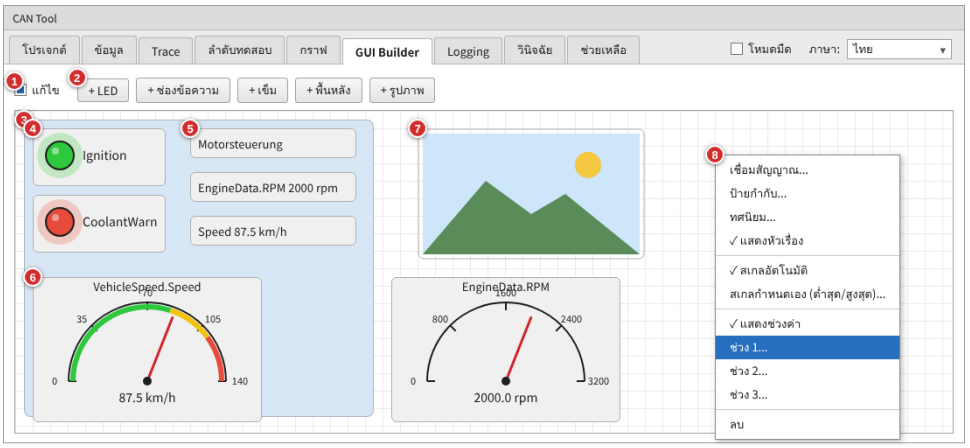
- ชนิดข้อมูล UBYTE, SBYTE, UWORD, SWORD, ULONG, SLONG, A_UINT64, A_INT64, FLOAT32_IEEE, FLOAT64_IEEE ลำดับไบต์ตามการตอบกลับ CONNECT ของ ECU
- การแปลง IDENTICAL, LINEAR, RAT_FUNC (ไม่รวมพจน์กำลังสอง) และ TAB_VERB (ค่าข้อความ) BIT_MASK จะถูกมาสก์ออกเมื่ออ่าน
- วัตถุที่ไม่รองรับ (เส้นโค้ง แผนที่ อาร์เรย์ ค่าที่วัดโดยไม่มีแอดเดรส) จะแสดงเป็นสีเทา

เมื่อฮาร์ดแวร์เป็น *การจำลอง* เครื่องมือจะเริ่ม XCP สเลฟจำลองที่สะท้อน A2L ของโปรเจกต์ (เดโม: `projects/demo/a2l/demo.a2l`) ไม่รวม: DAQ/STIM, seed&key, หน้าสอบเทียบ และการโปรแกรมแฟลช การที่ ECU จริงอนุญาตให้เข้าถึงทั้งหมดหรือไม่ขึ้นกับระดับการป้องกันของมัน

3.11 แท็บ "GUI Builder"

สร้างหน้าจอแสดงผลของคุณเองจากองค์ประกอบง่าย ๆ สำหรับสัญญาณที่มีอยู่ องค์ประกอบใช้สำหรับ**แสดงผล**เท่านั้น ไม่มีการป้อนค่า เชื่อมสัญญาณที่เลือกไว้ในแท็บข้อมูลได้ (รับหรือส่ง) เลย์เอาต์และการเชื่อมต่อถูกบันทึกในโปรเจกต์ (`builder`)

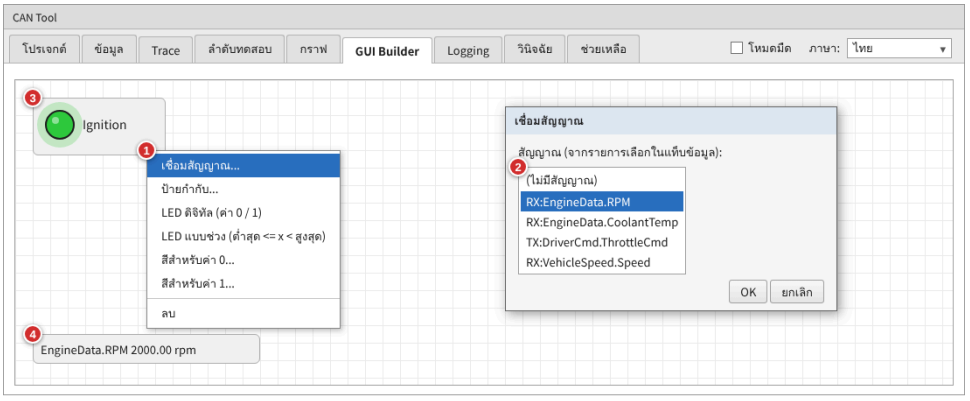
ภาพหน้าจอพร้อมคำอธิบาย



ภาพแสดงหน้าจอแบบแผนภาพ หมายเลขอ้างอิงถึงคำอธิบายด้านล่าง

1. ช่องติ๊กแก้ไข: แสดงตาราง เพิ่ม ย้าย และปรับขนาดได้
2. ปุ่มเพิ่ม LED ช่องข้อความ เข็ม พื้นหลัง และรูปภาพ
3. องค์ประกอบพื้นหลังที่เลือกสีได้: จัดกลุ่มองค์ประกอบด้วยภาพและอยู่ด้านหลัง
4. LED: ดิจิทัล (ค่า 0 / 1) และแบบช่วง (สีขึ้นกับช่วง ต่ำสุด <= x < สูงสุด)
5. ช่องข้อความ: ซ้ายไม่มีสัญญาณ (ข้อความธรรมดา) ขวามีสัญญาณ (ค่าและหน่วยแบบสด)
6. เข็มที่มีสเกล 180 องศา สเกลอัตโนมัติหรือกำหนดเอง และเลือกแสดงช่วงค่าได้
7. กรอบรูป: รูปที่เลือกถูกปรับขนาดให้พอดีกรอบอัตโนมัติ
8. เมนูคลิกขวาของเข็ม: สัญญาณ หัวเรื่อง สเกล แสดงช่วงค่า และลบ

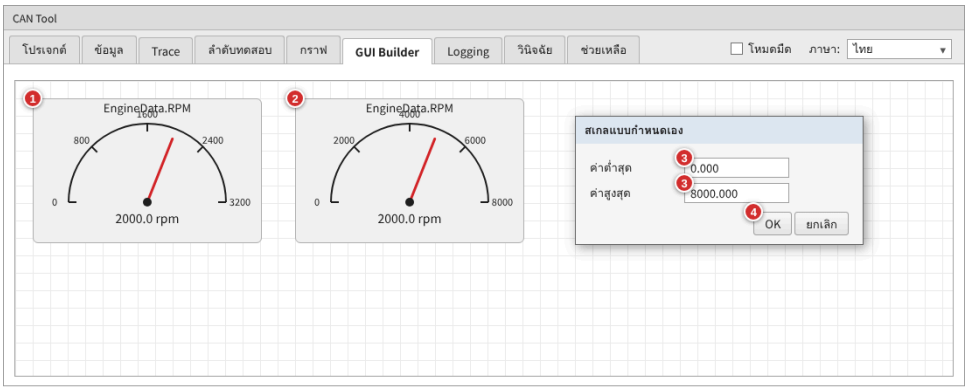
ภาพหน้าจอพร้อมคำอธิบาย



ภาพแสดงหน้าจอแบบแผนภาพ หมายเลขอ้างอิงถึงคำอธิบายด้านล่าง

1. คลิกขวาที่ LED: เชื่อมสัญญาณ ป้ายกำกับ โหมดดิจิทัล/ช่วง สี ลบ
2. รายการเลือก: สัญญาณจากแท็บข้อมูลในรูป RX:/TX:ข้อความ.สัญญาณ หรือ (ไม่มีสัญญาณ)
3. LED: สีขึ้นกับค่าสัญญาณ (ดิจิทัล 0 / 1 หรือตามช่วง 1 ถึง 3)
4. ช่องข้อความที่มีสัญญาณ: ข้อความ ค่า และหน่วย ตั้งทศนิยมได้

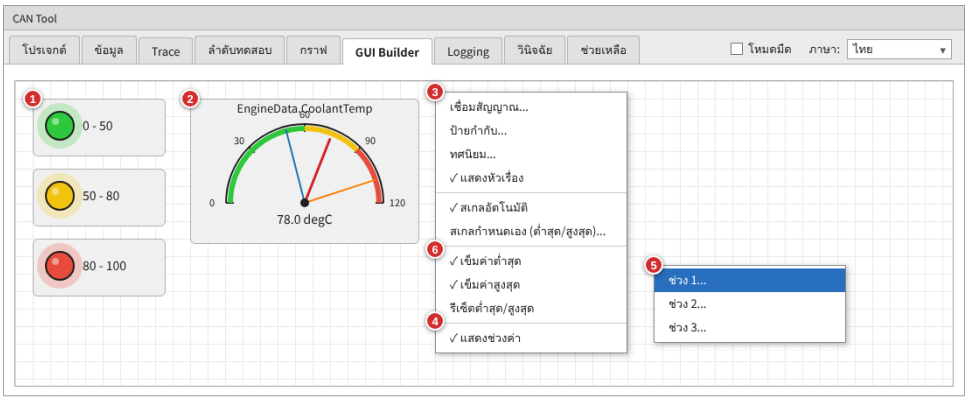
ภาพหน้าจอพร้อมคำอธิบาย



ภาพแสดงหน้าจอแบบแผนภาพ หมายเลขอ้างอิงถึงคำอธิบายด้านล่าง

1. เข็มที่มีสเกลอัตโนมัติ: ขอบเขตเป็นเลขกลม ที่นี่ 0 ถึง 3200
2. เข็มที่มีสเกลกำหนดเอง ที่นี่ 0 ถึง 8000
3. กล่องสเกลกำหนดเอง: ป้อนค่าต่ำสุดและสูงสุด
4. OK ใช้สเกล (เมื่อต่ำสุดน้อยกว่าสูงสุดเท่านั้น) ยกเลิกทิ้งค่า

ภาพหน้าจอพร้อมคำอธิบาย



ภาพแสดงหน้าจอแบบแผนภาพ หมายเลขอ้างอิงถึงคำอธิบายด้านล่าง

1. LED สามดวงแบบช่วง: แต่ละช่วงค่า (ต่ำสุด <= x < สูงสุด) กำหนดสี
2. เข็มที่แสดงช่วงค่า: สามช่วงสีบนสเกล (ช่วง 1 ถึง 3)
3. เมนู: แสดงหัวเรื่อง เปิดหรือปิดป้ายกำกับ
4. เมนู: เปิดหรือปิดการแสดงช่วงค่า
5. ช่วง 1 ถึง 3: แต่ละช่วงมีค่าต่ำสุด ค่าสูงสุด (กฎ ค่าต่ำสุด <= x < ค่าสูงสุด) สี และเปิด/ปิด
6. เมนู: เข็มที่สองและสามแบบเลือกได้สำหรับค่าต่ำสุด (น้ำเงิน) และสูงสุด (ส้ม) และการรีเซ็ต

องค์ประกอบ

องค์ประกอบ	ฟังก์ชัน
LED	แสดงสถานะของสัญญาณเป็นสี **ดิจิทัล**: ค่า 0 และค่า 1 มีสีที่เลือกได้อย่างละสี **ช่วง**: เหมือนสเกลของเข็ม มีสามช่วง (ต่ำสุด <= x < สูงสุด) ช่วงละหนึ่งสี หากค่าไม่อยู่ในช่วงใดหรือไม่มีค่า LED เป็นสีเทา
ช่องข้อความไม่มีสัญญาณ	ช่องข้อความธรรมดาที่กำหนดข้อความได้อิสระ เช่น หัวข้อหรือป้ายกำกับ
ช่องข้อความมีสัญญาณ	แสดงข้อความและค่าปัจจุบันของสัญญาณพร้อมหน่วย อัปเดตต่อเนื่อง ตั้งจำนวนทศนิยมได้
เข็ม	มาตรวัดแบบเข็มที่มีสเกล 180 องศา (ครึ่งวงกลม): ซ้ายคือค่าต่ำสุด ขวาคือค่าสูงสุด ด้านล่างแสดงค่าตัวเลขพร้อมหน่วย
พื้นหลัง	สี่เหลี่ยมสีธรรมดา (เลือกสีได้) ไม่มีฟังก์ชันอื่น ใช้จัดกลุ่มองค์ประกอบอื่นด้วยภาพ และอยู่หลังองค์ประกอบที่ใช้งานเสมอ แต่อยู่เหนือพื้นหลังมาตรฐาน
รูปภาพ	กรอบรูปที่เลือกรูปได้อิสระ (PNG, JPG, BMP, GIF, SVG) รูปจะถูกปรับให้พอดีกับกรอบอัตโนมัติ (คงอัตราส่วน) และปรับตามทุกการเปลี่ยนขนาด

การใช้งาน

ส่วนควบคุม	ฟังก์ชัน

แก้ไข	ติ๊ก: แสดงตาราง เพิ่ม ย้าย ปรับขนาด และลบองค์ประกอบได้ ไม่ติ๊ก: หน้าจอถูกล็อก (แสดงผลอย่างเดียว)
+ LED / + ช่องข้อความ / + เข็ม	เพิ่มองค์ประกอบใหม่ (หรือคลิกขวาบนพื้นที่ว่างเพื่อเพิ่ม ณ ตำแหน่งเมาส์)
ลาก	ย้ายองค์ประกอบ มุมขวาล่างใช้ปรับขนาด
คลิกขวาบนองค์ประกอบ	เมนู: เชื่อมสัญญาณ (หรือ "ไม่มีสัญญาณ") ป้ายกำกับหรือข้อความ เกณฑ์และสี LED ทศนิยม สเกล (เข็ม) ลบ

เข็มและสเกล

สเกลมีสองโหมด **อัตโนมัติ** (ค่าเริ่มต้น): ช่วงค่าขยายตามค่าที่สังเกตได้และปัดเป็นเลขกลม (เช่น 0 ถึง 100) ขีดและตัวเลขปรับตาม **กำหนดเอง**: **คลิกขวา**ที่เข็มแล้วเลือก **สเกลกำหนดเอง (ต่ำสุด/สูงสุด)** เพื่อกำหนดค่าต่ำสุดและสูงสุด (ค่าต่ำสุดต้องน้อยกว่าค่าสูงสุด) ช่องติ๊ก **สเกลอัตโนมัติ** ในเมนูเดียวกันใช้สลับกลับเป็นอัตโนมัติ ค่านอกสเกลทำให้เข็มหยุดที่ปลายสุด

เริ่มต้นอย่างรวดเร็ว

1. ติ๊ก **แก้ไข** ไว้ (แสดงตาราง)
2. คลิก **+ LED**, **+ ช่องข้อความ** หรือ **+ เข็ม** องค์ประกอบจะปรากฏบนพื้นที่วาด (หรือคลิกขวาบนพื้นที่ว่าง ณ ตำแหน่งเมาส์)
3. **คลิกขวา**ที่องค์ประกอบ เลือก **เชื่อมสัญญาณ** แล้วเลือกสัญญาณจากรายการ
4. ตั้งป้ายกำกับ เกณฑ์ LED ทศนิยม หรือสเกล และจัดตำแหน่งโดยการลาก มุมขวาล่างใช้ปรับขนาด
5. เอาเครื่องหมายติ๊ก **แก้ไข** ออก: หน้าจอถูกล็อกและใช้ดูอย่างเดียว

เชื่อมสัญญาณ

รายการแสดงสัญญาณที่เลือกไว้ในแท็บข้อมูลในรูป `RX:Msg.Signal` หรือ `TX:Msg.Signal` และมีรายการ "(ไม่มีสัญญาณ)" สำหรับยกเลิกการเชื่อม หากไม่มีสัญญาณที่ต้องการ ให้เลือกในแท็บข้อมูลก่อน ค่าจะปรากฏเมื่อมีการรับหรือส่งข้อมูล หน่วยมาจาก DBC เมื่อเปลี่ยนสัญญาณ สเกลอัตโนมัติจะถูกรีเซ็ต

ตั้งค่า LED และช่องข้อความ

การตั้งค่า	**ผล**
ป้ายกำกับ / ข้อความ	LED: ป้ายข้าง LED (ถ้าว่างใช้ชื่อสัญญาณ) ช่องข้อความ: ข้อความอิสระ เมื่อมีสัญญาณจะอยู่หน้าค่า
โหมด LED	ดิจิทัล (ค่า 0 / ค่า 1) หรือช่วง (ช่วง 1 ถึง 3) ดิจิทัล: ค่า 0 ให้สี 0 ค่าอื่นทั้งหมดให้สี 1
สี / ช่วง	ดิจิทัล: สีสำหรับค่า 0 และค่า 1 ช่วง: แต่ละช่วงมีเปิด/ปิด ค่าต่ำสุด ค่าสูงสุด และสี ตามกฎ **ค่าต่ำสุด <= x < ค่าสูงสุด** ช่วงแรกที่ตรงเงื่อนไขกำหนดสี เป็นการตั้งค่าเดียวกับการแสดงช่วงค่าของเข็ม
ทศนิยม	จำนวนทศนิยม 0 ถึง 9 สำหรับช่องข้อความและเข็ม
หัวเรื่อง (เข็ม)	ช่องติ๊ก **แสดงหัวเรื่อง** แสดงหรือซ่อนป้ายเหนือเข็ม พื้นที่จะถูกใช้สำหรับสเกลแทน
แสดงช่วงค่า (เข็ม)	ช่องติ๊ก **แสดงช่วงค่า** ใส่สีให้สเกลได้สูงสุดสามช่วง (ช่วง 1 ถึง 3) แต่ละช่วงมีค่าต่ำสุด ค่าสูงสุด สี และเปิด/ปิด ตามกฎ **ค่าต่ำสุด <= x < ค่าสูงสุด** จากนั้นทั้งสามช่วงจะปรากฏในเมนู
เข็มต่ำสุด/สูงสุด (เข็ม)	เมื่อติ๊ก **เข็มค่าต่ำสุด** และ **เข็มค่าสูงสุด** สเกลจะแสดงเข็มบางเพิ่มอีกสองเข็ม (สีน้ำเงินและสีส้ม) สำหรับค่าต่ำสุดและสูงสุดตั้งแต่เริ่มสังเกต **รีเซ็ตต่ำสุด/สูงสุด** เริ่มใหม่ การเปลี่ยนสัญญาณหรือสลับช่องติ๊กก็รีเซ็ตเช่นกัน ค่าไม่ถูกบันทึกในโปรเจกต์ บันทึกเฉพาะช่องติ๊ก

สเกลของเข็มโดยละเอียด

อัตโนมัติ: จนกว่าจะได้ค่าแรก ใช้ 0 ถึง 100 หลังจากนั้นเข็มจดจำค่าต่ำสุดและสูงสุดที่สังเกตได้ และปัดขอบเขตเป็นเลขกลม (1, 2, 2.5, 5 คูณเลขยกกำลังสิบ) หากทุกค่าไม่เป็นลบและใกล้ 0 สเกลจะเริ่มที่ 0 สเกลขยายได้แต่ไม่หดลง ช่องติ๊ก **สเกลอัตโนมัติ** ในเมนูคลิกขวา (ปิดแล้วเปิดใหม่) จะเริ่มสังเกตใหม่ สเกลแสดงขีดห้าขีดพร้อมตัวเลข

กำหนดเอง: คลิกขวาที่เข็ม เลือก **สเกลกำหนดเอง (ต่ำสุด/สูงสุด)**: ป้อนค่าต่ำสุดและสูงสุดในกล่องโต้ตอบ (ใส่ทศนิยมได้) หากค่าต่ำสุดไม่น้อยกว่าค่าสูงสุด จะมีข้อความแจ้งและไม่ใช้ค่า สเกลกำหนดเองมีผลทันที ถูกบันทึกในโปรเจกต์ และปิดโหมดอัตโนมัติ ค่านอกช่วงทำให้เข็มหยุดที่ปลายสุด

พื้นหลังและรูปภาพ

องค์ประกอบอยู่ในชั้น: พื้นหลังมาตรฐาน (พื้นที่วาด) ด้านบนคือองค์ประกอบพื้นหลัง และด้านบนสุดคือองค์ประกอบที่ใช้งาน (LED ช่องข้อความ เข็ม รูปภาพ) องค์ประกอบพื้นหลังจะถูกวางไว้หลังองค์ประกอบอื่นอัตโนมัติเมื่อเพิ่ม ย้าย และโหลด ดังนั้นกรอบสีหนึ่งกรอบจึงจัดกลุ่มการแสดงผลหลายอันได้ รูปที่เลือกจะถูกคัดลอกไปยังโฟลเดอร์โปรเจกต์ (`images`) เพื่อให้ย้ายโปรเจกต์ได้ หากไม่มีรูปจะแสดง "ไม่มีรูปภาพ"

สี LED และการแสดงช่วงค่า

ตัวอย่าง LED ดิจิทัล: ค่า 0 = แดง ค่า 1 = เขียว ตัวอย่าง LED แบบช่วง (อุณหภูมิ): ช่วง 1: 0 ถึง 80 เขียว ช่วง 2: 80 ถึง 100 เหลือง ช่วง 3: 100 ถึง 120 แดง นอกช่วง LED เป็นสีเทา

การแสดงช่วงค่าของเข็ม: มีสามช่วง แต่ละช่วงมีค่าต่ำสุด ค่าสูงสุด และสี ค่า x อยู่ในช่วงเมื่อ **ค่าต่ำสุด <= x < ค่าสูงสุด** เป็นจริง (รวมค่าต่ำสุด ไม่รวมค่าสูงสุด) ตัวอย่าง: ช่วง 1: 0 ถึง 80 เขียว ช่วง 2: 80 ถึง 100 เหลือง ช่วง 3: 100 ถึง 120 แดง แต่ละช่วงเปิดหรือปิดแยกกันได้ ส่วนที่อยู่นอกสเกลจะถูกตัดออก และค่าต่ำสุดต้องน้อยกว่าค่าสูงสุด เมื่อใช้สเกลอัตโนมัติ แถบสีจะปรับตามสเกลปัจจุบัน

การบันทึกและข้อจำกัด

องค์ประกอบ ตำแหน่ง ขนาด การเชื่อมต่อ และการตั้งค่าถูกบันทึกในโปรเจกต์ (`builder`) พื้นที่วาดเลื่อนได้ GUI Builder แสดงเฉพาะค่า ไม่มีองค์ประกอบป้อนข้อมูลหรือสวิตช์ แสดงได้เฉพาะสัญญาณที่เลือกไว้ในแท็บข้อมูล

หมายเหตุ: สัญญาณจะปรากฏเมื่อมีการรับหรือส่งข้อมูลแล้ว จนกว่าจะถึงตอนนั้นช่องข้อความและเข็มแสดง "--" หรือตำแหน่งศูนย์ และ LED ดับ

ภาคผนวก A: อ้างอิงคำสั่งลำดับทดสอบ

แต่ละขั้นตอนคือ JSON อ็อบเจกต์ที่มีฟิลด์ `type` และองค์ประกอบที่ระบุด้านล่าง ค่าทางกายภาพคือค่าหลังคำนวณ factor และ offset ของ DBC

set_signal - ตั้งค่าสัญญาณ

ตั้งค่าทางกายภาพของสัญญาณในข้อความส่ง ค่าใหม่จะออกบนบัสพร้อมเฟรมถัดไป (แบบเป็นรอบหรือส่งครั้งเดียว)

รูปแบบ

```
{"type": "set_signal", "message": "<ข้อความ>", "signal": "<สัญญาณ>", "value": <ตัวเลขหรือข้อความ>}
```

องค์ประกอบ

องค์ประกอบ	ชนิด	คำอธิบาย
`message`	ข้อความ, จำเป็น	ชื่อข้อความใน DBC หากไม่อยู่ในรายการส่งจะถูกเพิ่มให้ (ตอนแรกยังไม่ส่งเป็นรอบ)
`signal`	ข้อความ, จำเป็น	ชื่อสัญญาณในข้อความ
`value`	ตัวเลขหรือข้อความ, จำเป็น	ค่าทางกายภาพ (หลัง factor/offset เช่น 40 สำหรับ 40 %) สำหรับสัญญาณที่มีตารางค่าใช้ข้อความของตารางได้ เช่น "R" ค่านอกช่วงของสัญญาณจะถูกจำกัด

ตัวอย่าง

```
{
  "type": "set_signal",
  "message": "DriverCmd",
  "signal": "ThrottleCmd",
  "value": 40
}
```

หมายเหตุ: สัญญาณอื่นของข้อความที่ไม่ได้ระบุจะคงค่าเดิม (ค่าเริ่มต้น 0)

enable_tx - เปิด/ปิดการส่งเป็นรอบ

เริ่มหรือหยุดการส่งข้อความเป็นรอบตามคาบที่ตั้งไว้

รูปแบบ

```
{"type": "enable_tx", "message": "<ข้อความ>", "enabled": <true|false>}
```

องค์ประกอบ

องค์ประกอบ	ชนิด	คำอธิบาย
`message`	ข้อความ, จำเป็น	ชื่อข้อความใน DBC (เพิ่มในรายการส่งเมื่อจำเป็น คาบจาก DBC หรือ 100 ms หากไม่มี)
`enabled`	true/false, จำเป็น	true = ส่งเป็นรอบ, false = หยุดส่ง

ตัวอย่าง

```
{
  "type": "enable_tx",
  "message": "DriverCmd",
  "enabled": true
}
```

หมายเหตุ: เฟรมแรกส่งทันที จากนั้นส่งตามคาบที่ตั้งไว้

set_cycle - เปลี่ยนคาบ

เปลี่ยนคาบของข้อความส่ง (มีผลทันที แม้ขณะกำลังส่งเป็นรอบ)

รูปแบบ

```
{"type": "set_cycle", "message": "<ข้อความ>", "cycle_ms": <ตัวเลข>}
```

องค์ประกอบ

องค์ประกอบ	ชนิด	คำอธิบาย
`message`	ข้อความ, จำเป็น	ชื่อข้อความใน DBC
`cycle_ms`	ตัวเลข, จำเป็น	คาบเป็นมิลลิวินาที 0 = หยุดส่งเป็นรอบชั่วคราว

ตัวอย่าง

```
{
  "type": "set_cycle",
  "message": "DriverCmd",
  "cycle_ms": 20
}
```

send_once - ส่งข้อความหนึ่งครั้ง

ส่งหนึ่งเฟรมด้วยค่าสัญญาณปัจจุบัน โดยไม่ขึ้นกับการส่งเป็นรอบ

รูปแบบ

```
{"type": "send_once", "message": "<ข้อความ>"}
```

องค์ประกอบ

องค์ประกอบ	ชนิด	คำอธิบาย
`message`	ข้อความ, จำเป็น	ชื่อข้อความใน DBC

ตัวอย่าง

```
{
  "type": "send_once",
  "message": "DriverCmd"
}
```

send_raw - ส่งเฟรมดิบ

ส่งเฟรมดิบ (ID + ข้อมูล) บนบัส CAN ครั้งเดียวหรือวนซ้ำ เมื่อ cyclic=true และ enabled=false จะหยุดการส่งวนซ้ำของ ID นี้

รูปแบบ

```
{"type": "send_raw", "id": "<hex>", "data": "<hex>", "dlc": <n>, "cyclic": <true|false>, "cycle_ms": <n>}
```

องค์ประกอบ

องค์ประกอบ	ชนิด	คำอธิบาย
`id`	ตัวเลขหรือข้อความ, จำเป็น	CAN ID ฐานสิบหก (เช่น "0x100" หรือ "100")
`data`	ข้อความ Hex, จำเป็น	ข้อมูลเป็นไบต์ฐานสิบหก เช่น "00 01 02 03 04 05 06 07"
`dlc`	ตัวเลข, ไม่บังคับ	ความยาวข้อมูล (0-8, 12, 16, 20, 24, 32, 48, 64) ค่าเริ่มต้น: จำนวนไบต์ข้อมูล
`cyclic`	true/false, ไม่บังคับ	true = ส่งวนซ้ำ, false = ส่งครั้งเดียว (ค่าเริ่มต้น)
`cycle_ms`	ตัวเลข, ไม่บังคับ	รอบเวลาเป็น ms เมื่อ cyclic=true (ค่าเริ่มต้น 100)
`enabled`	true/false, ไม่บังคับ	false หยุดการส่งวนซ้ำของ ID นี้ (ใช้กับ cyclic=true เท่านั้น ค่าเริ่มต้น true)
`extended`	true/false, ไม่บังคับ	true บังคับใช้ ID แบบขยาย (ค่าเริ่มต้น: อัตโนมัติเมื่อมากกว่า 0x7FF)
`fd`	true/false, ไม่บังคับ	true บังคับใช้ CAN FD (ค่าเริ่มต้น: อัตโนมัติเมื่อ DLC มากกว่า 8)

ตัวอย่าง

```
{
  "type": "send_raw",
  "id": "0x100",
  "data": "00 01 02 03 04 05 06 07",
  "dlc": 8,
  "cyclic": true,
  "cycle_ms": 1000
}
```

หมายเหตุ: ไบต์ข้อมูลที่ขาดจะเติมด้วย 00 ID มากกว่า 0x7FF = เฟรมขยาย DLC มากกว่า 8 = CAN FD เฟรมที่ส่งวนซ้ำจะทำงานต่อหลังขั้นตอนนี้

wait - รอตามเวลาที่กำหนด

หยุดลำดับชั่วคราวตามเวลาที่กำหนด การส่งเป็นรอบและการรับยังทำงานต่อ

รูปแบบ

```
{"type": "wait", "ms": <ตัวเลข>}
```

องค์ประกอบ

องค์ประกอบ	ชนิด	คำอธิบาย

ms	ตัวเลข, จำเป็น	เวลารอเป็นมิลลิวินาที

ตัวอย่าง

```
{
  "type": "wait",
  "ms": 500
}
```

หมายเหตุ: การกดหยุดจะยุติการรอที่กำลังทำงานอยู่ทันที

wait_for - รอเงื่อนไขของสัญญาณ

รอจนสัญญาณเป็นไปตามเงื่อนไข นับเฉพาะค่าที่ได้รับหลังจากขั้นตอนนี้เริ่มแล้ว ขั้นตอนเป็น ok เมื่อพบครั้งแรก มิฉะนั้นเป็น error เมื่อหมดเวลา timeout

รูปแบบ

```
{"type": "wait_for", "direction": "RX", "message": "<ข้อความ>", "signal": "<สัญญาณ>", "op": "<ตัวดำเนินการ>", "value": <ตัวเลข>, "timeout_ms": <
```

องค์ประกอบ

องค์ประกอบ	ชนิด	คำอธิบาย
direction	"RX" หรือ "TX", จำเป็น	ประเมินข้อความที่รับหรือที่ส่ง
message	ข้อความ, จำเป็น	ชื่อข้อความใน DBC
signal	ข้อความ, จำเป็น	ชื่อสัญญาณ
op	ข้อความ, จำเป็น	การเปรียบเทียบ: == != < <= > >=
value	ตัวเลข, จำเป็น	ค่าเปรียบเทียบ (ค่าทางกายภาพ)
timeout_ms	ตัวเลข, ไม่บังคับ	เวลารอสูงสุดเป็น ms (ค่าเริ่มต้น 3000)

ตัวอย่าง

```
{
  "type": "wait_for",
  "direction": "RX",
  "message": "EngineData",
  "signal": "RPM",
  "op": ">",
  "value": 100,
  "timeout_ms": 3000
}
```

หมายเหตุ: การประเมินไม่ขึ้นกับการเลือกสัญญาณในแท็บ 'ข้อมูล'

check - ตรวจสอบค่าสัญญาณ

ตรวจสอบค่าล่าสุดที่ได้รับของสัญญาณทันที (ไม่รอ) ขั้นตอนจะล้มเหลวหากไม่เป็นไปตามเงื่อนไข หรือยังไม่เคยได้รับค่าตั้งแต่เริ่มลำดับ

รูปแบบ

```
{"type": "check", "direction": "RX", "message": "<ข้อความ>", "signal": "<สัญญาณ>", "op": "<ตัวดำเนินการ>", "value": <ตัวเลข>}
```

องค์ประกอบ

องค์ประกอบ	ชนิด	คำอธิบาย
direction	"RX" หรือ "TX", จำเป็น	ประเมินข้อความที่รับหรือที่ส่ง
message	ข้อความ, จำเป็น	ชื่อข้อความใน DBC
signal	ข้อความ, จำเป็น	ชื่อสัญญาณ
op	ข้อความ, จำเป็น	การเปรียบเทียบ: == != < <= > >=
value	ตัวเลข, จำเป็น	ค่าเปรียบเทียบ (ค่าทางกายภาพ)

ตัวอย่าง

```
{
  "type": "check",
  "direction": "RX",
  "message": "EngineData",
  "signal": "CoolantTemp",
  "op": "<",
  "value": 150
}
```

หมายเหตุ: หลังเริ่มลำดับมักยังไม่มีค่า - ควรซิงโครไนซ์ด้วย wait_for ก่อน

diag - Service วินิจฉัยจาก CDD

ส่ง Service วินิจฉัยที่นิยามใน CDD ผ่าน ISO-TP และประเมินการตอบกลับ

รูปแบบ

```
{"type": "diag", "service": "<ชื่อ Service>", "params": {"<พารามิเตอร์>": <ค่า>}, "expect": "<Hex นำหน้า>", "timeout_ms": <ตัวเลข>}
```

องค์ประกอบ

องค์ประกอบ	ชนิด	คำอธิบาย
service	ข้อความ, จำเป็น	ชื่อ Service ใน CDD (แท็บวินิจฉัย) เช่น ReadVIN
params	อ็อบเจกต์, ไม่บังคับ	ค่าสำหรับ {Platzhalter} ใน Request ตัวเลข = ฐานสิบ, ข้อความ = ฐานสิบหก ("F186")
expect	ข้อความ Hex, ไม่บังคับ	ส่วนต้นของการตอบกลับที่คาดหวัง เช่น "62 F1 90" ว่าง = การตอบกลับแบบ positive ใดก็ได้
timeout_ms	ตัวเลข, ไม่บังคับ	Timeout การตอบกลับเป็น ms (ค่าเริ่มต้น 1000) กรณี 'ResponsePending' (NRC 0x78) จะรอนานขึ้น

ตัวอย่าง

```
{
  "type": "diag",
  "service": "ExtendedSession",
  "params": {
    "session": 3
  },
  "expect": "50 03"
}
```

หมายเหตุ: การระบุที่อยู่ (Request/Response-ID, FD) มาจากแท็บ 'วินิจฉัย'

diag_raw - Request วินิจฉัยแบบ Hex

ส่ง UDS Request ใดก็ได้ (ไบต์ Hex) ผ่าน ISO-TP และประเมินการตอบกลับ

รูปแบบ

```
{"type": "diag_raw", "request": "<ไบต์ Hex>", "expect": "<Hex นำหน้า>", "timeout_ms": <ตัวเลข>}
```

องค์ประกอบ

องค์ประกอบ	ชนิด	คำอธิบาย
request	ข้อความ Hex, จำเป็น	ไบต์ของ Request เช่น "22 F1 86"
expect	ข้อความ Hex, ไม่บังคับ	ส่วนต้นของการตอบกลับที่คาดหวัง เช่น "62 F1 86" ว่าง = การตอบกลับแบบ positive ใดก็ได้
timeout_ms	ตัวเลข, ไม่บังคับ	Timeout การตอบกลับเป็น ms (ค่าเริ่มต้น 1000)

ตัวอย่าง

```
{
  "type": "diag_raw",
  "request": "3E 00",
  "expect": "7E 00"
}
```

หมายเหตุ: Request ที่ไม่มีการตอบกลับ (Sub-Function ที่ตั้งบิต 7 เช่น "3E 80") ถือว่า ok

loop - วนซ้ำ

รันขั้นตอนที่อยู่ภายในซ้ำหลายรอบต่อเนื่องกัน

รูปแบบ

```
{"type": "loop", "count": <ตัวเลข>, "steps": [ <ขั้นตอน>, <ขั้นตอน>, ... ]}
```

องค์ประกอบ

องค์ประกอบ	ชนิด	คำอธิบาย
count	ตัวเลข, จำเป็น	จำนวนรอบ
steps	รายการ, จำเป็น	ขั้นตอนซ้อน (ชนิดใดก็ได้ รวมถึงลูปซ้อนลูป)

ตัวอย่าง

```
{
  "type": "loop",
  "count": 3,
  "steps": [
    {
      "type": "set_signal",
      "message": "DriverCmd",
      "signal": "ThrottleCmd",
      "value": 20
    },
    {
      "type": "wait",
      "ms": 200
    }
  ]
}
```

หมายเหตุ: ในผลลัพธ์ ขั้นตอนภายในจะแสดงเช่น 3.2.1 (ขั้นตอนที่ 3, รอบที่ 2, ขั้นตอนที่ 1)

comment - หมายเหตุ

ไม่ทำอะไร ใช้บันทึกเอกสารของลำดับเท่านั้น

รูปแบบ

```
{"type": "comment", "text": "<ข้อความ>"}
```

องค์ประกอบ

องค์ประกอบ	ชนิด	คำอธิบาย
text	ข้อความ, ไม่บังคับ	หมายเหตุใดก็ได้

ตัวอย่าง

```
{
  "type": "comment",
  "text": "Hier beginnt der Hochlauftest"
}
```

ภาคผนวก B: หมายเหตุและข้อจำกัด

- **ความแม่นยำของเวลา**: การส่งเป็นรอบใช้ซอฟต์แวร์ตัวตั้งเวลา (ความแม่นยำในระดับไม่กี่มิลลิวินาที) ไม่ได้ trigger ด้วยฮาร์ดแวร์
- **DBC**: ไม่รองรับ Extended Multiplexing และ environment variable
- **CDD**: ไฟล์ CANdelaStudio ต้นฉบับอ่านแบบ heuristic เท่านั้น (บทที่ 2.2)
- **PEAK + CAN FD**: bit timing กำหนดตายตัว (80 MHz, 500k/2M) หากต้องการค่าอื่นใช้ `hardware.extra`
- **ความปลอดภัย**: การส่งเป็นรอบไม่เริ่มเองอัตโนมัติเมื่อโหลดโปรเจกต์ สำหรับบัสของรถหรือ test bench ให้ตรวจรายการส่งและค่าก่อนเชื่อมต่อ
- **แป้นพิมพ์**: ในตารางให้ดับเบิลคลิกเพื่อแก้ไข ในช่องวินิจฉัยแบบ raw การกด Enter จะส่ง Request

การทดสอบโดยไม่มีฮาร์ดแวร์

โปรเจกต์ตัวอย่าง `demo` ใช้การจำลองพร้อม ECU เสมือน จึงลองใช้ทุกแท็บได้ (ส่ง/รับ, Trace, กราฟ, Logging, ลำดับ *startup* และ *driving*, วินิจฉัย) โดยไม่ต้องมีฮาร์ดแวร์ การทดสอบแกนหลักรันด้วย `python -m unittest discover -s tests -v`

ภาคผนวก C: แพ็ก vFlash และ Manifest

แท็บ vFlash ในหมวด Diagnose จะโหลดแพ็กแฟลช ตีความ และรันด้วยลำดับการแฟลช UDS ในตัวบนฮาร์ดแวร์ที่เลือก (Vector, PEAK, SocketCAN, simulation) โดยไม่ต้องใช้ซอฟต์แวร์ vFlash ของ Vector

แหล่งที่มาที่รองรับ

- `*.vflashpack` หรือ `*.zip` ที่มี `flashpack.json` (หรือ `flashpack.yaml`) และไฟล์ข้อมูล: รองรับเต็มรูปแบบ
- แพ็ก vFlash จริงที่ไม่มี manifest: รูปแบบภายในไม่มีเอกสารเผยแพร่สู่สาธารณะ ระบบจะค้นหาไฟล์ HEX, S-Record และ BIN และอ่านโปรเจกต์แบบ heuristic โดยส่วนใหญ่ต้องเพิ่มลำดับขั้นตอน address และ Seed&Key ผ่าน manifest
- ไฟล์ manifest เดี่ยวหรือ image (`.hex`, `.s19`, `.srec`, `.mot`, `.bin`) พร้อมลำดับขั้นตอนมาตรฐานทั่วไป

Manifest flashpack.json

```
{
  "format": "canTool-flashpack-1",
  "name": "Demo Application",
  "ecu": "Engine_ECU",
  "request_id": "0x7E0",
  "response_id": "0x7E8",
  "extended": false,
  "fd": false,
  "security": {
    "type": "xor",
    "level": 1,
    "value": "0xFFFFFFFF"
  },
  "address_format": "0x44",
  "data": [
    {
      "file": "app.hex",
      "name": "Application"
    },
    {
      "file": "cal.bin",
      "address": "0x00020000",
      "name": "Calibration"
    }
  ],
  "steps": [
    {
      "type": "session",
      "session": 3
    },
    {
      "type": "comm_control",
      "control": 3,
      "comm_type": 1
    },
    {
      "type": "session",
      "session": 2
    },
    {
      "type": "security",
      "level": 1
    },
    {
      "type": "erase",
      "block": "all"
    },
    {
      "type": "download",
      "block": "all"
    },
    {
      "type": "check",
      "routine": "FF01"
    },
    {
      "type": "reset",
      "reset": 1
    }
  ]
}
```

องค์ประกอบ	ความหมาย
`name`, `ecu`	ชื่อที่แสดงและ ECU
`request_id`, `response_id`, `extended`, `fd`	การกำหนดแอดเดรส ISO-TP หากไม่ระบุ จะใช้การตั้งค่า Diagnose ของหมวด Diagnose
`security`	Seed&Key: `type` = `none`, `xor`, `add` หรือ `constant` พร้อม `level` (seed level, key = level + 1) และ `value`
`address_format`	addressAndLengthFormatIdentifier เช่น `0x44` = length 4 ไบต์ และ address 4 ไบต์
`max_chunk`	ขีดจำกัดสูงสุดของข้อมูลต่อ transfer request (มิฉะนั้นจะใช้ค่าจาก response ของ 0x34)
`data`	รายการไฟล์ข้อมูล (`file`, `name`) สำหรับไฟล์ BIN ต้องระบุ `address`

`steps`	ลำดับขั้นตอนของตัวเอง หากไม่ระบุ จะใช้ลำดับขั้นตอนมาตรฐาน

ขั้นตอนของลำดับ (steps)

ขั้นตอน	พารามิเตอร์และผล
`session`	`session`: diagnostic session (0x10)
`comm_control`	`control`, `comm_type`: communication control (0x28)
`security`	`level`: ขอ seed คำนวณและส่ง key (0x27)
`erase`	`block` (`all` หรือหมายเลข), `routine` (ค่าเริ่มต้น FF00): ลบหน่วยความจำ (0x31)
`download`	`block`: Request Download, Transfer Data, Transfer Exit (0x34, 0x36, 0x37)
`check`, `routine`	`routine`, `data` (hex), `control`, `timeout_ms`: เริ่ม routine (0x31)
`request`	`data` (hex), `expect` (ไบต์แรกของ response ที่คาดไว้)
`reset`	`reset`: ECU reset (0x11)
`wait`, `comment`	`ms` เวลารอ หรือ `text` เป็นความคิดเห็น

ลำดับขั้นตอนมาตรฐาน

หากไม่มี `steps` จะดำเนินการตามลำดับดังนี้: session 0x03, ปิดการสื่อสาร (0x28 03 01), session 0x02, security access (หากตั้ง `security`), ลบและโปรแกรมทุก block, ECU reset 0x11 01 ลำดับนี้เป็นแบบทั่วไปและไม่เฉพาะผู้ผลิต

ข้อควรระวังด้านความปลอดภัย

การแฟลชจะลบและเขียนทับหน่วยความจำของ ECU ห้ามตัดแหล่งจ่ายไฟและการเชื่อมต่อบัสในระหว่างนี้ โค้ดจากแพ็กจะไม่ถูกรันเด็ดขาด และไม่รองรับ DLL Seed&Key เฉพาะผู้ผลิต เมื่อเกิดข้อผิดพลาด ลำดับขั้นตอนจะหยุดและข้ามขั้นตอนที่เหลือ